

Lean 4 定理证明

Jeremy Avigad

Leonardo de Moura
Sebastian Ullrich

Soonho Kong

Contents

Contents	i
1 引言	1
1.1 计算机与定理证明	1
1.2 关于 Lean	2
1.3 关于本书	2
1.4 致谢	3
2 依值类型论	5
2.1 简单类型论	5
2.2 作为对象的类型	8
2.3 函数抽象与求值	11
2.4 定义	14
2.5 局部定义	16
2.6 变量与段落	17
2.7 命名空间	18
2.8 依值类型论为何“依值”？	21
2.9 隐式参数	23
3 命题与证明	29
3.1 命题即类型	29
3.2 使用命题即类型	31
3.3 命题逻辑	34
3.4 引入辅助子目标	39
3.5 经典逻辑	40
3.6 命题有效式示例	41
3.7 练习	44
4 量词与等式	45
4.1 全称量词	45
4.2 等式	48
4.3 计算式证明	51
4.4 存在量词	55
4.5 证明语言补充	61

4.6	练习	62
5	策略	65
5.1	进入策略模式	65
5.2	基本策略	68
5.3	更多策略	72
5.4	组织策略式证明	76
5.5	策略组合子	79
5.6	重写	81
5.7	使用化简器	83
5.8	split 策略	89
5.9	可扩展策略	89
5.10	练习	90
6	与 Lean 交互	91
6.1	消息	91
6.2	导入文件	93
6.3	关于节的更多说明	93
6.4	关于命名空间的更多说明	94
6.5	属性	97
6.6	关于隐式参数的更多说明	99
6.7	记法	101
6.8	强制转换	102
6.9	显示信息	103
6.10	设置选项	104
6.11	使用库	105
6.12	自动绑定的隐式参数	107
6.13	隐式 lambda 抽象	109
6.14	简单函数的语法糖	109
6.15	命名参数	110
7	归纳类型	113
7.1	枚举类型	114
7.2	带参数的构造子	119
7.3	归纳定义的命题	122
7.4	定义自然数	123
7.5	其他递归数据类型	126
7.6	归纳类型的策略	128
7.7	归纳族	133
7.8	公理细节	136
7.9	相互和嵌套归纳类型	136
7.10	练习	137
8	归纳与递归	139
8.1	模式匹配	139
8.2	通配符与重叠模式	143

8.3	结构递归与结构归纳	144
8.4	局部递归声明	148
8.5	良基递归与归纳	149
8.6	函数式归纳	154
8.7	相互递归	156
8.8	依值模式匹配	159
8.9	不可访问模式	162
8.10	匹配表达式	164
8.11	练习	165
9	结构与记录	169
9.1	声明结构	169
9.2	对象	172
9.3	继承	173
10	类型类	175
10.1	实例的链接	177
10.2	ToString (转为字符串)	178
10.3	数值字面量	179
10.4	输出参数	180
10.5	默认实例	182
10.6	局部实例	184
10.7	作用域实例	185
10.8	可判定命题	186
10.9	管理类型类推断	189
10.10	使用类型类的强制转换	191
11	转换策略模式	195
11.1	基本导航与重写	195
11.2	模式匹配	196
11.3	组织转换策略	197
11.4	转换模式中的其他策略	197
12	公理与计算	199
12.1	历史与哲学背景	200
12.2	命题外延性	200
12.3	函数外延性	201
12.4	商	203
12.5	选择	208
12.6	排中律	210

本书此版本假定你使用 Lean 4（具体为 4.29.0-rc7）。安装 Lean 请参见 Lean 文档中的 快速入门¹。本书的第一版是为 Lean 2 编写的，Lean 3 版本见此处²。

¹<https://lean-lang.org/documentation/setup/>

²https://leanprover.github.io/theorem_proving_in_lean/

1 引言

1.1 计算机与定理证明

形式化验证使用逻辑和计算方法来确立以精确数学术语表述的断言。这些断言既可以包括通常的数学定理，也可以包括关于硬件或软件、网络协议，以及机械系统和混合系统是否满足其规约的断言。在实践中，验证一段数学内容与验证一个系统的正确性之间并没有泾渭分明的界限：形式化验证需要用数学术语描述硬件和软件系统；在这一点上，确立关于其正确性的断言就成为一种定理证明。反过来，一个数学定理的证明可能需要冗长的计算；在这种情况下，验证该定理为真就需要验证该计算确实完成了它应当完成的任务。

支持一个数学断言的黄金标准是给出证明，而二十世纪逻辑学的发展表明，在多种基础系统中的任何一种之内，绝大多数（即便不是全部）传统证明方法都可以化约为一小组公理和规则。有了这种化约，计算机可以通过两种方式帮助确立一个断言：它首先可以帮助寻找证明，也可以帮助验证一个声称的证明是否正确。

自动定理证明关注“寻找”这一方面。归结定理证明器、语义表定理证明器、快速可满足性求解器等，为确立命题逻辑和一阶逻辑中公式的有效性提供了手段。其他系统则为特定语言和领域提供搜索过程和判定过程，例如整数或实数上的线性或非线性表达式。诸如 SMT（可满足性模理论，*satisfiability modulo theories*）这样的架构，将领域通用的搜索方法与领域专用的过程结合起来。计算机代数系统和专门的数学软件包提供了执行数学计算、确立数学界限或寻找数学对象的手段。计算也可以被视为一种证明，因此这些系统同样有助于确立数学断言。

自动推理系统追求能力和效率，往往以牺牲有保证的可靠性为代价。此类系统可能存在缺陷，也可能难以确保它们给出的结果是正确的。相比之下，交互式定理证明关注定理证明中“验证”的方面，要求每一个断言都由适当公理化基础中的一个证明来支持。这设定了非常高的标准：每一条推理规则、每一个计算步骤，都必须通过诉诸既有定义和定理而得到辩护，并一直追溯到基本公理和规则。事实上，大多数这样的系统会给出完全细化的“证明对象”，它们可以被传递给其他系统并独立检查。构造这样的证明通常需要用户提供更多输入并进行更多交互，但它使你能够获得更深入、更复杂的证明。

Lean Theorem Prover 旨在弥合交互式定理证明与自动定理证明之间的鸿沟：它把自动化工具和方法置于一个既支持用户交互、又支持构造完全指定的公理化证明的框架之中。其目标是同时支持数学推理和关于复杂系统的推理，并验证这两个领域中的断言。

Lean 的底层逻辑具有计算解释，因此 Lean 同样可以被看作一种编程语言。更切要地说，它可以被视为一个用于编写具有精确语义的程序的系统，也可以用于推理这些程序所计算的函数。Lean 还具有充当自身元编程语言的机制，这意味着你可以使用 Lean 本身实现自动化并扩展 Lean 的功能。Lean 的这些方面在免费在线书籍 *Functional Programming in Lean*¹ 中有所说明，不过本书也会涉及该系统的计算层面。

1.2 关于 Lean

Lean 项目由 Leonardo de Moura 于 2013 年在 Microsoft Research Redmond 发起。这是一项持续进行的长期工作，其自动化方面的许多潜力只会随着时间推移逐步实现。Lean 以 Apache 2.0 许可证² 发布；这是一种宽松的开源许可证，允许他人自由使用和扩展其代码与数学库。

若要在你的计算机上安装 Lean，可考虑使用 Quickstart³ 说明。Lean 的源代码以及构建 Lean 的说明可在 <https://github.com/leanprover/lean4>⁴ 获取。

本教程描述的是当前版本的 Lean，即 Lean 4。

1.3 关于本书

本书旨在教你在 Lean 中发展并验证证明。为此所需的许多背景知识其实并不专属于 Lean。首先，你将学习 Lean 所基于的逻辑系统：一种 依值类型论 的版本；它强大到足以证明几乎任何传统数学定理，也具有足够的表达力，能以自然的方式完成这些证明。更具体地说，Lean 基于一种被称为带归纳类型的构造演算的系统。Lean 不仅能够在依值类型论中定义数学对象、表达数学断言，还可以作为书写证明的语言使用。

由于完全详细的公理化证明极其复杂，定理证明的挑战在于让计算机尽可能填补更多细节。你将在 chapter 2 (p. 5) 中学习支持这一目标的多种方法。例如，项重写，以及 Lean 自动简化项和表达式的自动化方法。同样，你也会学习细化与类型推断的方法；它们可用于支持灵活形式的代数推理。

最后，你将学习 Lean 所特有的一些功能，包括你用来与系统通信的语言，以及 Lean 为管理复杂理论和数据所提供的机制。

在全文中，你会看到如下所示的 Lean 代码示例：

```
theorem and_commutative (p q : Prop) : p ∧ q → q ∧ p :=
  fun hpq : p ∧ q =>
    have hp : p := And.left hpq
    have hq : q := And.right hpq
    show q ∧ p from And.intro hq hp
```

在本书的每个代码示例旁，你都会看到一个标为“Copy to clipboard”的按钮。按下该按钮会复制示例，并附带足够的上下文，使代码能够正确编译。

¹https://lean-lang.org/functional_programming_in_lean/

²<https://github.com/leanprover/lean4/blob/master/LICENSE>

³<https://lean-lang.org/install/>

⁴<https://github.com/leanprover/lean4/>

你可以将示例代码粘贴到 VS Code⁵ 中并修改这些示例；Lean 会在你输入时持续检查结果并提供反馈。我们建议你在阅读后续章节时亲自运行这些示例，并对代码进行实验。你可以在 VS Code 中使用命令 “Lean 4: Docs: Show Documentation Resources”，并在打开的标签页中选择 “Theorem Proving in Lean 4” 来打开本书。

1.4 致谢

本教程是一个维护在 Github 上的开放获取项目。许多人为这项工作做出了贡献，提供了修正、建议、示例和文本。我们感谢 Ulrik Buchholz、Kevin Buzzard、Mario Carneiro、Nathan Carter、Eduardo Cavazos、Amine Chaieb、Joe Corneli、William DeMeo、Marcus Klaas de Vries、Ben Dyer、Gabriel Ebner、Anthony Hart、Simon Hudon、Sean Leather、Assia Mahboubi、Gihan Marasingha、Patrick Massot、Christopher John Mazey、Sebastian Ullrich、Floris van Doorn、Daniel Velleman、Théo Zimmerman、Paul Chisholm、Chris Lovett 和 Siddhartha Gadgil 的贡献。关于我们出色贡献者的最新名单，请参见 lean prover⁶ 和 lean community⁷。

⁵<https://code.visualstudio.com/>

⁶<https://github.com/leanprover/>

⁷<https://github.com/leanprover-community/>

2 依值类型论

依值类型论是一种强大而富有表现力的语言，使你能够表达复杂的数学断言，编写复杂的硬件与软件规约，并以自然且统一的方式对二者进行推理。**Lean** 基于依值类型论的一个版本，即构造演算，并带有可数的、非累积宇宙层级以及归纳类型。到本章结束时，你将理解这些说法的大部分含义。

2.1 简单类型论

“类型论”之所以得名，是因为每个表达式都有一个相关联的类型。例如，在给定上下文中， $x + 0$ 可以表示一个自然数，而 f 可以表示自然数上的一个函数。对于偏好精确定义的读者，**Lean** 中的自然数是任意精度的无符号整数。

下面给出一些示例，说明如何在 **Lean** 中声明对象并检查它们的类型。

```
/- Define some constants. -/

def m : Nat := 1      -- m is a natural number
def n : Nat := 0

def b1 : Bool := true -- b1 is a Boolean
def b2 : Bool := false

/- Check their types. -/

#check m      -- m : Nat
| m : Nat
#check n
| n : Nat
#check n + 0   -- n + 0 : Nat
| n + 0 : Nat
#check m * (n + 0) -- m * (n + 0) : Nat
| m * (n + 0) : Nat
#check b1      -- b1 : Bool
| b1 : Bool
-- "&&" is the Boolean and
#check b1 && b2 -- b1 && b2 : Bool
```

```

| b1 && b2 : Bool
-- Boolean or
#check b1 || b2      -- b1 || b2 : Bool
| b1 || b2 : Bool
-- Boolean "true"
#check true          -- Bool.true : Bool
| Bool.true : Bool
/- Evaluate -/

#eval 5 * 4          -- 20
| 20
#eval m + 2          -- 3
| 3
#eval b1 && b2        -- false
| false

```

位于 `/-` 和 `-/` 之间的任何文本构成一个注释块，Lean 会忽略它。类似地，两个短横线 `--` 表明该行余下部分是注释，也会被忽略。注释块可以嵌套，因此可以像许多编程语言中那样将一大段代码“注释掉”。

def 关键字在工作环境中声明新的常量符号。在上面的示例中，**def** `m : Nat := 1` 定义了一个新的常量 `m`，其类型为 `Nat`，值为 `1`。**#check** 命令要求 Lean 报告它们的类型；在 Lean 中，查询系统信息的辅助命令通常以井号 (`#`) 开头。**#eval** 命令要求 Lean 对给定表达式求值。你应当自己尝试声明一些常量，并对若干表达式做类型检查。以这种方式声明新对象，是试验该系统的一种好方法。

简单类型论的强大之处在于，你可以由已有类型构造新类型。例如，若 `a` 和 `b` 是类型，则 `a → b` 表示从 `a` 到 `b` 的函数类型，而 `a × b` 表示由一个 `a` 的元素与一个 `b` 的元素配对而成的类型，也称为笛卡尔积。注意，`×` 是一个 Unicode 符号。审慎使用 Unicode 可以提高可读性，所有现代编辑器也都很好地支持它。在 Lean 标准库中，你经常会看到用希腊字母表示类型，并用 Unicode 符号 `→` 作为 `->` 的更紧凑写法。

```

#check Nat → Nat      -- type the arrow as "\to" or "\r"
| Nat → Nat : Type
#check Nat -> Nat      -- alternative ASCII notation
| Nat → Nat : Type
#check Nat × Nat       -- type the product as "\times"
| Nat × Nat : Type
#check Prod Nat Nat    -- alternative notation
| Nat × Nat : Type
#check Nat → Nat → Nat
| Nat → Nat → Nat : Type

```

```

#check Nat → (Nat → Nat) -- same type as above
| Nat → Nat → Nat : Type
#check Nat × Nat → Nat
| Nat × Nat → Nat : Type
#check (Nat → Nat) → Nat -- a "functional"
| (Nat → Nat) → Nat : Type

#check Nat.succ      -- Nat.succ (n : Nat) : Nat
| Nat.succ (n : Nat) : Nat
#check (0, 1)        -- (0, 1) : Nat × Nat
| (0, 1) : Nat × Nat
#check Nat.add        -- Nat.add : Nat → Nat → Nat
| Nat.add : Nat → Nat → Nat
#check Nat.succ 2     -- Nat.succ 2 : Nat
| Nat.succ 2 : Nat
#check Nat.add 3      -- Nat.add 3 : Nat → Nat
| Nat.add 3 : Nat → Nat
#check Nat.add 5 2    -- Nat.add 5 2 : Nat
| Nat.add 5 2 : Nat
#check (5, 9).1       -- (5, 9).fst : Nat
| (5, 9).fst : Nat
#check (5, 9).2       -- (5, 9).snd : Nat
| (5, 9).snd : Nat
#eval Nat.succ 2      -- 3
| 3
#eval Nat.add 5 2     -- 7
| 7
#eval (5, 9).1        -- 5
| 5
#eval (5, 9).2        -- 9
| 9

```

同样，你应当自己尝试一些示例。

我们来看一些基本语法。你可以通过输入 `\to`、`\r` 或 `\->` 来输入 Unicode 箭头 $\rightarrow$ 。也可以使用 ASCII 替代写法 `->`，因此表达式 `Nat -> Nat` 与 `Nat → Nat` 含义相同。二者都表示以自然数为输入并以自然数为输出的函数类型。笛卡尔积的 Unicode 符号 $\times$ 可通过 `\times` 输入。通常会用小写希腊字母，如 α 、 β 和 γ ，在类型上取值。这些特定字母可分别通过 `\a`、`\b` 和 `\g` 输入。

这里还有几点需要注意。首先，将函数 f 应用于值 x ，记作 $f\ x$ （例如 `Nat.succ 2`）。其次，在书写类型表达式时，箭头向右结合；例如，`Nat.add` 的类型是 $\text{Nat} \rightarrow \text{Nat} \rightarrow \text{Nat}$ ，它等价于 $\text{Nat} \rightarrow (\text{Nat} \rightarrow \text{Nat})$ 。因此，可以把 `Nat.add` 看成一个函数：它接受一个自然数，并返回另一个接受自然数且返回自然数的函数。在类型论中，这通常比把 `Nat.add` 写成接受一对自然数作为输入并返回自然数的函数更方便。例如，这允许你对函数 `Nat.add` 进行“部分应用”。上面的例子表明，`Nat.add 3` 的类型是 $\text{Nat} \rightarrow \text{Nat}$ ；也就是说，`Nat.add 3` 返回一个“等待”第二个参数 n 的函数，这随后等价于写作 `Nat.add 3 n`。你已经看到，若有 $m : \text{Nat}$ 和 $n : \text{Nat}$ ，则 (m, n) 表示由 m 和 n 构成的有序对，其类型为 $\text{Nat} \times \text{Nat}$ 。这给出了创建自然数对的一种方式。反过来，若有 $p : \text{Nat} \times \text{Nat}$ ，则可以写 $p.1 : \text{Nat}$ 和 $p.2 : \text{Nat}$ ，这给出了提取其两个分量的方式。

2.2 作为对象的类型

Lean 的依值类型论扩展简单类型论的一种方式：类型本身——如 `Nat` 和 `Bool` 这样的实体——是一等公民；也就是说，它们自身也是对象。为使这一点成立，它们每一个也都必须有一个类型。

```
#check Nat
| Nat : Type
#check Bool
| Bool : Type
#check Nat → Bool
| Nat → Bool : Type
#check Nat × Bool
| Nat × Bool : Type
#check Nat → Nat
| Nat → Nat : Type
#check Nat × Nat → Nat
| Nat × Nat → Nat : Type
#check Nat → Nat → Nat
| Nat → Nat → Nat : Type
#check Nat → (Nat → Nat)
| Nat → Nat → Nat : Type
#check Nat → Nat → Bool
| Nat → Nat → Bool : Type
#check (Nat → Nat) → Nat
| (Nat → Nat) → Nat : Type
```

可以看到，上面每一个表达式都是类型 `Type` 的对象。也可以为类型声明新的常量：


```

def  $\alpha$  : Type := Nat
def  $\beta$  : Type := Bool
def F : Type  $\rightarrow$  Type := List
def G : Type  $\rightarrow$  Type  $\rightarrow$  Type := Prod

#check  $\alpha$           --  $\alpha$  : Type
|  $\alpha$  : Type
#check F  $\alpha$       -- F  $\alpha$  : Type
| F  $\alpha$  : Type
#check F Nat      -- F Nat : Type
| F Nat : Type
#check G  $\alpha$      -- G  $\alpha$  : Type  $\rightarrow$  Type
| G  $\alpha$  : Type  $\rightarrow$  Type
#check G  $\alpha$   $\beta$   -- G  $\alpha$   $\beta$  : Type
| G  $\alpha$   $\beta$  : Type
#check G  $\alpha$  Nat  -- G  $\alpha$  Nat : Type
| G  $\alpha$  Nat : Type

```

如上例所示，你已经见过类型为 $\text{Type} \rightarrow \text{Type} \rightarrow \text{Type}$ 的函数示例，即笛卡尔积 Prod:

```

def  $\alpha$  : Type := Nat
def  $\beta$  : Type := Bool

#check Prod  $\alpha$   $\beta$       --  $\alpha \times \beta$  : Type
|  $\alpha \times \beta$  : Type
#check  $\alpha \times \beta$       --  $\alpha \times \beta$  : Type
|  $\alpha \times \beta$  : Type
#check Prod Nat Nat     -- Nat  $\times$  Nat : Type
| Nat  $\times$  Nat : Type
#check Nat  $\times$  Nat      -- Nat  $\times$  Nat : Type
| Nat  $\times$  Nat : Type

```

再看另一个例子：给定任意类型 α ，类型 $\text{List } \alpha$ 表示由类型 α 的元素构成的列表的类型。

```

def  $\alpha$  : Type := Nat

#check List  $\alpha$       -- List  $\alpha$  : Type
| List  $\alpha$  : Type
#check List Nat     -- List Nat : Type

```

```

| List Nat : Type

```

既然 Lean 中每个表达式都有类型，一个自然的问题是：Type 本身具有何种类型？

```

#check Type      -- Type : Type 1

```

```

| Type : Type 1

```

你实际上已经触及 Lean 类型系统中最微妙的方面之一。Lean 的底层基础具有一个无限的类型层级：

```

#check Type      -- Type : Type 1

```

```

| Type : Type 1

```

```

#check Type 1    -- Type 1 : Type 2

```

```

| Type 1 : Type 2

```

```

#check Type 2    -- Type 2 : Type 3

```

```

| Type 2 : Type 3

```

```

#check Type 3    -- Type 3 : Type 4

```

```

| Type 3 : Type 4

```

```

#check Type 4    -- Type 4 : Type 5

```

```

| Type 4 : Type 5

```

可以把 Type 0 看作“小”类型或“普通”类型的宇宙。于是 Type 1 是一个更大的类型宇宙，它把 Type 0 作为元素包含在内；而 Type 2 又是一个更大的类型宇宙，它把 Type 1 作为元素包含在内。这个列表是无限的：对每个自然数 n ，都有一个 Type n 。Type 是 Type 0 的缩写：

```

#check Type

```

```

| Type : Type 1

```

```

#check Type 0

```

```

| Type : Type 1

```

下表有助于具体化正在讨论的关系。沿 x 轴移动表示宇宙的变化，而沿 y 轴移动表示有时称为“阶数”的变化。

种类	Prop (Sort 0)	Type (Sort 1)	Type 1 (Sort 2)	Type 2 (Sort 3)	...
类型	True	Bool	Nat -> Type	Type -> Type 1	...
项	True.intro	true	fun n => Fin n	fun (_ : Type) => Type	...

然而，有些操作需要在类型宇宙上是多态的。例如，对任意类型 α ，无论 α 位于哪个类型宇宙中，List α 都应当有意义。这解释了函数 List 的类型签名：

```

#check List      -- List.{u} ( $\alpha$  : Type u) : Type u

```

```

| List.{u} ( $\alpha$  : Type u) : Type u

```

这里 u 是一个在类型层级上取值的变量。`#check` 命令的输出意味着：只要 α 的类型是 `Type u`, `List  $\alpha$` 的类型也就是 `Type u`。函数 `Prod` 也类似地是多态的：

```
#check Prod    -- Prod.{u, v} ( $\alpha$  : Type u) ( $\beta$  : Type v) : Type
↪ (max u v)
| Prod.{u, v} ( $\alpha$  : Type u) ( $\beta$  : Type v) : Type (max u v)
```

为定义多态常量，Lean 允许使用 `universe` 命令显式声明宇宙变量：

```
universe u
```

```
def F ( $\alpha$  : Type u) : Type u := Prod  $\alpha$   $\alpha$ 
```

```
#check F      -- F.{u} ( $\alpha$  : Type u) : Type u
| F.{u} ( $\alpha$  : Type u) : Type u
```

也可以在定义 `F` 时给出宇宙参数，从而避免使用 `universe` 命令：

```
def F.{u} ( $\alpha$  : Type u) : Type u := Prod  $\alpha$   $\alpha$ 
```

```
#check F      -- F.{u} ( $\alpha$  : Type u) : Type u
| F.{u} ( $\alpha$  : Type u) : Type u
```

2.3 函数抽象与求值

Lean 提供 `fun`（或 `$\lambda$` ）关键字，用于如下从表达式创建函数：

```
#check fun x => x + 5    -- fun x => x + 5 : Nat → Nat
| fun x => x + 5 : Nat → Nat
--  $\lambda$  and fun mean the same thing
#check  $\lambda$  (x : Nat) => x + 5    -- fun x => x + 5 : Nat → Nat
| fun x => x + 5 : Nat → Nat
```

在这个例子中，类型 `Nat` 可以被推断出来：

```
#check fun x => x + 5    -- fun x => x + 5 : Nat → Nat
| fun x => x + 5 : Nat → Nat
#check  $\lambda$  x => x + 5      -- fun x => x + 5 : Nat → Nat
| fun x => x + 5 : Nat → Nat
```

可以通过传入所需参数来对 `lambda` 函数求值：

```
#eval ( $\lambda$  x : Nat => x + 5) 10    -- 15
| 15
```

由另一个表达式创建函数的过程称为 *lambda* 抽象。假设你有变量 $x : \alpha$ ，并且能够构造表达式 $t : \beta$ ，那么表达式 $\text{fun } (x : \alpha) \Rightarrow t$ ，或等价地 $\lambda (x : \alpha) \Rightarrow t$ ，就是类型 $\alpha \rightarrow \beta$ 的对象。可以把它看作从 α 到 β 的函数，该函数把任意值 x 映射到值 t 。

下面还有一些示例

```
#check fun x : Nat => fun y : Bool => if not y then x + 1 else
  ↪ x + 2
| fun x y => if (!y) = true then x + 1 else x + 2 : Nat → Bool
| ↪ → Nat

#check fun (x : Nat) (y : Bool) => if not y then x + 1 else x
  ↪ + 2
| fun x y => if (!y) = true then x + 1 else x + 2 : Nat → Bool
| ↪ → Nat

#check fun x y => if not y then x + 1 else x + 2 -- fun x y
  ↪ => if (!y) = true then x + 1 else x + 2 : Nat → Bool → Nat
| fun x y => if (!y) = true then x + 1 else x + 2 : Nat → Bool
| ↪ → Nat
```

Lean 将最后三个示例解释为同一个表达式；在最后一个表达式中，Lean 从表达式 $\text{if not } y \text{ then } x + 1 \text{ else } x + 2$ 推断 x 和 y 的类型。

一些数学上常见的函数操作示例可以用 *lambda* 抽象来描述：

```
def f (n : Nat) : String := toString n
def g (s : String) : Bool := s.length > 0

#check fun x : Nat => x -- fun x => x : Nat → Nat
| fun x => x : Nat → Nat

#check fun x : Nat => true -- fun x => true : Nat → Bool
| fun x => true : Nat → Bool

#check fun x : Nat => g (f x) -- fun x => g (f x) : Nat →
  ↪ Bool
| fun x => g (f x) : Nat → Bool

#check fun x => g (f x) -- fun x => g (f x) : Nat →
  ↪ Bool
| fun x => g (f x) : Nat → Bool
```

思考这些表达式的含义。表达式 $\text{fun } x : \text{Nat} \Rightarrow x$ 表示 Nat 上的恒等函数；表达式 $\text{fun } x : \text{Nat} \Rightarrow \text{true}$ 表示总是返回 true 的常值函数；而 $\text{fun } x : \text{Nat} \Rightarrow g (f x)$ 表示 f 与 g 的复合。一般而言，可以省略类型标注，让 Lean 为你推断。因此，例如可以写 $\text{fun } x \Rightarrow g (f x)$ ，而不写 $\text{fun } x : \text{Nat} \Rightarrow g (f x)$ 。

可以把函数作为参数传入，并通过给它们命名为 f 和 g ，在实现中使用这些函数：

```
#check fun (g : String → Bool) (f : Nat → String) (x : Nat) =>
  ↪ g (f x)
| fun g f x => g (f x) : (String → Bool) → (Nat → String) → Nat
  ↪ → Bool
```

也可以把类型作为参数传入：

```
#check fun (α β γ : Type) (g : β → γ) (f : α → β) (x : α) => g
  ↪ (f x)
| fun α β γ g f x => g (f x) : (α β γ : Type) → (β → γ) → (α →
  ↪ β) → α → γ
```

例如，最后一个表达式表示这样一个函数：它接受三个类型 α 、 β 和 γ ，以及两个函数 $g : \beta \rightarrow \gamma$ 和 $f : \alpha \rightarrow \beta$ ，并返回 g 与 f 的复合。（理解这个函数的类型需要理解依值积，这将在下文解释。）

lambda 表达式的一般形式是 `fun (x :  $\alpha$ ) => t`，其中变量 x 是一个“绑定变量”：它实际上是一个占位符，其“作用域”不会延伸到表达式 t 之外。例如，表达式 `fun (b :  $\beta$ ) (x :  $\alpha$ ) => b` 中的变量 b ，与先前声明的常量 b 没有任何关系。事实上，该表达式表示的函数与 `fun (u :  $\beta$ ) (z :  $\alpha$ ) => u` 相同。形式地说，只差绑定变量重命名的表达式称为 *alpha* 等价，并被认为是“相同的”。Lean 能识别这种等价。

注意，将项 $t : \alpha \rightarrow \beta$ 应用于项 $s : \alpha$ 会得到表达式 $t s : \beta$ 。回到前面的例子，并为清晰起见重命名绑定变量，请注意以下表达式的类型：

```
#check (fun x : Nat => x) 1      -- (fun x => x) 1 : Nat
| (fun x => x) 1 : Nat
#check (fun x : Nat => true) 1   -- (fun x => true) 1 : Bool
| (fun x => true) 1 : Bool
def f (n : Nat) : String := toString n
def g (s : String) : Bool := s.length > 0
#check
  (fun (α β γ : Type) (u : β → γ) (v : α → β) (x : α) => u (v
    ↪ x)) Nat String Bool g f 0
| (fun α β γ u v x => u (v x)) Nat String Bool g f 0 : Bool
```

如预期，表达式 `(fun x : Nat => x) 1` 的类型为 `Nat`。事实上，还应当有更强的结论：将表达式 `(fun x : Nat => x)` 应用于 `1` 应当“返回”值 `1`。而它确实如此：

```
#eval (fun x : Nat => x) 1      -- 1
| 1
#eval (fun x : Nat => true) 1   -- true
| true
```

稍后你会看到这些项是如何被求值的。现在请注意，这是依值类型论的一个重要特征：每个项都有计算行为，并支持规范化的概念。原则上，两个能化简到同一值的项称为定义等价。Lean 的类型检查器会把它们视为“相同”，并尽力识别和支持这些同一性。

Lean 是一门完整的编程语言。它有能够生成二进制可执行文件的编译器，也有交互式解释器。可以使用命令 **#eval** 执行表达式，这是测试函数的首选方式。

2.4 定义

回顾一下，**def** 关键字提供了一种声明新的具名对象的重要方式。

```
def double (x : Nat) : Nat :=
  x + x
```

如果你了解其他编程语言中的函数工作方式，这看起来可能会更熟悉。名称 **double** 被定义为一个函数，它接受一个类型为 **Nat** 的输入参数 **x**；调用结果是 **x + x**，因此返回类型为 **Nat**。随后可以这样调用此函数：

```
#eval double 3    -- 6
6
```

在这种情况下，可以把 **def** 看作一种具名的 **fun**。下面的写法得到相同结果：

```
def double : Nat → Nat :=
  fun x => x + x
```

```
#eval double 3    -- 6
6
```

当 Lean 有足够信息可以推断类型时，可以省略类型声明。类型推断是 Lean 的重要组成部分：

```
def double :=
  fun (x : Nat) => x + x
```

定义的一般形式是 **def foo : α := bar**，其中 α 是表达式 **bar** 返回的类型。Lean 通常可以推断类型 α ，但显式写出它往往是好习惯。这会澄清你的意图；如果定义右侧不具有匹配的类型，Lean 会报告错误。

右侧的 **bar** 可以是任意表达式，而不仅仅是 **lambda**。因此，**def** 也可以像这样仅仅为一个值命名：

```
def pi := 3.141592654
```

def 可以接受多个输入参数。下面创建一个将两个自然数相加的函数：

```
def add (x y : Nat) :=
  x + y

#eval add 3 2          -- 5
| 5
```

参数列表也可以像这样分开书写：

```
def add (x : Nat) (y : Nat) :=
  x + y

#eval add (double 3) (7 + 9) -- 22
| 22
```

注意，这里我们调用 `double` 函数来构造传给 `add` 的第一个参数。

可以在 `def` 中使用其他更有趣的表达式：

```
def greater (x y : Nat) :=
  if x > y then x
  else y
```

你大概可以猜出这个函数会做什么。

也可以定义一个以另一个函数为输入的函数。下面的函数会调用给定函数两次，并把第一次调用的输出传给第二次调用：

```
def doTwice (f : Nat → Nat) (x : Nat) : Nat :=
  f (f x)
```

```
#eval doTwice double 2 -- 8
| 8
```

现在再抽象一些，也可以指定类似类型参数的参数：

```
def compose (α β γ : Type) (g : β → γ) (f : α → β) (x : α) : γ
  ↪ :=
  g (f x)
```

这意味着 `compose` 是一个函数，它以任意两个函数作为输入参数，只要这两个函数各自都只接受一个输入。类型表达式 $\beta \rightarrow \gamma$ 与 $\alpha \rightarrow \beta$ 表明有一个要求：第二个函数的输出类型必须与第一个函数的输入类型相匹配。这是合理的；否则这两个函数就不能复合。

`compose` 还接受第三个参数，其类型为 α ；它用该参数调用第二个函数（局部命名为 f ），并把该函数的结果（类型为 β ）作为输入传给第一个函数（局部命名为 g ）。第一个函数返回类型 γ 的值，因此这也是 `compose` 函数的返回类型。

`compose` 也非常通用，因为它适用于任意类型 $\alpha \beta \gamma$ 。这意味着，只要两个函数各自接受一个参数，并且第二个函数的输出类型与第一个函数的输入类型相匹配，`compose` 几乎可以复合任意两个函数。例如：

```
def square (x : Nat) : Nat :=
  x * x

#eval compose Nat Nat Nat double square 3 -- 18
18
```

2.5 局部定义

Lean 也允许使用 **let** 关键字引入“局部”定义。表达式 **let a := t1; t2** 与把 **t2** 中每一次出现的 **a** 都替换为 **t1** 所得结果定义等价。

```
#check let y := 2 + 2; y * y -- let y := 2 + 2; y * y : Nat
let y := 2 + 2;
y * y : Nat

#eval let y := 2 + 2; y * y -- 16
16

def twice_double (x : Nat) : Nat :=
  let y := x + x; y * y

#eval twice_double 2 -- 16
16
```

这里，`twice_double x` 与项 $(x + x) * (x + x)$ 定义等价。
可以通过串联 **let** 语句来组合多个赋值：

```
#check let y := 2 + 2; let z := y + y; z * z
let y := 2 + 2;
let z := y + y;
z * z : Nat

#eval let y := 2 + 2; let z := y + y; z * z -- 64
64
```

使用换行时，可以省略 **;**。

```
def t (x : Nat) : Nat :=
  let y := x + x
  y * y
```

注意，表达式 **let a := t1; t2** 的含义与 **(fun a => t2) t1** 的含义非常相近，但二者并不相同。在第一个表达式中，应把 **t2** 中每一个 **a** 都看作 **t1** 的语法缩写。在第二个表达式中，**a** 是一个变量，而表达式 **fun a => t2** 必须在不依赖 **a** 的具体值的情况下有意义。**let** 构造是一种更强的缩写手段；有些形如 **let a := t1; t2** 的表达式不能表示为 **(fun a => t2) t1**。作为练习，请尝试理解为什么下面 **foo** 的定义能通过类型检查，而 **bar** 的定义不能。


```
def foo := let a := Nat; fun x : a => x + 2
/-
  def bar := (fun a => fun x : a => x + 2) Nat
-/
```

2.6 变量与段落

考虑下面三个函数定义：

```
def compose (α β γ : Type) (g : β → γ) (f : α → β) (x : α) : γ
  ↪ :=
  g (f x)
```

```
def doTwice (α : Type) (h : α → α) (x : α) : α :=
  h (h x)
```

```
def doThrice (α : Type) (h : α → α) (x : α) : α :=
  h (h (h x))
```

Lean 提供 **variable** 命令，使这类声明看起来更紧凑：

```
variable (α β γ : Type)
```

```
def compose (g : β → γ) (f : α → β) (x : α) : γ :=
  g (f x)
```

```
def doTwice (h : α → α) (x : α) : α :=
  h (h x)
```

```
def doThrice (h : α → α) (x : α) : α :=
  h (h (h x))
```

可以声明任意类型的变量，而不仅限于 Type 本身：

```
variable (α β γ : Type)
```

```
variable (g : β → γ) (f : α → β) (h : α → α)
```

```
variable (x : α)
```

```
def compose := g (f x)
```

```
def doTwice := h (h x)
```

```
def doThrice := h (h (h x))
```

```
#print compose
```

```
def compose : (α β γ : Type) → (β → γ) → (α → β) → α → γ :=
fun α β γ g f x => g (f x)
```

```
#print doTwice
```

```
def doTwice : (α : Type) → (α → α) → α → α :=
  fun α h x => h (h x)
#print doThrice
def doThrice : (α : Type) → (α → α) → α → α :=
  fun α h x => h (h (h x))
```

打印它们会显示，这三组定义具有完全相同的效果。

variable 命令指示 **Lean**：在按名称引用这些变量的定义中，把所声明的变量插入为绑定变量。**Lean** 足够智能，能够判断一个定义中哪些变量被显式或隐式使用。因此，在书写定义时，你可以像 α 、 β 、 γ 、 g 、 f 、 h 和 x 都是固定对象那样进行，并让 **Lean** 自动为你对定义进行抽象。

以这种方式声明时，变量会一直保持在作用域中，直到当前文件结束。不过，有时限制变量的作用域是有用的。为此，**Lean** 提供了 **section** 的概念：

```
section useful
  variable (α β γ : Type)
  variable (g : β → γ) (f : α → β) (h : α → α)
  variable (x : α)

  def compose := g (f x)
  def doTwice := h (h x)
  def doThrice := h (h (h x))
end useful
```

当段落关闭时，这些变量离开作用域，不能再被引用。

段落内部的行不必缩进。也不必给段落命名；也就是说，可以使用匿名的 **section/end** 对。不过，如果给段落命名，就必须使用相同名称关闭它。段落也可以嵌套，这允许你逐步声明新的变量。

2.7 命名空间

Lean 提供了将定义分组到嵌套的、层级化的命名空间中的能力：

```
namespace Foo
  def a : Nat := 5
  def f (x : Nat) : Nat := x + 7

  def fa : Nat := f a
  def ffa : Nat := f (f a)

  #check a
  Foo.a : Nat
  #check f
  Foo.f (x : Nat) : Nat
```

```

#check fa
| Foo.fa : Nat
#check ffa
| Foo.ffa : Nat
#check Foo.fa
| Foo.fa : Nat
end Foo

-- #check a -- error
-- #check f -- error
#check Foo.a
| Foo.a : Nat
#check Foo.f
| Foo.f (x : Nat) : Nat
#check Foo.fa
| Foo.fa : Nat
#check Foo.ffa
| Foo.ffa : Nat
open Foo

#check a
| Foo.a : Nat
#check f
| Foo.f (x : Nat) : Nat
#check fa
| Foo.fa : Nat
#check Foo.fa
| Foo.fa : Nat

```

当声明正在命名空间 **Foo** 中工作时，你声明的每个标识符都有一个带前缀“**Foo.**”的全名。在该命名空间内部，可以用较短名称引用标识符；但一旦结束该命名空间，就必须使用较长名称。与 **section** 不同，命名空间必须有名称。根层级只有一个匿名命名空间。

open 命令把较短名称引入当前上下文。通常，当导入一个模块时，你会想打开其中一个或多个命名空间，以便访问短标识符。但有时你会希望让这些信息由完全限定名保护起来，例如当它们与你想使用的另一个命名空间中的标识符冲突时。因此，命名空间为你提供了一种管理工作环境中名称的方式。

例如，Lean 将涉及列表的定义和定理归入命名空间 **List**。

```

#check List.nil
| List.nil.{u} {α : Type u} : List α

```

```
#check List.cons
```

```
List.cons.{u} {α : Type u} (head : α) (tail : List α) : List
  → α
```

```
#check List.map
```

```
List.map.{u_1, u_2} {α : Type u_1} {β : Type u_2} (f : α → β)
  → (l : List α) : List β
```

命令 **open** `List` 允许你使用较短名称:

```
open List
```

```
#check nil
```

```
List.nil.{u} {α : Type u} : List α
```

```
#check cons
```

```
List.cons.{u} {α : Type u} (head : α) (tail : List α) : List
  → α
```

```
#check map
```

```
List.map.{u_1, u_2} {α : Type u_1} {β : Type u_2} (f : α → β)
  → (l : List α) : List β
```

与段落一样，命名空间可以嵌套:

```
namespace Foo
```

```
def a : Nat := 5
```

```
def f (x : Nat) : Nat := x + 7
```

```
def fa : Nat := f a
```

```
namespace Bar
```

```
def ffa : Nat := f (f a)
```

```
#check fa
```

```
Foo.fa : Nat
```

```
#check ffa
```

```
Foo.Bar.ffa : Nat
```

```
end Bar
```

```
#check fa
```

```
Foo.fa : Nat
```

```
#check Bar.ffa
```

```
Foo.Bar.ffa : Nat
```

```
end Foo
```

```

#check Foo.fa
| Foo.fa : Nat
#check Foo.Bar.ffa
| Foo.Bar.ffa : Nat
open Foo

#check fa
| Foo.fa : Nat
#check Bar.ffa
| Foo.Bar.ffa : Nat

```

已经关闭的命名空间稍后可以重新打开，甚至可以在另一个文件中打开：

```

namespace Foo
  def a : Nat := 5
  def f (x : Nat) : Nat := x + 7

  def fa : Nat := f a
end Foo

#check Foo.a
| Foo.a : Nat
#check Foo.f
| Foo.f (x : Nat) : Nat
namespace Foo
  def ffa : Nat := f (f a)
end Foo

```

与段落一样，嵌套命名空间必须按打开的相反顺序关闭。命名空间和段落服务于不同目的：命名空间组织数据，而段落声明要插入定义中的变量。段落还可用于限定诸如 `set_option` 和 `open` 等命令的作用域。

然而，在许多方面，`namespace ... end` 块的行为与 `section ... end` 块相同。特别地，如果在命名空间内使用 `variable` 命令，其作用域会被限制在该命名空间内。类似地，如果在命名空间内使用 `open` 命令，当命名空间关闭时，其效果也会消失。

2.8 依值类型论为何“依值”？

简短的解释是：类型可以依赖于参数的值。你已经见过一个很好的例子：类型 `List  $\alpha$` 依赖于参数 α ，正是这种依赖区分了 `List Nat` 和 `List Bool`。再举一例，考虑类型 `Vector  $\alpha$  n`，它表示长度为 n 、元素类型为 α 的向量类型。这个类型依赖于两个参数：向量中元素的类型 ($\alpha : \text{Type}$) 以及向量的长度 $n : \text{Nat}$ 。

假设你想编写函数 `cons`，用于把一个新元素插入列表头部。`cons` 应当具有何种类型？这样的函数是多态的：你期望针对 `Nat`、`Bool` 或任意类型 α 的 `cons` 函数以同样方式行为。因此，把类型作为 `cons` 的第一个参数是合理的；这样，对任意类型 α ，`cons  $\alpha$` 就是类型为 α 的列表的插入函数。换言之，对每个 α ，`cons  $\alpha$` 都是这样一个函数：它接受一个元素 $a : \alpha$ 和一个列表 $as : \text{List } \alpha$ ，并返回一个新列表，因此有 `cons  $\alpha$   $a$   $as$  : List  $\alpha$` 。显然，`cons  $\alpha$` 应当具有类型 $\alpha \rightarrow \text{List } \alpha \rightarrow \text{List } \alpha$ 。但 `cons` 本身应当具有何种类型？第一猜想可能是 `Type  $\rightarrow$   $\alpha \rightarrow$  List  $\alpha \rightarrow$  List  $\alpha$` ，但仔细想来，这并不合理：此表达式中的 α 没有指向任何东西，而它本应指向类型为 `Type` 的那个参数。换句话说，假设 $\alpha : \text{Type}$ 是函数的第一个参数，那么接下来两个元素的类型分别是 α 和 `List  $\alpha$` 。这些类型会随着第一个参数 α 而变化。

```
def cons ( $\alpha : \text{Type}$ ) ( $a : \alpha$ ) ( $as : \text{List } \alpha$ ) : List  $\alpha$  :=
  List.cons  $a$   $as$ 
```

```
#check cons Nat          -- cons Nat : Nat  $\rightarrow$  List Nat  $\rightarrow$  List Nat
| cons Nat : Nat  $\rightarrow$  List Nat  $\rightarrow$  List Nat
#check cons Bool         -- cons Bool : Bool  $\rightarrow$  List Bool  $\rightarrow$  List
   $\hookrightarrow$  Bool
| cons Bool : Bool  $\rightarrow$  List Bool  $\rightarrow$  List Bool
#check cons              -- cons ( $\alpha : \text{Type}$ ) ( $a : \alpha$ ) ( $as : \text{List}$ 
   $\hookrightarrow$   $\alpha$ ) : List  $\alpha$ 
| cons ( $\alpha : \text{Type}$ ) ( $a : \alpha$ ) ( $as : \text{List } \alpha$ ) : List  $\alpha$ 
```

这是依值函数类型，或称依值箭头类型的一个实例。给定 $\alpha : \text{Type}$ 和 $\beta : \alpha \rightarrow \text{Type}$ ，可以把 β 看作 α 上的一个类型族，也就是说，对每个 $a : \alpha$ 都有一个类型 βa 。在这种情况下，类型 $(a : \alpha) \rightarrow \beta a$ 表示具有如下性质的函数 f 的类型：对每个 $a : \alpha$ ， $f a$ 都是 βa 的一个元素。换言之， f 返回值的类型依赖于其输入。

注意，对任意表达式 $\beta : \text{Type}$ ， $(a : \alpha) \rightarrow \beta$ 都有意义。当 β 的值依赖于 a 时（例如上一段中的表达式 βa ）， $(a : \alpha) \rightarrow \beta$ 表示一个依值函数类型。当 β 不依赖于 a 时， $(a : \alpha) \rightarrow \beta$ 与类型 $\alpha \rightarrow \beta$ 没有区别。事实上，在依值类型论（以及 Lean）中，当 β 不依赖于 a 时， $\alpha \rightarrow \beta$ 只是 $(a : \alpha) \rightarrow \beta$ 的记号。

回到列表的例子，可以使用命令 `#check` 查看以下 `List` 函数的类型。@ 符号以及圆括号与花括号之间的区别稍后会解释。

```
#check @List.cons        -- @List.cons : { $\alpha : \text{Type } u_1$ }  $\rightarrow$   $\alpha \rightarrow$  List
   $\hookrightarrow$   $\alpha \rightarrow$  List  $\alpha$ 
| @List.cons : { $\alpha : \text{Type } u_1$ }  $\rightarrow$   $\alpha \rightarrow$  List  $\alpha \rightarrow$  List  $\alpha$ 
#check @List.nil         -- @List.nil : { $\alpha : \text{Type } u_1$ }  $\rightarrow$  List  $\alpha$ 
| @List.nil : { $\alpha : \text{Type } u_1$ }  $\rightarrow$  List  $\alpha$ 
#check @List.length      -- @List.length : { $\alpha : \text{Type } u_1$ }  $\rightarrow$  List  $\alpha$ 
   $\hookrightarrow$   $\rightarrow$  Nat
| @List.length : { $\alpha : \text{Type } u_1$ }  $\rightarrow$  List  $\alpha \rightarrow$  Nat
```

```
#check @List.append -- @List.append : {α : Type u_1} → List α
  ↳ → List α → List α
| @List.append : {α : Type u_1} → List α → List α → List α
```

正如依值函数类型 $(a : \alpha) \rightarrow \beta$ a 通过允许 β 依赖于 a 来推广函数类型 $\alpha \rightarrow \beta$ 一样，依值笛卡尔积类型 $(a : \alpha) \times \beta$ a 也以同样方式推广笛卡尔积 $\alpha \times \beta$ 。依值积也称为 *sigma* 类型，并且也可以写作 $\Sigma a : \alpha, \beta$ a 。可以使用 $\langle a, b \rangle$ 或 `Sigma.mk a b` 创建依值对。字符 $\langle$ 和 $\rangle$ 可分别通过 `\langle` 与 `\rangle` 或 `\<` 与 `\>` 输入。

```
universe u v
```

```
def f (α : Type u) (β : α → Type v) (a : α) (b : β a) : (a :
  ↳ α) × β a :=
  ⟨a, b⟩
```

```
def g (α : Type u) (β : α → Type v) (a : α) (b : β a) : Σ a :
  ↳ α, β a :=
  Sigma.mk a b
```

```
def h1 (x : Nat) : Nat :=
  (f Type (fun α => α) Nat x).2
```

```
#eval h1 5 -- 5
```

```
| 5
```

```
def h2 (x : Nat) : Nat :=
  (g Type (fun α => α) Nat x).2
```

```
#eval h2 5 -- 5
```

```
| 5
```

上面的函数 `f` 与 `g` 表示同一个函数。

2.9 隐式参数

假设我们有如下列表实现：

```
#check Lst -- Lst.{u} (α : Type u) : Type u
| Lst.{u} (α : Type u) : Type u
#check Lst.cons -- Lst.cons.{u} (α : Type u) (a : α) (as :
  ↳ Lst α) : Lst α
| Lst.cons.{u} (α : Type u) (a : α) (as : Lst α) : Lst α
#check Lst.nil -- Lst.nil.{u} (α : Type u) : Lst α
| Lst.nil.{u} (α : Type u) : Lst α
```

```
#check Lst.append -- Lst.append.{u} (α : Type u) (as bs :
  ↳ Lst α) : Lst α
| Lst.append.{u} (α : Type u) (as bs : Lst α) : Lst α
```

那么，可以如下构造 `Nat` 的列表：

```
#check Lst.cons Nat 0 (Lst.nil Nat)
| Lst.cons Nat 0 (Lst.nil Nat) : Lst Nat
def as : Lst Nat := Lst.nil Nat
def bs : Lst Nat := Lst.cons Nat 5 (Lst.nil Nat)

#check Lst.append Nat as bs
| Lst.append Nat as bs : Lst Nat
```

由于构造子在类型上是多态的，我们必须反复把类型 `Nat` 作为参数插入。但这些信息是冗余的：在 `Lst.cons Nat 5 (Lst.nil Nat)` 中，可以根据第二个参数 `5` 具有类型 `Nat` 这一事实推断参数 α 。类似地，`Lst.nil Nat` 中的参数也可以被推断出来；这不是来自该表达式中的其他内容，而是来自它被作为参数传给函数 `Lst.cons` 这一事实，后者在该位置期望一个类型为 `Lst α` 的元素。

这是依值类型论的一个核心特征：项携带大量信息，而且其中一些信息常常可以从上下文推断出来。在 `Lean` 中，可以使用下划线 `_` 来指定系统应当自动填入这些信息。这称为“隐式参数”。

```
#check Lst.cons _ 0 (Lst.nil _)
| Lst.cons Nat 0 (Lst.nil Nat) : Lst Nat
def as : Lst Nat := Lst.nil _
def bs : Lst Nat := Lst.cons _ 5 (Lst.nil _)

#check Lst.append _ as bs -- Lst.append Nat as bs : Lst Nat
| Lst.append Nat as bs : Lst Nat
```

然而，输入所有这些下划线仍然很繁琐。当一个函数接受的某个参数通常可以从上下文推断时，`Lean` 允许你指定该参数默认应保持隐式。做法是把这些参数放在花括号中，如下所示：

```
universe u
def Lst (α : Type u) : Type u := List α

def Lst.cons {α : Type u} (a : α) (as : Lst α) : Lst α :=
  ↳ List.cons a as
def Lst.nil {α : Type u} : Lst α := List.nil
def Lst.append {α : Type u} (as bs : Lst α) : Lst α :=
  ↳ List.append as bs

#check Lst.cons 0 Lst.nil
```



```

| Lst.cons 0 Lst.nil : Lst Nat
def as : Lst Nat := Lst.nil
def bs : Lst Nat := Lst.cons 5 Lst.nil

#check Lst.append as bs
| as.append bs : Lst Nat

```

唯一改变的是变量声明中 $\alpha : \text{Type } u$ 周围的括号。也可以在函数定义中使用这种机制：

```

universe u
def ident { $\alpha : \text{Type } u$ } (x :  $\alpha$ ) := x

```

检查 `ident` 的类型时，需要将其置于圆括号中，以避免显示其签名：

```

#check (ident)      -- ident : ?m.22 → ?m.22
| ident : ?m.1 → ?m.1
#check ident 1      -- ident 1 : Nat
| ident 1 : Nat
#check ident "hello" -- ident "hello" : String
| ident "hello" : String
#check @ident      -- @ident : { $\alpha : \text{Type } u_1$ } →  $\alpha$  →  $\alpha$ 
| @ident : { $\alpha : \text{Type } u_1$ } →  $\alpha$  →  $\alpha$ 

```

这使 `ident` 的第一个参数成为隐式参数。从记号上看，这隐藏了对类型的指定，使 `ident` 看起来仿佛只是接受任意类型的一个参数。事实上，标准库中的函数 `id` 正是以这种方式定义的。这里选择一个非传统名称，只是为了避免名称冲突。

使用 **variable** 命令声明变量时，也可以把它们指定为隐式的：

```

universe u

section
  variable { $\alpha : \text{Type } u$ }
  variable (x :  $\alpha$ )
  def ident := x
end

#check ident
| ident.{u} { $\alpha : \text{Type } u$ } (x :  $\alpha$ ) :  $\alpha$ 
#check ident 4
| ident 4 : Nat
#check ident "hello"

```

```
ident "hello" : String
```

这里对 `ident` 的定义与上面的定义具有相同效果。

`Lean` 具有非常复杂的机制来实例化隐式参数；我们将看到，它们可用于推断函数类型、谓词，甚至证明。实例化项中的这些“空洞”或“占位符”的过程通常称为阐释。隐式参数的存在意味着，有时可能没有足够信息来精确确定一个表达式的含义。像 `id` 或 `List.nil` 这样的表达式称为多态的，因为它可以在不同上下文中具有不同含义。

总可以通过写 $(e : T)$ 来指定表达式 e 的类型 T 。这会指示 `Lean` 的阐释器在尝试解析隐式参数时，把值 T 用作 e 的类型。在下面第二组示例中，这一机制用于指定表达式 `id` 和 `List.nil` 的期望类型：

```
#check (List.nil)          -- [] : List ?m.2
[] : List ?m.1
#check (id)                -- id : ?m.1 → ?m.1
id : ?m.1 → ?m.1
#check (List.nil : List Nat) -- [] : List Nat
[] : List Nat
#check (id : Nat → Nat)     -- id : Nat → Nat
id : Nat → Nat
```

`Lean` 中的数码是重载的；但当无法推断一个数码的类型时，`Lean` 默认假定它是自然数。因此，下面前两个 `#check` 命令中的表达式会以相同方式阐释，而第三个 `#check` 命令将 `2` 解释为整数。

```
#check 2                   -- 2 : Nat
2 : Nat
#check (2 : Nat)           -- 2 : Nat
2 : Nat
#check (2 : Int)           -- 2 : Int
2 : Int
```

然而，有时会遇到这样的情况：我们已经把函数的某个参数声明为隐式，但现在又想显式提供该参数。如果 `foo` 是这样的函数，记号 `@foo` 表示同一个函数，只是所有参数都变为显式。

```
#check @id                -- @id : {α : Sort u_1} → α → α
@id : {α : Sort u_1} → α → α
#check @id Nat            -- id : Nat → Nat
id : Nat → Nat
#check @id Bool           -- id : Bool → Bool
id : Bool → Bool
#check @id Nat 1          -- id 1 : Nat
```

```
id 1 : Nat
#check @id Bool true -- id true : Bool
id true : Bool
```

注意，现在第一个 **#check** 命令给出了标识符 **id** 的类型，且没有插入任何占位符。此外，输出表明第一个参数是隐式的。

3 命题与证明

到目前为止，你已经见过一些在 **Lean** 中定义对象和函数的方法。在本章中，我们还将开始说明如何用依值类型论的语言书写数学断言和证明。

3.1 命题即类型

要证明关于依值类型论语言中所定义对象的断言，一种策略是在定义语言之上再叠加一套断言语言和一套证明语言。但没有理由以这种方式增殖语言：依值类型论灵活而富有表达力，我们没有理由不能在同一个一般框架中表示断言和证明。

例如，我们可以引入一个新类型 **Prop** 来表示命题，并引入构造子从已有命题构造新的命题。

```
#check And      -- And (a b : Prop) : Prop
| And (a b : Prop) : Prop
#check Or      -- Or (a b : Prop) : Prop
| Or (a b : Prop) : Prop
#check Not      -- Not (a : Prop) : Prop
| Not (a : Prop) : Prop
#check Implies -- Implies (p q : Prop) : Prop
| Implies (p q : Prop) : Prop
variable (p q r : Prop)

#check And p q      -- p ∧ q : Prop
| p ∧ q : Prop
#check Or (And p q) r      -- p ∧ q ∨ r : Prop
| p ∧ q ∨ r : Prop
#check Implies (And p q) (And q p) -- Implies (p ∧ q) (q ∧ p)
↔ : Prop
| Implies (p ∧ q) (q ∧ p) : Prop
```

于是，对于每个元素 $p : \text{Prop}$ ，我们还可以引入另一种类型 **Proof** p ，作为 p 的证明类型。所谓“公理”就是这种类型的一个常量。

```

#check Proof -- Proof (p : Prop) : Type
| Proof (p : Prop) : Type
axiom and_commut (p q : Prop) : Proof (Implies (And p q) (And
  ↪ q p))

variable (p q : Prop)

#check and_commut p q -- and_commut p q : Proof (Implies
  ↪ (p ∧ q) (q ∧ p))
| and_commut p q : Proof (Implies (p ∧ q) (q ∧ p))

```

不过，除了公理之外，我们还需要从旧证明构造新证明的规则。例如，在许多命题逻辑的证明系统中，我们有 *modus ponens*（肯定前件）规则：

从 $\text{Implies } p \ q$ 的一个证明和 p 的一个证明，可以得到 q 的一个证明。

我们可以将它表示如下：

```

axiom modus_ponens (p q : Prop) :
  Proof (Implies p q) → Proof p →
  Proof q

```

命题逻辑的自然演绎系统通常还依赖下列规则：

假设在把 p 作为假设的前提下，我们有 q 的一个证明。那么我们可以“取消”该假设，并得到 $\text{Implies } p \ q$ 的一个证明。

我们可以将它呈现如下：

```

axiom implies_intro (p q : Prop) :
  (Proof p → Proof q) → Proof (Implies p q)

```

这种方法会为我们提供一种构造断言和证明的合理方式。要判定表达式 t 是否是断言 p 的正确证明，就只需检查 t 是否具有类型 $\text{Proof } p$ 。

然而，还可以作一些简化。首先，我们可以把 $\text{Proof } p$ 与 p 本身合并起来，从而避免反复书写项 Proof 。换言之，只要有 $p : \text{Prop}$ ，我们就可以把 p 解释为一个类型，即它的证明所成的类型。于是，我们可以把 $t : p$ 读作断言 t 是 p 的一个证明。

此外，一旦作出这种认同，蕴含的规则就表明，我们可以在 $\text{Implies } p \ q$ 与 $p \rightarrow q$ 之间来回转换。换言之，命题 p 与 q 之间的蕴含，对应于拥有一个函数，它把 p 的任意元素映到 q 的一个元素。因此，引入联结词 Implies 完全是多余的：我们可以使用依值类型论中通常的函数空间构造子 $p \rightarrow q$ 作为我们的蕴含概念。

这正是构造演算（Calculus of Constructions）所采用的方法，因此 *Lean* 也采用这种方法。自然演绎证明系统中蕴含规则与函数抽象和应用规则完全对应这一事实，是 *Curry-Howard isomorphism*（Curry-Howard 同构）的一个实例；

它有时也称为 *propositions-as-types*（命题即类型）范式。事实上，类型 `Prop` 是 `Sort 0` 的语法糖，也就是上一章所描述的类型层级的最底层。此外，`Type u` 也只是 `Sort (u+1)` 的语法糖。`Prop` 有一些特殊性质，但像其他类型宇宙一样，它在箭头构造子下封闭：如果有 $p, q : \text{Prop}$ ，那么 $p \rightarrow q : \text{Prop}$ 。

至少有两种方式可以理解“命题即类型”。对于一些以构造性观点看待逻辑和数学的人来说，这忠实地表达了作为一个命题意味着什么：命题 p 表示某种数据类型，即对构成证明的数据类型的规范。于是， p 的证明就只是一个类型正确的对象 $t : p$ 。

不倾向于这种思想的人则可以把它看作一种简单的编码技巧。对于每个命题 p ，我们关联一个类型：如果 p 为假，该类型为空；如果 p 为真，该类型有一个元素，比如 $*$ 。在后一种情形下，我们说（与） p （相关联的类型）是有居项的。碰巧函数应用和抽象规则可以方便地帮助我们跟踪 `Prop` 的哪些元素是有居项的。因此，构造一个元素 $t : p$ 告诉我们 p 确实为真。你可以把 p 的居项看成“ p 为真这一事实”。 $p \rightarrow q$ 的一个证明使用“ p 为真这一事实”来得到“ q 为真这一事实”。

事实上，如果 $p : \text{Prop}$ 是任意命题，Lean 的内核会把任意两个元素 $t1, t2 : p$ 视为定义相等，这与它把 $(\text{fun } x \Rightarrow t) s$ 和 $t[s/x]$ 视为定义相等非常类似。这称为 *proof irrelevance*（证明无关性），并且与上一段的解释一致。它意味着，即使我们可以把证明 $t : p$ 当作依值类型论语言中的普通对象来处理，它们除了 p 为真这一事实之外不携带任何信息。

我们提出的关于 *propositions-as-types*（命题即类型）范式的两种理解方式，在根本上有所不同。从构造性的观点看，证明是抽象的数学对象，由依值类型论中合适的表达式来指称。相反，如果我们按照上面所描述的编码技巧来思考，那么表达式本身并不指称任何有趣的东西。真正起作用的是：我们能够把它们写下来，并检查它们是良类型的；这一事实保证了相关命题为真。换言之，表达式本身就是证明。

在下面的阐述中，我们会在这两种说法之间来回切换：有时说一个表达式“构造”“产生”或“返回”某个命题的证明，有时又直接说它“是”这样的证明。这类类似于计算机科学家有时会模糊语法与语义之间的区别：有时说一个程序“计算”某个函数，有时又仿佛该程序“就是”所讨论的函数。

无论如何，真正重要的是底线。要在依值类型论语言中形式化地表达一个数学断言，我们需要给出一个项 $p : \text{Prop}$ 。要证明该断言，我们需要给出一个项 $t : p$ 。Lean 作为证明助手的任务，是帮助我们构造这样的项 t ，并验证它形式良好且具有正确的类型。

3.2 使用命题即类型

在 *propositions-as-types*（命题即类型）范式中，只涉及 $\rightarrow$ 的定理可以用 `lambda` 抽象和应用来证明。在 Lean 中，`theorem` 命令引入一个新的定理：

```
set_option linter.unusedVariables false
---
variable {p : Prop}
variable {q : Prop}

theorem t1 : p → q → p := fun hp : p => fun hq : q => hp
```

将这个证明与类型为 $\alpha \rightarrow \beta \rightarrow \alpha$ 的表达式 `fun x :  $\alpha$  => fun y :  $\beta$  => x` 相比较，其中 α 和 β 是数据类型。这个表达式描述了一个函数，它分别接受类型为 α 和 β 的参数 x 与 y ，并返回 x 。`t1` 的证明具有相同的形式，唯一的区别是 p 和 q 是 `Prop` 的元素，而不是 `Type` 的元素。直观地说，我们对 $p \rightarrow q \rightarrow p$ 的证明假设 p 与 q 为真，并（平凡地）使用第一个假设来确立结论 p 为真。

注意，`theorem` 命令实际上是 `def` 命令的一个版本：在命题与类型的对应下，证明定理 $p \rightarrow q \rightarrow p$ 实际上等同于定义相关类型的一个元素。对于内核类型检查器来说，二者没有区别。

不过，定义与定理之间有一些实际差异。在通常情况下，永远没有必要展开一个定理的“定义”：由 `proof irrelevance`（证明无关性），该定理的任意两个证明都是定义相等的。一旦一个定理的证明完成，通常我们只需要知道证明存在；证明具体是什么并不重要。鉴于这一事实，Lean 将证明标记为不可约，这可作为给解析器（更准确地说，是 `elaborator`）的提示：处理文件时通常无需展开它们。事实上，Lean 通常能够并行处理和检查证明，因为评估一个证明的正确性并不需要知道另一个证明的细节。此外，在定义体中被引用的 `section 2.6` (p. 17) 会自动加入为参数，但对于定理，只有在定理类型中被引用的变量才会加入。这是因为一个命题被证明的方式不应影响所证明的命题本身。

与定义一样，`#print` 命令会显示一个定理的证明：

```
theorem t1 : p → q → p := fun hp : p => fun hq : q => hp

#print t1 -- theorem t1 : ∀ {p q : Prop}, p → q → p := fun {p
  ↪ q} hp hq => hp
theorem t1 : ∀ {p q : Prop}, p → q → p :=
fun {p q} hp hq => hp
```

注意，lambda 抽象 $hp : p$ 和 $hq : q$ 可以被看作 `t1` 的证明中的临时假设。Lean 还允许我们用 `show` 语句显式指定最终项 hp 的类型：

```
theorem t1 : p → q → p :=
  fun hp : p =>
    fun hq : q =>
      show p from hp
```

添加这类额外信息可以提高证明的清晰度，并帮助在书写证明时发现错误。`show` 命令所做的不过是标注类型；在内部，我们已经见过的 `t1` 的所有呈现都会产生同一个项。

与普通定义一样，我们可以把 lambda 抽象出来的变量移到冒号左侧：

```
theorem t1 (hp : p) (hq : q) : p := hp

#print t1 -- theorem t1 : ∀ {p q : Prop}, p → q → p := fun
  ↪ {p q} hp hq => hp
theorem t1 : ∀ {p q : Prop}, p → q → p :=
fun {p q} hp hq => hp
```


我们可以像进行函数应用一样使用定理 `t1`:

```
theorem t1 (hp : p) (hq : q) : p := hp
```

```
axiom hp : p
```

```
theorem t2 : q → p := t1 hp
```

axiom 声明假定给定类型的一个元素存在，并且可能破坏逻辑一致性。例如，我们可以用它假定空类型 `False` 有一个元素：

```
axiom unsound : False
```

```
-- Everything follows from false
```

```
theorem ex : 1 = 0 :=
```

```
False.elim unsound
```

声明一个“公理” $hp : p$ 等同于声明 p 为真，并由 hp 作为见证。将定理 `t1` : $p \rightarrow q \rightarrow p$ 应用于事实 $hp : p$ （即 p 为真）会得到定理 `t1 hp : q → p`。

回忆一下，我们也可以把定理 `t1` 写成如下形式：

```
theorem t1 {p q : Prop} (hp : p) (hq : q) : p := hp
```

```
#print t1
```

```
theorem t1 : ∀ {p q : Prop}, p → q → p :=  
fun {p q} hp hq => hp
```

现在 `t1` 的类型是 $\forall \{p q : \text{Prop}\}, p \rightarrow q \rightarrow p$ 。我们可以把它读作断言“对于每一对命题 $p q$ ，都有 $p \rightarrow q \rightarrow p$ 。”例如，我们可以把所有参数都移到冒号右侧：

```
theorem t1 : ∀ {p q : Prop}, p → q → p :=  
fun {p q : Prop} (hp : p) (hq : q) => hp
```

如果 p 和 q 已经被声明为 section 2.6 (p. 17)，Lean 会自动为我们将它们泛化：

```
variable {p q : Prop}
```

```
theorem t1 : p → q → p := fun (hp : p) (hq : q) => hp
```

当我们以这种方式泛化 `t1` 时，就可以把它应用于不同的命题对，从而得到这个一般定理的不同实例。

```
theorem t1 (p q : Prop) (hp : p) (hq : q) : p := hp
```

```
variable (p q r s : Prop)
```

```
#check t1 p q -- t1 p q : p → q → p
```

```
t1 p q : p → q → p
```

```

#check t1 r s -- t1 r s : r → s → r
| t1 r s : r → s → r
#check t1 (r → s) (s → r) -- t1 (r → s) (s → r) : (r → s) →
  ↪ (s → r) → r → s
| t1 (r → s) (s → r) : (r → s) → (s → r) → r → s
variable (h : r → s)

#check t1 (r → s) (s → r) h -- t1 (r → s) (s → r) h : (s → r)
  ↪ → r → s
| t1 (r → s) (s → r) h : (s → r) → r → s

```

再次利用 `propositions-as-types`（命题即类型）对应，类型为 $r \rightarrow s$ 的变量 h 可以被看作假设或前提，即 $r \rightarrow s$ 成立。

另一个例子是，让我们考虑上一章讨论过的复合函数，不过现在把类型换成命题。

```

variable (p q r s : Prop)

theorem t2 (h1 : q → r) (h2 : p → q) : p → r :=
  fun h3 : p =>
    show r from h1 (h2 h3)

```

作为命题逻辑的定理，`t2` 表达了什么？

注意，对于假设，使用数字 `Unicode` 下标通常很有用；它们可通过输入 `\0`、`\1`、`\2`、……得到，正如本例中所做的那样。

3.3 命题逻辑

Lean 定义了所有标准的逻辑联结词及其记号。命题联结词带有如下记号：

ASCII	Unicode	编辑器快捷输入	定义
True			True
False			False
Not	¬	\not, \neg	Not
/\	∧	\and	And
\/	∨	\or	Or
->	→	\to, \r, \imp	
<->	↔	\iff, \lr	Iff

它们的取值都在 `Prop` 中。

```

variable (p q : Prop)

#check p → q → p ∧ q
| p → q → p ∧ q : Prop
#check ¬p → p ↔ False
| ¬p → p ↔ False : Prop

```

```
#check p ∨ q → q ∨ p
| p ∨ q → q ∨ p : Prop
```

运算优先级如下：一元否定 $\neg$ 结合最强，其次是 $\wedge$ ，再其次是 $\vee$ ，然后是 $\rightarrow$ ，最后是 $\leftrightarrow$ 。例如， $a \wedge b \rightarrow c \vee d \wedge e$ 表示 $(a \wedge b) \rightarrow (c \vee (d \wedge e))$ 。请记住， $\rightarrow$ 右结合（现在参数是 `Prop` 的元素而不是某个其他 `Type`，这一点并没有改变），其他二元联结词也是如此。因此，如果有 $p \ q \ r : \text{Prop}$ ，表达式 $p \rightarrow q \rightarrow r$ 读作“若 p ，则若 q ，则 r 。”这正是 $p \wedge q \rightarrow r$ 的“柯里化”形式。

在上一章中，我们观察到 `lambda` 抽象可以看作 $\rightarrow$ 的“引入规则”。在当前语境中，它展示了如何“引入”或确立一个蕴含。应用可以看作“消去规则”，展示了如何在证明中“消去”或使用一个蕴含。其他命题联结词在 `Lean` 的库中定义，并会自动导入。每个联结词都带有其典范的引入规则和消去规则。

合取

表达式 `And.intro h1 h2` 使用证明 $h1 : p$ 和 $h2 : q$ 构造 $p \wedge q$ 的一个证明。通常把 `And.intro` 称为 *and-introduction*（合取引入）规则。在下一个例子中，我们使用 `And.intro` 创建 $p \rightarrow q \rightarrow p \wedge q$ 的一个证明。

```
variable (p q : Prop)
```

```
example (hp : p) (hq : q) : p ∧ q := And.intro hp hq
```

```
#check fun (hp : p) (hq : q) => And.intro hp hq
| fun hp hq => ⟨hp, hq⟩ : p → q → p ∧ q
```

`example` 命令陈述一个定理，但不给它命名，也不把它存入永久上下文。本质上，它只是检查给定项是否具有所标明的类型。它便于示例说明，我们会经常使用它。

表达式 `And.left h` 从证明 $h : p \wedge q$ 创建 p 的一个证明。类似地，`And.right h` 是 q 的一个证明。它们通常称为左、右 *and-elimination*（合取消去）规则。

```
variable (p q : Prop)
```

```
example (h : p ∧ q) : p := And.left h
```

```
example (h : p ∧ q) : q := And.right h
```

现在我们可以用下面的证明项来证明 $p \wedge q \rightarrow q \wedge p$ 。

```
variable (p q : Prop)
```

```
example (h : p ∧ q) : q ∧ p :=
  And.intro (And.right h) (And.left h)
```

注意，合取引入和合取消去类似于笛卡尔积的配对和投影操作。区别在于，给定 $hp : p$ 和 $hq : q$ 时，`And.intro hp hq` 具有类型 $p \wedge q : \text{Prop}$ ；而给定 $a : \alpha$ 和 $b : \beta$ 时，`Prod.mk a b` 具有类型 $\alpha \times \beta : \text{Type}$ 。`Prod` 不能用于 `Prop`，`And` 也不能用于 `Type`。 $\wedge$ 与 $\times$ 之间的相似性是 Curry-Howard isomorphism（Curry-Howard 同构）的又一个实例；但与蕴含和函数空间构造子不同，在 Lean 中 $\wedge$ 与 $\times$ 是分开处理的。不过按照这个类比，我们刚刚构造的证明类似于一个交换有序对两个分量的函数。我们将在 chapter 9 (p. 169) 中看到，Lean 中某些类型是结构，也就是说，该类型由一个单一的典范构造子定义，它从一系列合适的参数构造该类型的一个元素。对于每个 $p q : \text{Prop}$ ， $p \wedge q$ 就是一个例子：构造元素的典范方式是把 `And.intro` 应用于合适的参数 $hp : p$ 和 $hq : q$ 。当相关类型是归纳类型且可以从上下文推断出来时，Lean 允许我们在这类情形中使用匿名构造子记号 `{arg1, arg2, ...}`。特别地，我们通常可以写 `(hp, hq)`，而不写 `And.intro hp hq`：

```
variable (p q : Prop)
variable (hp : p) (hq : q)
```

```
#check ((hp, hq) : p ∧ q)
```

```
| (hp, hq) : p ∧ q
```

这些尖括号分别通过输入 `\<` 和 `\>` 获得。

Lean 提供了另一个有用的语法小工具。给定归纳类型 `Foo`（可能已应用若干参数）的一个表达式 `e`，记号 `e.bar` 是 `Foo.bar e` 的简写。这提供了一种无需打开命名空间即可方便访问函数的方式。例如，下面两个表达式含义相同：

```
variable (xs : List Nat)
```

```
#check List.length xs
```

```
| xs.length : Nat
```

```
#check xs.length
```

```
| xs.length : Nat
```

因此，给定 $h : p \wedge q$ ，我们可以写 `h.left` 表示 `And.left h`，写 `h.right` 表示 `And.right h`。于是，我们可以方便地把上面的示例证明改写如下：

```
variable (p q : Prop)
```

```
example (h : p ∧ q) : q ∧ p :=
  (h.right, h.left)
```

简洁与晦涩之间只有一线之隔：以这种方式省略信息有时会使证明更难阅读。但对于像上面这样直接的构造，当 h 的类型以及构造的目标都很显眼时，这种记号清晰而有效。

像 “And” 这样的构造经常会迭代出现。Lean 还允许你把右结合的嵌套构造子展平，因此下面两个证明是等价的：

```
variable (p q : Prop)
```

```
example (h : p ∧ q) : q ∧ p ∧ q :=
  (h.right, (h.left, h.right))
```

```
example (h : p ∧ q) : q ∧ p ∧ q :=
  (h.right, h.left, h.right)
```

这也常常很有用。

析取

表达式 `Or.intro_left q hp` 从证明 $hp : p$ 创建 $p \vee q$ 的一个证明。类似地, `Or.intro_right p hq` 使用证明 $hq : q$ 创建 $p \vee q$ 的一个证明。它们分别是左、右 *or-introduction* (析取引入) 规则。

```
variable (p q : Prop)
```

```
example (hp : p) : p ∨ q := Or.intro_left q hp
```

```
example (hq : q) : p ∨ q := Or.intro_right p hq
```

or-elimination (析取消去) 规则稍微复杂一些。其思想是, 我们可以通过说明 r 可由 p 推出, 并且 r 也可由 q 推出, 来从 $p \vee q$ 证明 r 。换言之, 这就是分情况证明。在表达式 `Or.elim hpq hpr hqr` 中, `Or.elim` 接受三个参数: $hpq : p \vee q$ 、 $hpr : p \rightarrow r$ 和 $hqr : q \rightarrow r$, 并产生 r 的一个证明。在下面的例子中, 我们使用 `Or.elim` 证明 $p \vee q \rightarrow q \vee p$ 。

```
variable (p q r : Prop)
```

```
example (h : p ∨ q) : q ∨ p :=
  Or.elim h
    (fun hp : p =>
      show q ∨ p from Or.intro_right q hp)
    (fun hq : q =>
      show q ∨ p from Or.intro_left p hq)
```

在大多数情形中, `Or.intro_right` 和 `Or.intro_left` 的第一个参数都可以由 Lean 自动推断。因此 Lean 提供了 `Or.inr` 和 `Or.inl`, 它们可看作 `Or.intro_right _` 与 `Or.intro_left _` 的简写。于是, 上面的证明项可以更简洁地写为:

```
variable (p q r : Prop)
```

```
example (h : p ∨ q) : q ∨ p :=
  Or.elim h (fun hp => Or.inr hp) (fun hq => Or.inl hq)
```

注意, 完整表达式中有足够的信息让 Lean 也推断出 hp 和 hq 的类型。不过, 在较长版本中使用类型标注会使证明更易读, 也有助于捕捉和调试错误。

由于 `Or` 有两个构造子, 我们不能使用匿名构造子记号。但我们仍然可以写 `h.elim`, 而不是 `Or.elim h`:

```
variable (p q r : Prop)
```

```
example (h : p ∨ q) : q ∨ p :=
  h.elim (fun hp => Or.inr hp) (fun hq => Or.inl hq)
```

再一次，你应当自行判断这类缩写是增强还是削弱了可读性。

否定与假命题

否定 $\neg p$ 实际上被定义为 $p \rightarrow \text{False}$ ，因此我们通过从 p 推导出矛盾来获得 $\neg p$ 。类似地，表达式 $\text{hnp } hp$ 从 $hp : p$ 和 $\text{hnp} : \neg p$ 产生 False 的一个证明。下一个例子使用这两条规则来产生 $(p \rightarrow q) \rightarrow \neg q \rightarrow \neg p$ 的一个证明。（符号 $\neg$ 可通过输入 `\not` 或 `\neg` 产生。）

```
variable (p q : Prop)
```

```
example (hpq : p → q) (hnq : ¬q) : ¬p :=
  fun hp : p =>
    show False from hnq (hpq hp)
```

联结词 `False` 有一条消去规则 `False.elim`，它表达了从矛盾可以推出任何事物这一事实。该规则有时称为 *ex falso* (*ex falso sequitur quodlibet* 的简称)，或爆炸原理。

```
variable (p q : Prop)
```

```
example (hp : p) (hnp : ¬p) : q := False.elim (hnp hp)
```

从假命题推出的任意事实 q 是 `False.elim` 的一个隐式参数，并会被自动推断。这个模式，即从相互矛盾的假设推出任意事实，非常常见，并由 `absurd` 表示。

```
variable (p q : Prop)
```

```
example (hp : p) (hnp : ¬p) : q := absurd hp hnp
```

例如，下面是 $\neg p \rightarrow q \rightarrow (q \rightarrow p) \rightarrow r$ 的一个证明：

```
variable (p q r : Prop)
```

```
example (hnp : ¬p) (hq : q) (hqp : q → p) : r :=
  absurd (hqp hq) hnp
```

顺便说一句，正如 `False` 只有一条消去规则，`True` 也只有一条引入规则 `True.intro : True`。换言之，`True` 就是真的，并且有一个典范证明 `True.intro`。

逻辑等价

表达式 `Iff.intro h1 h2` 从 $h1 : p \rightarrow q$ 和 $h2 : q \rightarrow p$ 产生 $p \leftrightarrow q$ 的一个证明。表达式 `Iff.mp h` 从 $h : p \leftrightarrow q$ 产生 $p \rightarrow q$ 的一个证明。类似地, `Iff.mpr h` 从 $h : p \leftrightarrow q$ 产生 $q \rightarrow p$ 的一个证明。下面是 $p \wedge q \leftrightarrow q \wedge p$ 的一个证明:

```
variable (p q : Prop)

theorem and_swap : p ∧ q ↔ q ∧ p :=
  Iff.intro
    (fun h : p ∧ q =>
      show q ∧ p from And.intro (And.right h) (And.left h))
    (fun h : q ∧ p =>
      show p ∧ q from And.intro (And.right h) (And.left h))

#check and_swap p q    -- and_swap p q : p ∧ q ↔ q ∧ p
| and_swap p q : p ∧ q ↔ q ∧ p
variable (h : p ∧ q)
example : q ∧ p := Iff.mp (and_swap p q) h
```

我们可以使用匿名构造子记号, 由正向和反向两个方向的证明构造 $p \leftrightarrow q$ 的一个证明; 也可以对 `mp` 和 `mpr` 使用 `.` 记号。因此, 前面的例子可以简洁地写成如下形式:

```
variable (p q : Prop)

theorem and_swap : p ∧ q ↔ q ∧ p :=
  ( fun h => {h.right, h.left}, fun h => {h.right, h.left} )

example (h : p ∧ q) : q ∧ p := (and_swap p q).mp h
```

3.4 引入辅助子目标

这里适合介绍 Lean 提供的另一个有助于组织长证明的工具, 即 **have** 构造, 它会在证明中引入一个辅助子目标。下面是一个改编自上一节的小例子:

```
variable (p q : Prop)

example (h : p ∧ q) : q ∧ p :=
  have hp : p := h.left
  have hq : q := h.right
  show q ∧ p from And.intro hq hp
```

在内部, 表达式 `have h : p := s; t` 产生项 `(fun (h : p) => t) s`。换言之, s 是 p 的一个证明, t 是在假设 $h : p$ 下所需结论的一个证明, 而二者通

过 `lambda` 抽象和应用组合起来。这个简单工具在组织长证明时极其有用，因为我们可以把中间的 **have** 作为通向最终目标的踏脚石。

Lean 还支持一种从目标反向推理的结构化方式，它模拟了普通数学中的“只需证明”构造。下一个例子只是调换了前一个证明的最后两行。

```
variable (p q : Prop)

example (h : p ∧ q) : q ∧ p :=
  have hp : p := h.left
  suffices hq : q from And.intro hq hp
  show q from And.right h
```

书写 `suffices hq : q` 会留下两个目标。首先，我们必须通过在额外假设 `hq : q` 下证明原始目标 $q \wedge p$ ，来证明“只需证明 q ”确实足够。最后，我们还必须证明 q 。

3.5 经典逻辑

到目前为止我们看到的引入规则和消去规则都是构造性的，也就是说，它们反映了基于 `propositions-as-types`（命题即类型）对应对逻辑联结词的计算性理解。通常的经典逻辑在此基础上加入排中律 $p \vee \neg p$ 。要使用这一原则，必须打开 `classical` 命名空间。

```
open Classical
```

```
variable (p : Prop)
```

```
#check em p
```

```
em p : p ∨ ¬p
```

直观地说，构造性的“Or”非常强：断言 $p \vee q$ 等同于知道究竟是哪一种情形。如果 RH 表示 Riemann 假设，经典数学家愿意断言 $RH \vee \neg RH$ ，即使我们还不能断言任一析取支。

排中律的一个推论是双重否定消去原则：

```
open Classical
```

```
theorem dne {p : Prop} (h : ¬¬p) : p :=
  Or.elim (em p)
    (fun hp : p => hp)
    (fun hnp : ¬p => absurd hnp h)
```

双重否定消去允许人们通过假设 $\neg p$ 并推出 `False` 来证明任意命题 p ，因为这等同于证明 $\neg\neg p$ 。换言之，双重否定消去允许人们进行反证法证明，而这在构造性逻辑中通常是不可能的。作为练习，你可以尝试证明其逆命题，也就是说明 `em` 可以从 `dne` 证明。经典公理还让你能够使用一些可通过诉诸 `em` 来辩护的额外证明模式。例如，可以进行分情况证明：


```

open Classical
variable (p : Prop)

example (h : ¬¬p) : p :=
  byCases
    (fun h1 : p => h1)
    (fun h1 : ¬p => absurd h1 h)

```

或者，你可以进行反证法证明：

```

open Classical
variable (p : Prop)

example (h : ¬¬p) : p :=
  byContradiction
    (fun h1 : ¬p =>
      show False from h h1)

```

如果你不习惯构造性地思考，可能需要一些时间才能形成对何处使用了经典推理的感觉。下面的例子需要经典推理，因为从构造性的立场看，知道 p 和 q 不可能同时为真，并不一定告诉你哪一个为假：

```

example (h : ¬(p ∧ q)) : ¬p ∨ ¬q :=
  Or.elim (em p)
    (fun hp : p =>
      Or.inr
        (show ¬q from
          fun hq : q =>
            h (hp, hq)))
    (fun hp : ¬p =>
      Or.inl hp)

```

稍后我们会看到，在构造性逻辑中确实存在一些情形，其中排中律和双重否定消去这样的原则是可允许的；Lean 支持在这类上下文中使用经典推理，而不依赖排中律。

Lean 用来支持经典推理的完整公理列表将在 chapter 12 (p. 199) 中讨论。

3.6 命题有效式示例

Lean 的标准库包含许多命题逻辑有效陈述的证明，你可以在自己的证明中自由使用它们。下面的列表包括若干常见恒等式。交换律：

$$1. p \wedge q \leftrightarrow q \wedge p$$

$$2. p \vee q \leftrightarrow q \vee p$$

结合律：

$$1. (p \wedge q) \wedge r \leftrightarrow p \wedge (q \wedge r)$$

$$2. (p \vee q) \vee r \leftrightarrow p \vee (q \vee r)$$

分配律:

$$1. p \wedge (q \vee r) \leftrightarrow (p \wedge q) \vee (p \wedge r)$$

$$2. p \vee (q \wedge r) \leftrightarrow (p \vee q) \wedge (p \vee r)$$

其他性质:

$$1. (p \rightarrow (q \rightarrow r)) \leftrightarrow (p \wedge q \rightarrow r)$$

$$2. ((p \vee q) \rightarrow r) \leftrightarrow (p \rightarrow r) \wedge (q \rightarrow r)$$

$$3. \neg(p \vee q) \leftrightarrow \neg p \wedge \neg q$$

$$4. \neg p \vee \neg q \rightarrow \neg(p \wedge q)$$

$$5. \neg(p \wedge \neg p)$$

$$6. p \wedge \neg q \rightarrow \neg(p \rightarrow q)$$

$$7. \neg p \rightarrow (p \rightarrow q)$$

$$8. (\neg p \vee q) \rightarrow (p \rightarrow q)$$

$$9. p \vee \text{False} \leftrightarrow p$$

$$10. p \wedge \text{False} \leftrightarrow \text{False}$$

$$11. \neg(p \leftrightarrow \neg p)$$

$$12. (p \rightarrow q) \rightarrow (\neg q \rightarrow \neg p)$$

这些需要经典推理:

$$1. (p \rightarrow r \vee s) \rightarrow ((p \rightarrow r) \vee (p \rightarrow s))$$

$$2. \neg(p \wedge q) \rightarrow \neg p \vee \neg q$$

$$3. \neg(p \rightarrow q) \rightarrow p \wedge \neg q$$

$$4. (p \rightarrow q) \rightarrow (\neg p \vee q)$$

$$5. (\neg q \rightarrow \neg p) \rightarrow (p \rightarrow q)$$

$$6. p \vee \neg p$$

$$7. (((p \rightarrow q) \rightarrow p) \rightarrow p)$$

sorry 标识符会神奇地产生任何命题的证明，或者提供任何数据类型的对象。当然，作为证明方法它是不可靠的——例如，你可以用它证明 **False**——并且当文件使用或导入依赖它的定理时，**Lean** 会产生严重警告。但它对于逐步构建长证明非常有用。可以从上到下开始书写证明，用 **sorry** 填充子证明。确保 **Lean** 接受包含所有 **sorry** 的项；如果不接受，就有需要修正的错误。然后回头把每个 **sorry** 替换为实际证明，直到不再剩下为止。还有一个有用技巧。除了使用 **sorry**，你也可以使用下划线 **_** 作为占位符。回忆一下，这会告诉 **Lean** 该参数是隐式的，应当自动填入。如果 **Lean** 尝试这样做但失败，它会返回错误消息 “don’t know how to synthesize placeholder”，随后给出它期望的项的类型，以及上下文中所有可用对象和假设。换言之，对于每个未解决的占位符，**Lean** 都会报告该处需要填充的子目标。随后你可以通过逐步填充这些占位符来构造证明。

作为参考，下面是从上面列表中取出的两个有效式的示例证明。

open Classical

```
-- distributivity
example (p q r : Prop) : p ∧ (q ∨ r) ↔ (p ∧ q) ∨ (p ∧ r) :=
  Iff.intro
    (fun h : p ∧ (q ∨ r) =>
      have hp : p := h.left
      Or.elim (h.right)
        (fun hq : q =>
          show (p ∧ q) ∨ (p ∧ r) from Or.inl ⟨hp, hq⟩)
        (fun hr : r =>
          show (p ∧ q) ∨ (p ∧ r) from Or.inr ⟨hp, hr⟩))
    (fun h : (p ∧ q) ∨ (p ∧ r) =>
      Or.elim h
        (fun hpq : p ∧ q =>
          have hp : p := hpq.left
          have hq : q := hpq.right
          show p ∧ (q ∨ r) from ⟨hp, Or.inl hq⟩)
        (fun hpr : p ∧ r =>
          have hp : p := hpr.left
          have hr : r := hpr.right
          show p ∧ (q ∨ r) from ⟨hp, Or.inr hr⟩))

-- an example that requires classical reasoning
example (p q : Prop) : ¬(p ∧ ¬q) → (p → q) :=
  fun h : ¬(p ∧ ¬q) =>
  fun hp : p =>
  show q from
    Or.elim (em q)
      (fun hq : q => hq)
      (fun hnq : ¬q => absurd (And.intro hp hnq) h)
```

3.7 练习

证明下列恒等式，将 **sorry** 占位符替换为实际证明。

```
variable (p q r : Prop)

-- commutativity of  $\wedge$  and  $\vee$ 
example : p  $\wedge$  q  $\leftrightarrow$  q  $\wedge$  p := sorry
example : p  $\vee$  q  $\leftrightarrow$  q  $\vee$  p := sorry

-- associativity of  $\wedge$  and  $\vee$ 
example : (p  $\wedge$  q)  $\wedge$  r  $\leftrightarrow$  p  $\wedge$  (q  $\wedge$  r) := sorry
example : (p  $\vee$  q)  $\vee$  r  $\leftrightarrow$  p  $\vee$  (q  $\vee$  r) := sorry

-- distributivity
example : p  $\wedge$  (q  $\vee$  r)  $\leftrightarrow$  (p  $\wedge$  q)  $\vee$  (p  $\wedge$  r) := sorry
example : p  $\vee$  (q  $\wedge$  r)  $\leftrightarrow$  (p  $\vee$  q)  $\wedge$  (p  $\vee$  r) := sorry

-- other properties
example : (p  $\rightarrow$  (q  $\rightarrow$  r))  $\leftrightarrow$  (p  $\wedge$  q  $\rightarrow$  r) := sorry
example : ((p  $\vee$  q)  $\rightarrow$  r)  $\leftrightarrow$  (p  $\rightarrow$  r)  $\wedge$  (q  $\rightarrow$  r) := sorry
example :  $\neg$ (p  $\vee$  q)  $\leftrightarrow$   $\neg$ p  $\wedge$   $\neg$ q := sorry
example :  $\neg$ p  $\vee$   $\neg$ q  $\rightarrow$   $\neg$ (p  $\wedge$  q) := sorry
example :  $\neg$ (p  $\wedge$   $\neg$ p) := sorry
example : p  $\wedge$   $\neg$ q  $\rightarrow$   $\neg$ (p  $\rightarrow$  q) := sorry
example :  $\neg$ p  $\rightarrow$  (p  $\rightarrow$  q) := sorry
example : ( $\neg$ p  $\vee$  q)  $\rightarrow$  (p  $\rightarrow$  q) := sorry
example : p  $\vee$  False  $\leftrightarrow$  p := sorry
example : p  $\wedge$  False  $\leftrightarrow$  False := sorry
example : (p  $\rightarrow$  q)  $\rightarrow$  ( $\neg$ q  $\rightarrow$   $\neg$ p) := sorry
```

证明下列恒等式，将 **sorry** 占位符替换为实际证明。这些需要经典推理。

```
open Classical

variable (p q r : Prop)

example : (p  $\rightarrow$  q  $\vee$  r)  $\rightarrow$  ((p  $\rightarrow$  q)  $\vee$  (p  $\rightarrow$  r)) := sorry
example :  $\neg$ (p  $\wedge$  q)  $\rightarrow$   $\neg$ p  $\vee$   $\neg$ q := sorry
example :  $\neg$ (p  $\rightarrow$  q)  $\rightarrow$  p  $\wedge$   $\neg$ q := sorry
example : (p  $\rightarrow$  q)  $\rightarrow$  ( $\neg$ p  $\vee$  q) := sorry
example : ( $\neg$ q  $\rightarrow$   $\neg$ p)  $\rightarrow$  (p  $\rightarrow$  q) := sorry
example : p  $\vee$   $\neg$ p := sorry
example : (((p  $\rightarrow$  q)  $\rightarrow$  p)  $\rightarrow$  p) := sorry
```

不用经典逻辑证明 $\neg(p \leftrightarrow \neg p)$ 。

4 量词与等式

上一章介绍了构造涉及命题联结词的陈述之证明的方法。本章将把逻辑构造的工具箱扩展到全称量词、存在量词以及等式关系。

4.1 全称量词

注意，如果 α 是任意类型，那么 α 上的一元谓词 p 可以表示为类型为 $\alpha \rightarrow \text{Prop}$ 的对象。在这种情况下，给定 $x : \alpha$ ， $p\ x$ 表示“ p 对 x 成立”这一断言。类似地，对象 $r : \alpha \rightarrow \alpha \rightarrow \text{Prop}$ 表示 α 上的二元关系：给定 $x\ y : \alpha$ ， $r\ x\ y$ 表示“ x 与 y 相关”这一断言。

全称量词 $\forall x : \alpha, p\ x$ 应表示断言“对每个 $x : \alpha$ ， $p\ x$ 成立”。与命题联结词一样，在自然演绎系统中，“全称量词”由引入规则和消去规则刻画。非形式地说，引入规则表述为：

在 $x : \alpha$ 任意的上下文中，给定 $p\ x$ 的一个证明，我们得到 $\forall x : \alpha, p\ x$ 的一个证明。

消去规则表述为：

给定 $\forall x : \alpha, p\ x$ 的一个证明以及任意项 $t : \alpha$ ，我们得到 $p\ t$ 的一个证明。

与蕴含的情形一样，命题即类型的解释现在发挥作用。回忆依值箭头类型的引入规则和消去规则：

在 $x : \alpha$ 任意的上下文中，给定类型为 $\beta\ x$ 的项 t ，我们有 $(\text{fun } x : \alpha \Rightarrow t) : (x : \alpha) \rightarrow \beta\ x$ 。

消去规则表述为：

给定项 $s : (x : \alpha) \rightarrow \beta\ x$ 以及任意项 $t : \alpha$ ，我们有 $s\ t : \beta\ t$ 。

在 $p\ x$ 具有类型 Prop 的情形下，如果把 $(x : \alpha) \rightarrow \beta\ x$ 替换为 $\forall x : \alpha, p\ x$ ，就可以把这些规则读作构造涉及全称量词之证明的正确规则。

因此，构造演算以这种方式把依值箭头类型与全称表达式等同起来。如果 p 是任意表达式， $\forall x : \alpha, p$ 只不过是 $(x : \alpha) \rightarrow p$ 的另一种记法；当 p 是

命题时，前一种记法通常比后一种更自然。典型地，表达式 p 会依赖于 $x : \alpha$ 。回忆普通函数空间的情形：我们可以把 $\alpha \rightarrow \beta$ 解释为 $(x : \alpha) \rightarrow \beta$ 的特殊情形，其中 β 不依赖于 x 。类似地，命题之间的蕴含 $p \rightarrow q$ 可看作 $\forall x : p, q$ 的特殊情形，其中表达式 q 不依赖于 x 。

下面的例子展示了 `propositions-as-types` 对应如何在实践中使用。

```
example ( $\alpha$  : Type) ( $p\ q$  :  $\alpha \rightarrow \text{Prop}$ ) :
  ( $\forall x : \alpha, p\ x \wedge q\ x$ )  $\rightarrow \forall y : \alpha, p\ y :=$ 
  fun  $h$  :  $\forall x : \alpha, p\ x \wedge q\ x \Rightarrow$ 
  fun  $y : \alpha \Rightarrow$ 
  show  $p\ y$  from ( $h\ y$ ).left
```

作为记号约定，我们赋予全称量词尽可能大的作用域，因此在上面的例子中需要用括号把对 x 的量化限制在假设内。证明 $\forall y : \alpha, p\ y$ 的标准方式是取任意的 y ，然后证明 $p\ y$ 。这就是引入规则。现在，既然 h 的类型是 $\forall x : \alpha, p\ x \wedge q\ x$ ，表达式 $h\ y$ 的类型就是 $p\ y \wedge q\ y$ 。这就是消去规则。取左合取项便得到所需结论 $p\ y$ 。

记住，只差一个绑定变量重命名的表达式被视为等价。因此，例如我们可以在假设和结论中都使用同一个变量 x ，而在证明中用另一个变量 z 来实例化它：

```
example ( $\alpha$  : Type) ( $p\ q$  :  $\alpha \rightarrow \text{Prop}$ ) :
  ( $\forall x : \alpha, p\ x \wedge q\ x$ )  $\rightarrow \forall x : \alpha, p\ x :=$ 
  fun  $h$  :  $\forall x : \alpha, p\ x \wedge q\ x \Rightarrow$ 
  fun  $z : \alpha \Rightarrow$ 
  show  $p\ z$  from And.left ( $h\ z$ )
```

再举一例，下面说明如何表达关系 r 具有传递性这一事实：

```
variable ( $\alpha$  : Type) ( $r$  :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$ )
variable ( $\text{trans}_r$  :  $\forall x\ y\ z, r\ x\ y \rightarrow r\ y\ z \rightarrow r\ x\ z$ )

variable ( $a\ b\ c$  :  $\alpha$ )
variable ( $\text{hab}$  :  $r\ a\ b$ ) ( $\text{hbc}$  :  $r\ b\ c$ )

#check  $\text{trans}_r$       --  $\text{trans}_r : \forall (x\ y\ z : \alpha), r\ x\ y \rightarrow r\ y\ z \rightarrow$ 
   $\hookrightarrow r\ x\ z$ 
|  $\text{trans}_r : \forall (x\ y\ z : \alpha), r\ x\ y \rightarrow r\ y\ z \rightarrow r\ x\ z$ 
#check  $\text{trans}_r\ a\ b\ c$  --  $\text{trans}_r\ a\ b\ c : r\ a\ b \rightarrow r\ b\ c \rightarrow r\ a\ c$ 
|  $\text{trans}_r\ a\ b\ c : r\ a\ b \rightarrow r\ b\ c \rightarrow r\ a\ c$ 
#check  $\text{trans}_r\ a\ b\ c\ \text{hab}$  --  $\text{trans}_r\ a\ b\ c\ \text{hab} : r\ b\ c \rightarrow r\ a\ c$ 
|  $\text{trans}_r\ a\ b\ c\ \text{hab} : r\ b\ c \rightarrow r\ a\ c$ 
#check  $\text{trans}_r\ a\ b\ c\ \text{hab}\ \text{hbc}$  --  $\text{trans}_r\ a\ b\ c\ \text{hab}\ \text{hbc} : r\ a\ c$ 
|  $\text{trans}_r\ a\ b\ c\ \text{hab}\ \text{hbc} : r\ a\ c$ 
```

思考这里发生了什么。当我们在值 $a\ b\ c$ 处实例化 $trans_r$ 时，最终得到 $r\ a\ b \rightarrow r\ b\ c \rightarrow r\ a\ c$ 的一个证明。把它应用到“假设” $hab : r\ a\ b$ ，我们得到蕴含 $r\ b\ c \rightarrow r\ a\ c$ 的一个证明。最后，把它应用到假设 hbc ，便得到结论 $r\ a\ c$ 的证明。

在这种情形中，如果参数 $a\ b\ c$ 可以由 $hab\ hbc$ 推断出来，那么显式提供它们会显得繁琐。因此，通常把这些参数设为隐式参数：

```
variable (α : Type) (r : α → α → Prop)
variable (trans_r : ∀ {x y z}, r x y → r y z → r x z)

variable (a b c : α)
variable (hab : r a b) (hbc : r b c)

#check trans_r
| trans_r : r ?m.4 ?m.5 → r ?m.5 ?m.6 → r ?m.4 ?m.6
#check trans_r hab
| trans_r hab : r b ?m.6 → r a ?m.6
#check trans_r hab hbc
| trans_r hab hbc : r a c
```

其优点是，我们可以直接把 $trans_r\ hab\ hbc$ 写作 $r\ a\ c$ 的证明。缺点是，对于表达式 $trans_r$ 和 $trans_r\ hab$ ，Lean 没有足够的信息来推断参数的类型。第一个 `#check` 命令的输出是 $r\ ?m.1\ ?m.2 \rightarrow r\ ?m.2\ ?m.3 \rightarrow r\ ?m.1\ ?m.3$ ，这表明在此情形中隐式参数尚未指定。

下面的例子说明如何用一个等价关系进行基本推理：

```
variable (α : Type) (r : α → α → Prop)

variable (refl_r : ∀ x, r x x)
variable (symm_r : ∀ {x y}, r x y → r y x)
variable (trans_r : ∀ {x y z}, r x y → r y z → r x z)

example (a b c d : α) (hab : r a b) (hcb : r c b) (hcd : r c
  ↪ d) : r a d :=
  trans_r (trans_r hab (symm_r hcb)) hcd
```

为了熟悉全称量词的用法，你应当尝试本节末尾的一些练习。

正是依值箭头类型，尤其是全称量词的类型规则，将 `Prop` 与其他类型区分开来。假设有 $\alpha : \text{Sort } i$ 和 $\beta : \text{Sort } j$ ，其中表达式 β 可以依赖于变量 $x : \alpha$ 。那么 $(x : \alpha) \rightarrow \beta$ 是 $\text{Sort } (\text{imax } i\ j)$ 的元素；这里若 j 不是 0 ，则 $\text{imax } i\ j$ 是 i 与 j 的最大值，否则为 0 。其思想如下。如果 j 不是 0 ，那么 $(x : \alpha) \rightarrow \beta$ 是 $\text{Sort } (\text{max } i\ j)$ 的元素。换言之，从 α 到 β 的依值函数类型“位于”下标为 i 与 j 最大值的宇宙中。然而，若 β 属于 $\text{Sort } 0$ ，即是 `Prop` 的元素，那么无论 α 位于哪个类型宇宙， $(x : \alpha) \rightarrow \beta$ 也都是 $\text{Sort } 0$ 的元素。换言之，如果 β 是依赖于 α 的命题，那么 $\forall x : \alpha, \beta$ 仍然是命题。这反映了把

Prop 解释为命题而非数据的类型，也正是它使 Prop 成为 *impredicative*。术语“predicative”源自二十世纪之交的基础研究。当时，Poincaré 和 Russell 等逻辑学家把集合论悖论归咎于一种“恶性循环”：我们通过对一个包含待定义性质本身的集合进行量化来定义某个性质。注意，如果 α 是任意类型，我们可以形成 α 上所有谓词的类型 $\alpha \rightarrow \text{Prop}$ （即“ α 的幂类型”）。Prop 的非谓性意味着，我们可以形成对 $\alpha \rightarrow \text{Prop}$ 量化的命题。特别地，我们可以通过对 α 上所有谓词量化来定义 α 上的谓词；这正是曾经被认为有问题的那种循环性。

4.2 等式

现在转向 Lean 库中定义的最基本关系之一，即等式关系。在关于 chapter 7 (p. 113) 的章节中，我们将解释等式是如何由 Lean 逻辑框架的基本构件定义出来的。这里则先说明如何使用它。

当然，等式的一个基本性质是它构成等价关系：

```
#check Eq.refl      -- Eq.refl.{u_1} {α : Sort u_1} (a : α) : a
↪ = a
| Eq.refl.{u_1} {α : Sort u_1} (a : α) : a = a
#check Eq.symm      -- Eq.symm.{u} {α : Sort u} {a b : α} (h : a
↪ = b) : b = a
| Eq.symm.{u} {α : Sort u} {a b : α} (h : a = b) : b = a
#check Eq.trans     -- Eq.trans.{u} {α : Sort u} {a b c : α} (h₁
↪ : a = b) (h₂ : b = c) : a = c
| Eq.trans.{u} {α : Sort u} {a b c : α} (h₁ : a = b) (h₂ : b =
↪ c) : a = c
```

可以告诉 Lean 不要插入隐式参数（这里显示为元变量），从而使输出更易读。

universe u

```
#check @Eq.refl.{u}  -- @Eq.refl : ∀ {α : Sort u} (a : α), a
↪ = a
| @Eq.refl : ∀ {α : Sort u} (a : α), a = a
#check @Eq.symm.{u}  -- @Eq.symm : ∀ {α : Sort u} {a b : α},
↪ a = b → b = a
| @Eq.symm : ∀ {α : Sort u} {a b : α}, a = b → b = a
#check @Eq.trans.{u} -- @Eq.trans : ∀ {α : Sort u} {a b c :
↪ α}, a = b → b = c → a = c
| @Eq.trans : ∀ {α : Sort u} {a b c : α}, a = b → b = c → a = c
```

标记 `.{u}` 告诉 Lean 在宇宙 u 处实例化这些常量。

因此，例如我们可以把上一节的例子特化到等式关系：

```
variable (α : Type) (a b c d : α)
```



```
variable (hab : a = b) (hcb : c = b) (hcd : c = d)
```

```
example : a = d :=
  Eq.trans (Eq.trans hab (Eq.symm hcb)) hcd
```

我们也可以使用投影记法:

```
example : a = d := (hab.trans hcb.symm).trans hcd
```

自反性比表面看起来更强大。回忆一下, 构造演算中的项具有计算解释, 而逻辑框架把具有共同约化结果的项视为相同。因此, 一些不平凡恒等式可以由自反性证明:

```
variable ( $\alpha$   $\beta$  : Type)
```

```
example (f :  $\alpha$   $\rightarrow$   $\beta$ ) (a :  $\alpha$ ) : (fun x => f x) a = f a :=
   $\hookrightarrow$  Eq.refl _
```

```
example (a :  $\alpha$ ) (b :  $\beta$ ) : (a, b).1 = a := Eq.refl _
```

```
example : 2 + 3 = 5 := Eq.refl _
```

该框架的这一特性非常重要, 因此库为 Eq.refl _ 定义了记法 rfl:

```
example (f :  $\alpha$   $\rightarrow$   $\beta$ ) (a :  $\alpha$ ) : (fun x => f x) a = f a := rfl
```

```
example (a :  $\alpha$ ) (b :  $\beta$ ) : (a, b).1 = a := rfl
```

```
example : 2 + 3 = 5 := rfl
```

然而, 等式远不只是等价关系。它还具有一个重要性质: 每个断言都尊重这种等价; 也就是说, 我们可以替换相等的表达式而不改变真值。换言之, 给定 $h1 : a = b$ 和 $h2 : p a$, 我们可以通过替换构造 $p b$ 的证明: Eq.subst h1 h2。

```
example ( $\alpha$  : Type) (a b :  $\alpha$ ) (p :  $\alpha$   $\rightarrow$  Prop)
  (h1 : a = b) (h2 : p a) : p b :=
  Eq.subst h1 h2
```

```
example ( $\alpha$  : Type) (a b :  $\alpha$ ) (p :  $\alpha$   $\rightarrow$  Prop)
  (h1 : a = b) (h2 : p a) : p b :=
  h1  $\triangleright$  h2
```

第二种写法中的三角符号是在 Eq.subst 和 Eq.symm 之上构建的宏; 可以通过输入 \t 得到它。

规则 Eq.subst 用于定义以下辅助规则, 它们执行更显式的替换。这些规则设计用来处理应用项, 即形如 $s t$ 的项。具体地说, congrArg 可用于替换参数, congrFun 可用于替换被应用的项, 而 congr 可同时替换二者。

```
variable ( $\alpha$  : Type)
```

```
variable (a b :  $\alpha$ )
```

```
variable (f g :  $\alpha$   $\rightarrow$  Nat)
```

```

variable (h1 : a = b)
variable (h2 : f = g)

example : f a = f b := congrArg f h1
example : f a = g a := congrFun h2 a
example : f a = g b := congr h2 h1

```

Lean 的库包含大量常用恒等式，例如：

```

variable (a b c : Nat)

example : a + 0 = a := Nat.add_zero a
example : 0 + a = a := Nat.zero_add a
example : a * 1 = a := Nat.mul_one a
example : 1 * a = a := Nat.one_mul a
example : a + b = b + a := Nat.add_comm a b
example : a + b + c = a + (b + c) := Nat.add_assoc a b c
example : a * b = b * a := Nat.mul_comm a b
example : a * b * c = a * (b * c) := Nat.mul_assoc a b c
example : a * (b + c) = a * b + a * c := Nat.mul_add a b c
example : a * (b + c) = a * b + a * c := Nat.left_distrib a b
  ↪ c
example : (a + b) * c = a * c + b * c := Nat.add_mul a b c
example : (a + b) * c = a * c + b * c := Nat.right_distrib a b
  ↪ c

```

注意，**Nat.mul_add** 和 **Nat.add_mul** 分别是 **Nat.left_distrib** 与 **Nat.right_distrib** 的别名。上述性质是针对自然数（类型 **Nat**）陈述的。

下面是一个自然数计算的例子，它结合使用替换、结合律和分配律。

```

example (x y : Nat) :
  (x + y) * (x + y) =
  x * x + y * x + x * y + y * y :=
have h1 : (x + y) * (x + y) = (x + y) * x + (x + y) * y :=
  Nat.mul_add (x + y) x y
have h2 : (x + y) * (x + y) = x * x + y * x + (x * y + y *
  ↪ y) :=
  (Nat.add_mul x y x) ▸ (Nat.add_mul x y y) ▸ h1
h2.trans (Nat.add_assoc (x * x + y * x) (x * y) (y *
  ↪ y)).symm

```

注意，**Eq.subst** 的第二个隐式参数给出了要发生替换的上下文，其类型为 $\alpha \rightarrow \text{Prop}$ 。因此，推断这个谓词需要一个高阶合一实例。在完全一般的情形下，判定是否存在高阶合一子是不可判定的，Lean 最多只能为该问题提供不完美的近似解。因此，**Eq.subst** 并不总能按你的期望工作。宏 $h \triangleright e$ 使用更有效

的启发式方法来计算这个隐式参数，常常能在直接应用 `Eq.subst` 失败的情况下成功。

由于等式推理非常常见且重要，**Lean** 提供了若干机制来更有效地进行等式推理。下一节给出一种语法，使你能够以更自然、更清晰的方式书写计算式证明。更重要的是，等式推理由项重写器、化简器以及其他自动化机制支持。项重写器和化简器将在下一节简要介绍，并在下一章更详细地讨论。

4.3 计算式证明

计算式证明只是由一系列中间结果组成的链条，这些结果意在通过等式传递性等基本原则组合起来。在 **Lean** 中，计算式证明以关键字 **calc** 开始，并具有如下语法：

```
calc
  <expr>_0 'op_1' <expr>_1 ':= ' <proof>_1
  ' _ ' 'op_2' <expr>_2 ':= ' <proof>_2
  ⋮
  ' _ ' 'op_n' <expr>_n ':= ' <proof>_n
```

注意，**calc** 中的各个关系具有相同的缩进。每个 `<proof>_i` 都是 `<expr>_{i-1}` `op_i <expr>_i` 的证明。

我们也可以在第一个关系中（紧接 `<expr>_0` 之后）使用 `_`，这有助于对齐一串关系/证明对：

```
calc <expr>_0
  ' _ ' 'op_1' <expr>_1 ':= ' <proof>_1
  ' _ ' 'op_2' <expr>_2 ':= ' <proof>_2
  ⋮
  ' _ ' 'op_n' <expr>_n ':= ' <proof>_n
```

下面是一个例子：

```
variable (a b c d e : Nat)
```

```
theorem T
  (h1 : a = b)
  (h2 : b = c + 1)
  (h3 : c = d)
  (h4 : e = 1 + d) :
  a = e :=
calc
  a = b      := h1
  _ = c + 1  := h2
  _ = d + 1  := congrArg Nat.succ h3
  _ = 1 + d  := Nat.add_comm d 1
  _ = e      := Eq.symm h4
```

这种证明书写风格在与 **simp** 和 **rw** 策略结合使用时最为有效：这些策略将在下一章更详细地讨论。例如，使用 **rw** 进行重写，上面的证明可以写成如下形式：

```
theorem T
  (h1 : a = b)
  (h2 : b = c + 1)
  (h3 : c = d)
  (h4 : e = 1 + d) :
  a = e :=
calc
  a = b      := by rw [h1]
  _ = c + 1  := by rw [h2]
  _ = d + 1  := by rw [h3]
  _ = 1 + d  := by rw [Nat.add_comm]
  _ = e      := by rw [h4]
```

本质上，**rw** 策略使用给定的等式（它可以是假设、定理名或复杂项）来“重写”目标。如果这样做把目标化为恒等式 $t = t$ ，该策略就应用自反性来证明它。

重写可以依次应用，因此上面的证明可以缩短为：

```
theorem T
  (h1 : a = b)
  (h2 : b = c + 1)
  (h3 : c = d)
  (h4 : e = 1 + d) :
  a = e :=
calc
  a = d + 1 := by rw [h1, h2, h3]
  _ = 1 + d := by rw [Nat.add_comm]
  _ = e     := by rw [h4]
```

甚至可以写成：

```
theorem T
  (h1 : a = b)
  (h2 : b = c + 1)
  (h3 : c = d)
  (h4 : e = 1 + d) :
  a = e :=
by rw [h1, h2, h3, Nat.add_comm, h4]
```

相比之下，**simp** 策略会反复应用给定恒等式，在项中任何适用位置、以任意顺序重写目标。它还会使用先前向系统声明的其他规则，并明智地应用交换律以避免循环。因此，我们也可以如下证明该定理：

```
theorem T
  (h1 : a = b)
  (h2 : b = c + 1)
```

```

    (h3 : c = d)
    (h4 : e = 1 + d) :
    a = e :=
  by simp [h1, h2, h3, Nat.add_comm, h4]

```

我们将在下一章讨论 **rw** 和 **simp** 的各种变体。

calc 命令可以为任何支持某种传递性的关系进行配置。它甚至可以组合不同的关系。

```

variable (a b c d : Nat)
example (h1 : a = b) (h2 : b ≤ c) (h3 : c + 1 < d) : a < d :=
  calc
    a = b      := h1
    _ < b + 1 := Nat.lt_succ_self b
    _ ≤ c + 1 := Nat.succ_le_succ h2
    _ < d      := h3

```

你可以通过添加 **Trans** 类型类的新实例，向 **calc** “教授”新的传递性定理。类型类稍后才会介绍，但下面的小例子展示了如何使用新的 **Trans** 实例扩展 **calc** 记法。

```

def divides (x y : Nat) : Prop :=
  ∃ k, k*x = y

def divides_trans (h1 : divides x y) (h2 : divides y z) :
  ⇨ divides x z :=
  let ⟨k1, d1⟩ := h1
  let ⟨k2, d2⟩ := h2
  {k1 * k2, by rw [Nat.mul_comm k1 k2, Nat.mul_assoc, d1, d2]}

def divides_mul (x : Nat) (k : Nat) : divides x (k*x) :=
  {k, rfl}

instance : Trans divides divides divides where
  trans := divides_trans

example (h1 : divides x y) (h2 : y = z) : divides x (2*z) :=
  calc
    divides x y      := h1
    _ = z            := h2
    divides _ (2*z) := divides_mul ..

infix:50 " | " => divides

example (h1 : divides x y) (h2 : y = z) : divides x (2*z) :=
  calc
    x | y      := h1
    _ = z      := h2
    _ | 2*z    := divides_mul ..

```

上面的例子也表明，即使你的关系没有中缀记法，也可以使用 **calc**。Lean 已经包含整除性的标准 Unicode 记法（使用 $\mid$ ，可输入为 `\dvd` 或 `\mid`），所以上例使用普通竖线以避免冲突。在实践中，这不是一个好主意，因为它可能与 **match ... with** 表达式中使用的 ASCII `|` 混淆。

借助 **calc**，我们可以把上一节中的证明写得更自然、更清晰。

```
variable (x y : Nat)
```

```
example : (x + y) * (x + y) = x * x + y * x + x * y + y * y :=
  calc
    (x + y) * (x + y) = (x + y) * x + (x + y) * y :=
      by rw [Nat.mul_add]
    _ = x * x + y * x + (x + y) * y :=
      by rw [Nat.add_mul]
    _ = x * x + y * x + (x * y + y * y) :=
      by rw [Nat.add_mul]
    _ = x * x + y * x + x * y + y * y :=
      by rw [←Nat.add_assoc]
```

这里值得考虑另一种 **calc** 记法。当第一个表达式占用这么多空间时，在第一个关系中使用 `_` 会自然地对齐所有关系：

```
variable (x y : Nat)
```

```
example : (x + y) * (x + y) = x * x + y * x + x * y + y * y :=
  calc (x + y) * (x + y)
    _ = (x + y) * x + (x + y) * y :=
      by rw [Nat.mul_add]
    _ = x * x + y * x + (x + y) * y :=
      by rw [Nat.add_mul]
    _ = x * x + y * x + (x * y + y * y) :=
      by rw [Nat.add_mul]
    _ = x * x + y * x + x * y + y * y :=
      by rw [←Nat.add_assoc]
```

这里 `Nat.add_assoc` 前的左箭头告诉重写器按相反方向使用该恒等式。（可以用 `\l` 输入它，或使用 ASCII 等价形式 `<-`。）如果追求简洁，**rw** 和 **simp** 都可以独立完成这项工作：

```
variable (x y : Nat)
```

```
example : (x + y) * (x + y) = x * x + y * x + x * y + y * y :=
  ↪ by
    rw [Nat.mul_add, Nat.add_mul, Nat.add_mul, ←Nat.add_assoc]
```

```
example : (x + y) * (x + y) = x * x + y * x + x * y + y * y :=
  ↪ by
    simp [Nat.mul_add, Nat.add_mul, Nat.add_assoc]
```

4.4 存在量词

最后，考虑存在量词，它可以写作 **exists** $x : \alpha, p\ x$ ，也可以写作 $\exists x : \alpha, p\ x$ 。这两种写法实际上都是 Lean 库中定义的较长表达式 **Exists** (**fun** $x : \alpha \Rightarrow p\ x$) 的便捷记法缩写。

如你现在应当预料到的那样，库同时包含引入规则和消去规则。引入规则很直接：要证明 $\exists x : \alpha, p\ x$ ，只需给出一个合适的项 t 以及 $p\ t$ 的证明。下面是一些例子：

```
example :  $\exists x : \text{Nat}, x > 0 :=$ 
  have  $h : 1 > 0 := \text{Nat.zero\_lt\_succ } 0$ 
  Exists.intro 1  $h$ 
```

```
example ( $x : \text{Nat}$ ) ( $h : x > 0$ ) :  $\exists y, y < x :=$ 
  Exists.intro 0  $h$ 
```

```
example ( $x\ y\ z : \text{Nat}$ ) ( $hxy : x < y$ ) ( $hyz : y < z$ ) :  $\exists w, x < w$ 
 $\hookrightarrow \wedge w < z :=$ 
  Exists.intro  $y$  (And.intro  $hxy\ hyz$ )
```

```
#check @Exists.intro -- @Exists.intro :  $\forall \{\alpha : \text{Sort } u_1\} \{p :$ 
 $\hookrightarrow \alpha \rightarrow \text{Prop}\} (w : \alpha), p\ w \rightarrow \text{Exists } p$ 
| @Exists.intro :  $\forall \{\alpha : \text{Sort } u_1\} \{p : \alpha \rightarrow \text{Prop}\} (w : \alpha), p\ w$ 
 $\hookrightarrow \rightarrow \text{Exists } p$ 
```

当类型可由上下文明确时，可以用匿名构造子记法 (t, h) 表示 **Exists.intro** $t\ h$ 。

```
example :  $\exists x : \text{Nat}, x > 0 :=$ 
  have  $h : 1 > 0 := \text{Nat.zero\_lt\_succ } 0$ 
  (1,  $h$ )
```

```
example ( $x : \text{Nat}$ ) ( $h : x > 0$ ) :  $\exists y, y < x :=$ 
  (0,  $h$ )
```

```
example ( $x\ y\ z : \text{Nat}$ ) ( $hxy : x < y$ ) ( $hyz : y < z$ ) :  $\exists w, x < w$ 
 $\hookrightarrow \wedge w < z :=$ 
  ( $y, hxy, hyz$ )
```

注意，**Exists.intro** 有隐式参数：Lean 必须在结论 $\exists x, p\ x$ 中推断谓词 $p : \alpha \rightarrow \text{Prop}$ 。这并非平凡之事。例如，如果有 $hg : g\ 0\ 0 = 0$ 并写下 **Exists.intro** 0 hg ，那么谓词 p 有许多可能取值，分别对应定理 $\exists x, g\ x\ x = x$ 、 $\exists x, g\ x\ x = 0$ 、 $\exists x, g\ x\ 0 = x$ 等。Lean 利用上下文来推断哪一个恰当的。下面的例子展示了这一点，其中我们把选项 **pp.explicit** 设为 **true**，请求 Lean 的美化打印机显示隐式参数。

```
variable ( $g : \text{Nat} \rightarrow \text{Nat} \rightarrow \text{Nat}$ )
```

```

theorem gex1 (hg : g 0 0 = 0) :  $\exists x, g\ x\ x = x := \langle 0, hg \rangle$ 
theorem gex2 (hg : g 0 0 = 0) :  $\exists x, g\ x\ 0 = x := \langle 0, hg \rangle$ 
theorem gex3 (hg : g 0 0 = 0) :  $\exists x, g\ 0\ 0 = x := \langle 0, hg \rangle$ 
theorem gex4 (hg : g 0 0 = 0) :  $\exists x, g\ x\ x = 0 := \langle 0, hg \rangle$ 

```

```

set_option pp.explicit true -- display implicit arguments

```

```

#print gex1

```

```

theorem gex1 :  $\forall (g : \text{Nat} \rightarrow \text{Nat} \rightarrow \text{Nat}),$ 
  @Eq Nat
  (g (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit
     $\hookrightarrow$  0)))
    (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit
       $\hookrightarrow$  0))))
    (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit
       $\hookrightarrow$  0)))  $\rightarrow$ 
    @Exists Nat fun x => @Eq Nat (g x x) x :=
fun g hg => @Exists.intro Nat (fun x => @Eq Nat (g x x) x)
   $\hookrightarrow$  (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0)))
   $\hookrightarrow$  hg

```

```

#print gex2

```

```

theorem gex2 :  $\forall (g : \text{Nat} \rightarrow \text{Nat} \rightarrow \text{Nat}),$ 
  @Eq Nat
  (g (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit
     $\hookrightarrow$  0)))
    (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit
       $\hookrightarrow$  0))))
    (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit
       $\hookrightarrow$  0)))  $\rightarrow$ 
    @Exists Nat fun x => @Eq Nat (g x (@OfNat.ofNat Nat
       $\hookrightarrow$  (nat_lit 0) (instOfNatNat (nat_lit 0)))) x :=
fun g hg =>
  @Exists.intro Nat (fun x => @Eq Nat (g x (@OfNat.ofNat Nat
     $\hookrightarrow$  (nat_lit 0) (instOfNatNat (nat_lit 0)))) x)
    (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0)))
     $\hookrightarrow$  hg

```

```

#print gex3

```

```

theorem gex3 :  $\forall (g : \text{Nat} \rightarrow \text{Nat} \rightarrow \text{Nat}),$ 
  @Eq Nat
  (g (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit
     $\hookrightarrow$  0)))
    (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit
       $\hookrightarrow$  0))))
    (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit
       $\hookrightarrow$  0)))  $\rightarrow$ 
    @Exists Nat fun x =>
    @Eq Nat

```



```

      (g (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat
        ↪ (nat_lit 0)))
        (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat
        ↪ (nat_lit 0))))
    x :=
  fun g hg =>
    @Exists.intro Nat
      (fun x =>
        @Eq Nat
          (g (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat
            ↪ (nat_lit 0)))
            (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat
            ↪ (nat_lit 0))))
          x)
        (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0)))
        ↪ hg)
#print gex4
theorem gex4 : ∀ (g : Nat → Nat → Nat),
  @Eq Nat
    (g (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit
    ↪ 0)))
    (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit
    ↪ 0))))
    (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit
    ↪ 0))) →
  @Exists Nat fun x => @Eq Nat (g x x) (@OfNat.ofNat Nat
    ↪ (nat_lit 0) (instOfNatNat (nat_lit 0))) :=
  fun g hg =>
    @Exists.intro Nat (fun x => @Eq Nat (g x x) (@OfNat.ofNat
    ↪ Nat (nat_lit 0) (instOfNatNat (nat_lit 0))))
    (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0)))
    ↪ hg

```

可以把 `Exists.intro` 看作一种信息隐藏操作，因为它隐藏了断言主体的见证。存在量词的消去规则 `Exists.elim` 执行相反操作。它允许我们通过证明对于任意值 w ， q 可由 $p\ w$ 推出，从而由 $\exists x : \alpha, p\ x$ 证明命题 q 。粗略地说，既然我们知道存在某个满足 $p\ x$ 的 x ，就可以给它取一个名字，比如 w 。如果 q 不提到 w ，那么证明 q 由 $p\ w$ 推出，等价于证明 q 由任意这种 x 的存在推出。下面是一个例子：

```
variable (α : Type) (p q : α → Prop)
```

```

example (h : ∃ x, p x ∧ q x) : ∃ x, q x ∧ p x :=
  Exists.elim h
  (fun w =>
    fun hw : p w ∧ q w =>
      show ∃ x, q x ∧ p x from ⟨w, hw.right, hw.left⟩)

```

把存在量词消去规则与析取消去规则比较会有所帮助：断言 $\exists x : \alpha, p x$ 可以看作命题 $p a$ 的“大析取”，其中 a 遍历 α 的所有元素。注意，匿名构造子记法 $\langle w, hw.right, hw.left \rangle$ 缩写了一个嵌套构造子应用；我们同样可以写成 $\langle w, \langle hw.right, hw.left \rangle \rangle$ 。注意，存在命题与依值类型一节中描述的 **Sigma** 类型非常相似。区别在于，存在命题是命题，而 **Sigma** 类型是类型。除此之外，它们非常相似。给定谓词 $p : \alpha \rightarrow \text{Prop}$ 和类型族 $\beta : \alpha \rightarrow \text{Type}$ ，对于项 $a : \alpha$ 以及 $h : p a$ 、 $h' : \beta a$ ，项 `Exists.intro a h` 的类型是 $(\exists x : \alpha, p x) : \text{Prop}$ ，而 `Sigma.mk a h'` 的类型是 $(\Sigma x : \alpha, \beta x)$ 。 $\exists$ 与 Σ 的相似性是 Curry-Howard isomorphism 的又一个实例。

Lean 提供了利用 **match** 表达式从存在量词中消去的更便捷方式：

```
variable (α : Type) (p q : α → Prop)
```

```
example (h : ∃ x, p x ∧ q x) : ∃ x, q x ∧ p x :=
  match h with
  | ⟨w, hw⟩ => ⟨w, hw.right, hw.left⟩
```

match 表达式是 Lean 函数定义系统的一部分，该系统为定义复杂函数提供了便捷而富有表达力的方式。再次，是 Curry-Howard isomorphism 使我们也能借用这一机制来书写证明。**match** 语句把存在断言“解构”为组成部分 w 和 hw ，随后即可在语句主体中使用它们来证明命题。为了更清楚，我们可以标注匹配中使用的类型：

```
example (h : ∃ x, p x ∧ q x) : ∃ x, q x ∧ p x :=
  match h with
  | ⟨(w : α), (hw : p w ∧ q w)⟩ => ⟨w, hw.right, hw.left⟩
```

我们甚至可以用 **match** 语句同时分解合取：

```
example (h : ∃ x, p x ∧ q x) : ∃ x, q x ∧ p x :=
  match h with
  | ⟨w, hpw, hqw⟩ => ⟨w, hqw, hpw⟩
```

Lean 还提供了模式匹配式的 **let** 表达式：

```
example (h : ∃ x, p x ∧ q x) : ∃ x, q x ∧ p x :=
  let ⟨w, hpw, hqw⟩ := h
  ⟨w, hqw, hpw⟩
```

这本质上只是上述 **match** 构造的另一种记法。Lean 甚至允许我们在 **fun** 表达式中使用隐式的 **match**：

```
example : (∃ x, p x ∧ q x) → ∃ x, q x ∧ p x :=
  fun ⟨w, hpw, hqw⟩ => ⟨w, hqw, hpw⟩
```

我们将在 chapter 8 (p. 139) 中看到，所有这些变体都是更一般的模式匹配构造的实例。

在下面的例子中，我们把 `IsEven a` 定义为 $\exists b, a = 2 * b$ ，然后证明两个偶数之和仍为偶数。

```

def IsEven (a : Nat) :=  $\exists b, a = 2 * b$ 

theorem even_plus_even (h1 : IsEven a) (h2 : IsEven b) :
  IsEven (a + b) :=
  Exists.elim h1 (fun w1 (hw1 : a = 2 * w1) =>
    Exists.elim h2 (fun w2 (hw2 : b = 2 * w2) =>
      Exists.intro (w1 + w2)
        (calc a + b
          _ = 2 * w1 + 2 * w2 := by rw [hw1, hw2]
          _ = 2 * (w1 + w2) := by rw [Nat.mul_add])))

```

利用本章介绍的各种工具——`match` 语句、匿名构造子以及 `rewrite` 策略——我们可以把这个证明简洁地写成：

```

theorem even_plus_even (h1 : IsEven a) (h2 : IsEven b) :
  IsEven (a + b) :=
  match h1, h2 with
  | ⟨w1, hw1⟩, ⟨w2, hw2⟩ =>
    (w1 + w2, by rw [hw1, hw2, Nat.mul_add])

```

正如构造性的“或”强于经典的“或”一样，构造性的“存在”也强于经典的“存在”。例如，下面的蕴含需要经典推理，因为从构造主义观点看，知道并非每个 x 都满足 $\neg p$ ，并不等同于拥有某个满足 p 的具体 x 。

```
open Classical
```

```

variable (p :  $\alpha \rightarrow \text{Prop}$ )

example (h :  $\neg \forall x, \neg p\ x$ ) :  $\exists x, p\ x$  :=
  byContradiction
  (fun h1 :  $\neg \exists x, p\ x$  =>
    have h2 :  $\forall x, \neg p\ x$  :=
      fun x =>
        fun h3 : p x =>
          have h4 :  $\exists x, p\ x$  := ⟨x, h3⟩
          show False from h1 h4
    show False from h h2)

```

下面是一些涉及存在量词的常见恒等式。在后面的练习中，我们鼓励你尽可能多地证明它们。我们也把判断哪些命题是非构造性的、因而需要某种经典推理这一任务留给你。

```
open Classical
```

```

variable ( $\alpha$  : Type) (p q :  $\alpha \rightarrow \text{Prop}$ )
variable (r : Prop)

```

```
example : ( $\exists x : \alpha, r$ )  $\rightarrow r$  := sorry
```

```
example (a :  $\alpha$ ) :  $r \rightarrow (\exists x : \alpha, r)$  := sorry
```

```
example : ( $\exists x, p\ x \wedge r$ )  $\leftrightarrow (\exists x, p\ x) \wedge r$  := sorry
```

example : $(\exists x, p x \vee q x) \leftrightarrow (\exists x, p x) \vee (\exists x, q x) := \text{sorry}$

example : $(\forall x, p x) \leftrightarrow \neg (\exists x, \neg p x) := \text{sorry}$

example : $(\exists x, p x) \leftrightarrow \neg (\forall x, \neg p x) := \text{sorry}$

example : $(\neg \exists x, p x) \leftrightarrow (\forall x, \neg p x) := \text{sorry}$

example : $(\neg \forall x, p x) \leftrightarrow (\exists x, \neg p x) := \text{sorry}$

example : $(\forall x, p x \rightarrow r) \leftrightarrow (\exists x, p x) \rightarrow r := \text{sorry}$

example $(a : \alpha) : (\exists x, p x \rightarrow r) \leftrightarrow (\forall x, p x) \rightarrow r := \text{sorry}$

example $(a : \alpha) : (\exists x, r \rightarrow p x) \leftrightarrow (r \rightarrow \exists x, p x) := \text{sorry}$

注意，第二个例子和最后两个例子需要假设类型 α 至少有一个元素 a 。

下面给出其中两个较困难命题的解答：

open Classical

variable $(\alpha : \text{Type}) (p q : \alpha \rightarrow \text{Prop})$

variable $(a : \alpha)$

variable $(r : \text{Prop})$

example : $(\exists x, p x \vee q x) \leftrightarrow (\exists x, p x) \vee (\exists x, q x) :=$

Iff.intro

(**fun** $\langle a, (h1 : p a \vee q a) \rangle \Rightarrow$

Or.elim h1

(**fun** $hpa : p a \Rightarrow \text{Or.inl } \langle a, hpa \rangle$)

(**fun** $hqa : q a \Rightarrow \text{Or.inr } \langle a, hqa \rangle$))

(**fun** $h : (\exists x, p x) \vee (\exists x, q x) \Rightarrow$

Or.elim h

(**fun** $\langle a, hpa \rangle \Rightarrow \langle a, (\text{Or.inl } hpa) \rangle$)

(**fun** $\langle a, hqa \rangle \Rightarrow \langle a, (\text{Or.inr } hqa) \rangle$))

example : $(\exists x, p x \rightarrow r) \leftrightarrow (\forall x, p x) \rightarrow r :=$

Iff.intro

(**fun** $\langle b, (hb : p b \rightarrow r) \rangle \Rightarrow$

fun $h2 : \forall x, p x \Rightarrow$

show r **from** $hb (h2 b)$)

(**fun** $h1 : (\forall x, p x) \rightarrow r \Rightarrow$

show $\exists x, p x \rightarrow r$ **from**

byCases

(**fun** $hap : \forall x, p x \Rightarrow \langle a, \lambda h' \Rightarrow h1 hap \rangle$)

(**fun** $hnap : \neg \forall x, p x \Rightarrow$

byContradiction

(**fun** $hnex : \neg \exists x, p x \rightarrow r \Rightarrow$

have $hap : \forall x, p x :=$

fun $x \Rightarrow$

byContradiction

(**fun** $hnp : \neg p x \Rightarrow$

```

      have hex :  $\exists x, p\ x \rightarrow r := \langle x, (\text{fun } hp \Rightarrow$ 
         $\hookrightarrow \text{absurd } hp\ hnp))$ 
      show False from hnex hex)
show False from hnap hap)))

```

4.5 证明语言补充

我们已经看到，**fun**、**have** 和 **show** 等关键字使我们能够书写形式化证明项，并使其反映非形式数学证明的结构。本节讨论证明语言中一些常用且方便的附加特性。

首先，可以使用匿名 **have** 表达式引入辅助目标，而无需为其命名。我们可以用关键字 **this** 引用最近以这种方式引入的表达式：

```

variable (f : Nat → Nat)
variable (h :  $\forall x : \text{Nat}, f\ x \leq f\ (x + 1)$ )

example :  $f\ 0 \leq f\ 3 :=$ 
  have :  $f\ 0 \leq f\ 1 := h\ 0$ 
  have :  $f\ 0 \leq f\ 2 := \text{Nat.le\_trans } \text{this } (h\ 1)$ 
  show  $f\ 0 \leq f\ 3$  from Nat.le_trans this (h 2)

```

证明常常从一个事实推进到下一个事实，因此这可以有效减少大量标签带来的杂乱。

当目标可以推断出来时，我们也可以写 **by assumption**，让 Lean 填入证明：

```

example :  $f\ 0 \leq f\ 3 :=$ 
  have :  $f\ 0 \leq f\ 1 := h\ 0$ 
  have :  $f\ 0 \leq f\ 2 := \text{Nat.le\_trans } (\text{by assumption})\ (h\ 1)$ 
  show  $f\ 0 \leq f\ 3$  from Nat.le_trans (by assumption) (h 2)

```

这会告诉 Lean 使用 **assumption** 策略：该策略会在局部上下文中寻找合适的假设来证明目标。下一章将进一步学习 **assumption** 策略。

我们也可以写 $\langle p \rangle$ ，请求 Lean 填入证明；这里 p 是我们希望 Lean 在上下文中找到其证明的命题。可以分别用 $\backslash f<$ 和 $\backslash f>$ 输入这些角引号。字母“f”代表“French”，因为这些 Unicode 符号也可用作法语引号。事实上，该记法在 Lean 中定义如下：

```

notation "<" p ">" => show p by assumption

```

这种方法比使用 **by assumption** 更稳健，因为需要推断的假设类型被显式给出。它也使证明更易读。下面是一个较详细的例子：

```

variable (f : Nat → Nat)
variable (h :  $\forall x : \text{Nat}, f\ x \leq f\ (x + 1)$ )

```

```

example :  $f\ 0 \geq f\ 1 \rightarrow f\ 1 \geq f\ 2 \rightarrow f\ 0 = f\ 2$  :=
  fun _ :  $f\ 0 \geq f\ 1 \Rightarrow$ 
    fun _ :  $f\ 1 \geq f\ 2 \Rightarrow$ 
      have :  $f\ 0 \geq f\ 2$  :=  $\text{Nat.le\_trans } \langle f\ 1 \geq f\ 2 \rangle \langle f\ 0 \geq f\ 1 \rangle$ 
      have :  $f\ 0 \leq f\ 2$  :=  $\text{Nat.le\_trans } (h\ 0) (h\ 1)$ 
      show  $f\ 0 = f\ 2$  from  $\text{Nat.le\_antisymm } \text{this } \langle f\ 0 \geq f\ 2 \rangle$ 

```

请记住，可以用这种法语引号来引用上下文中的任何对象，而不仅限于匿名引入的对象。它的使用也不限于命题，尽管用它来引用数据多少有些奇怪：

```

example (n : Nat) : Nat :=  $\langle \text{Nat} \rangle$ 

```

稍后，我们将展示如何使用 Lean 的宏系统扩展证明语言。

4.6 练习

1. 证明下列等价：

```

variable ( $\alpha$  : Type) ( $p\ q$  :  $\alpha \rightarrow \text{Prop}$ )

```

```

example :  $(\forall x, p\ x \wedge q\ x) \leftrightarrow (\forall x, p\ x) \wedge (\forall x, q\ x)$  :=
   $\hookrightarrow$  sorry

```

```

example :  $(\forall x, p\ x \rightarrow q\ x) \rightarrow (\forall x, p\ x) \rightarrow (\forall x, q\ x)$  :=
   $\hookrightarrow$  sorry

```

```

example :  $(\forall x, p\ x) \vee (\forall x, q\ x) \rightarrow \forall x, p\ x \vee q\ x$  :=
   $\hookrightarrow$  sorry

```

你还应当尝试理解为什么最后一个例子的逆蕴含不可导出。

2. 当公式的某个组成部分不依赖于被量化变量时，通常可以把它移到全称量词之外。尝试证明下列命题（其中第二个命题的一个方向需要经典逻辑）：

```

variable ( $\alpha$  : Type) ( $p\ q$  :  $\alpha \rightarrow \text{Prop}$ )

```

```

variable (r : Prop)

```

```

example :  $\alpha \rightarrow ((\forall x : \alpha, r) \leftrightarrow r)$  := sorry

```

```

example :  $(\forall x, p\ x \vee r) \leftrightarrow (\forall x, p\ x) \vee r$  := sorry

```

```

example :  $(\forall x, r \rightarrow p\ x) \leftrightarrow (r \rightarrow \forall x, p\ x)$  := sorry

```

3. 考虑“理发师悖论”：某个城镇中有一位（男性）理发师，他给且只给那些不给自己刮胡子的人刮胡子。证明这会导致矛盾：

```

variable (men : Type) (barber : men)
variable (shaves : men → men → Prop)

example (h : ∀ x : men, shaves barber x ↔ ¬ shaves x x) :
  → False :=
  sorry

```

4. 记住，在没有任何参数的情况下，类型为 **Prop** 的表达式就是一个断言。补全下面的 **prime** 和 **Fermat_prime** 定义，并构造给出的每个断言。例如，可以通过断言对每个自然数 n ，都存在一个大于 n 的素数，来表达“存在无限多个素数”。Goldbach 弱猜想断言，每个大于 5 的奇数都是三个素数之和。如有必要，请查阅 Fermat 素数或其他陈述的定义。

```

def even (n : Nat) : Prop := sorry

def prime (n : Nat) : Prop := sorry

def infinitely_many_primes : Prop := sorry

def Fermat_prime (n : Nat) : Prop := sorry

def infinitely_many_Fermat_primes : Prop := sorry

def goldbach_conjecture : Prop := sorry

def Goldbach's_weak_conjecture : Prop := sorry

def Fermat's_last_theorem : Prop := sorry

```

5. 尽可能多地证明“存在量词”一节中列出的恒等式。

5 策略

本章介绍一种构造证明的替代方法：使用策略。证明项是数学证明的表示；策略则是描述如何构造这种证明的命令或指令。非形式地说，你可能会这样开始一个数学证明：“为了证明正向方向，展开定义，应用前面的引理，然后化简。”正如这些话是在告诉读者如何找到相应证明，策略也是告诉 **Lean** 如何构造证明项的指令。策略天然支持一种增量式的证明书写方式：将证明分解，并一次处理一个目标。

我们把由一系列策略组成的证明称为“策略式”证明，以区别于此前见过的证明项写法，后者称为“项式”证明。每种风格各有优缺点。例如，策略式证明可能更难阅读，因为读者需要预测或猜测每条指令的结果。但它们也可能更短、更容易书写。此外，策略提供了使用 **Lean** 自动化能力的入口，因为自动化过程本身也是策略。

5.1 进入策略模式

从概念上说，陈述一个定理或引入一个 **have** 语句都会创建一个目标，即构造具有预期类型的项这一目标。例如，下面的代码在包含常量 $p, q : \text{Prop}$ 、 $hp : p$ 和 $hq : q$ 的上下文中，创建了构造类型为 $p \wedge q \wedge p$ 的项这一目标：

```
theorem test (p q : Prop) (hp : p) (hq : q) : p ∧ q ∧ p := by  
  sorry
```

可以把这个目标写作如下形式：

```
p : Prop  
q : Prop  
hp : p  
hq : q  
⊢ p ∧ q ∧ p
```

事实上，如果把上例中的“**sorry**”替换为下划线，**Lean** 会报告说尚未解决的正是这个目标。

通常，你通过写出一个显式项来满足这样的目标。但在任何需要项的位置，**Lean** 都允许我们改为插入一个 **by <tactics>** 块，其中 **<tactics>** 是由分号或换行分隔的一系列命令。可以用这种方式证明上面的定理：

```
theorem test (p q : Prop) (hp : p) (hq : q) : p ∧ q ∧ p :=  
  by apply And.intro  
    exact hp
```

```

apply And.intro
exact hq
exact hp

```

我们经常把 **by** 关键字放在前一行，并把上面的例子写成：

```

theorem test (p q : Prop) (hp : p) (hq : q) : p ∧ q ∧ p := by
apply And.intro
exact hp
apply And.intro
exact hq
exact hp

```

apply 策略应用一个表达式，该表达式被看作表示一个带有零个或多个参数的函数。它把当前目标中的结论与该表达式进行合一，并为剩余参数创建新目标，前提是后续参数不依赖于这些参数。在上面的例子中，命令 **apply** And.intro 产生两个子目标：

case left	case right
p : Prop	p : Prop
q : Prop	q : Prop
hp : p	hp : p
hq : q	hq : q
⊢ p	⊢ q ∧ p

第一个目标由命令 **exact** hp 解决。**exact** 命令只是 **apply** 的一个变体，它表明给定表达式应当精确填充目标。在策略证明中使用它是一种良好做法，因为它失败时意味着出现了问题。它也比 **apply** 更稳健，因为精化器在处理被应用的表达式时，会考虑由目标给出的预期类型。不过在此例中，**apply** 同样可以工作。

可以用 **#print** 命令查看得到的证明项：

```

#print test
theorem test : ∀ (p q : Prop), p → q → p ∧ q ∧ p :=
fun p q hp hq => ⟨hp, ⟨hq, hp⟩⟩

```

可以增量地书写策略脚本。在 VS Code 中，可以按 **Ctrl Shift Enter** 打开显示消息的窗口；当光标位于策略块中时，该窗口会显示当前目标。如果证明尚未完成，标记 **by** 会带有红色波浪线，错误消息中会包含剩余目标。

策略命令可以接受复合表达式，而不只是单个标识符。下面是前一个证明的较短版本：

```

theorem test (p q : Prop) (hp : p) (hq : q) : p ∧ q ∧ p := by
apply And.intro hp
exact And.intro hq hp

```

不出所料，它产生完全相同的证明项：

```

#print test
theorem test : ∀ (p q : Prop), p → q → p ∧ q ∧ p :=
fun p q hp hq => ⟨hp, ⟨hq, hp⟩⟩

```

多个策略应用可以用分号连接，写在同一行中。

```
theorem test (p q : Prop) (hp : p) (hq : q) : p ∧ q ∧ p := by
  apply And.intro hp; exact And.intro hq hp
```

可能产生多个子目标的策略通常会给它们加上标签。例如，策略 **apply** **And.intro** 把第一个子目标标记为 **left**，把第二个标记为 **right**。对于 **apply** 策略而言，这些标签是从 **And.intro** 声明中所用参数名推断出来的。可以使用记法 **case** <tag> => <tactics> 来组织策略。下面是本章第一个策略证明的结构化版本。

```
theorem test (p q : Prop) (hp : p) (hq : q) : p ∧ q ∧ p := by
  apply And.intro
  case left => exact hp
  case right =>
    apply And.intro
    case left => exact hq
    case right => exact hp
```

可以使用 **case** 记法，在 **left** 之前先解决子目标 **right**：

```
theorem test (p q : Prop) (hp : p) (hq : q) : p ∧ q ∧ p := by
  apply And.intro
  case right =>
    apply And.intro
    case left => exact hq
    case right => exact hp
  case left => exact hp
```

注意，在 **case** 块内部，Lean 会隐藏其他目标。在 **case** left => 之后，证明状态为：

```
p : Prop
q : Prop
hp : p
hq : q
⊢ q
```

我们说 **case** 正在“聚焦”所选目标。此外，如果在 **case** 块结束时所选目标尚未完全解决，Lean 会报告错误。

对于简单子目标，使用标签选择子目标也许不值得，但你可能仍希望组织证明结构。Lean 还提供“项目符号”记法 . <tactics>（或 · <tactics>）来组织证明：

```
theorem test (p q : Prop) (hp : p) (hq : q) : p ∧ q ∧ p := by
  apply And.intro
  . exact hp
  . apply And.intro
    . exact hq
    . exact hp
```

5.2 基本策略

除 **apply** 和 **exact** 之外，另一个有用的策略是 **intro**，它用于引入假设。下面是前一章证明过的命题逻辑恒等式，现在用策略来证明。

```
example (p q r : Prop) : p ∧ (q ∨ r) ↔ (p ∧ q) ∨ (p ∧ r) := by
  apply Iff.intro
  . intro h
    apply Or.elim (And.right h)
    . intro hq
      apply Or.inl
      apply And.intro
      . exact And.left h
      . exact hq
    . intro hr
      apply Or.inr
      apply And.intro
      . exact And.left h
      . exact hr
  . intro h
    apply Or.elim h
    . intro hpq
      apply And.intro
      . exact And.left hpq
      . apply Or.inl
        exact And.right hpq
    . intro hpr
      apply And.intro
      . exact And.left hpr
      . apply Or.inr
        exact And.right hpr
```

更一般地，**intro** 命令可用于引入任意类型的变量：

```
example (α : Type) : α → α := by
  intro a
  exact a
```

```
example (α : Type) : ∀ x : α, x = x := by
  intro x
  exact Eq.refl x
```

可以用它引入多个变量：

```
example : ∀ a b c : Nat, a = b → a = c → c = b := by
  intro a b c h1 h2
  exact Eq.trans (Eq.symm h2) h1
```

正如 **apply** 策略是交互式构造函数应用的命令，**intro** 策略则是交互式构造函数抽象的命令（即形如 **fun x => e** 的项）。与 λ 抽象记法一样，**intro** 策略允许我们使用隐式的 **match**。

```

example (p q :  $\alpha \rightarrow \text{Prop}$ ) : ( $\exists x, p\ x \wedge q\ x$ )  $\rightarrow \exists x, q\ x \wedge p\ x$ 
  ↪ := by
    intro ⟨w, hpw, hqw⟩
    exact ⟨w, hqw, hpw⟩

```

也可以像在 **match** 表达式中那样提供多个分支。

```

example (p q :  $\alpha \rightarrow \text{Prop}$ ) : ( $\exists x, p\ x \vee q\ x$ )  $\rightarrow \exists x, q\ x \vee p\ x$ 
  ↪ := by
    intro
    | ⟨w, Or.inl h⟩ => exact ⟨w, Or.inr h⟩
    | ⟨w, Or.inr h⟩ => exact ⟨w, Or.inl h⟩

```

intros 策略可以不带任何参数使用；在这种情况下，它会选择名称并尽可能地引入变量。稍后会看到一个例子。**assumption** 策略会查看当前目标上下文中的假设；如果其中有一个与结论匹配，它就应用该假设。

```
variable (x y z w : Nat)
```

```

example (h1 : x = y) (h2 : y = z) (h3 : z = w) : x = w := by
  apply Eq.trans h1
  apply Eq.trans h2
  assumption -- applied h3

```

必要时，它会对结论中的元变量进行合一：

```
variable (x y z w : Nat)
```

```

example (h1 : x = y) (h2 : y = z) (h3 : z = w) : x = w := by
  apply Eq.trans
  assumption -- solves x = ?b with h1
  apply Eq.trans
  assumption -- solves y = ?h2.b with h2
  assumption -- solves z = w with h3

```

下面的例子使用 **intros** 命令自动引入三个变量和两个假设：

```

example :  $\forall a\ b\ c : \text{Nat}, a = b \rightarrow a = c \rightarrow c = b$  := by
  intros
  apply Eq.trans
  apply Eq.symm
  assumption
  assumption

```

注意，Lean 自动生成的名称默认不可访问。这样做的动机是确保你的策略证明不依赖自动生成的名称，从而更加稳健。不过，可以使用组合子 **unhygienic** 来禁用这一限制。

```

example :  $\forall a\ b\ c : \text{Nat}, a = b \rightarrow a = c \rightarrow c = b$  := by
  ↪ unhygienic
  intros

```

```

apply Eq.trans
apply Eq.symm
exact a_2
exact a_1

```

还可以使用 **rename_i** 策略来重命名上下文中最近的不可访问名称。在下面的例子中，策略 **rename_i h1 _ h2** 重命名了上下文中最后三个假设中的两个。

```

example :  $\forall a b c d : \text{Nat}, a = b \rightarrow a = d \rightarrow a = c \rightarrow c = b := \text{by}$ 
  intros
  rename_i h1 _ h2
  apply Eq.trans
  apply Eq.symm
  exact h2
  exact h1

```

rfl 策略解决那些把自反关系应用于定义相等参数的目标。等式是自反的：

```

example (y : Nat) : (fun x : Nat => 0) y = 0 := by
  rfl

```

repeat 组合子可用于多次应用一个策略：

```

example :  $\forall a b c : \text{Nat}, a = b \rightarrow a = c \rightarrow c = b := \text{by}$ 
  intros
  apply Eq.trans
  apply Eq.symm
  repeat assumption

```

另一个有时有用的策略是 **revert**；在某种意义上，它是 **intro** 的逆操作：

```

example (x : Nat) : x = x := by
  revert x
  intro y
  rfl

```

在 **revert x** 之后，证明状态为： $\vdash \forall (x : \text{Nat}), x = x$ 在 **intro y** 之后，

证明状态为： $y : \text{Nat}$
 $\vdash y = y$

把一个假设移入目标会得到一个蕴含：

```

example (x y : Nat) (h : x = y) : y = x := by
  revert h
  intro h1
  -- goal is x y : Nat, h1 : x = y  $\vdash$  y = x
  apply Eq.symm
  assumption

```

在 **revert h** 之后，证明状态为：

```

x : Nat
y : Nat
⊢ x = y → y = x

```

在 **intro** h_1 之后，证明状态为：

```

x : Nat
y : Nat
h1 : x = y
⊢ y = x

```

但 **revert** 更聪明：它不仅会还原上下文中的某个元素，还会同时还原上下文中所有位于其后的、依赖于它的元素。例如，在上面的例子中还原 x 时，会把 h 一并带上：

```

example (x y : Nat) (h : x = y) : y = x := by
  revert x
  intros
  apply Eq.symm
  assumption

```

在 **revert** x 之后，目标为：

$$\text{y : Nat} \\ \vdash \forall (x : \text{Nat}), x = y \rightarrow y = x$$

也可以一次还原上下文中的多个元素：

```

example (x y : Nat) (h : x = y) : y = x := by
  revert x y
  intros
  apply Eq.symm
  assumption

```

在 **revert** x y 之后，目标为：

$$\vdash \forall (x y : \text{Nat}), x = y \rightarrow y = x$$

只能对局部上下文中的元素使用 **revert**，也就是局部变量或假设。但可以使用 **generalize** 策略，用一个新变量替换目标中的任意表达式：

```

example : 3 = 3 := by
  generalize 3 = x
  revert x
  intro y
  rfl

```

特别地，在 **generalize** 之后，目标为

$$\text{x : Nat} \\ \vdash x = x$$

上述记法的助记方式是：通过把 3 设为任意变量 x 来泛化目标。要小心：并非每次泛化都会保持目标的有效性。这里，**generalize** 把一个可用 **rfl** 证明的目标替换成一个不可证明的目标：

```

example : 2 + 3 = 5 := by
  generalize 3 = x
  sorry

```

在这个例子中，**sorry** 策略类似于 **sorry** 证明项。它关闭当前目标，并产生通常的警告，说明使用了 **sorry**。为了保持原目标的有效性，**generalize** 策略允许我们记录 3 已被 x 替换这一事实。只需提供一个标签，**generalize** 就会用它把该赋值存入局部上下文：

```
example : 2 + 3 = 5 := by
  generalize h : 3 = x
  rw [← h]
```

在 **generalize** $h : 3 = x$ 之后， h 是 $3 = x$ 的证明：

```
x : Nat
h : 3 = x
⊢ 2 + x = 5
```

这里，重写策略 **rw** 使用 h 再次把 x 替换为 3。**rw** 策略将在下文讨论。

5.3 更多策略

还有一些策略可用于构造和析构命题与数据。例如，当目标形如 $p \vee q$ 时，可以使用 **apply** **Or.inl** 和 **apply** **Or.inr** 等策略。反过来，**cases** 策略可用于分解析取：

```
example (p q : Prop) : p ∨ q → q ∨ p := by
  intro h
  cases h with
  | inl hp => apply Or.inr; exact hp
  | inr hq => apply Or.inl; exact hq
```

注意，其语法与 **match** 表达式中使用的语法类似。新的子目标可以按任意顺序解决：

```
example (p q : Prop) : p ∨ q → q ∨ p := by
  intro h
  cases h with
  | inr hq => apply Or.inl; exact hq
  | inl hp => apply Or.inr; exact hp
```

也可以使用不带 **with** 的（非结构化）**cases**，然后为每种情况给出策略：

```
example (p q : Prop) : p ∨ q → q ∨ p := by
  intro h
  cases h
  apply Or.inr
  assumption
  apply Or.inl
  assumption
```

当可以用同一个策略关闭多个子目标时，（非结构化的）**cases** 尤其有用：

```
example (p : Prop) : p ∨ p → p := by
  intro h
```



```

cases h
repeat assumption

```

还可以使用组合子 `tac1 <;> tac2`, 把 `tac2` 应用于策略 `tac1` 产生的每个子目标:

```

example (p : Prop) : p ∨ p → p := by
  intro h
  cases h <;> assumption

```

可以把非结构化的 `cases` 策略与 `case` 和 `.` 记法结合使用:

```

example (p q : Prop) : p ∨ q → q ∨ p := by
  intro h
  cases h
  . apply Or.inr
  assumption
  . apply Or.inl
  assumption

```

```

example (p q : Prop) : p ∨ q → q ∨ p := by
  intro h
  cases h
  case inr h =>
    apply Or.inl
    assumption
  case inl h =>
    apply Or.inr
    assumption

```

```

example (p q : Prop) : p ∨ q → q ∨ p := by
  intro h
  cases h
  case inr h =>
    apply Or.inl
    assumption
  . apply Or.inr
  assumption

```

`cases` 策略也可以用来分解合取:

```

example (p q : Prop) : p ∧ q → q ∧ p := by
  intro h
  cases h with
  | intro hp hq => constructor; exact hq; exact hp

```

在这个例子中, 应用 `cases` 策略之后只剩一个目标, 其中 $h : p \wedge q$ 被一对假设 $hp : p$ 和 $hq : q$ 取代:

```

case intro
  p : Prop
  q : Prop
  hp : p
  hq : q
  ⊢ q ∧ p

```

constructor 策略应用合取的唯一构造子 `And.intro`。

利用这些策略，上一节中的一个例子可以改写如下：

```

example (p q r : Prop) : p ∧ (q ∨ r) ↔ (p ∧ q) ∨ (p ∧ r) := by
  apply Iff.intro
  . intro h
    cases h with
    | intro hp hqr =>
      cases hqr
      . apply Or.inl; constructor <=> assumption
      . apply Or.inr; constructor <=> assumption
  . intro h
    cases h with
    | inl hpq =>
      cases hpq with
      | intro hp hq =>
        constructor; exact hp; apply Or.inl; exact hq
    | inr hpr =>
      cases hpr with
      | intro hp hr =>
        constructor; exact hp; apply Or.inr; exact hr

```

你将在 chapter 7 (p. 113) 中看到，这些策略相当一般。**cases** 策略可用于分解任意归纳定义类型的元素；**constructor** 总是应用归纳定义类型中第一个适用的构造子。例如，可以把 **cases** 和 **constructor** 用于存在量词：

```

example (p q : Nat → Prop) : (∃ x, p x) → ∃ x, p x ∨ q x := by
  intro h
  cases h with
  | intro x px => constructor; apply Or.inl; exact px

```

这里，**constructor** 策略把存在断言的第一个组成部分，即 x 的值，保留为隐式。它由一个元变量表示，稍后应被实例化。在前面的例子中，元变量的适当值由策略 **exact** px 决定，因为 px 的类型是 $p\ x$ 。如果想显式指定存在量词的见证，可以改用 **exists** 策略：

```

example (p q : Nat → Prop) : (∃ x, p x) → ∃ x, p x ∨ q x := by
  intro h
  cases h with
  | intro x px => exists x; apply Or.inl; exact px

```

下面是另一个例子：

```

example (p q : Nat → Prop) : (∃ x, p x ∧ q x) → ∃ x, q x ∧ p x
↪ := by
  intro h
  cases h with
  | intro x hpq =>
    cases hpq with
    | intro hp hq =>
      exists x

```

这些策略既可用于命题，也同样可用于数据。在下一个例子中，它们被用来定义交换乘积类型和和类型分量的函数：

```

def swap_pair : α × β → β × α := by
  intro p
  cases p
  constructor <.> assumption

def swap_sum : Sum α β → Sum β α := by
  intro p
  cases p
  . apply Sum.inr; assumption
  . apply Sum.inl; assumption

```

注意，除了我们为变量选择的名称之外，这些定义与合取和析取的类似命题证明完全相同。**cases** 策略还会对自然数进行分类讨论：

```

open Nat
example (P : Nat → Prop)
  (h₀ : P 0) (h₁ : ∀ n, P (succ n))
  (m : Nat) : P m := by
  cases m with
  | zero    => exact h₀
  | succ m' => exact h₁ m'

```

cases 策略及其配套的 **induction** 策略将在 section 7.6 (p. 128) 一节中更详细地讨论。

contradiction 策略会在当前目标的假设中搜索矛盾：

```

example (p q : Prop) : p ∧ ¬ p → q := by
  intro h
  cases h
  contradiction

```

也可以在策略块中使用 **match**。

```

example (p q r : Prop) : p ∧ (q ∨ r) ↔ (p ∧ q) ∨ (p ∧ r) := by
  apply Iff.intro
  . intro h
    match h with
    | (⟦_, Or.inl _⟧) =>
      apply Or.inl; constructor <.> assumption

```

```

| {_, Or.inr _} =>
  apply Or.inr; constructor <;> assumption
. intro h
match h with
| Or.inl {hp, hq} =>
  constructor; exact hp; apply Or.inl; exact hq
| Or.inr {hp, hr} =>
  constructor; exact hp; apply Or.inr; exact hr

```

可以把 **intro** 与 **match** “结合”起来，把前面的例子写成如下形式：

```

example (p q r : Prop) : p ∧ (q ∨ r) ↔ (p ∧ q) ∨ (p ∧ r) := by
  apply Iff.intro
. intro
  | {hp, Or.inl hq} =>
    apply Or.inl; constructor <;> assumption
  | {hp, Or.inr hr} =>
    apply Or.inr; constructor <;> assumption
. intro
  | Or.inl {hp, hq} =>
    constructor; assumption; apply Or.inl; assumption
  | Or.inr {hp, hr} =>
    constructor; assumption; apply Or.inr; assumption

```

5.4 组织策略式证明

策略常常提供一种高效构造证明的方式，但很长的指令序列可能会掩盖论证结构。本节介绍一些为策略式证明提供结构的方法，使此类证明更易读、更稳健。

Lean 的证明书写语法有一个优点：可以混合项式证明与策略式证明，并在二者之间自由切换。例如，策略 **apply** 和 **exact** 期望任意项，而这些项可以用 **have**、**show** 等书写。反过来，在书写任意 Lean 项时，也总能通过插入 **by** 块来调用策略模式。下面是一个略显玩具化的例子：

```

example (p q r : Prop) : p ∧ (q ∨ r) → (p ∧ q) ∨ (p ∧ r) := by
  intro h
  exact
    have hp : p := h.left
    have hqr : q ∨ r := h.right
    show (p ∧ q) ∨ (p ∧ r) by
      cases hqr with
      | inl hq => exact Or.inl {hp, hq}
      | inr hr => exact Or.inr {hp, hr}

```

下面是一个更自然的例子：

```

example (p q r : Prop) : p ∧ (q ∨ r) ↔ (p ∧ q) ∨ (p ∧ r) := by
  apply Iff.intro
. intro h
  cases h.right with

```

```

| inl hq => exact Or.inl ⟨h.left, hq⟩
| inr hr => exact Or.inr ⟨h.left, hr⟩
. intro h
cases h with
| inl hpq => exact ⟨hpq.left, Or.inl hpq.right⟩
| inr hpr => exact ⟨hpr.left, Or.inr hpr.right⟩

```

事实上，有一个 **show** 策略，它类似于证明项中的 **show** 表达式。它只是在保持策略模式的同时，声明即将被解决的目标类型。

```

example (p q r : Prop) : p ∧ (q ∨ r) ↔ (p ∧ q) ∨ (p ∧ r) := by
  apply Iff.intro
  . intro h
    cases h.right with
    | inl hq =>
      show (p ∧ q) ∨ (p ∧ r)
      exact Or.inl ⟨h.left, hq⟩
    | inr hr =>
      show (p ∧ q) ∨ (p ∧ r)
      exact Or.inr ⟨h.left, hr⟩
  . intro h
    cases h with
    | inl hpq =>
      show p ∧ (q ∨ r)
      exact ⟨hpq.left, Or.inl hpq.right⟩
    | inr hpr =>
      show p ∧ (q ∨ r)
      exact ⟨hpr.left, Or.inr hpr.right⟩

```

show 策略实际上可用于把目标改写为定义相等的形式：

```

example (n : Nat) : n + 1 = Nat.succ n := by
  show Nat.succ n = Nat.succ n
  rfl

```

还有一个 **have** 策略，它像书写证明项时一样引入一个新的子目标：

```

example (p q r : Prop) : p ∧ (q ∨ r) → (p ∧ q) ∨ (p ∧ r) := by
  intro ⟨hp, hqr⟩
  show (p ∧ q) ∨ (p ∧ r)
  cases hqr with
  | inl hq =>
    have hpq : p ∧ q := And.intro hp hq
    apply Or.inl
    exact hpq
  | inr hr =>
    have hpr : p ∧ r := And.intro hp hr
    apply Or.inr
    exact hpr

```

与证明项一样，可以在 **have** 策略中省略标签；此时会使用默认标签 *this*：

```
example (p q r : Prop) : p ∧ (q ∨ r) → (p ∧ q) ∨ (p ∧ r) := by
intro {hp, hqr}
show (p ∧ q) ∨ (p ∧ r)
cases hqr with
| inl hq =>
  have : p ∧ q := And.intro hp hq
  apply Or.inl
  exact this
| inr hr =>
  have : p ∧ r := And.intro hp hr
  apply Or.inr
  exact this
```

have 策略中的类型也可以省略，因此可以写 **have** hp := h.left 和 **have** hqr := h.right。事实上，使用这种记法时，甚至可以同时省略类型和标签；此时新事实会以标签 *this* 引入：

```
example (p q r : Prop) : p ∧ (q ∨ r) → (p ∧ q) ∨ (p ∧ r) := by
intro {hp, hqr}
cases hqr with
| inl hq =>
  have := And.intro hp hq
  apply Or.inl; exact this
| inr hr =>
  have := And.intro hp hr
  apply Or.inr; exact this
```

Lean 还有一个 **let** 策略，它类似于 **have** 策略，但用于引入局部定义，而不是辅助事实。它是证明项中 **let** 的策略对应物：

```
example : ∃ x, x + 2 = 8 := by
let a : Nat := 3 * 2
exists a
```

与 **have** 一样，可以通过写 **let** a := 3 * 2 来省略类型。**let** 与 **have** 的区别在于，**let** 在上下文中引入局部定义，因此该局部声明的定义可以在证明中展开。

我们已经使用 **·** 来创建嵌套策略块。在嵌套块中，Lean 聚焦于第一个目标；如果该目标在块结束时尚未完全解决，就会生成错误。这有助于标明由某个策略引入的多个子目标的各自证明。记法 **·** 对空白敏感，并依靠缩进判断策略块是否结束。另一种做法是使用花括号和分号定义策略块：

```
example (p q r : Prop) : p ∧ (q ∨ r) ↔ (p ∧ q) ∨ (p ∧ r) := by
apply Iff.intro
{ intro h;
  cases h.right;
  { show (p ∧ q) ∨ (p ∧ r);
    exact Or.inl {h.left, <q>} }
  { show (p ∧ q) ∨ (p ∧ r);
```

```

    exact Or.inr ⟨h.left, <r>⟩ } }
{ intro h;
  cases h;
  { show p ∧ (q ∨ r);
    rename_i hpq;
    exact ⟨hpq.left, Or.inl hpq.right⟩ }
  { show p ∧ (q ∨ r);
    rename_i hpr;
    exact ⟨hpr.left, Or.inr hpr.right⟩ } }

```

用缩进来组织证明很有用：每当一个策略留下多个子目标时，我们通过把剩余子目标放入块并缩进来区分它们。因此，如果把定理 `foo` 应用于单个目标产生四个子目标，证明通常应类似如下形式：

```

apply foo
. <proof of first goal>
. <proof of second goal>
. <proof of third goal>
. <proof of final goal>

```

或者：

```

apply foo
case <tag of first goal> => <proof of first goal>
case <tag of second goal> => <proof of second goal>
case <tag of third goal> => <proof of third goal>
case <tag of final goal> => <proof of final goal>

```

或者：

```

apply foo
{ <proof of first goal> }
{ <proof of second goal> }
{ <proof of third goal> }
{ <proof of final goal> }

```

5.5 策略组合子

策略组合子是由已有策略形成新策略的操作。`by` 块中已经隐含了一个顺序组合子：

```

example (p q : Prop) (hp : p) : p ∨ q :=
  by apply Or.inl; assumption

```

这里, **apply Or.inl**; **assumption** 在功能上等价于一个单一策略: 先应用 **apply Or.inl**, 再应用 **assumption**。

在 $t_1 <;> t_2$ 中, $<;>$ 运算符提供顺序操作的并行版本: 先把 t_1 应用于当前目标, 然后把 t_2 应用于产生的所有子目标:

```
example (p q : Prop) (hp : p) (hq : q) : p ∧ q :=
  by constructor <;> assumption
```

当所得目标可以用统一方式完成, 或者至少可以统一地在所有目标上取得进展时, 这尤其有用。

first | t_1 | t_2 | ... | t_n 会依次应用每个 t_i , 直到其中一个成功; 如果都不成功, 则失败:

```
example (p q : Prop) (hp : p) : p ∨ q := by
  first | apply Or.inl; assumption | apply Or.inr; assumption
```

```
example (p q : Prop) (hq : q) : p ∨ q := by
  first | apply Or.inl; assumption | apply Or.inr; assumption
```

在第一个例子中, 左分支成功; 而在第二个例子中, 成功的是右分支。在接下来的三个例子中, 同一个复合策略在每种情况下都成功:

```
example (p q r : Prop) (hp : p) : p ∨ q ∨ r := by
  repeat (first | apply Or.inl; assumption | apply Or.inr |
    ↪ assumption)
```

```
example (p q r : Prop) (hq : q) : p ∨ q ∨ r := by
  repeat (first | apply Or.inl; assumption | apply Or.inr |
    ↪ assumption)
```

```
example (p q r : Prop) (hr : r) : p ∨ q ∨ r := by
  repeat (first | apply Or.inl; assumption | apply Or.inr |
    ↪ assumption)
```

该策略首先尝试用假设立即解决左侧析取项; 如果失败, 就尝试聚焦到右侧析取项; 如果仍不可行, 就调用 **assumption** 策略。

到现在你无疑已经注意到, 策略可能失败。事实上, 正是“失败”状态使 **first** 组合子回溯并尝试下一个策略。**try** 组合子构造一个总是成功的策略, 尽管它可能只是平凡地成功: **try** t 执行 t , 即使 t 失败也报告成功。它等价于 **first** | t | **skip**, 其中 **skip** 是一个什么也不做 (并因此成功) 的策略。在下一个例子中, 第二个 **constructor** 在右侧合取项 $q \wedge r$ 上成功 (记住析取和合取向右结合), 但在第一个目标上失败。**try** 策略确保顺序组合成功:

```
example (p q r : Prop) (hp : p) (hq : q) (hr : r) : p ∧ q ∧ r
  ↪ := by
  constructor <;> (try constructor) <;> assumption
```

要小心: **repeat** (**try** t) 会永远循环, 因为内部策略永不失败。

在一个证明中，常常会有多个尚未解决的目标。并行顺序组合是一种安排单个策略应用于多个目标的方式，但还有其他方法。例如，**all_goals t** 会把 **t** 应用于所有打开的目标：

```
example (p q r : Prop) (hp : p) (hq : q) (hr : r) : p ∧ q ∧ r
  ↪ := by
    constructor
    all_goals (try constructor)
    all_goals assumption
```

在这种情况下，**any_goals** 策略提供了更稳健的解决方案。它类似于 **all_goals**，不同之处在于：只要其参数在至少一个目标上成功，它就成功：

```
example (p q r : Prop) (hp : p) (hq : q) (hr : r) : p ∧ q ∧ r
  ↪ := by
    constructor
    any_goals constructor
    any_goals assumption
```

下面 **by** 块中的第一个策略会反复拆分合取：

```
example (p q r : Prop) (hp : p) (hq : q) (hr : r) :
  p ∧ ((p ∧ q) ∧ r) ∧ (q ∧ r ∧ p) := by
  repeat (any_goals constructor)
  all_goals assumption
```

事实上，可以把完整策略压缩成一行：

```
example (p q r : Prop) (hp : p) (hq : q) (hr : r) :
  p ∧ ((p ∧ q) ∧ r) ∧ (q ∧ r ∧ p) := by
  repeat (any_goals (first | constructor | assumption))
```

组合子 **focus t** 确保 **t** 只影响当前目标，并暂时把其他目标从作用域中隐藏。因此，如果 **t** 通常只影响当前目标，那么 **focus (all_goals t)** 与 **t** 具有相同效果。

5.6 重写

rw 策略和 **simp** 策略已在 section 4.3 (p. 51) 中简要介绍。本节和下一节将更详细地讨论它们。

rw 策略提供了把替换应用于目标和假设的基本机制，从而为处理等式提供一种方便而高效的方式。该策略最基本的形式是 **rw [t]**，其中 **t** 是一个类型断言某个等式的项。例如，**t** 可以是上下文中的假设 $h : x = y$ ；也可以是一般引理，如 **add_comm** : $\forall x y, x + y = y + x$ ，此时重写策略会尝试为 x 和 y 寻找合适的实例；还可以是任何断言具体或一般等式的复合项。下面的例子中，我们使用这种基本形式，用一个假设来重写目标。

```
variable (k : Nat) (f : Nat → Nat)
```

```

example (h1 : f 0 = 0) (h2 : k = 0) : f k = 0 := by
  rw [h2] -- replace k with 0
  rw [h1] -- replace f 0 with 0

```

在上面的例子中，第一次使用 **rw** 会在目标 $f\ k = 0$ 中把 k 替换为 0 。随后第二次使用会把 $f\ 0$ 替换为 0 。该策略会自动关闭任何形如 $t = t$ 的目标。下面是使用复合表达式进行重写的例子：

```

example (x y : Nat) (p : Nat → Prop) (q : Prop) (h : q → x =
  ↪ y)
  (h' : p y) (hq : q) : p x := by
  rw [h hq]; assumption

```

这里， $h\ hq$ 建立了等式 $x = y$ 。

多个重写可以用记法 **rw** [t₁, ..., t_n] 组合；它只是 **rw**[t₁]; ...; **rw** [t_n] 的缩写。前一个例子可写成如下形式：

```

variable (k : Nat) (f : Nat → Nat)

```

```

example (h1 : f 0 = 0) (h2 : k = 0) : f k = 0 := by
  rw [h2, h1]

```

默认情况下，**rw** 按正向使用等式：将左端与某个表达式匹配，并用右端替换它。记法 **←t** 可用于指示策略按反向使用等式 t 。

```

variable (a b : Nat) (f : Nat → Nat)

```

```

example (h1 : a = b) (h2 : f a = 0) : f b = 0 := by
  rw [←h1, h2]

```

在这个例子中，项 **←h₁** 指示重写器把 b 替换为 a 。在编辑器中，可以输入 **\l** 得到反向箭头。也可以使用 ASCII 等价形式 **<-**。

有时，一个恒等式的左端可以匹配模式中的多个子项；此时 **rw** 策略会选择遍历项时找到的第一个匹配。如果这不是你想要的匹配，可以使用额外参数来指定合适的子项。

```

example (a b c : Nat) : a + b + c = a + c + b := by
  rw [Nat.add_assoc, Nat.add_comm b, ← Nat.add_assoc]

```

```

example (a b c : Nat) : a + b + c = a + c + b := by
  rw [Nat.add_assoc, Nat.add_assoc, Nat.add_comm b]

```

```

example (a b c : Nat) : a + b + c = a + c + b := by
  rw [Nat.add_assoc, Nat.add_assoc, Nat.add_comm _ b]

```

在上面的第一个例子中，第一步把 $a + b + c$ 重写为 $a + (b + c)$ 。下一步对项 $b + c$ 应用交换律；如果不指定参数，策略反而会把 $a + (b + c)$ 重写为 $(b + c) + a$ 。最后一步按反向应用结合律，把 $a + (c + b)$ 重写为 $a + c + b$ 。接下来的两个例子则在两边应用结合律，把括号移到右侧，然后交换 b

与 `c`。注意，最后一个例子通过指定 `Nat.add_comm` 的第二个参数，说明重写应发生在右端。

默认情况下，`rw` 策略只影响目标。记法 `rw [t] at h` 会把重写应用于假设：

```
example (f : Nat → Nat) (a : Nat) (h : a + 0 = 0) : f a = f 0
↪ := by
  rw [Nat.add_zero] at h
  rw [h]
```

第一步 `rw [Nat.add_zero] at h` 把假设 $a + 0 = 0$ 重写为 $a = 0$ 。然后用新的假设 $a = 0$ 把目标重写为 $f\ 0 = f\ 0$ 。

`rw` 策略并不限于命题。在下面的例子中，我们使用 `rw [h] at t`，把假设 $t : \text{Tuple } \alpha\ n$ 重写为 $t : \text{Tuple } \alpha\ 0$ 。

```
def Tuple (α : Type) (n : Nat) :=
  { as : List α // as.length = n }
```

```
example (n : Nat) (h : n = 0) (t : Tuple α n) : Tuple α 0 :=
↪ by
  rw [h] at t
  exact t
```

5.7 使用化简器

`rw` 被设计为操纵目标的精细工具，而化简器则提供更强大的自动化形式。Lean 库中的许多恒等式都带有 `[simp]` 属性，`simp` 策略会使用它们来迭代表达式中的子项。

```
example (x y z : Nat) : (x + 0) * (0 + y * 1 + z * 0) = x * y
↪ := by
  simp
```

```
example (x y z : Nat) (p : Nat → Prop) (h : p (x * y))
      : p ((x + 0) * (0 + y * 1 + z * 0)) := by
  simp; assumption
```

在第一个例子中，目标中等式的左端使用涉及 0 和 1 的通常恒等式被化简，将目标化为 $x * y = x * y$ 。此时，`simp` 应用自反性完成证明。在第二个例子中，`simp` 把目标化为 $p\ (x * y)$ ，此时假设 h 完成证明。下面是一些关于列表的更多例子：

```
open List
```

```
example (xs : List Nat)
      : reverse (xs ++ [1, 2, 3]) = [3, 2, 1] ++ reverse xs
↪ := by
  simp
```

```

example (xs ys : List  $\alpha$ )
  : length (reverse (xs ++ ys)) = length xs + length ys
   $\hookrightarrow$  := by
simp [Nat.add_comm]

```

与 **rw** 一样，可以使用关键字 **at** 来化简一个假设：

```

example (x y z : Nat) (p : Nat  $\rightarrow$  Prop)
  (h : p ((x + 0) * (0 + y * 1 + z * 0))) : p (x * y) :=
   $\hookrightarrow$  by
simp at h; assumption

```

此外，可以使用“通配符”星号来化简所有假设和目标：

```

attribute [local simp] Nat.mul_comm Nat.mul_assoc
 $\hookrightarrow$  Nat.mul_left_comm
attribute [local simp] Nat.add_assoc Nat.add_comm
 $\hookrightarrow$  Nat.add_left_comm

```

```

example (w x y z : Nat) (p : Nat  $\rightarrow$  Prop)
  (h : p (x * y + z * w * x)) : p (x * w * z + y * x) :=
   $\hookrightarrow$  by
simp at *; assumption

```

```

example (x y z : Nat) (p : Nat  $\rightarrow$  Prop)
  (h1 : p (1 * x + y)) (h2 : p (x * z * 1))
  : p (y + 0 + x)  $\wedge$  p (z * x) := by
simp at * <;> constructor <;> assumption

```

对于交换且结合的运算，例如自然数乘法，化简器会使用这两个事实来重写表达式，同时也使用左交换律。在乘法的情形中，后者表述为： $x * (y * z) = y * (x * z)$ 。**local** 修饰符告诉化简器在当前文件（或视情况在当前节、命名空间）中使用这些规则。交换律和左交换律看起来可能有问题，因为反复应用任一规则都会导致循环。但化简器会检测那些置换参数的恒等式，并使用一种称为有序重写的技术。这意味着系统维护项的内部次序，并且只有当应用恒等式会降低该次序时才应用它。对于上面提到的三个恒等式，其效果是表达式中的所有括号都向右结合，并且表达式以一种规范（尽管有些任意）的方式排序。于是，在结合律和交换律意义下等价的两个表达式会被重写为同一个规范形式。

```

example (w x y z : Nat) (p : Nat  $\rightarrow$  Prop)
  : x * y + z * w * x = x * w * z + y * x := by
simp

```

```

example (w x y z : Nat) (p : Nat  $\rightarrow$  Prop)
  (h : p (x * y + z * w * x)) : p (x * w * z + y * x) :=
   $\hookrightarrow$  by
simp; simp at h; assumption

```

与 `rw` 一样，可以向 `simp` 传递一个事实列表供其使用，其中包括一般引理、局部假设、要展开的定义以及复合表达式。`simp` 策略也识别 `rewrite` 所识别的 `←t` 语法。无论如何，额外规则都会被加入用于化简项的恒等式集合中。

```
def f (m n : Nat) : Nat :=
  m + n + m
```

```
example {m n : Nat} (h : n = 1) (h' : 0 = m) : (f m n) = n :=
  ↪ by
  simp [h, ←h', f]
```

一个常见习惯用法是使用局部假设来化简目标：

```
variable (k : Nat) (f : Nat → Nat)
```

```
example (h1 : f 0 = 0) (h2 : k = 0) : f k = 0 := by
  simp [h1, h2]
```

若要在化简时使用局部上下文中的所有假设，可以使用通配符 `*`：

```
variable (k : Nat) (f : Nat → Nat)
```

```
example (h1 : f 0 = 0) (h2 : k = 0) : f k = 0 := by
  simp [*]
```

下面是另一个例子：

```
example (u w x y z : Nat) (h1 : x = y + z) (h2 : w = u + x)
  : w = z + y + u := by
  simp [*, Nat.add_comm]
```

化简器也会进行命题重写。例如，使用假设 p 时，它会把 $p \wedge q$ 重写为 q ，并把 $p \vee q$ 重写为 `True`，随后平凡地证明它。迭代这类重写会产生非平凡的命题推理。

```
example (p q : Prop) (hp : p) : p ∧ q ↔ q := by
  simp [*]
```

```
example (p q : Prop) (hp : p) : p ∨ q := by
  simp [*]
```

```
example (p q r : Prop) (hp : p) (hq : q) : p ∧ (q ∨ r) := by
  simp [*]
```

下一个例子化简所有假设，然后用它们证明目标。

```
example (u w x x' y y' z : Nat) (p : Nat → Prop)
  (h1 : x + 0 = x') (h2 : y + 0 = y')
  : x + y + 0 = x' + y' := by
  simp at *
  simp [*]
```

化简器尤其有用的一个原因是，它的能力会随着库的发展而增长。例如，假设我们定义一个列表操作，通过追加输入的反转来使其对称化：

```
def mk_symm (xs : List  $\alpha$ ) :=
  xs ++ xs.reverse
```

那么对于任意列表 xs ， $(mk_symm\ xs).reverse$ 等于 $mk_symm\ xs$ ；这可以通过展开定义轻易证明：

```
theorem reverse_mk_symm (xs : List  $\alpha$ )
  : (mk_symm xs).reverse = mk_symm xs := by
  simp [mk_symm]
```

现在可以使用这个定理证明新的结果：

```
example (xs ys : List Nat)
  : (xs ++ mk_symm ys).reverse = mk_symm ys ++
     $\hookrightarrow$  xs.reverse := by
  simp [reverse_mk_symm]
```

```
example (xs ys : List Nat) (p : List Nat  $\rightarrow$  Prop)
  (h : p (xs ++ mk_symm ys).reverse)
  : p (mk_symm ys ++ xs.reverse) := by
  simp [reverse_mk_symm] at h; assumption
```

但使用 `reverse_mk_symm` 通常是正确做法；如果用户不必显式调用它会更好。可以在定义该定理时把它标记为化简规则来实现这一点：

```
@[simp] theorem reverse_mk_symm (xs : List  $\alpha$ )
  : (mk_symm xs).reverse = mk_symm xs := by
  simp [mk_symm]
```

```
example (xs ys : List Nat)
  : (xs ++ mk_symm ys).reverse = mk_symm ys ++
     $\hookrightarrow$  xs.reverse := by
  simp
```

```
example (xs ys : List Nat) (p : List Nat  $\rightarrow$  Prop)
  (h : p (xs ++ mk_symm ys).reverse)
  : p (mk_symm ys ++ xs.reverse) := by
  simp at h; assumption
```

记法 `@[simp]` 声明 `reverse_mk_symm` 具有 `[simp]` 属性，也可以更显式地写出：

```
theorem reverse_mk_symm (xs : List  $\alpha$ )
  : (mk_symm xs).reverse = mk_symm xs := by
  simp [mk_symm]
```

```
attribute [simp] reverse_mk_symm
```

```
example (xs ys : List Nat)
  : (xs ++ mk_symm ys).reverse = mk_symm ys ++
    ↪ xs.reverse := by
  simp
```

```
example (xs ys : List Nat) (p : List Nat → Prop)
  (h : p (xs ++ mk_symm ys).reverse)
  : p (mk_symm ys ++ xs.reverse) := by
  simp at h; assumption
```

该属性也可以在定理声明后的任何时候应用：

```
theorem reverse_mk_symm (xs : List  $\alpha$ )
  : (mk_symm xs).reverse = mk_symm xs := by
  simp [mk_symm]
```

```
example (xs ys : List Nat)
  : (xs ++ mk_symm ys).reverse = mk_symm ys ++
    ↪ xs.reverse := by
  simp [reverse_mk_symm]
```

```
attribute [simp] reverse_mk_symm
```

```
example (xs ys : List Nat) (p : List Nat → Prop)
  (h : p (xs ++ mk_symm ys).reverse)
  : p (mk_symm ys ++ xs.reverse) := by
  simp at h; assumption
```

然而，一旦应用了该属性，就无法永久移除它；任何导入赋予该属性之文件的文件都会继承它。正如我们将在 section 6.5 (p. 97) 中进一步讨论的，可以使用 **local** 修饰符将属性的作用域限制在当前文件或当前节中：

```
theorem reverse_mk_symm (xs : List  $\alpha$ )
  : (mk_symm xs).reverse = mk_symm xs := by
  simp [mk_symm]
```

section

```
attribute [local simp] reverse_mk_symm
```

```
example (xs ys : List Nat)
  : (xs ++ mk_symm ys).reverse = mk_symm ys ++
    ↪ xs.reverse := by
  simp
```

```
example (xs ys : List Nat) (p : List Nat → Prop)
  (h : p (xs ++ mk_symm ys).reverse)
```

```

      : p (mk_symm ys ++ xs.reverse) := by
    simp at h; assumption
end

```

在该节之外，化简器默认不再使用 `reverse_mk_symm`。

注意，我们讨论过的各种 **simp** 选项——给出显式规则列表，以及使用 **at** 指定位置——可以组合使用，但它们列出的顺序是固定的。可以在编辑器中把光标放在 **simp** 标识符上，查看与之关联的文档字符串，从而看到正确顺序。

还有两个有用的附加修饰符。默认情况下，**simp** 包含所有标记了 **[simp]** 属性的定理。写作 **simp only** 会排除这些默认规则，使你可以使用更明确构造的规则列表。在下面的例子中，减号和 **only** 被用来阻止应用 `reverse_mk_symm`。

```

def mk_symm (xs : List  $\alpha$ ) :=
  xs ++ xs.reverse
@[simp] theorem reverse_mk_symm (xs : List  $\alpha$ )
  : (mk_symm xs).reverse = mk_symm xs := by
  simp [mk_symm]

example (xs ys : List Nat) (p : List Nat  $\rightarrow$  Prop)
  (h : p (xs ++ mk_symm ys).reverse)
  : p (mk_symm ys ++ xs.reverse) := by
  simp at h; assumption

example (xs ys : List Nat) (p : List Nat  $\rightarrow$  Prop)
  (h : p (xs ++ mk_symm ys).reverse)
  : p ((mk_symm ys).reverse ++ xs.reverse) := by
  simp [-reverse_mk_symm] at h; assumption

example (xs ys : List Nat) (p : List Nat  $\rightarrow$  Prop)
  (h : p (xs ++ mk_symm ys).reverse)
  : p ((mk_symm ys).reverse ++ xs.reverse) := by
  simp only [List.reverse_append] at h; assumption

```

simp 策略有许多配置选项。例如，可以如下启用上下文化简：

```

example : if x = 0 then y + x = y else x  $\neq$  0 := by
  simp +contextual

```

使用 `+contextual` 时，**simp** 策略在化简 $y + x = y$ 时会使用事实 $x = 0$ ，而在化简另一个分支时会使用 $x \neq 0$ 。下面是另一个例子：

```

example :  $\forall$  (x : Nat) (h : x = 0), y + x = y := by
  simp +contextual

```

另一个有用的配置选项是 `+arith`，它启用算术化简。

```

example : 0 < 1 + x  $\wedge$  x + y + 2  $\geq$  y + 1 := by
  simp +arith

```


5.8 split 策略

split 策略对于按情况拆分嵌套的 **if-then-else** 和 **match** 表达式很有用。对于具有 n 个情况的 **match** 表达式，**split** 策略最多生成 n 个子目标。下面是一个例子：

```
def f (x y z : Nat) : Nat :=
  match x, y, z with
  | 5, _, _ => y
  | _, 5, _ => y
  | _, _, 5 => y
  | _, _, _ => 1
```

```
example (x y z : Nat) : x ≠ 5 → y ≠ 5 → z ≠ 5 → z = w → f x y
  ↪ w = 1 := by
  intros
  simp [f]
  split
  . contradiction
  . contradiction
  . contradiction
  . rfl
```

可以把上面的策略证明压缩为如下形式。

```
example (x y z : Nat) :
  x ≠ 5 → y ≠ 5 → z ≠ 5 → z = w →
  f x y w = 1 := by
  intros; simp [f]; split <|> first | contradiction | rfl
```

策略 **split <|> first | contradiction | rfl** 先应用 **split** 策略，然后对每个生成的目标尝试 **contradiction**；如果 **contradiction** 失败，再尝试 **rfl**。像 **simp** 一样，我们也可以把 **split** 应用于某个特定假设：

```
def g (xs ys : List Nat) : Nat :=
  match xs, ys with
  | [a, b], _ => a+b+1
  | _, [b, _] => b+1
  | _, _      => 1
```

```
example (xs ys : List Nat) (h : g xs ys = 0) : False := by
  simp [g] at h; split at h <|> simp +arith at h
```

5.9 可扩展策略

在下面的例子中，我们使用命令 **syntax** 定义记法 **triv**。然后，使用命令 **macro_rules** 指定当使用 **triv** 时应执行什么操作。可以提供不同展开，策略解释器会逐一尝试，直到其中一个成功：

```

-- Define a new tactic notation
syntax "triv" : tactic

macro_rules
  | `(tactic| triv) => `(tactic| assumption)

example (h : p) : p := by
  triv

-- You cannot prove the following theorem using `triv`
-- example (x :  $\alpha$ ) : x = x := by
--   triv

-- Let's extend `triv`. The tactic interpreter
-- tries all possible macro extensions for `triv` until one
-- ↪ succeeds
macro_rules
  | `(tactic| triv) => `(tactic| rfl)

example (x :  $\alpha$ ) : x = x := by
  triv

example (x :  $\alpha$ ) (h : p) : x = x  $\wedge$  p := by
  apply And.intro <|> triv

-- We now add a (recursive) extension
macro_rules | `(tactic| triv) => `(tactic| apply And.intro <|>
  ↪ triv)

example (x :  $\alpha$ ) (h : p) : x = x  $\wedge$  p := by
  triv

```

5.10 练习

1. 回到 chapter 3 (p. 29) 和 chapter 4 (p. 45) 中的练习，现在尽可能多地用策略证明重做，并在适当时也使用 **rw** 和 **simp**。
2. 使用策略组合子，为下面的命题给出一行证明：

```

example (p q r : Prop) (hp : p)
  : (p  $\vee$  q  $\vee$  r)  $\wedge$  (q  $\vee$  p  $\vee$  r)  $\wedge$  (q  $\vee$  r  $\vee$  p) := by
  sorry

```

6 与 Lean 交互

你现在已经熟悉了依值类型论的基础：它既是一种定义数学对象的语言，也是一种构造证明的语言。你目前还缺少的一点，是一种定义新数据类型的机制。我们将在下一章填补这一空白，届时将引入归纳数据类型这一概念。不过在此之前，本章先暂时离开类型论的机制性内容，转而探讨一些与 Lean 交互时更具实践性的方面。

这里给出的信息并不一定会立刻对你有用。我们建议你先快速浏览本节，对 Lean 的特性形成整体印象，然后在需要时再回来查阅。

6.1 消息

Lean 会产生三类消息：

错误 当代码中存在不一致、导致其无法被处理时，就会产生错误。例如，语法错误（如缺少 `)`）以及类型错误（例如试图把一个自然数与函数相加）。

警告 警告描述的是代码中潜在的问题，例如出现了 **sorry**。与错误不同，这并不意味着代码毫无意义；不过，警告同样值得认真对待。

信息 信息并不表示代码存在问题，其中包括 **#check**、**#eval** 等命令的输出。

Lean 可以检查某条命令是否产生了预期的消息。如果消息匹配，那么其中的错误会被忽略；这可用于确保出现的正是我们想要的错误。如果不匹配，则会再产生一个错误。你可以使用 **#guard_msgs** 命令来说明哪些消息是预期的。

下面是一个例子：

```
/--
error: Type mismatch
  "Not a number"
has type
  String
but is expected to have type
  Nat
-/
#guard_msgs in
def x : Nat := "Not a number"
```

在 `#guard_msgs` 后面的括号中写入消息类别时，它只会检查指定的类别，而让其他消息照常显示。在这个例子中，`#eval` 由于存在 `sorry` 而发出错误，但针对 `sorry` 总会产生的警告仍会像往常一样显示出来：

```
/--
error: aborting evaluation since the expression depends on the
  ↳ 'sorry' axiom, which can lead to runtime instability and
  ↳ crashes.
```

To attempt to evaluate anyway despite the risks, use the
↳ `'#eval!'` command.

```
-/
```

```
#guard_msgs(error) in
#eval (sorry : Nat)
```

```
└─ declaration uses `sorry`
```

如果不作这样的配置，两条消息都会被捕获：

```
/--
error: aborting evaluation since the expression depends on the
  ↳ 'sorry' axiom, which can lead to runtime instability and
  ↳ crashes.
```

To attempt to evaluate anyway despite the risks, use the
↳ `'#eval!'` command.

```
---
```

```
warning: declaration uses `sorry`
```

```
-/
```

```
#guard_msgs in
#eval (sorry : Nat)
```

```
└─ Docstring on `#guard_msgs` does not match generated
  ↳ message:
```

```
+ warning: declaration uses `sorry`
```

```
+ ---
```

```
error: aborting evaluation since the expression depends on
  ↳ the 'sorry' axiom, which can lead to runtime
  ↳ instability and crashes.
```

```
To attempt to evaluate anyway despite the risks, use the
  ↳ '#eval!' command.
```

```
- ---
```

```
- warning: declaration uses `sorry`
```

本书中的一些例子会使用 `#guard_msgs` 来标明预期出现的错误。

6.2 导入文件

Lean 前端的目标是解释用户输入、构造形式化表达式，并检查它们是否良构且类型正确。Lean 还支持多种编辑器，它们能够提供持续的检查与反馈。更多信息可参见 Lean 的文档页面¹。

Lean 标准库中的定义和定理分散在多个文件中。用户也可能希望使用额外的库，或者在多个文件中开发自己的项目。Lean 启动时，会自动导入库 `Init` 文件夹中的内容，其中包含若干基础定义与构造。因此，我们在这里给出的多数例子都可以“开箱即用”。

不过，如果你想使用额外的文件，就需要在文件开头通过 `import` 语句手动导入。命令

```
import Bar.Baz.Blah
```

会导入文件 `Bar/Baz/Blah.olean`，其中这些描述是相对于 Lean 的搜索路径来解释的。关于搜索路径如何确定，可参见 文档页面²。默认情况下，它包括标准库目录，以及（在某些上下文中）用户本地项目的根目录。

导入具有传递性。换言之，如果你导入 `Foo`，而 `Foo` 又导入了 `Bar`，那么你也可以访问 `Bar` 的内容，无需再显式导入它。

6.3 关于节的更多说明

Lean 提供了多种分节机制来帮助组织理论结构。你已经在 section 2.6 (p. 17) 中看到，`section` 命令不仅可以将彼此相关的理论内容组织在一起，还可以声明变量，并在需要时将它们自动插入为定理和定义的参数。请记住，`variable` 命令的用途是为定理声明待用变量，如下例所示：

```
section
variable (x y : Nat)

def double := x + x

#check double y
| double y : Nat
#check double (2 * x)
| double (2 * x) : Nat
attribute [local simp] Nat.add_assoc Nat.add_comm
  ↪ Nat.add_left_comm

theorem t1 : double (x + y) = double x + double y := by
  simp [double]

#check t1 y
```

¹<https://lean-lang.org/documentation/>

²<https://lean-lang.org/documentation/>

```

| t1 y : ∀ (y_1 : Nat), double (y + y_1) = double y + double
|   ↪ y_1

#check t1 (2 * x)

| t1 (2 * x) : ∀ (y : Nat), double (2 * x + y) = double (2 * x)
|   ↪ + double y

theorem t2 : double (x * y) = double x * y := by
  simp [double, Nat.add_mul]

end

```

`double` 的定义不必显式将 `x` 声明为参数；Lean 会检测到这种依赖关系并自动插入它。同样，Lean 也会检测 `t1` 与 `t2` 中 `x` 的出现，并在那里自动插入该参数。注意，`double` 并不会把 `y` 当作参数。变量只会被加入到那些实际使用了它们的声明中。

6.4 关于命名空间的更多说明

在 Lean 中，标识符由层级化的名称给出，例如 `Foo.Bar.baz`。我们已经在 section 2.7 (p. 18) 中看到，Lean 提供了处理层级名称的机制。命令 `namespace Foo` 会在每个定义和定理的名称前加上 `Foo`，直到遇到 `end Foo` 为止。随后，命令 `open Foo` 会为那些以 `Foo` 为前缀的定义和定理创建临时的别名。

```

namespace Foo
def bar : Nat := 1
end Foo

open Foo

#check bar
| Foo.bar : Nat

#check Foo.bar
| Foo.bar : Nat

```

下面这个定义

```
def Foo.bar : Nat := 1
```

会被当作一个宏，并展开为

```

namespace Foo
def bar : Nat := 1
end Foo

```

虽然定理和定义的名称必须唯一，但用于引用它们的别名却不必如此。当我们打开一个命名空间时，某个标识符可能会变得有歧义。**Lean** 会尝试利用类型信息在上下文中消解歧义，但你始终可以通过给出全名来明确所指。为此，字符串 `_root_` 就是空前缀的显式写法。

```
def String.add (a b : String) : String :=
  a ++ b

def Bool.add (a b : Bool) : Bool :=
  a != b

def add (α β : Type) : Type := Sum α β

open Bool
open String

-- This reference is ambiguous:
-- #check add

#check String.add          -- String.add (a b : String) :
  ↪ String
| String.add (a b : String) : String
#check Bool.add            -- Bool.add (a b : Bool) : Bool
| Bool.add (a b : Bool) : Bool
#check _root_.add          -- _root_.add (α β : Type) : Type
| _root_.add (α β : Type) : Type
#check add "hello" "world" -- "hello".add "world" : String
| "hello".add "world" : String
#check add true false      -- true.add false : Bool
| true.add false : Bool
#check add Nat Nat         -- _root_.add Nat Nat : Type
| _root_.add Nat Nat : Type
```

我们可以使用 **protected** 关键字来阻止创建这种较短的别名：

```
protected def Foo.bar : Nat := 1

open Foo

/-- error: Unknown identifier `bar` -/
#guard_msgs in
#check bar -- error

#check Foo.bar
```

```
Foo.bar : Nat
```

这种做法经常用于 `Nat.rec` 和 `Nat.recOn` 之类的名称，以避免常见名称发生重载。

`open` 命令还有若干变体。命令

```
open Nat (succ zero gcd)
```

```
#check zero      -- Nat.zero : Nat
```

```
Nat.zero : Nat
```

```
#eval gcd 15 6   -- 3
```

```
3
```

只会为列出的标识符创建别名。命令

```
open Nat hiding succ gcd
```

```
#check zero      -- Nat.zero : Nat
```

```
Nat.zero : Nat
```

```
/-- error: Unknown identifier `gcd` -/
```

```
#guard_msgs in
```

```
#eval gcd 15 6   -- error
```

```
#eval Nat.gcd 15 6 -- 3
```

```
3
```

会为 `Nat` 命名空间中除所列标识符之外的所有内容创建别名。

```
open Nat renaming mul → times, add → plus
```

```
#eval plus (times 2 2) 3 -- 7
```

```
7
```

会创建别名，并将 `Nat.mul` 重命名为 `times`，将 `Nat.add` 重命名为 `plus`。

有时，把别名从一个命名空间 `export` 到另一个命名空间，或导出到顶层，会很有用。命令

```
export Nat (succ add sub)
```

会在当前命名空间中为 `succ`、`add` 和 `sub` 创建别名，因此只要该命名空间被打开，这些别名就可用。如果在命名空间之外使用此命令，那么这些别名会被导出到顶层。

6.5 属性

Lean 的主要功能是将用户输入翻译为形式化表达式，由内核检查其正确性，然后存入环境中以备后用。但有些命令还会对环境产生其他影响，例如为环境中的对象赋予属性、定义记法，或者像 chapter 10 (p. 175) 一章中所述那样，声明类型类的实例。这些命令大多具有全局效果，也就是说，它们不仅在当前文件中生效，在任何导入该文件的文件中也继续生效。不过，这类命令通常支持 **local** 修饰符，表示它们只在当前 **section** 或 **namespace** 关闭之前，或直到当前文件结束之前生效。

在 section 5.7 (p. 83) 中，我们看到可以用 **[simp]** 属性来标注定理，从而使化简器能够使用它们。下面的例子在列表上定义了前缀关系，证明该关系是自反的，并把 **[simp]** 属性赋给这个定理。

```
def isPrefix (l₁ : List α) (l₂ : List α) : Prop :=
  ∃ t, l₁ ++ t = l₂

@[simp] theorem List.isPrefix_self (as : List α) : isPrefix as
  ↪ as :=
  ([], by simp)

example : isPrefix [1, 2, 3] [1, 2, 3] := by
  simp
```

随后，化简器会通过把 `isPrefix [1, 2, 3] [1, 2, 3]` 重写成 `True` 来证明它。

也可以在定义完成后的任意时刻再赋予该属性：

```
theorem List.isPrefix_self (as : List α) : isPrefix as as :=
  ([], by simp)

attribute [simp] List.isPrefix_self
```

在上述所有情形中，该属性都会在任何导入此声明所在文件的文件中继续生效。加入 **local** 修饰符则会限制其作用域：

section

```
theorem List.isPrefix_self (as : List α) : isPrefix as as :=
  ([], by simp)
```

```
attribute [local simp] List.isPrefix_self
```

```
example : isPrefix [1, 2, 3] [1, 2, 3] := by
  simp
```

end

```

/-- error: `simp` made no progress -/
#guard_msgs in
example : isPrefix [1, 2, 3] [1, 2, 3] := by
  simp

```

再举一个例子,我们可以使用 **instance** 命令,把记号 $\leq$ 赋给关系 `isPrefix`。这个命令将在 chapter 10 (p. 175) 一章中解释;它的工作方式是给相关定义赋予一个 **[instance]** 属性。

```

def isPrefix (l1 : List α) (l2 : List α) : Prop :=
  ∃ t, l1 ++ t = l2

instance : LE (List α) where
  le := isPrefix

theorem List.isPrefix_self (as : List α) : as ≤ as :=
  ([], by simp)

```

这种赋值同样也可以局部化:

```

def instLe : LE (List α) := { le := isPrefix }

```

```

section
attribute [local instance] instLe

example (as : List α) : as ≤ as :=
  ([], by simp)

```

```

end

```

```

/--
error: failed to synthesize instance of type class
      LE (List α)

```

```

Hint: Type class instance resolution failures can be inspected
↪ with the `set_option trace.Meta.synthInstance true`
↪ command.
-/
#guard_msgs in
example (as : List α) : as ≤ as :=
  ([], by simp)

```

在下面的 section 6.7 (p. 101) 一节中,我们将讨论 Lean 定义记法的机制,并看到它们也支持 **local** 修饰符。不过,在 section 6.10 (p. 104) 一节中,我们将讨论 Lean 设置选项的机制;这一点并不遵循上述模式:选项只能局部设置,也就是说,它们的作用域总是限制在当前节或当前文件内。

6.6 关于隐式参数的更多说明

在 section 2.9 (p. 23) 中，我们看到，如果 Lean 将某个项 t 的类型显示为 $\{x : \alpha\} \rightarrow \beta$ ，那么花括号表示 x 已被标记为 t 的一个隐式参数。这意味着，每当你写下 t 时，都会自动插入一个占位符，也就是“洞”，从而把 t 替换为 $@t _$ 。如果你不希望发生这种情况，就必须改写成 $@t$ 。

请注意，隐式参数会被急切地插入。假设我们定义了一个函数 $f : (x : \text{Nat}) \rightarrow \{y : \text{Nat}\} \rightarrow (z : \text{Nat}) \rightarrow \text{Nat}$ 。那么，当我们在没有提供更多参数的情况下写表达式 $f \ 7$ 时，它会被解析为 $@f \ 7 _$ 。

Lean 还提供了一种更弱的标注，用来说明：只有在后面紧跟显式参数之前，才应插入占位符。它可以写作双花括号，因此 f 的类型可写为 $f : (x : \text{Nat}) \rightarrow \{\{y : \text{Nat}\}\} \rightarrow (z : \text{Nat}) \rightarrow \text{Nat}$ 。使用这种标注时，表达式 $f \ 7$ 会按原样解析，而 $f \ 7 \ 3$ 则会像使用强标注时一样，被解析为 $@f \ 7 _ \ 3$ 。这种标注也可以写成 $\square y : \text{Nat}$ ，其中 Unicode 括号分别通过 $\backslash\{\{$ 与 $\backslash\}$ 输入。

为了说明这种差异，请看下面的例子，它表明：一个自反且欧几里得的关系同时也是对称的和传递的。

```
def reflexive {α : Type u} (r : α → α → Prop) : Prop :=
  ∀ (a : α), r a a
```

```
def symmetric {α : Type u} (r : α → α → Prop) : Prop :=
  ∀ {a b : α}, r a b → r b a
```

```
def transitive {α : Type u} (r : α → α → Prop) : Prop :=
  ∀ {a b c : α}, r a b → r b c → r a c
```

```
def Euclidean {α : Type u} (r : α → α → Prop) : Prop :=
  ∀ {a b c : α}, r a b → r a c → r b c
```

```
theorem th1 {α : Type u} {r : α → α → Prop}
  (reflr : reflexive r) (euclr : Euclidean r)
  : symmetric r :=
  fun {a b : α} =>
  fun (h : r a b) =>
  show r b a from euclr h (reflr _)
```

```
theorem th2 {α : Type u} {r : α → α → Prop}
  (symmr : symmetric r) (euclr : Euclidean r)
  : transitive r :=
  fun {a b c : α} =>
  fun (rab : r a b) (rbc : r b c) =>
  euclr (symmr rab) rbc
```

```
theorem th3 {α : Type u} {r : α → α → Prop}
  (reflr : reflexive r) (euclr : Euclidean r)
  : transitive r :=
  th2 (th1 reflr @euclr) @euclr
```

```

variable (r :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$ )
variable (euclr : Euclidean r)

#check euclr
| euclr : r ?m.3 ?m.4  $\rightarrow$  r ?m.3 ?m.5  $\rightarrow$  r ?m.4 ?m.5

```

这个结果被分解成若干小步骤: **th1** 说明一个既自反又欧几里得的关系是对称的, 而 **th2** 说明一个既对称又欧几里得的关系是传递的。随后, **th3** 将这两个结果组合起来。不过请注意, 我们必须手动禁用 **euclr** 中的隐式参数, 否则会插入过多的隐式参数。如果改用弱隐式参数, 这个问题就会消失:

```

def reflexive { $\alpha$  : Type u} (r :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$ ) : Prop :=
   $\forall$  (a :  $\alpha$ ), r a a

def symmetric { $\alpha$  : Type u} (r :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$ ) : Prop :=
   $\forall$  {a b :  $\alpha$ }, r a b  $\rightarrow$  r b a

def transitive { $\alpha$  : Type u} (r :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$ ) : Prop :=
   $\forall$  {a b c :  $\alpha$ }, r a b  $\rightarrow$  r b c  $\rightarrow$  r a c

def Euclidean { $\alpha$  : Type u} (r :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$ ) : Prop :=
   $\forall$  {a b c :  $\alpha$ }, r a b  $\rightarrow$  r a c  $\rightarrow$  r b c

theorem th1 { $\alpha$  : Type u} {r :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$ }
  (reflr : reflexive r) (euclr : Euclidean r)
  : symmetric r :=
  fun {a b :  $\alpha$ } =>
  fun (h : r a b) =>
  show r b a from euclr h (reflr _)

theorem th2 { $\alpha$  : Type u} {r :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$ }
  (symmr : symmetric r) (euclr : Euclidean r)
  : transitive r :=
  fun {a b c :  $\alpha$ } =>
  fun (rab : r a b) (rbc : r b c) =>
  euclr (symmr rab) rbc

theorem th3 { $\alpha$  : Type u} {r :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$ }
  (reflr : reflexive r) (euclr : Euclidean r)
  : transitive r :=
  th2 (th1 reflr euclr) euclr

variable (r :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$ )
variable (euclr : Euclidean r)

#check euclr -- euclr : Euclidean r
| euclr : Euclidean r

```

还有第三种隐式参数，用方括号 [与] 表示。它们用于类型类，这将在 chapter 10 (p. 175) 一章中说明。

6.7 记法

Lean 中的标识符可以包含任意字母数字字符，也包括希腊字母（除了 α 、 Σ 和 λ ，它们如我们所见，在依值类型论中具有特殊含义）。标识符还可以包含下标；输入方法是先键入 `\_`，再输入所需的下标字符。

Lean 的解析器是可扩展的，也就是说，我们可以定义新的记法。

Lean 的语法可以由用户在各个层次上进行扩展和定制，范围从基础的“`mixfix`”记法一直到自定义精化器。事实上，所有内建语法也都是通过同样面向用户开放的机制和 API 来解析和处理的。本节将介绍并解释这些不同的扩展点。

在编程语言中，引入新记法是一项相对少见的特性，有时甚至会因为可能使代码晦涩而不受欢迎。然而在形式化工作中，它却是一种极其宝贵的工具，能够让相应领域中已经约定俗成的符号与记法以简洁的方式体现在代码里。更进一步地，除了基础记法外，Lean 还能把常见样板代码抽象成行为良好的宏，并嵌入完整的自定义领域特定语言（DSL），以高效且可读的文本形式编码子问题；这对程序员和证明工程师都大有裨益。

记法与优先级

最基础的语法扩展命令允许我们引入新的（或重载已有的）前缀、中缀和后缀运算符。

```
infixl:65 " + " => HAdd.hAdd -- left-associative
infix:50  " = " => Eq       -- non-associative
infixr:80 " ^ " => HPow.hPow -- right-associative
prefix:100 "-"      => Neg.neg
postfix:max "-¹"    => Inv.inv
```

在描述运算符种类（即其“fixity”）的命令名之后，我们先给出运算符的解析优先级，其前面带一个冒号 `:`；然后写上用双引号括起来的新 `token` 或已有 `token`（其中的空格用于美观输出）；最后在箭头 `=>` 后写出该运算符应被翻译成的函数。

优先级是一个自然数，用来描述运算符与其参数结合得有多“紧”，也即编码了运算顺序。通过观察上述命令展开后的形式，我们可以更精确地理解这一点：

```
notation:65 lhs:65 " + " rhs:66 => HAdd.hAdd lhs rhs
notation:50 lhs:51 " = " rhs:51 => Eq lhs rhs
notation:80 lhs:81 " ^ " rhs:80 => HPow.hPow lhs rhs
notation:100 "-" arg:100 => Neg.neg arg
-- `max` is a shorthand for precedence 1024:
notation:1024 arg:1024 "-¹" => Inv.inv arg
```

事实证明，第一个代码块中的所有命令实际上都是命令宏，会被翻译为更通用的 **notation** 命令。我们稍后会学习如何编写这样的宏。**notation** 命令不只接受单个 **token**，而是接受由 **token** 与带优先级的具名项占位符交错组成的序列：这些占位符可以在 $\Rightarrow$ 右侧被引用，并替换为该位置解析得到的相应项。优先级为 p 的占位符，在该位置只接受优先级至少为 p 的记法。因此，字符串 $a + b + c$ 不能被解析为 $a + (b + c)$ ，因为 **infixl** 记法右侧操作数的优先级比该记法自身高 1。相对地，**infixr** 会把该记法自身的优先级复用于右侧操作数，因此 $a \wedge b \wedge c$ 可以被解析为 $a \wedge (b \wedge c)$ 。注意，如果我们直接用 **notation** 来引入如下的中缀记法

```
notation:65 lhs:65 " ~ " rhs:65 => wobble lhs rhs
```

在这种优先级不足以充分决定结合性的情形下，Lean 的解析器默认采用右结合。更准确地说，当语法存在歧义时，Lean 的解析器遵循局部的最长解析规则：在解析 $a \sim b \sim c$ 中 $a \sim$ 的右侧时，只要当前优先级允许，它就会尽可能继续向后解析，不会在 b 处停止，而会把 $\sim c$ 也一并解析进去。因此，该项等价于 $a \sim (b \sim c)$ 。

如上所述，**notation** 命令允许我们自由地混合 **token** 与占位符，从而定义任意的 **mixfix** 语法。

```
notation:max "(" e ")" => e
```

```
notation:10 Γ " ⊢ " e " : " τ => Typing Γ e τ
```

未标注优先级的占位符默认使用 0，也就是说，它们在自身位置上接受任意优先级的记法。如果两个记法发生重叠，我们仍然应用最长解析规则：

```
notation:65 a " + " b:66 " + " c:66 => a + b - c
```

```
#eval 1 + 2 + 3 -- 0
```

```
0
```

这里会优先选择新的记法，而不是二元记法，因为后者在形成链式解析之前，会在 $1 + 2$ 之后就停止。如果有多个记法都接受同样的最长解析，那么最终选择会延迟到精化阶段：除非恰好只有一个重载在类型上是正确的，否则精化就会失败。

6.8 强制转换

在 Lean 中，自然数类型 **Nat** 与整数类型 **Int** 是不同的。不过，存在一个函数 **Int.ofNat**，它把自然数嵌入到整数中，这意味着在需要时我们可以把任意自然数看作一个整数。Lean 具有检测并插入这类强制转换的机制。我们也可以用重载的 $\uparrow$ 运算符显式请求强制转换。

```
variable (m n : Nat)
```

```
variable (i j : Int)
```

```
#check i + m -- i + ↑m : Int
```

```

| i + ↑m : Int
#check i + m + j -- i + ↑m + j : Int
| i + ↑m + j : Int
#check i + m + n -- i + ↑m + ↑n : Int
| i + ↑m + ↑n : Int

```

6.9 显示信息

你可以通过多种方式向 Lean 查询其当前状态，以及当前上下文中可用的对象与定理。你已经见过其中最常见两种，即 `#check` 和 `#eval`。请记住，`#check` 经常与 `@` 运算符配合使用，后者会把定理或定义的所有参数都显式写出。此外，你还可以使用 `#print` 命令获取任意标识符的信息。如果该标识符表示一个定义或定理，Lean 会打印该符号的类型及其定义；如果它是常量或公理，Lean 会说明这一点，并显示它的类型。

```

-- examples with equality
#check Eq
| Eq.{u_1} {α : Sort u_1} : α → α → Prop
#check @Eq
| @Eq : {α : Sort u_1} → α → α → Prop
#check Eq.symm
| Eq.symm.{u} {α : Sort u} {a b : α} (h : a = b) : b = a
#check @Eq.symm
| @Eq.symm : ∀ {α : Sort u_1} {a b : α}, a = b → b = a
#print Eq.symm
| theorem Eq.symm.{u} : ∀ {α : Sort u} {a b : α}, a = b → b = a
  ↳ :=
  fun {α} {a b} h => h ▸ rfl
-- examples with And
#check And
| And (a b : Prop) : Prop
#check And.intro
| And.intro {a b : Prop} (left : a) (right : b) : a ∧ b
#check @And.intro
| @And.intro : ∀ {a b : Prop}, a → b → a ∧ b
-- a user-defined function
def foo {α : Type u} (x : α) : α := x

#check foo
| foo.{u} {α : Type u} (x : α) : α

```

```
#check @foo
| @foo : {α : Type u_1} → α → α
#print foo
| def foo.{u} : {α : Type u} → α → α :=
| fun {α} x => x
```

6.10 设置选项

Lean 维护着若干内部变量，用户可以设置它们来控制 Lean 的行为。其语法如下：

```
set_option <name> <value>
```

有一类非常实用的选项用于控制 Lean 的美观打印机如何显示项。下面这些选项都接受 true 或 false 作为输入：

```
pp.explicit   : 显示隐式参数
pp.universes  : 显示隐藏的宇宙参数
pp.notation   : 使用已定义的记法显示输出
```

例如，下面的设置会产生长得多的输出：

```
set_option pp.explicit true
set_option pp.universes true
set_option pp.notation false
```

```
#check 2 + 2 = 4
| @Eq.{1} Nat
|   (@HAdd.hAdd.{0, 0, 0} Nat Nat Nat (@instHAdd.{0} Nat
|     ↳ instAddNat)
|     (@OfNat.ofNat.{0} Nat (nat_lit 2) (instOfNatNat (nat_lit
|       ↳ 2)))
|     (@OfNat.ofNat.{0} Nat (nat_lit 2) (instOfNatNat (nat_lit
|       ↳ 2))))
|     (@OfNat.ofNat.{0} Nat (nat_lit 4) (instOfNatNat (nat_lit
|       ↳ 4))) : Prop
#reduce (fun x => x + 2) = (fun x => x + 3)
| @Eq.{1} (Nat → Nat)
|   (fun x =>
|     @HAdd.hAdd.{0, 0, 0} Nat Nat Nat (@instHAdd.{0} Nat
|       ↳ instAddNat) x
|     (@OfNat.ofNat.{0} Nat (nat_lit 2) (instOfNatNat
|       ↳ (nat_lit 2))))
|   fun x =>
|     @HAdd.hAdd.{0, 0, 0} Nat Nat Nat (@instHAdd.{0} Nat
|       ↳ instAddNat) x
|     (@OfNat.ofNat.{0} Nat (nat_lit 3) (instOfNatNat (nat_lit
|       ↳ 3))))
```



```
#check (fun x => x + 1) 1
(fun x =>
  @HAdd.hAdd.{0, 0, 0} Nat Nat Nat (@instHAdd.{0} Nat
    ↳ instAddNat) x
    (@OfNat.ofNat.{0} Nat (nat_lit 1) (instOfNatNat
      ↳ (nat_lit 1))))
  (@OfNat.ofNat.{0} Nat (nat_lit 1) (instOfNatNat (nat_lit
    ↳ 1))) : Nat
```

命令 `set_option pp.all true` 会一次性启用这些设置，而 `set_option pp.all false` 则会恢复到先前的取值。在调试证明、或者试图理解晦涩的错误信息时，打印更多附加信息往往非常有用。不过，信息过多也可能令人不堪重负，而 Lean 的默认设置通常已足以应对日常交互。

6.11 使用库

要高效地使用 Lean，你不可避免地需要借助库中的定义和定理。回想一下，文件开头的 `import` 命令会导入其他文件中先前编译好的结果，而且导入具有传递性；如果你导入 `Foo`，而 `Foo` 又导入了 `Bar`，那么来自 `Bar` 的定义和定理也会对你可用。不过，打开命名空间这件事——也就是获得较短名称——并不会随导入自动延续。在每个文件中，你都需要显式打开自己想使用的命名空间。

总的来说，熟悉这个库及其内容非常重要，这样你才能知道有哪些定理、定义、记法和资源可供使用。下面我们会看到，Lean 的编辑器模式也可以帮助你找到所需内容，但直接研究库本身的内容往往仍然不可避免。Lean 的标准库可以在 GitHub 上在线查看：

- <https://github.com/leanprover/lean4/tree/master/src/Init>³
- <https://github.com/leanprover/lean4/tree/master/src/Std>⁴

你可以通过 GitHub 的浏览器界面查看这些目录和文件的内容。如果你已经在自己的计算机上安装了 Lean，那么可以在 `lean` 文件夹中找到这个库，并用文件管理器浏览它。每个文件顶部的注释头还提供了更多信息。

Lean 的库开发者遵循一套通用的命名准则，以便你更容易猜出所需定理的名称，或者在支持此功能的 Lean 编辑器模式中利用 Tab 补全来找到它——下一节会讨论这一点。标识符通常采用 `camelCase`，类型采用 `CamelCase`。至于定理名，我们倾向于使用描述性的名称，并用 `_` 将不同部分分隔开来。很多时候，定理的名字本身就直接描述了其结论：

```
#check Nat.succ_ne_zero
| Nat.succ_ne_zero (n : Nat) : n.succ ≠ 0
#check Nat.zero_add
| Nat.zero_add (n : Nat) : 0 + n = n
```

³<https://github.com/leanprover/lean4/tree/master/src/Init>

⁴<https://github.com/leanprover/lean4/tree/master/src/Std>

```
#check Nat.mul_one
| Nat.mul_one (n : Nat) : n * 1 = n
#check Nat.le_of_succ_le_succ
| Nat.le_of_succ_le_succ {n m : Nat} : n.succ ≤ m.succ → n ≤ m
```

请记住, Lean 中的标识符可以组织在层级化的命名空间中。例如, 命名空间 `Nat` 中名为 `le_of_succ_le_succ` 的定理, 其全名是 `Nat.le_of_succ_le_succ`; 但通过命令 `open Nat` (对未标记为 **protected** 的名称而言), 我们可以使用较短的名字。我们会在 chapter 7 (p. 113) 和 chapter 9 (p. 169) 两章中看到, 在 Lean 中定义结构和归纳数据类型时, 会自动生成相关操作; 这些操作被存放在一个与所定义类型同名的命名空间中。例如, 积类型附带了如下操作:

```
#check @Prod.mk
| @Prod.mk : {α : Type u_1} → {β : Type u_2} → α → β → α × β
#check @Prod.fst
| @Prod.fst : {α : Type u_1} → {β : Type u_2} → α × β → α
#check @Prod.snd
| @Prod.snd : {α : Type u_1} → {β : Type u_2} → α × β → β
#check @Prod.rec
| @Prod.rec : {α : Type u_2} →
  {β : Type u_3} → {motive : α × β → Sort u_1} → ((fst : α) →
    → (snd : β) → motive (fst, snd)) → (t : α × β) → motive t
```

第一个用于构造一个序对, 接下来的两个 `Prod.fst` 与 `Prod.snd` 则分别投影出这两个分量。最后的 `Prod.rec` 提供了另一种在积类型上定义函数的机制: 它把问题归结为在两个分量上的一个函数。像 `Prod.rec` 这样的名称是 **protected** 的, 这意味着即使打开了 `Prod` 命名空间, 也必须使用它们的全名。

按照“命题即类型”的对应, 逻辑联结词也都是归纳类型的实例, 因此我们也倾向于对它们使用点记法:

```
#check @And.intro
| @And.intro : ∀ {a b : Prop}, a → b → a ∧ b
#check @And.casesOn
| @And.casesOn : {a b : Prop} →
  {motive : a ∧ b → Sort u_1} → (t : a ∧ b) → ((left : a) →
    → (right : b) → motive ...) → motive t
#check @And.left
| @And.left : ∀ {a b : Prop}, a ∧ b → a
#check @And.right
| @And.right : ∀ {a b : Prop}, a ∧ b → b
#check @Or.inl
| @Or.inl : ∀ {a b : Prop}, a → a ∨ b
```

```

#check @Or.inr
| @Or.inr : ∀ {a b : Prop}, b → a ∨ b
#check @Or.elim
| @Or.elim : ∀ {a b c : Prop}, a ∨ b → (a → c) → (b → c) → c
#check @Exists.intro
| @Exists.intro : ∀ {α : Sort u_1} {p : α → Prop} (w : α), p w
  ↪ → Exists p
#check @Exists.elim
| @Exists.elim : ∀ {α : Sort u_1} {p : α → Prop} {b : Prop}, (∃
  ↪ x, p x) → (∀ (a : α), p a → b) → b
#check @Eq.refl
| @Eq.refl : ∀ {α : Sort u_1} (a : α), a = a
#check @Eq.subst
| @Eq.subst : ∀ {α : Sort u_1} {motive : α → Prop} {a b : α}, a
  ↪ = b → motive a → motive b

```

6.12 自动绑定的隐式参数

在上一节中，我们已经展示了隐式参数如何让函数使用起来更方便。然而，像 `compose` 这样的函数在定义时仍然相当冗长。请注意，带宇宙多态的 `compose` 比我们先前定义版本还要更冗长。

```
universe u v w
```

```

def compose {α : Type u} {β : Type v} {γ : Type w}
  (g : β → γ) (f : α → β) (x : α) : γ :=
  g (f x)

```

在定义 `compose` 时显式给出宇宙参数，就可以避免使用 `universe` 命令。

```

def compose.{u, v, w}
  {α : Type u} {β : Type v} {γ : Type w}
  (g : β → γ) (f : α → β) (x : α) : γ :=
  g (f x)

```

Lean 4 支持一项名为自动绑定的隐式参数的新特性。它使得编写 `compose` 这样的函数方便得多。当 Lean 处理一个声明的头部时，任何未绑定的标识符都会自动被加入为隐式参数。有了这一特性，我们可以把 `compose` 写成

```

def compose (g : β → γ) (f : α → β) (x : α) : γ :=
  g (f x)

```

```

#check @compose -- @compose : {β : Sort u_1} → {γ : Sort u_2}
  ↪ → {α : Sort u_3} → (β → γ) → (α → β) → α → γ

```

```
@compose : { $\beta$  : Sort u_1} → { $\gamma$  : Sort u_2} → { $\alpha$  : Sort u_3} →
  ↪ ( $\beta$  →  $\gamma$ ) → ( $\alpha$  →  $\beta$ ) →  $\alpha$  →  $\gamma$ 
```

注意，Lean 在这里推断出了一个更一般的类型，使用的是 `Sort` 而不是 `Type`。

尽管我们非常喜欢这一特性，并且在实现 Lean 时大量使用它，但我们也意识到，有些用户可能会对此感到不适。因此，你可以使用命令 `set_option autoImplicit false` 将其关闭。

```
set_option autoImplicit false
```

```
/--
error: Unknown identifier `β`

Note: It is not possible to treat `β` as an implicitly bound
↪ variable here because the `autoImplicit` option is set to
↪ `false`.
---
error: Unknown identifier `γ`

Note: It is not possible to treat `γ` as an implicitly bound
↪ variable here because the `autoImplicit` option is set to
↪ `false`.
---
error: Unknown identifier `α`

Note: It is not possible to treat `α` as an implicitly bound
↪ variable here because the `autoImplicit` option is set to
↪ `false`.
---
error: Unknown identifier `β`

Note: It is not possible to treat `β` as an implicitly bound
↪ variable here because the `autoImplicit` option is set to
↪ `false`.
---
error: Unknown identifier `α`

Note: It is not possible to treat `α` as an implicitly bound
↪ variable here because the `autoImplicit` option is set to
↪ `false`.
---
error: Unknown identifier `γ`

Note: It is not possible to treat `γ` as an implicitly bound
↪ variable here because the `autoImplicit` option is set to
↪ `false`.
-/
#guard_msgs in
def compose ( $g$  :  $\beta$  →  $\gamma$ ) ( $f$  :  $\alpha$  →  $\beta$ ) ( $x$  :  $\alpha$ ) :  $\gamma$  :=
   $g$  ( $f$   $x$ )
```

6.13 隐式 lambda 抽象

当一个表达式的预期类型是一个尚在等待隐式参数的函数时，精化器会自动引入相应的 lambda。例如，`pure` 的类型表明其第一个参数是一个隐式类型 α ，但 `ReaderT.pure` 的第一个参数却是 `Reader` 单子的上下文类型 ρ 。系统会自动在它外层加上一层 `fun {α} => ...`，从而使精化器能够在函数体中正确填充隐式参数。

```
instance : Monad (ReaderT ρ m) where
  pure := ReaderT.pure
  bind := ReaderT.bind
```

用户可以通过使用 `@`，或者书写带有 `{}` 或 `[]` 绑定标注的 lambda 表达式，来禁用隐式 lambda 特性。下面是几个例子：

```
def id1 : {α : Type} → α → α :=
  fun x => x

def listId : List ({α : Type} → α → α) :=
  (fun x => x) :: []

-- In this example, implicit lambda introduction has been
-- ↪ disabled because
-- we use '@' before {kw}`fun`
def id2 : {α : Type} → α → α :=
  @fun α (x : α) => id1 x

def id3 : {α : Type} → α → α :=
  @fun α x => id1 x

def id4 : {α : Type} → α → α :=
  fun x => id1 x

-- In this example, implicit lambda introduction has been
-- ↪ disabled
-- because we used the binder annotation `{...}`
def id5 : {α : Type} → α → α :=
  fun {α} x => id1 x
```

6.14 简单函数的语法糖

Lean 提供了一种用于描述简单函数的记法，它使用匿名占位符而不是 `fun`。当 `·` 作为某个项的一部分出现时，离它最近的一对外围括号就会变成一个函数，并把 `·` 作为它的参数。如果同一对括号中包含多个占位符，且它们之间没有其他中间括号，那么这些占位符会按从左到右的顺序依次成为参数。下面是一些例子：

```

#check (· + 1) -- fun x => x + 1 : Nat → Nat
| fun x => x + 1 : Nat → Nat
#check (2 - ·) -- fun x => 2 - x : Nat → Nat
| fun x => 2 - x : Nat → Nat
#eval [1, 2, 3, 4, 5].foldl (· * ·) 1 -- 120
| 120
def f (x y z : Nat) :=
  x + y + z

#check (f · 1 ·) -- fun x1 x2 => f x1 1 x2 : Nat → Nat → Nat
| fun x1 x2 => f x1 1 x2 : Nat → Nat → Nat
#eval [(1, 2), (3, 4), (5, 6)].map (·.1) -- [1, 3, 5]
| [1, 3, 5]

```

嵌套括号会引入新的函数。在下面这个例子中，会创建两个不同的 lambda 表达式：

```

#check (Prod.mk · (· + 1)) -- fun x => (x, fun x => x + 1) :
  ↳ ?m.2 → ?m.2 × (Nat → Nat)
| fun x => (x, fun x => x + 1) : ?m.1 → ?m.1 × (Nat → Nat)

```

6.15 命名参数

命名参数让你可以通过参数名而不是参数列表中的位置，来为某个形参指定实参。如果你不记得参数的顺序，但知道它们的名字，就可以按任意顺序传入实参。当 Lean 无法推断某个隐式参数时，你也可以借此显式提供它的值。命名参数还能通过明确每个参数所代表的含义，提升代码的可读性。

```

def sum (xs : List Nat) :=
  xs.foldl (init := 0) (·+·)

#eval sum [1, 2, 3, 4]
| 10
-- 10

example {a b : Nat} {p : Nat → Nat → Nat → Prop}
  (h₁ : p a b b) (h₂ : b = a) :
  p a a b :=
  Eq.subst (motive := fun x => p a x b) h₂ h₁

```

在下面的例子中，我们将说明命名参数与默认参数之间的相互作用。

```

def f (x : Nat) (y : Nat := 1) (w : Nat := 2) (z : Nat) :=
  x + y + w - z

```

```

example (x z : Nat) : f (z := z) x = x + 1 + 2 - z := rfl

example (x z : Nat) : f x (z := z) = x + 1 + 2 - z := rfl

example (x y : Nat) : f x y = fun z => x + y + 2 - z := rfl

example : f = (fun x z => x + 1 + 2 - z) := rfl

example (x : Nat) : f x = fun z => x + 1 + 2 - z := rfl

example (y : Nat) : f (y := 5) = fun x z => x + 5 + 2 - z :=
  ↪ rfl

def g {α} [Add α] (a : α) (b? : Option α := none) (c : α) : α
  ↪ :=
  match b? with
  | none    => a + c
  | some b => a + b + c

variable {α} [Add α]

example : g = fun (a c : α) => a + c := rfl

example (x : α) : g (c := x) = fun (a : α) => a + x := rfl

example (x : α) : g (b? := some x) = fun (a c : α) => a + x +
  ↪ c := rfl

example (x : α) : g x = fun (c : α) => x + c := rfl

example (x y : α) : g x y = fun (c : α) => x + y + c := rfl

```

你可以使用 `..` 把缺失的显式参数统一补成 `_`。这一特性与命名参数结合起来，在编写模式时非常有用。下面是一个例子：

```

inductive Term where
  | var      (name : String)
  | num      (val : Nat)
  | app      (fn : Term) (arg : Term)
  | lambda   (name : String) (type : Term) (body : Term)

def getBinderName : Term → Option String
  | Term.lambda (name := n) .. => some n
  | _           => none

def getBinderType : Term → Option Term
  | Term.lambda (type := t) .. => some t
  | _           => none

```

当显式参数可以由 Lean 自动推断，而我们又希望避免写出一串 `_` 时，省略号同样也很有用。

```
example (f : Nat → Nat) (a b c : Nat) : f (a + b + c) = f (a +  
  ↪ (b + c)) :=  
  congrArg f (Nat.add_assoc ..)
```


7 归纳类型

我们已经看到，Lean 的形式化基础包括基本类型 `Prop`、`Type 0`、`Type 1`、`Type 2`、……，并允许形成依值函数类型 $(x : \alpha) \rightarrow \beta$ 。在例子中，我们还使用了诸如 `Bool`、`Nat` 和 `Int` 这样的其他类型，以及诸如 `List` 和乘积 $\times$ 这样的类型构造子。事实上，在 Lean 的库中，除宇宙之外的每个具体类型，以及除依值箭头之外的每个类型构造子，都是称为归纳类型的一般类型构造族的实例。值得注意的是，仅凭类型宇宙、依值箭头类型和归纳类型，就可以构筑起相当宏大的数学大厦；其余一切都由此而来。

直观地说，归纳类型由一组指定的构造子构成。在 Lean 中，指定这种类型的语法如下：

```
inductive Foo where
| constructor1 : ... → Foo
| constructor2 : ... → Foo
...
| constructorn : ... → Foo
```

直观含义是，每个构造子都指定了一种构造 `Foo` 的新对象的方法，这些对象可能依赖于先前构造出的值。类型 `Foo` 只包含以这种方式构造出的对象。我们将在下文看到，构造子的参数可以包含类型 `Foo` 的对象，但须满足某种“正性”约束：该约束保证 `Foo` 的元素是自底向上构造的。粗略地说，每个 `...` 都可以是由 `Foo` 和先前定义的类型构成的任意箭头类型，其中 `Foo` 即便出现，也只能作为依值箭头类型的“目标”。

我们将给出若干归纳类型的例子。我们还会考察上述模式的轻微推广，包括相互定义的归纳类型，以及所谓的归纳族。

与逻辑联结词一样，每个归纳类型都配有引入规则和消去规则：前者说明如何构造该类型的元素，后者说明如何在另一个构造中“使用”该类型的元素。它们与逻辑联结词的类比并不令人意外：正如下文将看到的，逻辑联结词本身也是归纳类型构造的例子。你已经见过归纳类型的引入规则：它们正是类型定义中指定的构造子。消去规则给出了该类型上的递归原则，而归纳原则也是它的一个特例。

在下一章中，我们将介绍 Lean 的函数定义机制，它提供了更便捷的方法在归纳类型上定义函数并进行归纳证明。不过，由于归纳类型这一概念如此基础，我们认为从底层的、动手式的理解开始非常重要。我们将从一些基本的归纳类型例子出发，逐步推进到更精细、更复杂的例子。

7.1 枚举类型

最简单的归纳类型，是具有有限个、逐一枚举的元素的类型。

```
inductive Weekday where
  | sunday : Weekday
  | monday : Weekday
  | tuesday : Weekday
  | wednesday : Weekday
  | thursday : Weekday
  | friday : Weekday
  | saturday : Weekday
```

inductive 命令创建一个新类型 `Weekday`。所有构造子都位于 `Weekday` 命名空间中。

```
#check Weekday.sunday
| Weekday.sunday : Weekday
#check Weekday.monday
| Weekday.monday : Weekday
open Weekday

#check sunday
| Weekday.sunday : Weekday
#check monday
| Weekday.monday : Weekday
```

在声明 `Weekday` 归纳类型时，可以省略 `: Weekday`。

```
inductive Weekday where
  | sunday
  | monday
  | tuesday
  | wednesday
  | thursday
  | friday
  | saturday
```

可以把 `sunday`、`monday`、……、`saturday` 看成 `Weekday` 的彼此不同的元素，除此之外没有其他区分性质。消去原则 `Weekday.rec` 会与类型 `Weekday` 及其构造子一同定义。它也称为递归子，并且正是它使这个类型成为“归纳”的：它允许我们通过为每个构造子指定相应的值，来定义 `Weekday` 上的函数。直观地说，归纳类型由其构造子穷尽生成，除了这些构造子构造出的元素之外没有其他元素。

```
Weekday.rec.{u} {motive : Weekday → Sort u}
  (sunday : motive Weekday.sunday)
  (monday : motive Weekday.monday)
```

```

(tuesday : motive Weekday.tuesday)
(wednesday : motive Weekday.wednesday)
(thursday : motive Weekday.thursday)
(friday : motive Weekday.friday)
(saturday : motive Weekday.saturday)
(t : Weekday) :
  motive t

```

我们将使用 **match** 表达式来定义一个从 `Weekday` 到自然数的函数：

```
open Weekday
```

```

def numberOfDay (d : Weekday) : Nat :=
  match d with
  | sunday    => 1
  | monday    => 2
  | tuesday   => 3
  | wednesday => 4
  | thursday  => 5
  | friday    => 6
  | saturday  => 7

```

```
#eval numberOfDay Weekday.sunday -- 1
```

```
1
```

```
#eval numberOfDay Weekday.monday -- 2
```

```
2
```

```
#eval numberOfDay Weekday.tuesday -- 3
```

```
3
```

在使用 Lean 的逻辑时，**match** 表达式会通过声明归纳类型时生成的递归子 `Weekday.rec` 编译。这保证所得项在类型论中是良定义的。对于编译后的代码，**match** 则像其他函数式编程语言中那样编译。

```
open Weekday
```

```

def numberOfDay (d : Weekday) : Nat :=
  match d with
  | sunday    => 1
  | monday    => 2
  | tuesday   => 3
  | wednesday => 4
  | thursday  => 5
  | friday    => 6
  | saturday  => 7

```

```
set_option pp.all true
```

```
#print numberOfDay
```

```

def numberOfDay : (d : Weekday) → Nat :=
  fun (d : Weekday) =>
    numberOfDay.match_1.{1} (fun (d : Weekday) => Nat) d
    (fun (_ : Unit) => @OfNat.ofNat.{0} Nat (nat_lit 1)
      ↪ (instOfNatNat (nat_lit 1)))
    (fun (_ : Unit) => @OfNat.ofNat.{0} Nat (nat_lit 2)
      ↪ (instOfNatNat (nat_lit 2)))
    (fun (_ : Unit) => @OfNat.ofNat.{0} Nat (nat_lit 3)
      ↪ (instOfNatNat (nat_lit 3)))
    (fun (_ : Unit) => @OfNat.ofNat.{0} Nat (nat_lit 4)
      ↪ (instOfNatNat (nat_lit 4)))
    (fun (_ : Unit) => @OfNat.ofNat.{0} Nat (nat_lit 5)
      ↪ (instOfNatNat (nat_lit 5)))
    (fun (_ : Unit) => @OfNat.ofNat.{0} Nat (nat_lit 6)
      ↪ (instOfNatNat (nat_lit 6))) fun (_ : Unit) =>
    @OfNat.ofNat.{0} Nat (nat_lit 7) (instOfNatNat (nat_lit
      ↪ 7))

#print numberOfDay.match_1

def numberOfDay.match_1.{u_1} : (motive : Weekday → Sort u_1)
  ↪ →
  (d : Weekday) →
  (h_1 : (a : Unit) → motive Weekday.sunday) →
  (h_2 : (a : Unit) → motive Weekday.monday) →
  (h_3 : (a : Unit) → motive Weekday.tuesday) →
  (h_4 : (a : Unit) → motive Weekday.wednesday) →
  (h_5 : (a : Unit) → motive Weekday.thursday) →
  (h_6 : (a : Unit) → motive Weekday.friday) →
  ↪ (h_7 : (a : Unit) → motive
  ↪ Weekday.saturday) → motive d :=
  fun (motive : Weekday → Sort u_1) (d : Weekday) (h_1 : (a :
  ↪ Unit) → motive Weekday.sunday)
  (h_2 : (a : Unit) → motive Weekday.monday) (h_3 : (a :
  ↪ Unit) → motive Weekday.tuesday)
  (h_4 : (a : Unit) → motive Weekday.wednesday) (h_5 : (a :
  ↪ Unit) → motive Weekday.thursday)
  (h_6 : (a : Unit) → motive Weekday.friday) (h_7 : (a :
  ↪ Unit) → motive Weekday.saturday) =>
  @Weekday.casesOn.{u_1} (fun (x : Weekday) => motive x) d
  ↪ (h_1 Unit.unit) (h_2 Unit.unit) (h_3 Unit.unit)
  (h_4 Unit.unit) (h_5 Unit.unit) (h_6 Unit.unit) (h_7
  ↪ Unit.unit)

#print Weekday.casesOn

@[reducible] def Weekday.casesOn.{u} : {motive : (t :
  ↪ Weekday) → Sort u} →
  (t : Weekday) →
  (sunday : motive Weekday.sunday) →
  (monday : motive Weekday.monday) →
  (tuesday : motive Weekday.tuesday) →

```

```

        (wednesday : motive Weekday.wednesday) →
        (thursday : motive Weekday.thursday) →
        (friday : motive Weekday.friday) → (saturday :
        ↪ motive Weekday.saturday) → motive t :=
fun {motive : (t : Weekday) → Sort u} (t : Weekday) (sunday :
    ↪ motive Weekday.sunday) (monday : motive Weekday.monday)
    (tuesday : motive Weekday.tuesday) (wednesday : motive
    ↪ Weekday.wednesday) (thursday : motive
    ↪ Weekday.thursday)
    (friday : motive Weekday.friday) (saturday : motive
    ↪ Weekday.saturday) =>
    @Weekday.rec.{u} motive sunday monday tuesday wednesday
    ↪ thursday friday saturday t
#check @Weekday.rec
@Weekday.rec.{u_1} : {motive : (t : Weekday) → Sort u_1} →
    (sunday : motive Weekday.sunday) →
    (monday : motive Weekday.monday) →
    (tuesday : motive Weekday.tuesday) →
    (wednesday : motive Weekday.wednesday) →
    (thursday : motive Weekday.thursday) →
    (friday : motive Weekday.friday) → (saturday :
    ↪ motive Weekday.saturday) → (t : Weekday) →
    ↪ motive t

```

声明归纳数据类型时，可以使用 **deriving Repr** 指示 Lean 生成一个函数，把 **Weekday** 对象转换为文本。**#eval** 命令使用这个函数来显示 **Weekday** 对象。如果不存在 **Repr**，**#eval** 会尝试当场派生一个。

```
inductive Weekday where
```

```

| sunday
| monday
| tuesday
| wednesday
| thursday
| friday
| saturday

```

```
deriving Repr
```

```
open Weekday
```

```
#eval tuesday -- Weekday.tuesday
```

```
Weekday.tuesday
```

把与某个结构相关的定义和定理放入同名命名空间中通常很有用。例如，我们可以把 **numberOfDay** 函数放在 **Weekday** 命名空间中。这样，当打开该命名空间后，就可以使用较短的名称。

我们可以定义从 **Weekday** 到 **Weekday** 的函数：

```

namespace Weekday
def next (d : Weekday) : Weekday :=
  match d with
  | sunday    => monday
  | monday    => tuesday
  | tuesday   => wednesday
  | wednesday => thursday
  | thursday  => friday
  | friday    => saturday
  | saturday  => sunday

def previous (d : Weekday) : Weekday :=
  match d with
  | sunday    => saturday
  | monday    => sunday
  | tuesday   => monday
  | wednesday => tuesday
  | thursday  => wednesday
  | friday    => thursday
  | saturday  => friday

#eval next (next tuesday)      -- Weekday.thursday
| Weekday.thursday
#eval next (previous tuesday)  -- Weekday.tuesday
| Weekday.tuesday
example : next (previous tuesday) = tuesday :=
  rfl

end Weekday

```

如何证明对任意 `Weekday d` 都有 `next (previous d) = d` 这一定理？
 可以使用 `match` 为每个构造子分别给出该断言的证明：

```

theorem next_previous (d : Weekday) : next (previous d) = d :=
  match d with
  | sunday    => rfl
  | monday    => rfl
  | tuesday   => rfl
  | wednesday => rfl
  | thursday  => rfl
  | friday    => rfl
  | saturday  => rfl

```

使用策略证明时，我们可以更加简洁：

```

theorem next_previous (d : Weekday) : next (previous d) = d :=
  ↪ by
    cases d <|> rfl

```

下文的 section 7.6 (p. 128) 将介绍更多专门用于利用归纳类型的策略。

注意，在 `propositions-as-types` 对应之下，我们既可以用 `match` 来定义函数，也可以用它来证明定理。换言之，在 `propositions-as-types` 对应之下，分情况证明就是一种分情况定义，只不过被“定义”的是一个证明，而不是一段数据。

Lean 库中的 `Bool` 类型就是枚举类型的一个实例。

```
inductive Bool where
| false : Bool
| true  : Bool
```

(为了运行这些例子，我们把它们放在名为 `Hidden` 的命名空间中，使得像 `Bool` 这样的名称不会与标准库中的 `Bool` 冲突。这是必要的，因为这些类型是 Lean “prelude”的一部分，会在系统启动时自动导入。)

作为练习，你应该思考这些类型的引入规则和消去规则分别做什么。进一步地，我们建议在 `Bool` 类型上定义布尔运算 `and`、`or`、`not`，并验证常见恒等式。注意，可以用 `match` 定义像 `and` 这样的二元运算：

```
def and (a b : Bool) : Bool :=
  match a with
  | true  => b
  | false => false
```

类似地，大多数恒等式都可以通过引入合适的 `match`，然后使用 `rfl` 来证明。

7.2 带参数的构造子

枚举类型是归纳类型的一个非常特殊的情形，其中构造子完全不接受参数。一般而言，一个“构造”可以依赖于数据，而这些数据随后体现在被构造出的参数中。考虑库中乘积类型和和类型的定义：

```
inductive Prod (α : Type u) (β : Type v)
| mk : α → β → Prod α β
```

```
inductive Sum (α : Type u) (β : Type v) where
| inl : α → Sum α β
| inr : β → Sum α β
```

考察这些例子中发生了什么。乘积类型有一个构造子 `Prod.mk`，它接受两个参数。要在 `Prod α β` 上定义函数，可以假设输入形如 `Prod.mk a b`，并且必须用 `a` 和 `b` 来指定输出。我们可以利用这一点为 `Prod` 定义两个投影。请记住，标准库把 `Prod α β` 记作 $\alpha \times \beta$ ，把 `Prod.mk a b` 记作 (a, b) 。

```
def fst {α : Type u} {β : Type v} (p : Prod α β) : α :=
  match p with
  | Prod.mk a b => a
```

```
def snd {α : Type u} {β : Type v} (p : Prod α β) : β :=
  match p with
  | Prod.mk a b => b
```

函数 `fst` 接受一个对 p 。`match` 将 p 解释为一个对 `Prod.mk a b`。还请回忆 chapter 2 (p. 5) 中的内容：为了使这些定义尽可能一般，我们允许类型 α 和 β 属于任意宇宙。

下面是另一个例子，其中我们使用递归子 `Prod.casesOn`，而不是 `match`。

```
def prod_example (p : Bool × Nat) : Nat :=
  Prod.casesOn (motive := fun _ => Nat) p
    (fun b n => cond b (2 * n) (2 * n + 1))
```

```
#eval prod_example (true, 3)
```

```
6
```

```
#eval prod_example (false, 3)
```

```
7
```

参数 `motive` 用来指定你想构造的对象的类型；它是一个函数，因为该类型可能依赖于这个对。函数 `cond` 是布尔条件表达式：如果 b 为真，`cond b t1 t2` 返回 $t1$ ，否则返回 $t2$ 。函数 `prod_example` 接受一个由布尔值 b 和数 n 组成的对，并根据 b 为真还是为假，返回 $2 * n$ 或 $2 * n + 1$ 。

相比之下，和类型有两个构造子 `inl` 和 `inr`（分别表示“插入左边”和“插入右边”），每个构造子接受一个（显式）参数。要在 `Sum  $\alpha$   $\beta$` 上定义函数，我们必须处理两种情况：输入要么形如 `inl a`，此时必须用 a 指定输出值；要么形如 `inr b`，此时必须用 b 指定输出值。

```
def sum_example (s : Sum Nat Nat) : Nat :=
  Sum.casesOn (motive := fun _ => Nat) s
    (fun n => 2 * n)
    (fun n => 2 * n + 1)
```

```
#eval sum_example (Sum.inl 3)
```

```
6
```

```
#eval sum_example (Sum.inr 3)
```

```
7
```

这个例子与前一个类似，但现在 `sum_example` 的输入隐含地要么形如 `inl n`，要么形如 `inr n`。在第一种情况下，函数返回 $2 * n$ ；在第二种情况下，它返回 $2 * n + 1$ 。

注意，乘积类型依赖于参数 $\alpha \beta : \text{Type}$ ；这些参数既是构造子的参数，也是 `Prod` 的参数。当 `Lean` 检测到这些参数可以从构造子的后续参数或返回类型中推断出来时，就会把它们设为隐式参数。在 section 7.4 (p. 123) 中，我们将看到当归纳类型的构造子接受来自该归纳类型自身的参数时会发生什么。本节所考察例子的特征在于：每个构造子只依赖于先前指定的类型。注意，具有多个构造子的类型具有析取性：`Sum  $\alpha$   $\beta$` 的元素要么形如 `inl a`，要么形如 `inl b`。具有多个参数的构造子引入合取性信息：从 `Prod  $\alpha$   $\beta$` 的元素 `Prod.mk a b` 中，我们可以同时提取 a 和 b 。任意归纳类型都可以同时包含这两种特征：它可以有任意多个构造子，而每个构造子又可以接受任意多个参数。

与函数定义一样，`Lean` 的归纳定义语法允许你把构造子的具名参数放在冒号之前：


```
inductive Prod ( $\alpha$  : Type u) ( $\beta$  : Type v) where
  | mk (fst :  $\alpha$ ) (snd :  $\beta$ ) : Prod  $\alpha$   $\beta$ 
```

```
inductive Sum ( $\alpha$  : Type u) ( $\beta$  : Type v) where
  | inl (a :  $\alpha$ ) : Sum  $\alpha$   $\beta$ 
  | inr (b :  $\beta$ ) : Sum  $\alpha$   $\beta$ 
```

这些定义得到的结果与本节前面给出的定义本质上相同。

像 **Prod** 这样只有一个构造子的类型是纯粹合取性的：构造子只是把参数列表打包成一份数据，本质上是一个元组，其中后续参数的类型可以依赖于初始参数的类型。我们也可以把这种类型看作一个“记录”或“结构”。在 **Lean** 中，关键字 **structure** 可以用来同时定义这样的归纳类型及其投影。

```
structure Prod ( $\alpha$  : Type u) ( $\beta$  : Type v) where
  mk ::
  fst :  $\alpha$ 
  snd :  $\beta$ 
```

这个例子同时引入了归纳类型 **Prod**、它的构造子 **mk**、通常的消去器（**rec** 和 **recOn**），以及上面定义的投影 **fst** 和 **snd**。

如果没有给构造子命名，**Lean** 默认使用 **mk**。例如，下面的定义把颜色存储为 RGB 值的三元组记录：

```
structure Color where
  red : Nat
  green : Nat
  blue : Nat
deriving Repr
```

```
def yellow := Color.mk 255 255 0
```

```
#eval Color.red yellow
```

```
255
```

yellow 的定义用所示的三个值构造了这个记录，而投影 **Color.red** 返回红色分量。

structure 命令对于定义代数结构尤其有用，**Lean** 也提供了大量基础设施来支持对它们的处理。例如，下面是半群的定义：

```
structure Semigroup where
  carrier : Type u
  mul : carrier → carrier → carrier
  mul_assoc :  $\forall a b c, mul (mul a b) c = mul a (mul b c)$ 
```

我们将在 **chapter 9** (p. 169) 一章中看到更多例子。

我们已经讨论过依值积类型 **Sigma**：

```
inductive Sigma { $\alpha$  : Type u} ( $\beta$  :  $\alpha \rightarrow$  Type v) where
  | mk : (a :  $\alpha$ ) →  $\beta$  a → Sigma  $\beta$ 
```

库中另外两个归纳类型的例子如下：

```
inductive Option ( $\alpha$  : Type u) where
  | none : Option  $\alpha$ 
  | some :  $\alpha \rightarrow$  Option  $\alpha$ 
```

```
inductive Inhabited ( $\alpha$  : Type u) where
  | mk :  $\alpha \rightarrow$  Inhabited  $\alpha$ 
```

在依值类型论的语义中，没有内建的偏函数概念。函数类型 $\alpha \rightarrow \beta$ 或依值函数类型 $(a : \alpha) \rightarrow \beta$ 的每个元素，都被假定在每个输入上有值。Option 类型提供了一种表示偏函数的方法。Option β 的元素要么是 none，要么形如 some b ，其中 $b : \beta$ 。因此，我们可以把类型 $\alpha \rightarrow$ Option β 的元素 f 看作从 α 到 β 的偏函数：对每个 $a : \alpha$ ， $f a$ 要么返回 none，表示 $f a$ “未定义”，要么返回 some b 。Inhabited α 的元素只是一个见证，表明 α 中存在元素。稍后我们会看到，Inhabited 是 Lean 中类型类的一个例子：可以告知 Lean 某些合适的基本类型是有元素的，Lean 随后可以据此自动推断其他构造出的类型也是有元素的。作为练习，我们鼓励你为从 α 到 β 以及从 β 到 γ 的偏函数发展出复合的概念，并证明它具有预期的行为。我们还鼓励你证明 Bool 和 Nat 是有元素的，两个有元素类型的乘积也是有元素的，而以有元素类型为值域的函数类型也是有元素的。

7.3 归纳定义的命题

归纳定义的类型可以位于任意类型宇宙中，包括最底层的 Prop。事实上，逻辑联结词正是这样定义的。

```
inductive False : Prop
```

```
inductive True : Prop where
  | intro : True
```

```
inductive And ( $a b$  : Prop) : Prop where
  | intro :  $a \rightarrow b \rightarrow$  And  $a b$ 
```

```
inductive Or ( $a b$  : Prop) : Prop where
  | inl :  $a \rightarrow$  Or  $a b$ 
  | inr :  $b \rightarrow$  Or  $a b$ 
```

你应该思考这些定义如何产生你已经见过的引入规则和消去规则。有一些规则支配着归纳类型的消去器可以消去到什么，也就是说，哪些类型可以作为递归子的目标。粗略地说，Prop 中归纳类型的特征在于：它通常只能消去到 Prop 中的其他类型。这与如下理解是一致的：如果 $p : \text{Prop}$ ，那么元素 $hp : p$ 不携带数据。不过，这条规则有一个小例外，我们将在下文 section 7.7 (p. 133) 中讨论。甚至存在量词也是归纳定义的：

```
inductive Exists { $\alpha$  : Sort u} ( $p$  :  $\alpha \rightarrow$  Prop) : Prop where
  | intro ( $w : \alpha$ ) ( $h : p w$ ) : Exists  $p$ 
```

请记住，记号 $\exists x : \alpha, p$ 是 Exists ($\text{fun } x : \alpha \Rightarrow p$) 的语法糖。False、True、And 和 Or 的定义与 Empty、Unit、Prod 和 Sum 的定义完全类似。区别在于，前一组给出 Prop 的元素，而后一组给出 Type u 的元素（对某个 u）。类似地， $\exists x : \alpha, p$ 是 $\Sigma x : \alpha, \beta$ 的 Prop 值变体。

这里适合提及另一种归纳类型，记作 $\{x : \alpha // p\}$ ，它有些像 $\exists x : \alpha$ ， p 与 $\Sigma x : \alpha, \beta$ 的混合体。

```
inductive Subtype { $\alpha$  : Type u} ( $p$  :  $\alpha \rightarrow \text{Prop}$ ) where
  | mk : ( $x$  :  $\alpha$ )  $\rightarrow p$   $x \rightarrow$  Subtype  $p$ 
```

事实上，在 Lean 中，Subtype 是用结构命令定义的：

```
structure Subtype { $\alpha$  : Sort u} ( $p$  :  $\alpha \rightarrow \text{Prop}$ ) where
  val :  $\alpha$ 
  property :  $p$  val
```

记号 $\{x : \alpha // p\}$ 是 Subtype (fun x : $\alpha \Rightarrow p\ x$) 的语法糖。它模仿集合论中的子集记号：其含义是， $\{x : \alpha // p\}$ 表示 α 中具有性质 p 的元素所组成的集合。

7.4 定义自然数

到目前为止，我们见到的归纳定义类型都是“扁平的”：构造子包装数据并将其插入某个类型中，相应的递归子则解包这些数据并作用于它们。当构造子作用于正在定义的类型自身的元素时，情形会有趣得多。一个典型例子是自然数类型 Nat：

```
inductive Nat where
  | zero : Nat
  | succ : Nat  $\rightarrow$  Nat
```

这里有两个构造子。我们从 $\text{zero} : \text{Nat}$ 开始：它不接受参数，所以一开始就拥有它。相比之下，构造子 succ 只能应用于先前构造出的 Nat 。把它应用于 zero 得到 $\text{succ zero} : \text{Nat}$ 。再次应用则得到 $\text{succ (succ zero)} : \text{Nat}$ ，依此类推。直观地说， Nat 是具有这些构造子的“最小”类型，也就是说，它从 zero 出发、反复应用 succ 而被穷尽地（且自由地）生成。和前面一样， Nat 的递归子旨在定义从 Nat 到任意论域的依值函数 f ，也就是对某个 $\text{motive} : \text{Nat} \rightarrow \text{Sort } u$ 而言，类型 $(n : \text{Nat}) \rightarrow \text{motive } n$ 的元素 f 。它必须处理两种情况：输入为 zero 的情况，以及输入形如 $\text{succ } n$ 、其中 $n : \text{Nat}$ 的情况。在第一种情况下，我们像以前一样，只需指定一个具有适当类型的目标值。然而在第二种情况下，递归子可以假设 f 在 n 处的值已经计算出来。因此，递归子的下一个参数会根据 n 和 $f\ n$ 来指定 $f\ (\text{succ } n)$ 的值。如果检查递归子的类型，会看到如下内容：

```
Nat.rec.{u} :
  {motive : Nat  $\rightarrow$  Sort u}  $\rightarrow$ 
  (zero : motive Nat.zero)  $\rightarrow$ 
  (succ : ( $n$  : Nat)  $\rightarrow$  motive  $n \rightarrow$  motive (Nat.succ  $n$ ))  $\rightarrow$ 
  ( $t$  : Nat)  $\rightarrow$  motive  $t$ 
```

隐式参数 motive 是正在定义的函数的余定义域。在类型论中，通常说 motive 是消去/递归的动机，因为它描述了我们希望构造的对象种类。接下来的两个参数指定如何计算零和后继两种情况，如上所述。它们也称为小前提。最后， $t : \text{Nat}$ 是该函数的输入。它也称为大前提。

Nat.recOn 类似于 Nat.rec ，但大前提出现在小前提之前。

```
Nat.recOn.{u} :
  {motive : Nat  $\rightarrow$  Sort u}  $\rightarrow$ 
  ( $t$  : Nat)  $\rightarrow$ 
```

```
(zero : motive Nat.zero) →
(succ : ((n : Nat) → motive n → motive (Nat.succ n))) →
motive t
```

例如，考虑自然数上的加法函数 `add m n`。固定 m 后，我们可以对 n 递归地定义加法。在基本情形中，把 `add m zero` 设为 m 。在后继步骤中，假设 `add m n` 的值已经确定，我们把 `add m (succ n)` 定义为 `succ (add m n)`。

```
inductive Nat where
  | zero : Nat
  | succ : Nat → Nat
deriving Repr
```

```
def add (m n : Nat) : Nat :=
  match n with
  | Nat.zero    => m
  | Nat.succ n  => Nat.succ (add m n)
```

```
open Nat
```

```
#eval add (succ (succ zero)) (succ zero)
Hidden.Nat.succ (Hidden.Nat.succ (Hidden.Nat.succ
  ↳ (Hidden.Nat.zero)))
```

把这样的定义放入命名空间 `Nat` 中很有用。随后可以在该命名空间中定义熟悉的记号。此时，加法的两个定义方程按定义成立：

```
namespace Nat
```

```
def add (m n : Nat) : Nat :=
  match n with
  | Nat.zero    => m
  | Nat.succ n  => Nat.succ (add m n)
```

```
instance : Add Nat where
  add := add
```

```
theorem add_zero (m : Nat) : m + zero = m := rfl
```

```
theorem add_succ (m n : Nat) : m + succ n = succ (m + n) :=
  ↳ rfl
```

```
end Nat
```

我们将在 chapter 10 (p. 175) 一章中解释 `instance` 命令如何工作。在下面的例子中，我们将使用 Lean 自带的自然数版本。

然而，证明像 $0 + n = n$ 这样的事实需要归纳证明。正如上面所观察到的，当余定义域 `motive n` 是 `Prop` 的元素时，归纳原则只是递归原则的一个特例。它体现了熟悉的归纳证明模式：要证明 $\forall n, \text{motive } n$ ，先证明 `motive 0`；然后对任意 n ，假设 $ih : \text{motive } n$ ，并证明 `motive (n + 1)`。

```
open Nat
```

```

theorem zero_add (n : Nat) : 0 + n = n :=
  Nat.recOn (motive := fun x => 0 + x = x)
    n
    (show 0 + 0 = 0 from rfl)
    (fun (n : Nat) (ih : 0 + n = n) =>
      show 0 + (n + 1) = n + 1 from
        calc 0 + (n + 1)
          _ = (0 + n) + 1 := rfl
          _ = n + 1      := by rw [ih])

```

再次注意，当 `Nat.recOn` 用在证明语境中时，它实际上就是伪装起来的归纳原则。`rw` 和 `simp` 策略在这类证明中通常非常有效。在这个例子中，二者都可以用来把证明化简为：

```
open Nat
```

```

theorem zero_add (n : Nat) : 0 + n = n :=
  Nat.recOn (motive := fun x => 0 + x = x) n
    rfl
    (fun n ih => by simp [ih])

```

再举一例，让我们证明加法的结合律： $\forall m\ n\ k, m + n + k = m + (n + k)$ 。（按照我们的定义，记号 $+$ 左结合，所以 $m + n + k$ 实际上是 $(m + n) + k$ 。）最困难的部分是弄清楚应该对哪个变量做归纳。由于加法是按第二个参数递归定义的， k 是一个不错的猜测；一旦作出这个选择，证明几乎会自行写出：

```
open Nat
```

```

theorem add_assoc (m n k : Nat) : m + n + k = m + (n + k) :=
  Nat.recOn (motive := fun k => m + n + k = m + (n + k)) k
    (show m + n + 0 = m + (n + 0) from rfl)
    (fun k (ih : m + n + k = m + (n + k)) =>
      show m + n + (k + 1) = m + (n + (k + 1)) from
        calc m + n + (k + 1)
          _ = (m + n + k) + 1 := rfl
          _ = (m + (n + k)) + 1 := by rw [ih]
          _ = m + ((n + k) + 1) := rfl
          _ = m + (n + (k + 1)) := rfl)

```

同样地，可以把证明化简为：

```
open Nat
```

```

theorem add_assoc (m n k : Nat) : m + n + k = m + (n + k) :=
  Nat.recOn (motive := fun k => m + n + k = m + (n + k)) k
    rfl
    (fun k ih => by simp [add_succ (m + n) k, ih]; rfl)

```

假设我们尝试证明加法的交换律。若选择对第二个参数做归纳，可以从如下证明开始：

```
open Nat
```

```

theorem add_comm (m n : Nat) : m + n = n + m :=
  Nat.recOn (motive := fun x => m + x = x + m) n
    (show m + 0 = 0 + m by rw [Nat.zero_add, Nat.add_zero])

```

```

(fun (n : Nat) (ih : m + n = n + m) =>
  show m + succ n = succ n + m from
  calc m + succ n
    _ = succ (m + n) := rfl
    _ = succ (n + m) := by rw [ih]
    _ = succ n + m   := sorry)

```

此时我们看到，还需要另一个辅助事实，即 $\text{succ } (n + m) = \text{succ } n + m$ 。可以对 m 做归纳来证明它：

```
open Nat
```

```

theorem succ_add (n m : Nat) : succ n + m = succ (n + m) :=
  Nat.recOn (motive := fun x => succ n + x = succ (n + x)) m
  (show succ n + 0 = succ (n + 0) from rfl)
  (fun (m : Nat) (ih : succ n + m = succ (n + m)) =>
    show succ n + succ m = succ (n + succ m) from
    calc succ n + succ m
      _ = succ (succ n + m)   := rfl
      _ = succ (succ (n + m)) := by rw [ih]
      _ = succ (n + succ m)   := rfl)

```

然后可以用 `succ_add` 替换前一个证明中的 `sorry`。证明仍然可以进一步压缩：

```
open Nat
```

```

theorem succ_add (n m : Nat) : succ n + m = succ (n + m) :=
  Nat.recOn (motive := fun x => succ n + x = succ (n + x)) m
  rfl
  (fun m ih => by simp [add_succ (succ n)])

```

```

theorem add_comm (m n : Nat) : m + n = n + m :=
  Nat.recOn (motive := fun x => m + x = x + m) n
  (by simp [add_zero, zero_add])
  (fun m ih => by simp_all [succ_add, add_succ])

```

7.5 其他递归数据类型

让我们再考察一些归纳定义类型的例子。对于任意类型 α ，库中定义了由 α 的元素组成的列表类型 `List  $\alpha$` 。

```

inductive List ( $\alpha$  : Type u) where
  | nil   : List  $\alpha$ 
  | cons (h :  $\alpha$ ) (t : List  $\alpha$ ) : List  $\alpha$ 

```

```
namespace List
```

```

def append (as bs : List  $\alpha$ ) : List  $\alpha$  :=
  match as with

```

```

| nil      => bs
| cons a as => cons a (append as bs)

```

```

theorem nil_append (as : List  $\alpha$ ) : append nil as = as :=
  rfl

```

```

theorem cons_append (a :  $\alpha$ ) (as bs : List  $\alpha$ ) :
  append (cons a as) bs = cons a (append as bs) :=
  rfl

```

end List

类型 α 的元素列表要么是空列表 `nil`，要么是一个元素 $h : \alpha$ 后接一个列表 $t : \text{List } \alpha$ 。第一个元素 h 通常称为列表的“头”，其余部分 t 称为“尾”。

作为练习，请证明以下命题：

```

theorem append_nil (as : List  $\alpha$ ) :
  append as nil = as :=
  sorry

```

```

theorem append_assoc (as bs cs : List  $\alpha$ ) :
  append (append as bs) cs = append as (append bs cs) :=
  sorry

```

也请尝试定义函数 `length : { $\alpha$  : Type u} → List  $\alpha$  → Nat` 来返回列表长度，并证明它具有预期的行为（例如，`length (append as bs) = length as + length bs`）。

作为另一个例子，我们可以定义二叉树类型：

```

inductive BinaryTree where
| leaf : BinaryTree
| node : BinaryTree → BinaryTree → BinaryTree

```

事实上，我们甚至可以定义可数分支树的类型：

```

inductive CBTree where
| leaf : CBTree
| sup : (Nat → CBTree) → CBTree

```

namespace CBTree

```

def succ (t : CBTree) : CBTree :=
  sup (fun _ => t)

```

```

def toCBTree : Nat → CBTree
| 0 => leaf
| n+1 => succ (toCBTree n)

```

```

def omega : CBTree :=
  sup toCBTree

```

end CBTree

7.6 归纳类型的策略

鉴于归纳类型在 **Lean** 中的基础重要性，存在若干专门为有效处理它们而设计的策略并不令人惊讶。这里我们介绍其中一些。

cases 策略作用于归纳定义类型的元素，并且顾名思义：它按照所有可能的构造子分解该元素。在最基本的形式中，它应用于局部上下文中的元素 x 。随后，它把目标化为若干情形，其中 x 分别被每一种构造替换。

```
example (p : Nat → Prop)
  (hz : p 0) (hs : ∀ n, p (Nat.succ n)) :
  ∀ n, p n := by
intro n
cases n
. exact hz
. apply hs
```

在第一个分支中，证明状态为：

```
case zero
p : Nat → Prop
hz : p 0
hs : ∀ (n : Nat), p n.succ
⊢ p 0
```

在第二个分支中，则为：

```
case succ
p : Nat → Prop
hz : p 0
hs : ∀ (n : Nat), p n.succ
n+ : Nat
⊢ p (n+ + 1)
```

它还有一些额外功能。首先，**cases** 允许你用 **with** 子句为每个分支选择名称。例如，在下一个例子中，我们为 **succ** 的参数选择名称 m ，使得第二种情形指的是 **succ** m 。更重要的是，**cases** 策略会检测局部上下文中依赖目标变量的项。它会还原这些元素，进行分情况讨论，然后再重新引入它们。在下面的例子中，请注意假设 $h : n \neq 0$ 在第一个分支中变为 $h : 0 \neq 0$ ，在第二个分支中变为 $h : m + 1 \neq 0$ 。

open Nat

```
example (n : Nat) (h : n ≠ 0) : succ (pred n) = n := by
cases n with
| zero =>
  apply absurd rfl h
| succ m =>
  rfl
```


注意，**cases** 既可以用来产生数据，也可以用来证明命题。

```
def f (n : Nat) : Nat := by
  cases n; exact 3; exact 7
```

```
example : f 0 = 3 := rfl
```

```
example : f 5 = 7 := rfl
```

同样，**cases** 会还原、拆分，然后重新引入上下文中的依赖项。

```
def Tuple (α : Type) (n : Nat) :=
  { as : List α // as.length = n }
```

```
def f {n : Nat} (t : Tuple α n) : Nat := by
  cases n; exact 3; exact 7
```

```
def myTuple : Tuple Nat 3 :=
  ([0, 1, 2], rfl)
```

```
example : f myTuple = 7 :=
  rfl
```

下面是一个含有多个带参数构造子的例子。

```
inductive Foo where
  | bar1 : Nat → Nat → Foo
  | bar2 : Nat → Nat → Nat → Foo
```

```
def silly (x : Foo) : Nat := by
  cases x with
  | bar1 a b => exact b
  | bar2 c d e => exact e
```

各个构造子对应的分支不需要按照构造子声明的顺序来解决。

```
def silly (x : Foo) : Nat := by
  cases x with
  | bar2 c d e => exact e
  | bar1 a b => exact b
```

with 的语法便于书写结构化证明。Lean 还提供了互补的 **case** 策略，使你能够聚焦于目标并指定变量名。

```
def silly (x : Foo) : Nat := by
  cases x
  case bar1 a b => exact b
  case bar2 c d e => exact e
```

case 策略很聪明，因为它会把构造子与相应目标匹配起来。例如，我们可以按相反顺序填充上面的目标：

```
def silly (x : Foo) : Nat := by
  cases x
  case bar2 c d e => exact e
  case bar1 a b => exact b
```

也可以对任意表达式使用 **cases**。假设该表达式出现在目标中，**cases** 策略会对该表达式进行泛化，引入得到的全称量化变量，并对它做分情况讨论。

```
open Nat
```

```
example (p : Nat → Prop) (hz : p 0) (hs : ∀ n, p (succ n)) (m
↪ k : Nat)
  : p (m + 3 * k) := by
  cases m + 3 * k
  exact hz -- goal is p 0
  apply hs -- goal is a : Nat ⊢ p (succ a)
```

可以把这理解为“按 $m + 3 * k$ 是零还是某个数的后继来分情况讨论”。其结果在功能上等价于以下证明：

```
open Nat
```

```
example (p : Nat → Prop) (hz : p 0) (hs : ∀ n, p (succ n)) (m
↪ k : Nat)
  : p (m + 3 * k) := by
  generalize m + 3 * k = n
  cases n
  exact hz
  apply hs
```

注意，表达式 $m + 3 * k$ 会被 **generalize** 抹去；重要的只是它是形如 0 ，还是形如 $n+1$ 。这种形式的 **cases** 不会还原那些同样提及该等式中表达式（本例中为 $m + 3 * k$ ）的假设。如果这样的项出现在某个假设中，而你也想对它进行泛化，就需要显式地 **revert** 它。

如果要分情况讨论的表达式没有出现在目标中，**cases** 策略会使用 **have** 把该表达式的类型放入上下文。下面是一个例子：

```
example (p : Prop) (m n : Nat)
  (h1 : m < n → p) (h2 : m ≥ n → p) : p := by
  cases Nat.lt_or_ge m n
  case inl hlt => exact h1 hlt
  case inr hge => exact h2 hge
```

定理 `Nat.lt_or_ge m n` 说明 $m < n \vee m \geq n$ ，因此很自然地把上面的证明看作在这两种情形上拆分。在第一个分支中，我们有假设 $hlt : m < n$ ；在第二个分支中，我们有假设 $hge : m \geq n$ 。上面的证明在功能上等价于如下证明：

```
example (p : Prop) (m n : Nat)
  (h1 : m < n → p) (h2 : m ≥ n → p) : p := by
```

```

have h : m < n ∨ m ≥ n := Nat.lt_or_ge m n
cases h
case inl hlt => exact h₁ hlt
case inr hge => exact h₂ hge

```

前两行之后，我们得到假设 $h : m < n \vee m \geq n$ ，然后只需对它做分情况讨论。

下面是另一个例子，其中我们使用自然数相等性的可判定性，把证明拆分为 $m = n$ 和 $m \neq n$ 两种情况。

```
#check Nat.sub_self
```

```
! Nat.sub_self (n : Nat) : n - n = 0
```

```

example (m n : Nat) : m - n = 0 ∨ m ≠ n := by
  cases Decidable.em (m = n) with
  | inl heq => rw [heq]; apply Or.inl; exact Nat.sub_self n
  | inr hne => apply Or.inr; exact hne

```

请记住，如果你 `open Classical`，就可以对任意命题使用排中律。但是借助类型类推断（见 chapter 10 (p. 175)），Lean 实际上可以找到相关的判定过程，这意味着你可以在可计算函数中使用这种分情况拆分。

正如 `cases` 策略可以用于进行分情况证明，`induction` 策略也可以用于进行归纳证明。其语法类似于 `cases`，区别在于参数只能是局部上下文中的项。下面是一个例子：

```

theorem zero_add (n : Nat) : 0 + n = n := by
  induction n with
  | zero => rfl
  | succ n ih => rw [Nat.add_succ, ih]

```

与 `cases` 一样，我们可以使用 `case` 策略来替代 `with`。

```

theorem zero_add (n : Nat) : 0 + n = n := by
  induction n
  case zero => rfl
  case succ n ih => rw [Nat.add_succ, ih]

```

下面是一些额外的例子：

```
open Nat
```

```

theorem zero_add (n : Nat) : 0 + n = n := by
  induction n <=> simp [*, add_zero, add_succ]

```

```

theorem succ_add (m n : Nat) : succ m + n = succ (m + n) := by
  induction n <=> simp [*, add_zero, add_succ]

```

```

theorem add_comm (m n : Nat) : m + n = n + m := by
  induction n <=> simp [*, add_zero, add_succ, succ_add,
    ↪ zero_add]

```

```

theorem add_assoc (m n k : Nat) : m + n + k = m + (n + k) :=
  ↪ by
  induction k <=> simp [*, add_zero, add_succ]

```

induction 策略还支持具有多个目标（也称为大前提）的用户自定义归纳原则。此例使用 `Nat.mod.inductionOn`，它具有如下签名：

```
Nat.mod.inductionOn
  {motive : Nat → Nat → Sort u}
  (x y : Nat)
  (ind : ∀ x y, 0 < y ∧ y ≤ x → motive (x - y) y → motive x
    ↪ y)
  (base : ∀ x y, ¬(0 < y ∧ y ≤ x) → motive x y) :
  motive x y
example (x : Nat) {y : Nat} (h : y > 0) : x % y < y := by
induction x, y using Nat.mod.inductionOn with
| ind x y h₁ ih =>
  rw [Nat.mod_eq_sub_mod h₁.2]
  exact ih h
| base x y h₁ =>
  have : ¬ 0 < y ∨ ¬ y ≤ x := Iff.mp
    ↪ (Decidable.not_and_iff_or_not ..) h₁
  match this with
| Or.inl h₁ => exact absurd h h₁
| Or.inr h₁ =>
  have hgt : y > x := Nat.gt_of_not_le h₁
  rw [← Nat.mod_eq_of_lt hgt] at hgt
  assumption
```

在策略中也可以使用 **match** 记号：

```
example : p ∨ q → q ∨ p := by
intro h
match h with
| Or.inl _ => apply Or.inr; assumption
| Or.inr h2 => apply Or.inl; exact h2
```

为方便起见，模式匹配已经集成到 **intro** 和 **funext** 等策略中。

```
example : s ∧ q ∧ r → p ∧ r → q ∧ p := by
intro {_, {hq, _}} {hp, _}
exact {hq, hp}
```

```
example :
  (fun (x : Nat × Nat) (y : Nat × Nat) => x.1 + y.2)
  =
  (fun (x : Nat × Nat) (z : Nat × Nat) => z.2 + x.1) := by
funext (a, b) (c, d)
show a + d = d + a
rw [Nat.add_comm]
```

本节最后介绍一个旨在方便处理归纳类型的策略，即 **injection** 策略。按照设计，归纳类型的元素是自由生成的；也就是说，构造子是单射的，并且其值域两两不交。**injection** 策略正是为了利用这一事实：

```
open Nat
```

```
example (m n k : Nat) (h : succ (succ m) = succ (succ n))
  : n + k = m + k := by
  injection h with h'
  injection h' with h''
  rw [h'']
```

第一次使用该策略会把 $h' : m.succ = n.succ$ 加入上下文，第二次则加入 $h'' : m = n$ 。

injection 策略还会检测由不同构造子被设为相等所产生的矛盾，并利用这些矛盾关闭目标。

```
open Nat
```

```
example (m n : Nat) (h : succ m = 0) : n = n + 7 := by
  injection h
```

```
example (m n : Nat) (h : succ m = 0) : n = n + 7 := by
  contradiction
```

```
example (h : 7 = 4) : False := by
  contradiction
```

如第二个例子所示，**contradiction** 策略也会检测这种形式的矛盾。

7.7 归纳族

我们已经快要完整描述 Lean 接受的各种归纳定义了。到目前为止，你已经看到 Lean 允许引入带有任意数量递归构造子的归纳类型。事实上，单个归纳定义还可以按下面要介绍的方式，引入一个带索引的归纳类型族。

归纳族是一个带索引的类型族，由如下形式的同步归纳定义给出：

```
inductive foo : ... → Sort u where
| constructor1 : ... → foo ...
| constructor2 : ... → foo ...
...
| constructorn : ... → foo ...
```

普通归纳定义构造某个 $\text{Sort } u$ 的元素；与之不同，更一般的版本构造一个函数 $\dots \rightarrow \text{Sort } u$ ，其中“...”表示一系列参数类型，也称为索引。随后，每个构造子都构造该族中某个成员的元素。一个例子是 $\text{Vect } \alpha \ n$ 的定义，它是长度为 n 、元素来自 α 的向量类型：

```
inductive Vect (α : Type u) : Nat → Type u where
| nil : Vect α 0
| cons : α → {n : Nat} → Vect α n → Vect α (n + 1)
```

注意，构造子 `cons` 接受 `Vect  $\alpha$   $n$` 的一个元素，并返回 `Vect  $\alpha$  ( $n + 1$ )` 的一个元素，从而用该族某个成员的元素来构造另一个成员的元素。

一个更特殊的例子是 `Lean` 中相等类型的定义：

```
inductive Eq { $\alpha$  : Sort u} ( $a$  :  $\alpha$ ) :  $\alpha$  → Prop where
| refl : Eq  $a$   $a$ 
```

对每个固定的 α : Sort u 和 a : α ，这个定义构造了一个由 x : α 索引的类型族 `Eq  $a$   $x$` 。然而值得注意的是，它只有一个构造子 `refl`，后者是 `Eq  $a$   $a$` 的元素。直观地说，构造 `Eq  $a$   $x$` 的唯一的证明方法，是在 x 为 a 的情况下使用自反性。注意，在类型族 `Eq  $a$   $x$` 中，`Eq  $a$   $a$` 是唯一有元素的类型。`Lean` 生成的消去原则如下：

```
universe u v
```

```
#check (@Eq.rec : { $\alpha$  : Sort u} → { $a$  :  $\alpha$ } →
           {motive : ( $x$  :  $\alpha$ ) →  $a = x$  → Sort v} →
           motive  $a$  rfl →
           { $b$  :  $\alpha$ } → ( $h$  :  $a = b$ ) → motive  $b$   $h$ )

@Eq.rec : { $\alpha$  : Sort u} →
  { $a$  :  $\alpha$ } → {motive : ( $a_1$  :  $\alpha$ ) →  $a = a_1$  → Sort v} → motive
  →  $a$  (Eq.refl  $a$ ) → { $a_1$  :  $\alpha$ } → ( $t$  :  $a = a_1$ ) → motive  $a_1$ 
  →  $t$ 
```

一个值得注意的事实是，关于相等性的所有基本公理都可由构造子 `refl` 和消去器 `Eq.rec` 推出。不过，相等性的定义并不典型；参见 section 7.8 (p. 136) 中的讨论。

递归子 `Eq.rec` 还用于定义替换：

```
theorem subst { $\alpha$  : Type u} { $a$   $b$  :  $\alpha$ } { $p$  :  $\alpha$  → Prop}
  ( $h_1$  : Eq  $a$   $b$ ) ( $h_2$  :  $p$   $a$ ) :  $p$   $b$  :=
  Eq.rec (motive := fun  $x$  _ =>  $p$   $x$ )  $h_2$   $h_1$ 
```

也可以使用 `match` 定义 `subst`。

```
theorem subst { $\alpha$  : Type u} { $a$   $b$  :  $\alpha$ } { $p$  :  $\alpha$  → Prop}
  ( $h_1$  : Eq  $a$   $b$ ) ( $h_2$  :  $p$   $a$ ) :  $p$   $b$  :=
  match  $h_1$  with
| rfl =>  $h_2$ 
```

实际上，`Lean` 会用基于所生成辅助定义的方式编译 `match` 表达式，例如 `Eq.casesOn` 和 `Eq.ndrec`；而这些辅助定义本身又是用 `Eq.rec` 定义的。

```
theorem subst { $\alpha$  : Type u} { $a$   $b$  :  $\alpha$ } { $p$  :  $\alpha$  → Prop}
  ( $h_1$  :  $a = b$ ) ( $h_2$  :  $p$   $a$ ) :  $p$   $b$  :=
  match  $h_1$  with
| rfl =>  $h_2$ 
```

```
set_option pp.all true
```

```
#print subst
```

```

theorem Hidden.subst.{u} :  $\forall \{ \alpha : \text{Type } u \} \{ a b : \alpha \} \{ p : \alpha \rightarrow \text{Prop} \}$ 
  ( $h_1 : @Eq.\{u + 1\} \alpha a b$ ) ( $h_2 : p a$ ),  $p b :=$ 
fun { $\alpha : \text{Type } u \}$  { $a b : \alpha \}$  { $p : \alpha \rightarrow \text{Prop} \}$  ( $h_1 : @Eq.\{u + 1\} \alpha$ 
   $\hookrightarrow a b$ ) ( $h_2 : p a$ ) =>
  @Hidden.subst.match_1.{u}  $\alpha a$  (fun ( $b : \alpha$ ) ( $h_1 : @Eq.\{u + 1\} \alpha$ 
     $\hookrightarrow b a$ ) =>  $p b$ )  $b h_1$  fun ( $_ : \text{Unit}$ ) =>  $h_2$ 

#print subst.match_1_1

#print Eq.casesOn

@[reducible] def Eq.casesOn.{u, u_1} : { $\alpha : \text{Sort } u_1$ }  $\rightarrow$ 
  { $a : \alpha$ }  $\rightarrow$ 
  { $motive : (a_1 : \alpha) \rightarrow (t : @Eq.\{u_1\} \alpha a a_1) \rightarrow \text{Sort } u$ }  $\rightarrow$ 
  { $a_1 : \alpha$ }  $\rightarrow (t : @Eq.\{u_1\} \alpha a a_1) \rightarrow (refl : motive a$ 
     $\hookrightarrow (@Eq.refl.\{u_1\} \alpha a)) \rightarrow motive a_1 t :=$ 
fun { $\alpha : \text{Sort } u_1$ } { $a : \alpha$ } { $motive : (a_1 : \alpha) \rightarrow (t :$ 
   $\hookrightarrow @Eq.\{u_1\} \alpha a a_1) \rightarrow \text{Sort } u$ } { $a_1 : \alpha$ } ( $t : @Eq.\{u_1\} \alpha a$ 
   $\hookrightarrow a_1$ )
  ( $refl : motive a (@Eq.refl.\{u_1\} \alpha a)$ ) =>
  @Eq.rec.{u, u_1}  $\alpha a motive refl a_1 t$ 

#print Eq.ndrec

@[reducible] def Eq.ndrec.{u1, u2} : { $\alpha : \text{Sort } u_2$ }  $\rightarrow$ 
  { $a : \alpha$ }  $\rightarrow \{motive : \alpha \rightarrow \text{Sort } u_1\} \rightarrow (m : motive a) \rightarrow \{b : \alpha\}$ 
   $\hookrightarrow \rightarrow (h : @Eq.\{u_2\} \alpha a b) \rightarrow motive b :=$ 
fun { $\alpha : \text{Sort } u_2$ } { $a : \alpha$ } { $motive : \alpha \rightarrow \text{Sort } u_1$ } ( $m : motive$ 
   $\hookrightarrow a$ ) { $b : \alpha$ } ( $h : @Eq.\{u_2\} \alpha a b$ ) =>
  @Eq.rec.{u1, u2}  $\alpha a$  (fun ( $x : \alpha$ ) ( $x_1 : @Eq.\{u_2\} \alpha a x$ ) =>
     $\hookrightarrow motive x$ )  $m b h$ 

```

对 $h_1 : a = b$ 使用递归子或 **match** 时，我们可以假定 a 和 b 相同；在这种情况下， $p b$ 与 $p a$ 也相同。

证明 **Eq** 具有对称性和传递性并不困难。在下面的例子中，我们证明 **symm**，并把定理 **trans** 和 **congr**（同余）留作练习。

```
variable { $\alpha \beta : \text{Type } u \}$  { $a b c : \alpha \}$ 
```

```
theorem symm (h : Eq a b) : Eq b a :=
  match h with
  | rfl => rfl
```

```
theorem trans (h1 : Eq a b) (h2 : Eq b c) : Eq a c :=
  sorry
```

```
theorem congr (f :  $\alpha \rightarrow \beta$ ) (h : Eq a b) : Eq (f a) (f b) :=
  sorry
```

在类型文献中，还有对归纳定义的进一步推广，例如归纳-递归和归纳-归纳原则。Lean 不支持这些原则。

7.8 公理细节

我们已经通过例子描述了归纳类型及其语法。本节为对公理基础感兴趣的读者提供更多信息。

我们已经看到，归纳类型的构造子接受参数——直观地说，就是在整个归纳构造过程中保持固定的参数——以及索引，即为同时正在构造的类型族作参数化的参数。每个构造子都应具有一个类型，其中参数类型由先前定义的类型、参数类型与索引类型，以及当前正在定义的归纳族构成。要求是：如果后者出现，它只能严格正向地出现。简单来说，这意味着凡是包含它的构造子参数，都必须是一个依值箭头类型，并且正在定义的归纳类型只出现在该依值箭头类型的结果类型位置；其中索引由常量和先前的参数给出。

由于归纳类型位于某个 u 下的 $\text{Sort } u$ 中，自然要问： u 可以实例化为哪些宇宙层级。归纳类型族 C 的定义中，每个构造子 c 都具有如下形式：

$$c : (a : \alpha) \rightarrow (b : \beta[a]) \rightarrow C \ a \ p[a,b]$$

其中 a 是一列数据类型参数， b 是构造子的参数序列，而 $p[a, b]$ 是索引，它们决定该构造所栖居的是归纳族中的哪一个元素。（注意，这种描述有些误导，因为只要依赖关系有意义，构造子的参数可以按任意顺序出现。）对 C 的宇宙层级的约束分为两种情况，取决于该归纳类型是否被指定落在 Prop （即 $\text{Sort } 0$ ）中。

先考虑归纳类型没有被指定落在 Prop 中的情况。此时宇宙层级 u 受到如下约束：

对于如上的每个构造子 c ，以及序列 $\beta[a]$ 中的每个 $\beta_k[a]$ ，若 $\beta_k[a] : \text{Sort } v$ ，则有 $u \geq v$ 。

换言之，要求宇宙层级 u 至少与表示构造子参数的每个类型所在的宇宙层级一样大。

当归纳类型被指定落在 Prop 中时，构造子参数的宇宙层级没有约束。但这些宇宙层级确实会影响消去规则。一般而言，对于 Prop 中的归纳类型，消去规则的动机要求位于 Prop 中。

最后这条规则有一个例外：当只有一个构造子，并且每个构造子参数要么位于 Prop 中、要么是索引时，我们允许从一个归纳定义的 Prop 消去到任意 Sort 。直观理由是，在这种情况下，消去并没有使用任何超出“参数类型有元素”这一事实之外的信息。这个特例称为单例消去。

我们已经在归纳定义的相等类型的消去器 Eq.rec 的应用中看到单例消去发挥作用。我们可以用元素 $h : \text{Eq } a \ b$ 将元素 $h_2 : p \ a$ 强制转换为 $p \ b$ ，即使 $p \ a$ 和 $p \ b$ 是任意类型也可以；因为这种转换不会产生新数据，它只是重新解释我们已有的数据。单例消去也用于异质相等和良基递归；这些内容将在 section 8.5 (p. 149) 一章中讨论。

7.9 相互和嵌套归纳类型

现在我们考虑归纳类型的两种常用推广。**Lean** 通过把它们“编译”为上面描述的更原始的归纳类型来支持它们。换言之，**Lean** 解析更一般的定义，基于它们

定义辅助归纳类型，然后再用这些辅助类型定义我们真正想要的类型。要有效使用这些类型，需要下一章将介绍的 Lean 方程编译器。尽管如此，在这里描述这些声明仍然有意义，因为它们是普通归纳定义的直接变体。

首先，Lean 支持相互定义的归纳类型。其思想是，可以同时定义两个（或更多）归纳类型，其中每一个都引用其他类型。

```
mutual
  inductive Even : Nat → Prop where
    | even_zero : Even 0
    | even_succ : (n : Nat) → Odd n → Even (n + 1)

  inductive Odd : Nat → Prop where
    | odd_succ : (n : Nat) → Even n → Odd (n + 1)
end
```

在这个例子中，两个类型被同时定义：自然数 n 若为 0 或比某个 Odd 数大一，则为 Even；若比某个 Even 数大一，则为 Odd。在下面的练习中，你需要把细节写出来。

相互归纳定义也可用于定义一种有限树的表示，其节点由 α 的元素标记：

```
mutual
  inductive Tree ( $\alpha$  : Type u) where
    | node :  $\alpha$  → TreeList  $\alpha$  → Tree  $\alpha$ 

  inductive TreeList ( $\alpha$  : Type u) where
    | nil : TreeList  $\alpha$ 
    | cons : Tree  $\alpha$  → TreeList  $\alpha$  → TreeList  $\alpha$ 
end
```

有了这个定义，可以通过给出一个 α 的元素以及一个可能为空的子树列表，来构造 Tree α 的元素。子树列表由类型 TreeList α 表示；它被定义为要么是空列表 nil，要么是由一棵树和一个 TreeList α 元素构成的 cons。

然而，使用这个定义并不方便。如果子树列表由类型 List (Tree α) 给出会好得多，尤其是因为 Lean 的库包含许多处理列表的函数和定理。可以证明类型 TreeList α 与 List (Tree α) 同构，但沿着这个同构来回转换结果很繁琐。事实上，Lean 允许我们定义真正想要的归纳类型：

```
inductive Tree ( $\alpha$  : Type u) where
  | mk :  $\alpha$  → List (Tree  $\alpha$ ) → Tree  $\alpha$ 
```

这称为嵌套归纳类型。它超出了上一节给出的归纳类型严格规范，因为 Tree 并不是严格正向地出现在 mk 的参数中，而是嵌套在 List 类型构造子内部。于是 Lean 会在其内核中自动构造 TreeList α 与 List (Tree α) 之间的同构，并依据该同构定义 Tree 的构造子。

7.10 练习

1. 尝试定义自然数上的其他运算，例如乘法、前驱函数（满足 $\text{pred } 0 = 0$ ）、截断减法（当 m 大于或等于 n 时满足 $n - m = 0$ ）以及乘方。然后基

于我们已经证明的定理，尝试证明它们的一些基本性质。由于其中许多已经在 `Lean` 的核心库中定义，为避免名称冲突，你应该在名为 `Hidden` 或类似名称的命名空间中工作。

2. 定义一些列表上的运算，例如 `length` 函数或 `reverse` 函数。证明一些性质，例如：
a. `length (xs ++ ys) = length xs + length ys`.
`length (reverse xs) = length xs`.
c. `reverse (reverse xs) = xs`
3. 定义一个归纳数据类型，其项由以下构造子构成：
 - `const n`，表示自然数 n 的常量
 - `var n`，编号为 n 的变量
 - `plus s t`，表示 s 与 t 的和
 - `times s t`，表示 s 与 t 的积

递归地定义一个函数，使其能够相对于变量赋值来求值任意这样的项。

4. 类似地，定义命题公式的类型，以及该类型上的函数：求值函数、度量公式复杂度的函数，以及把另一个公式替换给定变量的函数。

8 归纳与递归

在上一章中，我们看到归纳定义为在 **Lean** 中引入新类型提供了强有力的手段。此外，构造子和递归子是定义这些类型上函数的唯一方式。由 **propositions-as-types** 对应可知，这意味着归纳是证明的基本方法。

Lean 提供了定义递归函数、进行模式匹配以及书写归纳证明的自然方式。它允许你通过指定函数应满足的等式来定义函数，也允许你通过指定如何处理可能出现的各种情形来证明定理。在幕后，这些描述会经由一个我们称为“方程编译器”的过程，被“编译”到原始递归子。方程编译器并不是受信代码基的一部分；它的输出由一些项组成，这些项会由内核独立检查。

8.1 模式匹配

对示意性模式的解释是编译过程的第一步。我们已经看到，**casesOn** 递归子可以根据归纳定义类型中涉及的构造子，通过分情形来定义函数并证明定理。但是，复杂的定义可能会使用多个嵌套的 **casesOn** 应用，因而难以阅读和理解。模式匹配提供了一种更方便的方法，也为函数式编程语言的用户所熟悉。

考虑自然数这一归纳定义的类型。每个自然数要么是 **zero**，要么是 **succ x**，因此你可以通过在这两种情形中分别指定一个值，来定义从自然数到任意类型的函数：

```
open Nat
```

```
def sub1 : Nat → Nat
| zero   => zero
| succ x => x
```

```
def isZero : Nat → Bool
| zero   => true
| succ x => false
```

用于定义这些函数的等式在定义上成立：

```
example : sub1 0 = 0 := rfl
```

```
example (x : Nat) : sub1 (succ x) = x := rfl
```

```
example : isZero 0 = true := rfl
```

```
example (x : Nat) : isZero (succ x) = false := rfl
```

```
example : sub1 7 = 6 := rfl
```

```
example (x : Nat) : isZero (x + 3) = false := rfl
```

除了 zero 和 succ，我们也可以使用更熟悉的记号：

```
def sub1 : Nat → Nat
| 0      => 0
| x + 1 => x
```

```
def isZero : Nat → Bool
| 0      => true
| x + 1 => false
```

由于加法和零记号已经被赋予 **[match_pattern]** 属性，它们可以用于模式匹配。Lean 只是将这些表达式规范化，直到构造子 zero 和 succ 显现出来。

模式匹配适用于任何归纳类型，例如乘积类型和选项类型：

```
def swap :  $\alpha \times \beta \rightarrow \beta \times \alpha$ 
| (a, b) => (b, a)
```

```
def foo : Nat × Nat → Nat
| (m, n) => m + n
```

```
def bar : Option Nat → Nat
| some n => n + 1
| none   => 0
```

这里我们不仅用它来定义函数，也用它来进行分情形证明：

```
def not : Bool → Bool
| true  => false
| false => true
```

```
theorem not_not :  $\forall (b : \text{Bool}), \text{not} (\text{not } b) = b$ 
| true  => show not (not true) = true from rfl
| false => show not (not false) = false from rfl
```

模式匹配也可用于析构归纳定义的命题：

```
example (p q : Prop) :  $p \wedge q \rightarrow q \wedge p$ 
| And.intro h1 h2 => And.intro h2 h1
```

```
example (p q : Prop) :  $p \vee q \rightarrow q \vee p$ 
| Or.inl hp => Or.inr hp
| Or.inr hq => Or.inl hq
```

这为展开使用逻辑联结词的假设提供了一种紧凑方式。

在所有这些例子中，模式匹配都用于进行一次单一的情形区分。更有意思的是，模式可以包含嵌套的构造子，如下列例子所示。

```
def sub2 : Nat → Nat
| 0      => 0
| 1      => 0
| x + 2 => x
```

方程编译器首先根据输入是 `zero` 还是形如 `succ x` 来分情形。然后它再根据 `x` 是形如 `zero` 还是 `succ x` 进行情形区分。它从给出的模式中确定必要的情形区分；如果这些模式未能穷尽所有情形，则会报错。同样，我们也可以使用算术记号，如下一个版本所示。无论哪种情形，定义等式都在定义上成立。

```
example : sub2 0 = 0 := rfl
example : sub2 1 = 0 := rfl
example : sub2 (x+2) = x := rfl
```

```
example : sub2 5 = 3 := rfl
```

你可以写 `#print sub2` 来查看该函数是如何被编译为递归子的。（Lean 会告诉你，`sub2` 是根据一个内部辅助函数 `sub2.match_1` 定义的，但你也可以把它打印出来。）Lean 使用这些辅助函数来编译 `match` 表达式。实际上，上述定义会展开为

```
def sub2 : Nat → Nat :=
  fun x =>
    match x with
    | 0      => 0
    | 1      => 0
    | x + 2 => x
```

下面是更多嵌套模式匹配的例子：

```
example (p q : α → Prop) :
  (∃ x, p x ∨ q x) →
  (∃ x, p x) ∨ (∃ x, q x)
| Exists.intro x (Or.inl px) => Or.inl (Exists.intro x px)
| Exists.intro x (Or.inr qx) => Or.inr (Exists.intro x qx)
```

```
def foo : Nat × Nat → Nat
| (0, n)      => 0
| (m+1, 0)    => 1
| (m+1, n+1) => 2
```

方程编译器可以按顺序处理多个参数。例如，把前一个例子定义为二元函数会更自然：

```
def foo : Nat → Nat → Nat
| 0,      n      => 0
| m + 1, 0      => 1
| m + 1, n + 1 => 2
```

下面是另一个例子：

```
def bar : List Nat → List Nat → Nat
| [], [] => 0
| a :: as, [] => a
| [], b :: bs => b
| a :: as, b :: bs => a + b
```

注意，模式之间用逗号分隔。

在下面每个例子中，尽管其他参数也出现在模式列表中，但情形区分只发生在第一个参数上。

```
def and : Bool → Bool → Bool
| true, a => a
| false, _ => false
```

```
def or : Bool → Bool → Bool
| true, _ => true
| false, a => a
```

```
def cond : Bool →  $\alpha$  →  $\alpha$  →  $\alpha$ 
| true, x, y => x
| false, x, y => y
```

还请注意，当定义中不需要某个参数的值时，你可以改用下划线。这个下划线称为通配符模式，或匿名变量。与方程编译器之外的用法相反，这里的下划线并不表示隐式参数。用下划线表示通配符是函数式编程语言中的常见做法，因此 Lean 也采用了这一记号。section 8.2 (p. 143) 一节进一步阐述了通配符的概念，而 section 8.9 (p. 162) 的说明则解释了你也可以如何在模式中使用隐式参数。

如 chapter 7 (p. 113) 中所述，归纳数据类型可以依赖于参数。下面的例子使用模式匹配定义 tail 函数。参数 $\alpha : \text{Type } u$ 出现在冒号之前，表示它不参与模式匹配。Lean 也允许参数出现在：之后，但对它们进行模式匹配需要显式的 match。

```
def tail1 { $\alpha$  : Type u} : List  $\alpha$  → List  $\alpha$ 
| [] => []
| a :: as => as
```

```
def tail2 : { $\alpha$  : Type u} → List  $\alpha$  → List  $\alpha$ 
|  $\alpha$ , [] => []
|  $\alpha$ , a :: as => as
```

尽管在这两个例子中参数 α 的位置不同，但两种情况下它都以同样的方式处理：它不参与情形区分。

Lean 还可以处理更复杂的模式匹配形式，其中依值类型的参数会对各种情形施加额外约束。这类依值模式匹配的例子将在 section 8.8 (p. 159) 一节中讨论。

8.2 通配符与重叠模式

考虑上一节中的一个例子：

```
def foo : Nat → Nat → Nat
| 0,      n      => 0
| m + 1, 0       => 1
| m + 1, n + 1 => 2
```

另一种写法是：

```
def foo : Nat → Nat → Nat
| 0, n => 0
| m, 0 => 1
| m, n => 2
```

在第二种写法中，模式会发生重叠；例如，参数对 `0, 0` 匹配全部三个情形。但是 Lean 通过使用第一个适用的等式来处理这种歧义，因此在这个例子中最终结果相同。特别地，以下等式在定义上成立：

```
example : foo 0      0      = 0 := rfl
example : foo 0      (n + 1) = 0 := rfl
example : foo (m + 1) 0      = 1 := rfl
example : foo (m + 1) (n + 1) = 2 := rfl
```

由于不需要 m 和 n 的值，我们也完全可以改用通配符模式。

```
def foo : Nat → Nat → Nat
| 0, _ => 0
| _, 0 => 1
| _, _ => 2
```

你可以检查，`foo` 的这个定义满足与之前相同的定义等式。

一些函数式编程语言支持 不完全模式。在这些语言中，对于未覆盖的情形，解释器会产生异常，或返回某个任意值。我们可以用 `Inhabited` 类型类来模拟返回任意值的方法。粗略地说，`Inhabited  $\alpha$` 的一个元素见证了 α 中有元素这一事实；在 chapter 10 (p. 175) 中，我们将看到可以指示 Lean 适当的基本类型是有居留元的，并且它可以自动推断其他构造出的类型也是有居留元的。在此基础上，标准库为任何有居留元的类型提供了默认元素 `default`。我们也可以使用类型 `Option  $\alpha$` 来模拟不完全模式。思路是对已给出的模式返回 `some a`，而对未覆盖的情形使用 `none`。下面的例子展示了这两种方法。

```
def f1 : Nat → Nat → Nat
| 0, _  => 1
| _, 0  => 2
| _, _  => default -- “不完全” 情形

example : f1 0      0      = 1      := rfl
example : f1 0      (a+1) = 1      := rfl
example : f1 (a+1) 0      = 2      := rfl
```

```
example : f1 (a+1) (b+1) = default := rfl
```

```
def f2 : Nat → Nat → Option Nat
| 0, _ => some 1
| _, 0 => some 2
| _, _ => none      -- “不完全” 情形
```

```
example : f2 0 0 = some 1 := rfl
```

```
example : f2 0 (a+1) = some 1 := rfl
```

```
example : f2 (a+1) 0 = some 2 := rfl
```

```
example : f2 (a+1) (b+1) = none := rfl
```

方程编译器很聪明。如果你在下面的定义中漏掉任何情形，错误信息会告诉你哪些情形尚未被覆盖。

```
def bar : Nat → List Nat → Bool → Nat
| 0, _, false => 0
| 0, b :: _, _ => b
| 0, [], true => 7
| a+1, [], false => a
| a+1, [], true => a + 1
| a+1, b :: _, _ => a + b
```

在适当的情况下，它还会使用 **if ... then ... else** 而不是 **casesOn**。

```
def foo : Char → Nat
| 'A' => 1
| 'B' => 2
| _ => 3
```

```
#print foo.match_1
```

```
def foo.match_1.{u_1} : (motive : Char → Sort u_1) →
  (x : Char) → (Unit → motive 'A') → (Unit → motive 'B') →
  ↳ ((x : Char) → motive x) → motive x :=
fun motive x h_1 h_2 h_3 =>
  dite (x = 'A') (Eq.ndrec_symm (h_1 ())) fun h_1 => dite (x
  ↳ = 'B') (Eq.ndrec_symm (h_2 ())) fun h_2 => h_3 x
```

8.3 结构递归与结构归纳

方程编译器之所以强大，是因为它还支持递归定义。在接下来的三节中，我们将分别介绍：

- 结构递归定义
- 良基递归定义

- 相互递归定义

一般而言，方程编译器处理如下形式的输入：

```
def foo (a :  $\alpha$ ) : (b :  $\beta$ )  $\rightarrow$   $\gamma$ 
| [patterns1] => t1
...
| [patternsn] => tn
```

这里 $(a : \alpha)$ 是一串参数， $(b : \beta)$ 是进行模式匹配的一串参数，而 γ 是任意类型，它可以依赖于 a 和 b 。每一行都应当包含相同数量的模式，即 β 的每个元素各有一个模式。正如我们所见，模式要么是变量，要么是应用于其他模式的构造子，要么是可规范化为这种形式的表达式（其中非构造子带有 `[match_pattern]` 属性）。构造子的出现会引发情形区分，而构造子的参数由给定变量表示。在 section 8.8 (p. 159) 一节中，我们将看到，为了使表达式通过类型检查，模式中的某些显式项会被强制为特定形式，尽管它们并不参与模式匹配。因此它们被称为“inaccessible patterns”（不可访问模式）。不过在介绍 section 8.8 (p. 159) 之前，我们还不需要使用这种不可访问模式。

如上一节所见，项 $t_1, \dots, t_n$ 可以使用任意参数 a ，也可以使用相应模式中引入的任意变量。使递归与归纳成为可能的是，它们还可以包含对 `foo` 的递归调用。本节将讨论 结构递归，其中出现在 $\Rightarrow$ 右侧、传给 `foo` 的参数，是左侧模式的子项。其思想是：它们在结构上更小，因此在归纳类型中出现在更早的阶段。下面是上一章中的一些结构递归例子，现在用方程编译器来定义：

```
open Nat

def add : Nat  $\rightarrow$  Nat  $\rightarrow$  Nat
| m, zero    => m
| m, succ n => succ (add m n)

theorem add_zero (m : Nat) : add m zero = m := rfl
theorem add_succ (m n : Nat) : add m (succ n) = succ (add m n)
   $\hookrightarrow$  := rfl

theorem zero_add :  $\forall$  n, add zero n = n
| zero    => rfl
| succ n => congrArg succ (zero_add n)

def mul : Nat  $\rightarrow$  Nat  $\rightarrow$  Nat
| n, zero    => zero
| n, succ m => add (mul n m) n
```

`zero_add` 的证明清楚表明，在 `Lean` 中归纳证明实际上是一种递归。

上面的例子表明，`add` 的定义等式在定义上成立，`mul` 也是如此。方程编译器会尽可能确保这一点；直接的结构归纳正是这种情形。然而在其他情形中，归约只在 命题上成立，也就是说，它们是必须显式应用的等式定理。方程编译器会在内部生成这样的定理。它们并不意在由用户直接使用；相反，`simp` 策略会被配置为在必要时使用它们。下面的 `zero_add` 证明就是这样工作的：

```

theorem zero_add :  $\forall n$ , add zero  $n$  =  $n$ 
| zero   => by simp [add]
| succ  $n$  => by simp [add, zero_add]

```

与通过模式匹配进行定义一样，结构递归或结构归纳的参数也可以出现在冒号之前。这样的参数只是在处理定义之前被加入局部上下文。例如，加法的定义也可以写成：

```

open Nat
def add ( $m$  : Nat) : Nat → Nat
| zero   =>  $m$ 
| succ  $n$  => succ (add  $m$   $n$ )

```

你也可以用 **match** 来书写上面的例子。

```

open Nat
def add ( $m$   $n$  : Nat) : Nat :=
  match  $n$  with
| zero   =>  $m$ 
| succ  $n$  => succ (add  $m$   $n$ )

```

Fibonacci 函数 **fib** 给出了一个更有趣的结构递归例子。

```

def fib : Nat → Nat
| 0    => 1
| 1    => 1
|  $n+2$  => fib ( $n+1$ ) + fib  $n$ 

```

```

example : fib 0 = 1 := rfl

```

```

example : fib 1 = 1 := rfl

```

```

example : fib ( $n + 2$ ) = fib ( $n + 1$ ) + fib  $n$  := rfl

```

```

example : fib 7 = 21 := rfl

```

这里，**fib** 函数在 $n + 2$ （它在定义上等于 **succ** (**succ** n)) 处的值，是用它在 $n + 1$ （它在定义上等价于 **succ** n ）处的值以及在 n 处的值来定义的。不过，这是计算 Fibonacci 函数的一种众所周知的低效方式，其运行时间关于 n 呈指数增长。下面是一种更好的方法：

```

def fibFast ( $n$  : Nat) : Nat :=
  (loop  $n$ ).2
where
  loop : Nat → Nat × Nat
  | 0    => (0, 1)
  |  $n+1$  => let  $p$  := loop  $n$ ; ( $p$ .2,  $p$ .1 +  $p$ .2)

```

```

#eval fibFast 100 -- 573147844013817084101

```

573147844013817084101

下面是使用 **let rec** 而不是 **where** 的同一定义。

```
def fibFast (n : Nat) : Nat :=
  let rec loop : Nat → Nat × Nat
    | 0    => (0, 1)
    | n+1 => let p := loop n; (p.2, p.1 + p.2)
  (loop n).2
```

在这两种情形中，Lean 都会生成辅助函数 `fibFast.loop`。

为了处理结构递归，方程编译器使用 值历程递归：它使用随每个归纳定义类型自动生成的常量 `below` 和 `brecOn`。通过查看 `Nat.below` 和 `Nat.brecOn` 的类型，你可以大致了解其工作方式：

```
variable (C : Nat → Type u)
```

```
#check (@Nat.below C : Nat → Type u)
```

```
Nat.below : Nat → Type u
```

```
#reduce @Nat.below C (3 : Nat)
```

```
Nat.below 3
```

```
#check (@Nat.brecOn C : (n : Nat) → ((n : Nat) → @Nat.below C
  ↪ n → C n) → C n)
```

```
Nat.brecOn : (t : Nat) → ((t : Nat) → Nat.below t → C t) → C
  ↪ t
```

类型 `@Nat.below C (3 : Nat)` 是一个数据结构，存储 `C 0`、`C 1` 和 `C 2` 的元素。值历程递归由 `Nat.brecOn` 实现。它使我们能够依据函数此前的所有值，来定义类型为 $(n : \text{Nat}) \rightarrow C\ n$ 的依值函数在特定输入 n 处的值：这些此前的值以 `@Nat.below C n` 的一个元素给出。

使用值历程递归，是方程编译器向 Lean 内核证明函数终止的技术之一。它并不影响代码生成器：后者会像其他函数式编程语言编译器那样编译递归函数。回忆一下，`#eval fib <n>` 关于 `<n>` 是指数时间的。另一方面，`#reduce fib <n>` 却很高效，因为它使用发送给内核、基于 `brecOn` 构造的定义。

```
def fib : Nat → Nat
  | 0    => 1
  | 1    => 1
  | n+2 => fib (n+1) + fib n
```

```
-- 慢:
```

```
-- #eval fib 50
```

```
-- 快:
```

```
#reduce fib 50
```

```
20365011074
```

```
#print fib
```

```
def fib : Nat → Nat :=
  fun x =>
    Nat.brecOn x fun x f =>
      (match (motive := (x : Nat) → Nat.below x → Nat) x with
      | 0 => fun x => 1
      | 1 => fun x => 1
      | n.succ.succ => fun x => x.1 + x.2.1)
      f
```

递归定义的另一个好例子是列表的 `append` 函数。

```
def append : List α → List α → List α
| [], bs => bs
| a::as, bs => a :: append as bs
```

```
example : append [1, 2, 3] [4, 5] = [1, 2, 3, 4, 5] := rfl
```

再看另一个例子：它把第一个列表的元素与第二个列表的元素相加，直到两个列表中有一个耗尽为止。

```
def listAdd [Add α] : List α → List α → List α
| [], _ => []
| _, [] => []
| a::as, b::bs => (a + b) :: listAdd as bs
```

```
#eval listAdd [1, 2, 3] [4, 5, 6, 6, 9, 10] -- [5, 7, 9]
[5, 7, 9]
```

建议你在下面的练习中尝试类似的例子。

8.4 局部递归声明

你可以使用 `let rec` 关键字定义局部递归声明。

```
def replicate (n : Nat) (a : α) : List α :=
  let rec loop : Nat → List α → List α
  | 0, as => as
  | n+1, as => loop n (a::as)
  loop n []
```

```
#check @replicate.loop -- @replicate.loop : {α : Type u_1} → α
→ Nat → List α → List α
@replicate.loop : {α : Type u_1} → α → Nat → List α → List α
```

Lean 会为每个 `let rec` 创建一个辅助声明。在上面的例子中，它为 `replicate` 中出现的 `let rec loop` 创建了声明 `replicate.loop`。请注意，Lean 会把 `let rec` 声明中出现的任何局部变量作为额外参数加入，从而“闭合”该声明。例如，局部变量 `a` 出现在 `let rec loop` 中。

你也可以在策略模式中使用 `let rec`，并用它创建归纳证明。

```

theorem length_replicate (n : Nat) (a :  $\alpha$ ) :
  (replicate n a).length = n := by
  let rec aux (n : Nat) (as : List  $\alpha$ ) :
    (replicate.loop a n as).length = n + as.length := by
    match n with
    | 0   => simp [replicate.loop]
    | n+1 => simp +arith [replicate.loop, aux n]
  exact aux n []

```

你还可以在定义之后使用 **where** 子句引入辅助递归声明。Lean 会把它转换为 **let rec**。

```

def replicate (n : Nat) (a :  $\alpha$ ) : List  $\alpha$  :=
  loop n []
where
  loop : Nat → List  $\alpha$  → List  $\alpha$ 
  | 0,   as => as
  | n+1, as => loop n (a::as)

theorem length_replicate (n : Nat) (a :  $\alpha$ ) :
  (replicate n a).length = n := by
  exact aux n []
where
  aux (n : Nat) (as : List  $\alpha$ ) :
    (replicate.loop a n as).length = n + as.length := by
    match n with
    | 0   => simp [replicate.loop]
    | n+1 => simp +arith [replicate.loop, aux n]

```

8.5 良基递归与归纳

当无法使用结构递归时，我们可以用良基递归来证明终止性。我们需要一个良基关系，以及一个证明，说明每个递归应用相对于这个关系都是递减的。依值类型论强大到足以编码并论证良基递归的正当性。让我们从理解其工作方式所需的逻辑背景开始。

Lean 的标准库定义了两个谓词：**Acc** r a 和 **WellFounded** r 。其中 r 是类型 α 上的二元关系，而 a 是类型 α 的一个元素。

```

variable ( $\alpha$  : Sort u)
variable (r :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$ )

```

```

#check (Acc r :  $\alpha \rightarrow \text{Prop}$ )

```

```

| Acc r :  $\alpha \rightarrow \text{Prop}$ 

```

```

#check (WellFounded r : Prop)

```

```

| WellFounded r : Prop

```

第一个谓词 `Acc` 是一个归纳定义的谓词。按照它的定义, `Acc r x` 等价于 $\forall y, r y x \rightarrow \text{Acc } r y$ 。如果把 $r y x$ 理解为某种序关系 $y < x$, 那么 `Acc r x` 表示 x 可从下方到达, 也就是说它的所有前驱都是可达的。特别地, 如果 x 没有前驱, 那么它就是可达的。给定任意类型 α , 我们应当能够以递归方式为 α 的每个可达元素赋值: 先为其所有前驱赋值, 再为它本身赋值。

断言 r 是良基的, 记为 `WellFounded r`, 正是断言该类型的每个元素都是可达的。根据上述考虑, 如果 r 是类型 α 上的良基关系, 那么相对于关系 r , 我们就应当有一个 α 上的良基递归原理。事实上也确实如此: 标准库定义了 `WellFounded.fix`, 它正是为此目的服务的。

noncomputable

```
def f {α : Sort u}
  (r : α → α → Prop)
  (h : WellFounded r)
  (C : α → Sort v)
  (F : (x : α) → ((y : α) → r y x → C y) → C x) :
  (x : α) → C x :=
WellFounded.fix h F
```

这里出现了一长串角色, 但第一组我们已经见过: 类型 α 、关系 r , 以及假设 h , 即 r 是良基的。变量 C 表示递归定义的动机: 对于每个元素 $x : \alpha$, 我们希望构造一个 $C x$ 的元素。函数 F 提供了完成此事的归纳配方: 给定 x 的每个前驱 y 的 $C y$ 元素, 它告诉我们如何构造一个 $C x$ 的元素。

注意, `WellFounded.fix` 同样可以作为归纳原理使用。它说明, 如果 $<$ 是良基的, 而你想证明 $\forall x, C x$, 那么只需证明: 对任意 x , 若有 $\forall y, r y x \rightarrow C y$, 则有 $C x$ 。

在上面的例子中, 我们使用修饰符 **noncomputable**, 因为代码生成器目前不支持 `WellFounded.fix`。函数 `WellFounded.fix` 是 `Lean` 用来论证函数终止的另一种工具。

`Lean` 知道自然数上的通常次序 $<$ 是良基的。它还知道多种从已有良基序构造新良基序的方法, 例如使用字典序。

下面基本上就是标准库中自然数除法的定义。

open Nat

```
theorem div_lemma {x y : Nat} : 0 < y ∧ y ≤ x → x - y < x :=
  fun h => sub_lt (Nat.lt_of_lt_of_le h.left h.right) h.left

def div.F (x : Nat) (f : (x₁ : Nat) → x₁ < x → Nat → Nat) (y :
  → Nat) : Nat :=
  if h : 0 < y ∧ y ≤ x then
    f (x - y) (div_lemma h) y + 1
  else
    zero

noncomputable def div := WellFounded.fix (measure id).wf div.F

#reduce div 8 2 -- 4
```

4

这个定义有些难以看懂。这里递归发生在 x 上，而 $\text{div.F } x \text{ } f : \text{Nat} \rightarrow \text{Nat}$ 会为这个固定的 x 返回“除以 y ”的函数。你必须记住， div.F 的第二个参数，即递归的配方，是一个函数；它应当为所有小于 x 的值 x_1 返回相应的除以 y 的函数。

精化器的设计目标之一就是让这类定义更方便。它接受如下写法：

```
def div (x y : Nat) : Nat :=
  if h : 0 < y ∧ y ≤ x then
    have : x - y < x := Nat.sub_lt (Nat.lt_of_lt_of_le h.1
      ↪ h.2) h.1
    div (x - y) y + 1
  else
    0
```

当 Lean 遇到递归定义时，它会首先尝试结构递归；只有在失败时，才退回到良基递归。Lean 使用策略 **decreasing_tactic** 来证明递归应用更小。上例中的辅助命题 $x - y < x$ 应当被视为给该策略的提示。

div 的定义等式并不在定义上成立，但我们可以使用 **unfold** 策略展开 div 。我们使用 chapter 11 (p. 195) 来选择要展开哪一个 div 应用。

```
example (x y : Nat) :
  div x y =
    if 0 < y ∧ y ≤ x then
      div (x - y) y + 1
    else 0 := by
  -- 展开等式左侧出现的项：
  conv => lhs; unfold div
  rfl
```

```
example (x y : Nat) (h : 0 < y ∧ y ≤ x) :
  div x y = div (x - y) y + 1 := by
  conv => lhs; unfold div
  simp [h]
```

下面的例子类似：它把任意自然数转换为二进制表达式，表示为由 0 和 1 组成的列表。我们必须提供证据说明递归调用是递减的；这里我们用 **sorry** 完成这一点。**sorry** 不会阻止解释器成功求值该函数，但当项中含有 **sorry** 时，必须使用 **#eval!** 而不是 **#eval**。

```
def natToBin : Nat → List Nat
| 0      => [0]
| 1      => [1]
| n + 2 =>
  have : (n + 2) / 2 < n + 2 := sorry
  natToBin ((n + 2) / 2) ++ [n % 2]
```

```
#eval! natToBin 1234567
```

作为最后一个例子，我们注意到 Ackermann 函数可以直接定义，因为它由自然数上的字典序的良基性来保证。**termination_by** 子句指示 Lean 使用字典序。这个子句实际上把函数参数映射到类型 $\text{Nat} \times \text{Nat}$ 的元素。然后，Lean 使用类型类解析合成一个类型为 $\text{WellFoundedRelation} (\text{Nat} \times \text{Nat})$ 的元素。

```
def ack : Nat → Nat → Nat
| 0, y   => y+1
| x+1, 0 => ack x 1
| x+1, y+1 => ack x (ack (x+1) y)
termination_by x y => (x, y)
```

在许多情况下，Lean 可以自动确定一个适当的字典序。Ackermann 函数就是这样的情形之一，因此 **termination_by** 子句是可选的：

```
def ack : Nat → Nat → Nat
| 0, y   => y+1
| x+1, 0 => ack x 1
| x+1, y+1 => ack x (ack (x+1) y)
```

注意，上面的例子之所以使用字典序，是因为实例 $\text{WellFoundedRelation} (\alpha \times \beta)$ 使用字典序。Lean 还定义了实例

```
instance (priority := low) [SizeOf  $\alpha$ ] : WellFoundedRelation  $\alpha$ 
↪ :=
  sizeOfWFRel
```

在下面的例子中，我们通过说明递归应用中 $as.size - i$ 是递减的，来证明终止性。

```
def takeWhile (p :  $\alpha \rightarrow \text{Bool}$ ) (as : Array  $\alpha$ ) : Array  $\alpha$  :=
  go 0 #[]
where
  go (i : Nat) (r : Array  $\alpha$ ) : Array  $\alpha$  :=
    if h : i < as.size then
      let a := as[i]
      if p a then
        go (i+1) (r.push a)
      else
        r
    else
      r
  termination_by as.size - i
```

注意，在这个例子中辅助函数 `go` 是递归的，而 `takeWhile` 并不是。再一次，Lean 可以自动识别这种模式，因此 **termination_by** 子句并非必要：

```
def takeWhile (p :  $\alpha \rightarrow \text{Bool}$ ) (as : Array  $\alpha$ ) : Array  $\alpha$  :=
  go 0 #[]
where
  go (i : Nat) (r : Array  $\alpha$ ) : Array  $\alpha$  :=
```



```

if h : i < as.size then
  let a := as[i]
  if p a then
    go (i+1) (r.push a)
  else
    r
else
  r

```

默认情况下, Lean 使用策略 **decreasing_tactic** 来证明递归应用是递减的。修饰符 **decreasing_by** 允许我们提供自己的策略。下面是一个例子。

```

theorem div_lemma {x y : Nat} : 0 < y ∧ y ≤ x → x - y < x :=
fun ⟨ypos, ylex⟩ => Nat.sub_lt (Nat.lt_of_lt_of_le ypos
  ↪ ylex) ypos

```

```

def div (x y : Nat) : Nat :=
if h : 0 < y ∧ y ≤ x then
  div (x - y) y + 1
else
  0
decreasing_by apply div_lemma; assumption

```

注意, **decreasing_by** 不是 **termination_by** 的替代品; 二者是相辅相成的。**termination_by** 用于指定良基关系, 而 **decreasing_by** 用于提供我们自己的策略, 以证明递归应用是递减的。在下面的例子中, 我们同时使用二者。

```

def ack : Nat → Nat → Nat
| 0, y => y+1
| x+1, 0 => ack x 1
| x+1, y+1 => ack x (ack (x+1) y)
termination_by x y => (x, y)
decreasing_by
  -- 展开良基递归的辅助定义:
  all_goals simp_wf
  • apply Prod.Lex.left; simp +arith
  • apply Prod.Lex.right; simp +arith
  • apply Prod.Lex.left; simp +arith

```

我们可以使用 **decreasing_by sorry** 来指示 Lean “相信”该函数会终止。

```

def natToBin : Nat → List Nat | 0 => [0] | 1 => [1] | n + 2 => natToBin ((n + 2) /

```

```

#eval! natToBin 1234567

```

回忆一下, 使用 **sorry** 等同于使用一个新的公理, 应当避免。下面的例子中, 我们用 **sorry** 证明了 False。命令 **#print axioms unsound** 表明, **unsound** 依赖于用于实现 **sorry** 的不可靠公理 **sorryAx**。

```

def unsound (x : Nat) : False :=
  unsound (x + 1)
decreasing_by sorry

#check unsound 0
| unsound 0 : False
-- `unsound 0` 是 `False` 的证明

#print axioms unsound -- 'unsound' 依赖于公理: [sorryAx]
| 'unsound' depends on axioms: [sorryAx]

```

总结:

- 如果没有 **termination_by**, Lean 会 (在可能时) 通过选择一个参数, 然后使用类型类解析为该参数的类型合成一个良基关系, 从而导出良基关系。
- 如果指定了 **termination_by**, 它会把函数参数映射到某个类型 α , 然后再次使用类型类解析。回忆一下, $\beta \times \gamma$ 的默认实例是基于 β 和 γ 的良基关系构造的字典序。
- **Nat** 的默认良基关系实例是 $(\cdot < \cdot)$ 。
- 默认情况下, 策略 **decreasing_tactic** 用于说明递归应用相对于所选的良基关系更小。如果 **decreasing_tactic** 失败, 错误信息会包含剩余目标 $\dots \mid - G$ 。注意, **decreasing_tactic** 会使用 **assumption**。因此, 你可以包含一个 **have** 表达式来证明目标 G 。你也可以使用 **decreasing_by** 提供自己的策略。

8.6 函数式归纳

Lean 会为递归函数生成专门的归纳原理。这些归纳原理遵循函数定义的递归结构, 而不是数据类型的结构。关于函数的证明通常遵循函数自身的递归结构, 因此这些归纳原理使得关于函数的陈述可以更方便地得到证明。

例如, 使用 **ack** 的函数式归纳原理来证明结果总是大于 0, 需要为 **ack** 中模式匹配的每个分支处理一个情形:

```

def ack : Nat → Nat → Nat
| 0, y    => y+1
| x+1, 0  => ack x 1
| x+1, y+1 => ack x (ack (x+1) y)

theorem ack_gt_zero : ack n m > 0 := by
  fun_induction ack with
  | case1 y =>
    simp
  | case2 x ih =>
    exact ih

```

```
| case3 x y ih1 ih2 =>
  simp [ack, *]
```

在 `case1` 中，目标是：

```
case case1
y : Nat
⊢ y + 1 > 0
```

目标中的 $y + 1$ 对应于 `ack` 第一种情形返回的值。

在 `case2` 中，目标是：

```
case case2
x : Nat
ih : ack x 1 > 0
⊢ ack x 1 > 0
```

目标中的 `ack x 1` 对应于把 `ack` 应用于模式变量 $x + 1$ 和 `0` 后，在 `ack` 第二种情形中返回的值。这个项会自动简化为右侧。令人满意的是，归纳假设 $ih : \text{ack } x \ 1 > 0$ 对应于递归调用，而它正是该情形返回的答案。

在 `case3` 中，目标是：

```
case case3
x : Nat
y : Nat
ih1 : ack (x + 1) y > 0
ih2 : ack x (ack (x + 1) y) > 0
⊢ ack x (ack (x + 1) y) > 0
```

目标中的 `ack x (ack (x + 1) y)` 对应于 `ack` 的第三种情形返回的值，此时应用于 $x + 1$ 和 $y + 1$ 的 `ack` 已经被归约。归纳假设 $ih1 : \text{ack } (x + 1) \ y > 0$ 和 $ih2 : \text{ack } x \ (\text{ack } (x + 1) \ y) > 0$ 对应于递归调用，其中 $ih1$ 匹配嵌套的递归调用。归纳假设再次正好适用。

使用 `fun_induction` `ack` 会得到与 `ack` 的递归结构相匹配的目标和归纳假设。因此，证明可以写成一行：

```
theorem ack_gt_zero : ack n m > 0 := by
  fun_induction ack <=> simp [*, ack]
```

还有一个 `fun_cases` 策略，它类似于 `cases` 策略。它会为函数控制流中的每个分支生成一个情形。它和 `fun_induction` 还会额外提供假设，以排除未被采用的路径。函数 `f` 表示一个五路布尔析取：

```
def f : Bool → Bool → Bool → Bool → Bool → Bool
| true, _, _, _, _ => true
| _, true, _, _, _ => true
| _, _, true, _, _ => true
| _, _, _, true, _ => true
| _, _, _, _, x => x
```

为了证明它确实是析取，最后一个情形需要知道所有参数都不是 `true`。这一信息由该策略提供：

```
theorem f_or : f b1 b2 b3 b4 b5 = (b1 || b2 || b3 || b4 || b5)
```

```
↪ := by
  fun_cases f
  all_goals sorry
```

每个情形都包含一个排除先前情形的假设:

```

case case1
  b2: Bool
  b3: Bool
  b4: Bool
  b5: Bool
  ⊢ true = (true || b2 || b3 || b4 || b5)

case case2
  b1: Bool
  b3: Bool
  b4: Bool
  b5: Bool
  x+: b1 = true → False
  ⊢ true = (b1 || true || b3 || b4 || b5)

case case3
  b1: Bool
  b2: Bool
  b4: Bool
  b5: Bool
  x+1: b1 = true → False
  x+: b2 = true → False
  ⊢ true = (b1 || b2 || true || b4 || b5)

case case4
  b1: Bool
  b2: Bool
  b3: Bool
  b5: Bool
  x+2: b1 = true → False
  x+1: b2 = true → False
  x+: b3 = true → False
  ⊢ true = (b1 || b2 || b3 || true || b5)

case case5
  b1: Bool
  b2: Bool
  b3: Bool
  b4: Bool
  b5: Bool
  x+3: b1 = true → False
  x+2: b2 = true → False
  x+1: b3 = true → False
  x+: b4 = true → False
  ⊢ b5 = (b1 || b2 || b3 || b4 || b5)
```

simp_all 策略会同时简化所有假设和目标, 它可以处理全部情形:

```
theorem f_or : f b1 b2 b3 b4 b5 = (b1 || b2 || b3 || b4 || b5)
```

```
↪ := by
  fun_cases f <|> simp_all
```

8.7 相互递归

Lean 也支持相互递归定义。其语法类似于相互归纳类型的语法。下面是一个例子:

```
mutual
  def even : Nat → Bool
```

```

| 0    => true
| n+1  => odd n

def odd : Nat → Bool
| 0    => false
| n+1  => even n
end

example : even (a + 1) = odd a := by
  simp [even]

example : odd (a + 1) = even a := by
  simp [odd]

theorem even_eq_not_odd : ∀ a, even a = not (odd a) := by
  intro a; induction a
  . simp [even, odd]
  . simp [even, odd, *]
```

之所以这是一个相互定义，是因为 `even` 递归地用 `odd` 定义，而 `odd` 又递归地用 `even` 定义。在幕后，这会被编译为一个单一的递归定义。内部定义的函数以一个和类型的元素为参数：它要么是 `even` 的输入，要么是 `odd` 的输入。然后它返回适合该输入的输出。为了定义这个函数，`Lean` 使用一个适当的良基度量。内部细节意在对用户隐藏；使用这类定义的规范方式是像上面那样使用 `simp`（或 `unfold`）。

相互递归定义也为处理相互归纳类型和嵌套归纳类型提供了自然方式。回忆一下，前面曾把 `Even` 和 `Odd` 定义为相互归纳谓词。

```

mutual
  inductive Even : Nat → Prop where
    | even_zero : Even 0
    | even_succ : ∀ n, Odd n → Even (n + 1)

  inductive Odd : Nat → Prop where
    | odd_succ : ∀ n, Even n → Odd (n + 1)
end
```

构造子 `even_zero`、`even_succ` 和 `odd_succ` 提供了证明一个数为偶数或奇数的正向手段。我们需要利用归纳类型由这些构造子生成这一事实，来知道零不是奇数，并且后两个蕴含可以反向使用。照例，构造子保存在以所定义类型命名的命名空间中，而命令 `open Even Odd` 使我们能够更方便地访问它们。

```

theorem not_odd_zero : ¬ Odd 0 :=
  fun h => nomatch h

theorem even_of_odd_succ : ∀ n, Odd (n + 1) → Even n
| _, odd_succ n h => h
```

```
theorem odd_of_even_succ :  $\forall n, \text{Even } (n + 1) \rightarrow \text{Odd } n$ 
| _, even_succ n h => h
```

再看一个例子。假设我们使用嵌套归纳类型来归纳地定义一组项，使得一个项要么是常量（其名称由字符串给出），要么是把一个常量应用于一系列常量所得的结果。

```
inductive Term where
| const : String  $\rightarrow$  Term
| app    : String  $\rightarrow$  List Term  $\rightarrow$  Term
```

于是我们可以使用相互递归定义来计算一个项中出现的常量个数，以及一系列项中出现的常量个数。

```
namespace Term
```

```
mutual
def numConsts : Term  $\rightarrow$  Nat
| const _ => 1
| app _ cs => numConstsLst cs

def numConstsLst : List Term  $\rightarrow$  Nat
| [] => 0
| c :: cs => numConsts c + numConstsLst cs
end
```

```
def sample := app "f" [app "g" [const "x"], const "y"]
```

```
#eval numConsts sample
```

```
2
```

```
end Term
```

作为最后一个例子，我们定义函数 `replaceConst a b e`，它在项 `e` 中把常量 `a` 替换为 `b`，然后证明常量个数保持不变。注意，我们的证明使用相互递归（也即归纳）。

```
mutual
def replaceConst (a b : String) : Term  $\rightarrow$  Term
| const c => if a == c then const b else const c
| app f cs => app f (replaceConstLst a b cs)

def replaceConstLst (a b : String) : List Term  $\rightarrow$  List Term
| [] => []
| c :: cs => replaceConst a b c :: replaceConstLst a b cs
end
```

```
mutual
theorem numConsts_replaceConst (a b : String) (e : Term) :
  numConsts (replaceConst a b e) = numConsts e := by
```

```

match e with
| const c =>
  simp [replaceConst]; split <;> simp [numConsts]
| app f cs =>
  simp [replaceConst, numConsts, numConsts_replaceConstLst
    ↪ a b cs]

theorem numConsts_replaceConstLst (a b : String) (es : List
  ↪ Term) :
  numConstsLst (replaceConstLst a b es) = numConstsLst es
  ↪ := by
match es with
| [] => simp [replaceConstLst, numConstsLst]
| c :: cs =>
  simp [replaceConstLst, numConstsLst,
    ↪ numConsts_replaceConst a b c,
    ↪ numConsts_replaceConstLst a b cs]
end

```

8.8 依值模式匹配

我们在 section 8.1 (p. 139) 一节中讨论过的所有模式匹配例子，都可以很容易地用 `casesOn` 和 `recOn` 写出。然而，对于 `Vect  $\alpha$  n` 这样的带索引归纳族，情况通常并非如此，因为情形区分会对索引值施加约束。若没有方程编译器，我们就需要大量样板代码，才能用递归子定义 `map`、`zip` 和 `unzip` 这样非常简单的函数。为理解其中困难，考虑定义一个函数 `tail` 需要做什么：它接受一个向量 $v : \text{Vect } \alpha (n + 1)$ 并删除第一个元素。

```

inductive Vect ( $\alpha : \text{Type } u$ ) : Nat  $\rightarrow$  Type  $u$ 
| nil : Vect  $\alpha$  0
| cons :  $\alpha \rightarrow \{n : \text{Nat}\} \rightarrow \text{Vect } \alpha n \rightarrow \text{Vect } \alpha (n + 1)$ 

```

第一个想法可能是使用 `Vect.casesOn` 函数：

```

Vect.casesOn.{u, v}
{ $\alpha : \text{Type } v$ } {motive : ( $a : \text{Nat}$ )  $\rightarrow$  Vect  $\alpha$   $a \rightarrow$  Sort  $u$ }
{ $a : \text{Nat}$ }
(t : Vect  $\alpha$   $a$ )
(nil : motive 0 nil)
(cons : ( $a : \alpha$ )  $\rightarrow$  { $n : \text{Nat}$ }  $\rightarrow$  (a_1 : Vect  $\alpha$   $n$ )  $\rightarrow$ 
  motive (n + 1) (cons a a_1)) :
motive a t

```

但是在 `nil` 情形中我们应当返回什么值？这里出现了一个微妙之处：如果 v 的类型是 `Vect  $\alpha$  (n + 1)`，它不可能是 `nil`，但并不清楚如何把这一点告诉 `Vect.casesOn`。

一种解决方案是定义一个辅助函数：

```

def tailAux (v : Vect  $\alpha$   $m$ ) :  $m = n + 1 \rightarrow \text{Vect } \alpha n :=$ 
  Vect.casesOn (motive := fun x _ => x = n + 1  $\rightarrow$  Vect  $\alpha$   $n$ ) v

```

```

(fun h : 0 = n + 1 => Nat.noConfusion h)
(fun (a :  $\alpha$ ) (m : Nat) (as : Vect  $\alpha$  m) =>
  fun (h : m + 1 = n + 1) =>
    Nat.noConfusion h (fun h1 : m = n => h1 ▸ as))

```

```

def tail (v : Vect  $\alpha$  (n+1)) : Vect  $\alpha$  n :=
  tailAux v rfl

```

在 `nil` 情形中, m 被实例化为 0 , 而 `Nat.noConfusion` 利用 $0 = n + 1$ 不可能发生这一事实。否则, v 形如 `cons a as`, 我们只需把 `as` 从长度为 m 的向量强制转换为长度为 n 的向量, 然后返回它。

定义 `tail` 的困难在于维持索引之间的关系。`tailAux` 中的假设 $m = n + 1$ 用来传达 n 与小前提相关索引之间的关系。此外, $0 = n + 1$ 情形不可达, 丢弃这类情形的规范方式是使用 `Nat.noConfusion`。

不过, 使用递归方程定义 `tail` 函数很容易, 方程编译器会为我们自动生成所有样板代码。下面还有若干类似的例子:

```

def head : {n : Nat} → Vect  $\alpha$  (n+1) →  $\alpha$ 
| n, cons a as => a

```

```

def tail : {n : Nat} → Vect  $\alpha$  (n+1) → Vect  $\alpha$  n
| n, cons a as => as

```

```

theorem eta :  $\forall$  {n : Nat} (v : Vect  $\alpha$  (n+1)), cons (head v)
  ↪ (tail v) = v
| n, cons a as => rfl

```

```

def map (f :  $\alpha \rightarrow \beta \rightarrow \gamma$ ) : {n : Nat} → Vect  $\alpha$  n → Vect  $\beta$  n →
  ↪ Vect  $\gamma$  n
| 0, nil, nil => nil
| n+1, cons a as, cons b bs => cons (f a b) (map f as bs)

```

```

def zip : {n : Nat} → Vect  $\alpha$  n → Vect  $\beta$  n → Vect ( $\alpha \times \beta$ ) n
| 0, nil, nil => nil
| n+1, cons a as, cons b bs => cons (a, b) (zip as bs)

```

注意, 我们可以省略诸如 `head nil` 这类“不可达”情形的递归方程。为带索引族自动生成的定义远非直截了当。例如:

```

def zipWith (f :  $\alpha \rightarrow \beta \rightarrow \gamma$ ) : {n : Nat} → Vect  $\alpha$  n → Vect  $\beta$  n
  ↪ Vect  $\gamma$  n
| 0, nil, nil => nil
| n+1, cons a as, cons b bs => cons (f a b) (zipWith f as
  ↪ bs)

```

```

#print zipWith

```

```

def Vect.zipWith.{u_1, u_2, u_3} : { $\alpha$  : Type u_1} →
  { $\beta$  : Type u_2} → { $\gamma$  : Type u_3} → ( $\alpha \rightarrow \beta \rightarrow \gamma$ ) → {n : Nat} →
  ↪ Vect  $\alpha$  n → Vect  $\beta$  n → Vect  $\gamma$  n :=
  fun { $\alpha$ } { $\beta$ } { $\gamma$ } f x x_1 x_2 =>

```



```

Vect.brecOn (motive := fun x x_3 => Vect β x → Vect γ x)
  ↪ x_1
  (fun x x_3 f_1 x_4 =>
    (match (motive :=
      (x : Nat) →
        (x_5 : Vect α x) → Vect β x → Vect.below (motive
          ↪ := fun x x_7 => Vect β x → Vect γ x) x_5 →
          Vect γ x)
      x, x_3, x_4 with
    | 0, nil, nil => fun x => nil
    | n.succ, cons a as, cons b bs => fun x => cons (f a
      ↪ b) (x.1 bs))
    f_1)
  x_2

#print zipWith.match_1
def Vect.zipWith.match_1.{u_1, u_2, u_3} : {α : Type u_1} →
  {β : Type u_2} →
  (motive : (x : Nat) → Vect α x → Vect β x → Sort u_3) →
  (x : Nat) →
  (x_1 : Vect α x) →
  (x_2 : Vect β x) →
  (Unit → motive 0 nil nil) →
  ((n : Nat) →
    (a : α) → (as : Vect α n) → (b : β) → (bs :
      ↪ Vect β n) → motive n.succ (cons a as)
      ↪ (cons b bs)) →
    motive x x_1 x_2 :=
fun {α} {β} motive x x_1 x_2 h_1 h_2 =>
Nat.casesOn (motive := fun x => (x_3 : Vect α x) → (x_4 :
  ↪ Vect β x) → motive x x_3 x_4) x
  (fun x x_3 =>
    casesOn (motive := fun a x_4 => Nat.zero = a → x ≈ x_4
      ↪ → motive Nat.zero x x_3) x
      (fun h h_3 =>
        ... ▶
        casesOn (motive := fun a x => Nat.zero = a → x_3
          ↪ ≈ x → motive Nat.zero nil x_3) x_3
          (fun h h_4 => ... ▶ h_1 ()) (fun a {n} a_1 h =>
            ↪ False.elim ...) ... ...)
          (fun a {n} a_1 h => False.elim ...) ... ...)
      (fun n x x_3 =>
        casesOn (motive := fun a x_4 => n.succ = a → x ≈ x_4 →
          ↪ motive n.succ x x_3) x (fun h => False.elim ...)
          (fun a {n_1} a_1 h =>
            n.elimOffset n_1 1 h fun x_4 =>
              Eq.ndrec (motive := fun {n_2} => (a_2 : Vect α
                ↪ n_2) → x ≈ cons a a_2 → motive n.succ x x_3)
                (fun a_2 h =>
                  ... ▶

```

```

cases0n (motive := fun a_3 x => n.succ =
  ↪ a_3 → x_3 ≈ x → motive n.succ (cons a
  ↪ a_2) x_3) x_3
  (fun h => False.elim ...)
  (fun a_3 {n_2} a_4 h =>
    n.elimOffset n_2 1 h fun x =>
      Eq.ndrec (motive := fun {n_3} =>
        (a_5 : Vect β n_3) → x_3 ≈ cons a_3
        ↪ a_5 → motive n.succ (cons a
        ↪ a_2) x_3)
        (fun a_5 h => ... ▶ h_2 n a a_2 a_3
        ↪ a_5) x a_4)
    ... ...)
  x_4 a_1)
  ... ...)
x_1 x_2

```

`zipWith` 函数手工定义起来甚至比 `tail` 函数更繁琐。我们鼓励你使用 `Vect.recOn`、`Vect.casesOn` 和 `Vect.noConfusion` 试一试。

8.9 不可访问模式

有时，依值匹配模式中的某个参数对于定义本身并非必要，但仍必须包含它，以便适当地特化表达式的类型。`Lean` 允许用户把这样的子项标记为对模式匹配不可访问。例如，当左侧出现的某个项既不是变量也不是构造子应用时，这些标注就是必要的，因为这类项并不是模式匹配的合适目标。我们可以把这种不可访问模式视为模式中“无关紧要”的组成部分。你可以通过写 `.(t)` 来声明某个子项不可访问。如果不可访问模式可以被推断，也可以写 `_`。

在下面的例子中，我们声明一个归纳类型，用来定义“属于 f 的像”这一性质。你可以把类型 `ImageOf f b` 的元素看作证据，说明 b 属于 f 的像；构造子 `imf` 用于构造这样的证据。然后，我们可以为任意函数 f 定义一个“逆”，它把 f 的像中的任何东西送到一个映到它的元素。类型规则迫使我们为第一个参数写出 $f a$ ，但这个项既不是变量也不是构造子应用，并且在模式匹配定义中不起作用。为了定义下面的函数 `inverse`，我们必须把 $f a$ 标记为不可访问。

```

inductive ImageOf {α β : Type u} (f : α → β) : β → Type u

```

```

  ↪ where
    | imf : (a : α) → ImageOf f (f a)

```

```

open ImageOf

```

```

def inverse {f : α → β} : (b : β) → ImageOf f b → α
  | .(f a), imf a => a

```

```

def inverse' {f : α → β} : (b : β) → ImageOf f b → α
  | _, imf a => a

```

在上面的例子中，不可访问标注清楚表明 f 不是模式匹配变量。

不可访问模式可用于澄清并控制使用依值模式匹配的定义。考虑函数 `Vect.add` 的如下定义：在假设某类型带有相应加法函数的前提下，它将该类型元素构成的两个向量相加：

```
inductive Vect ( $\alpha$  : Type u) : Nat  $\rightarrow$  Type u
| nil : Vect  $\alpha$  0
| cons :  $\alpha \rightarrow \{n : \text{Nat}\} \rightarrow \text{Vect } \alpha \ n \rightarrow \text{Vect } \alpha \ (n+1)$ 

def Vect.add [Add  $\alpha$ ] : { $n : \text{Nat}$ }  $\rightarrow$  Vect  $\alpha$   $n \rightarrow$  Vect  $\alpha$   $n \rightarrow$  Vect
 $\hookrightarrow \alpha \ n$ 
| 0, nil, nil => nil
|  $n+1$ , cons a as, cons b bs => cons (a + b) (add as bs)
```

参数 $\{n : \text{Nat}\}$ 出现在冒号之后，因为在整个定义中它不能保持固定。实现这个定义时，方程编译器首先根据第一个参数是 0 还是形如 $n+1$ 进行情形区分。随后它对接下来的两个参数进行嵌套情形区分；在每个情形中，方程编译器都会排除与第一个模式不兼容的情形。

但事实上，并不需要对第一个参数作情形区分：当我们将第二个参数作情形区分时，`Vect` 的 `casesOn` 消去子会自动抽象这个参数，并把它替换为 0 和 $n+1$ 。使用不可访问模式，我们可以提示方程编译器避免对 n 作情形区分。

```
def add [Add  $\alpha$ ] : { $n : \text{Nat}$ }  $\rightarrow$  Vect  $\alpha$   $n \rightarrow$  Vect  $\alpha$   $n \rightarrow$  Vect  $\alpha$   $n$ 
| .(_), nil, nil => nil
| .(_), cons a as, cons b bs => cons (a + b) (add as bs)
```

将该位置标记为不可访问模式，会告诉方程编译器两件事：第一，该参数的形式应当由其他参数施加的约束推断出来；第二，第一个参数不应参与模式匹配。

为方便起见，不可访问模式 `.(_)` 可以写作 `_`。

```
def add [Add  $\alpha$ ] : { $n : \text{Nat}$ }  $\rightarrow$  Vect  $\alpha$   $n \rightarrow$  Vect  $\alpha$   $n \rightarrow$  Vect  $\alpha$   $n$ 
| _, nil, nil => nil
| _, cons a as, cons b bs => cons (a + b) (add as bs)
```

如上所述，参数 $\{n : \text{Nat}\}$ 是模式匹配的一部分，因为它不能在整个定义中保持固定。`Lean` 并不要求显式提供这些判别项，而是会为我们自动隐式包含这些额外判别项。

```
def add [Add  $\alpha$ ] { $n : \text{Nat}$ } : Vect  $\alpha$   $n \rightarrow$  Vect  $\alpha$   $n \rightarrow$  Vect  $\alpha$   $n$ 
| nil, nil => nil
| cons a as, cons b bs => cons (a + b) (add as bs)
```

结合自动绑定隐式参数功能后，你可以进一步简化声明，写成：

```
def add [Add  $\alpha$ ] : Vect  $\alpha$   $n \rightarrow$  Vect  $\alpha$   $n \rightarrow$  Vect  $\alpha$   $n$ 
| nil, nil => nil
| cons a as, cons b bs => cons (a + b) (add as bs)
```

利用这些新功能，你可以把前几节定义的其他向量函数更紧凑地写成如下形式：

```
def head : Vect  $\alpha$  (n+1)  $\rightarrow$   $\alpha$ 
| cons a as => a
```

```
def tail : Vect  $\alpha$  (n+1) → Vect  $\alpha$  n
| cons a as => as
```

```
theorem eta : (v : Vect  $\alpha$  (n+1)) → cons (head v) (tail v) = v
| cons a as => rfl
```

```
def map (f :  $\alpha$  →  $\beta$  →  $\gamma$ ) : Vect  $\alpha$  n → Vect  $\beta$  n → Vect  $\gamma$  n
| nil,      nil      => nil
| cons a as, cons b bs => cons (f a b) (map f as bs)
```

```
def zip : Vect  $\alpha$  n → Vect  $\beta$  n → Vect ( $\alpha \times \beta$ ) n
| nil,      nil      => nil
| cons a as, cons b bs => cons (a, b) (zip as bs)
```

8.10 匹配表达式

Lean 还为许多函数式语言中出现的 **match-with** 表达式提供了编译器:

```
def isNotZero (m : Nat) : Bool :=
  match m with
  | 0      => false
  | n + 1 => true
```

这看起来与普通的模式匹配定义差别不大，但关键在于，**match** 可以在表达式中的任意位置使用，并且可以带任意参数。

```
def isNotZero (m : Nat) : Bool :=
  match m with
  | 0      => false
  | n + 1 => true

def filter (p :  $\alpha$  → Bool) : List  $\alpha$  → List  $\alpha$ 
| []      => []
| a :: as =>
  match p a with
  | true => a :: filter p as
  | false => filter p as
```

```
example : filter isNotZero [1, 0, 0, 3, 0] = [1, 3] := rfl
```

下面是另一个例子:

```
def foo (n : Nat) (b c : Bool) :=
  5 + match n - 5, b && c with
  | 0,      true  => 0
  | m + 1, true  => m + 7
  | 0,      false => 5
  | m + 1, false => m + 3
```

```
#eval foo 7 true false
```

```
9
```

```
example : foo 7 true false = 9 := rfl
```

Lean 在内部使用 **match** 构造来实现系统各处的模式匹配。因此，下面四个定义具有相同的最终效果：

```
def bar1 : Nat × Nat → Nat
| (m, n) => m + n
```

```
def bar2 (p : Nat × Nat) : Nat :=
  match p with
  | (m, n) => m + n
```

```
def bar3 : Nat × Nat → Nat :=
  fun (m, n) => m + n
```

```
def bar4 (p : Nat × Nat) : Nat :=
  let (m, n) := p; m + n
```

这些变体对于析构命题同样有用：

```
variable (p q : Nat → Prop)
```

```
example : (∃ x, p x) → (∃ y, q y) → ∃ x y, p x ∧ q y
| (x, px), (y, qy) => (x, y, px, qy)
```

```
example (h0 : ∃ x, p x) (h1 : ∃ y, q y)
  : ∃ x y, p x ∧ q y :=
  match h0, h1 with
  | (x, px), (y, qy) => (x, y, px, qy)
```

```
example : (∃ x, p x) → (∃ y, q y) → ∃ x y, p x ∧ q y :=
  fun (x, px) (y, qy) => (x, y, px, qy)
```

```
example (h0 : ∃ x, p x) (h1 : ∃ y, q y)
  : ∃ x y, p x ∧ q y :=
  let (x, px) := h0
  let (y, qy) := h1
  (x, y, px, qy)
```

8.11 练习

1. 打开一个命名空间 **Hidden** 以避免命名冲突，并使用方程编译器定义自然数上的加法、乘法和幂运算。然后使用方程编译器导出它们的一些基本性质。

2. 类似地，使用方程编译器定义列表上的一些基本操作（如 `reverse` 函数），并通过归纳证明关于列表的定理（例如对任意列表 `xs`，都有 `reverse (reverse xs) = xs`）。
3. 定义你自己的函数，在自然数上执行值历程递归。类似地，看看你是否能自己想出如何定义 `WellFounded.fix`。
4. 仿照 section 8.8 (p. 159) 一节中的例子，定义一个能连接两个向量的函数。这有些棘手；你需要定义一个辅助函数。
5. 考虑下面的算术表达式类型。其思想是，`var n` 是一个变量 v_n ，而 `const n` 是值为 n 的常量。

```
inductive Expr where
  | const : Nat → Expr
  | var : Nat → Expr
  | plus : Expr → Expr → Expr
  | times : Expr → Expr → Expr
deriving Repr
```

```
open Expr
```

```
def sampleExpr : Expr :=
  plus (times (var 0) (const 7)) (times (const 2) (var 1))
```

这里 `sampleExpr` 表示 $(v_0 * 7) + (2 * v_1)$ 。写一个函数对此类表达式求值，将每个 `var n` 求值为 v_n 。

```
def eval (v : Nat → Nat) : Expr → Nat
  | const n      => sorry
  | var n        => v n
  | plus e1 e2   => sorry
  | times e1 e2  => sorry
```

```
def sampleVal : Nat → Nat
  | 0 => 5
  | 1 => 6
  | _ => 0
```

```
-- 试一试。这里应当得到 47。
-- #eval eval sampleVal sampleExpr
```

实现“常量融合”：这是一个把 $5 + 7$ 这样的子项简化为 12 的过程。使用辅助函数 `simpConst`，定义一个函数“fuse”：为了简化加法或乘法，先递归地简化参数，然后应用 `simpConst` 尝试简化结果。

```
def simpConst : Expr → Expr
| plus (const  $n_1$ ) (const  $n_2$ ) => const ( $n_1 + n_2$ )
| times (const  $n_1$ ) (const  $n_2$ ) => const ( $n_1 * n_2$ )
| e => e

def fuse : Expr → Expr := sorry

theorem simpConst_eq (v : Nat → Nat)
: ∀ e : Expr, eval v (simpConst e) = eval v e :=
  sorry

theorem fuse_eq (v : Nat → Nat)
: ∀ e : Expr, eval v (fuse e) = eval v e :=
  sorry
```

最后两个定理表明这些定义保持取值不变。

9 结构与记录

我们已经看到，Lean 的基础系统包含归纳类型。此外，我们还注意到一个引人注目的事实：仅凭类型宇宙、依值箭头类型和归纳类型，就可以构造起相当宏大的数学体系；其余一切都由此而来。Lean 标准库包含许多归纳类型的实例（例如 `Nat`、`Prod`、`List`），甚至逻辑联结词也是用归纳类型定义的。

回忆一下，只包含一个构造子的非递归归纳类型称为结构或记录。积类型是一种结构，依值积（`Sigma`）类型也是如此。一般而言，每当我们定义一个结构 `S` 时，通常也会定义投影函数，使我们能够“析构”`S` 的每个实例，并取回存储在其字段中的值。函数 `Prod.fst` 和 `Prod.snd` 分别返回一个有序对的第一、第二个元素，它们就是这类投影的例子。

在编写程序或形式化数学时，定义包含许多字段的结构并不少见。Lean 中可用的 `structure` 命令提供了支持这一过程的基础设施。当我们用该命令定义结构时，Lean 会自动生成所有投影函数。`structure` 命令还允许我们在既有结构的基础上定义新结构。此外，Lean 还提供了便捷记法，用于定义给定结构的实例。

9.1 声明结构

结构命令本质上是用于定义归纳数据类型的“前端”。每个 `structure` 声明都会引入一个同名命名空间。其一般形式如下：

```
structure <name> <parameters> <parent-structures> where
  <constructor> :: <fields>
```

其中大多数部分都是可选的。下面是一个例子：

```
structure Point (α : Type u) where
  mk ::
  x : α
  y : α
```

`Point` 类型的值用 `Point.mk a b` 创建，而点 `p` 的字段可用 `Point.x p` 和 `Point.y p` 访问（不过 `p.x` 和 `p.y` 也可以，见下文）。结构命令还会生成有用的递归子和定理。下面是上述声明所生成的一些构造。

```
-- 一个类型
#check Point
```

```

| Point.{u} (α : Type u) : Type u
-- 消去子
#check @Point.rec
| @Point.rec : {α : Type u_2} →
  {motive : Point α → Sort u_1} → ((x y : α) → motive { x :=
    → x, y := y }) → (t : Point α) → motive t
-- 构造子
#check @Point.mk -- @Point.mk : {α : Type u_1} → α → α → Point
  ↪ α
| @Point.mk : {α : Type u_1} → α → α → Point α
-- 一个投影
#check @Point.x -- @Point.x : {α : Type u_1} → Point α → α
| @Point.x : {α : Type u_1} → Point α → α
-- 一个投影
#check @Point.y -- @Point.y : {α : Type u_1} → Point α → α
| @Point.y : {α : Type u_1} → Point α → α

```

如果没有给出构造子的名称，则构造子默认命名为 `mk`。

下面是一些使用生成构造的简单定理和表达式。和往常一样，可以使用命令 `open Point` 来省略前缀 `Point`。

```

#eval Point.x (Point.mk 10 20) -- 10
| 10
#eval Point.y (Point.mk 10 20) -- 20
| 20
open Point

example (a b : α) : x (mk a b) = a :=
  rfl

example (a b : α) : y (mk a b) = b :=
  rfl

```

给定 $p : \text{Point Nat}$ ，点记法 $p.x$ 是 `Point.x p` 的简写。这提供了一种访问结构字段的便捷方式。

```

def p := Point.mk 10 20

#check p.x -- p.x : Nat
| p.x : Nat
#eval p.x -- 10
| 10
#eval p.y -- 20

```

20

点记法不仅便于访问记录的投影，也便于应用在同名命名空间中定义的函数。回忆 section 3.3 (p. 35) 中的内容：如果 p 的类型是 `Point`，那么在 `foo` 的第一个非隐式参数类型为 `Point` 的前提下，表达式 `p.foo` 会被解释为 `Point.foo p`。因此，在下面的例子中，表达式 `p.add q` 是 `Point.add p q` 的简写。

```
structure Point ( $\alpha$  : Type  $u$ ) where
```

```
   $x$  :  $\alpha$ 
```

```
   $y$  :  $\alpha$ 
```

```
deriving Repr
```

```
def Point.add ( $p$   $q$  : Point Nat) :=
  mk ( $p.x$  +  $q.x$ ) ( $p.y$  +  $q.y$ )
```

```
def p : Point Nat := Point.mk 1 2
```

```
def q : Point Nat := Point.mk 3 4
```

```
#eval p.add q -- {  $x$  := 4,  $y$  := 6 }
```

```
! {  $x$  := 4,  $y$  := 6 }
```

在下一章中，你将学习如何定义像 `add` 这样的函数，使它不仅适用于 `Point Nat`，而是在假设 α 带有相应加法运算时，能一般地适用于 `Point  $\alpha$` 的元素。

更一般地说，给定表达式 `p.foo x y z`，其中 `p : Point`，Lean 会把 `p` 插入到 `Point.foo` 中类型为 `Point` 的第一个参数位置。例如，对于下面定义的标量乘法，`p.smul 3` 会被解释为 `Point.smul 3 p`。

```
def Point.smul ( $n$  : Nat) ( $p$  : Point Nat) :=
  Point.mk ( $n$  *  $p.x$ ) ( $n$  *  $p.y$ )
```

```
def p : Point Nat := Point.mk 1 2
```

```
#eval p.smul 3 -- {  $x$  := 3,  $y$  := 6 }
```

```
! {  $x$  := 3,  $y$  := 6 }
```

```
example { $p$  : Point Nat} : p.smul 3 = Point.smul 3 p := rfl
```

类似的技巧常用于 `List.map` 函数；该函数把列表作为其第二个非隐式参数：

```
#check @List.map
```

```
@List.map : { $\alpha$  : Type  $u_1$ } → { $\beta$  : Type  $u_2$ } → ( $\alpha$  →  $\beta$ ) → List
  ↪  $\alpha$  → List  $\beta$ 
```

```
def xs : List Nat := [1, 2, 3]
```

```
def f : Nat → Nat := fun x => x * x
```

```
#eval xs.map f -- [1, 4, 9]
```

■ [1, 4, 9]

```
example {xs : List  $\alpha$ } {f :  $\alpha \rightarrow \beta$ } :
  xs.map f = List.map f xs :=
  rfl
```

这里，`xs.map f` 会被解释为 `List.map f xs`。

9.2 对象

我们一直在使用构造子来创建结构类型的元素。对于包含许多字段的结构而言，这通常不方便，因为我们必须记住字段定义的顺序。因此，Lean 提供了以下替代记法，用于定义结构类型的元素。

```
{ (<field-name> := <expr>)* : structure-type }
or
{ (<field-name> := <expr>)* }
```

只要结构的名称可以由期望类型推断出来，后缀 `: structure-type` 就可以省略。例如，我们使用这种记法来定义“点”。字段给出的顺序并不重要，因此下面所有表达式都定义了同一个点。

```
structure Point ( $\alpha$  : Type u) where
```

```
  x :  $\alpha$ 
  y :  $\alpha$ 
```

```
#check { x := 10, y := 20 : Point Nat } -- { x := 10, y := 20
  ↪ } : Point Nat
```

```
■ { x := 10, y := 20 } : Point Nat
```

```
#check { y := 20, x := 10 : Point _ } -- { x := 10, y := 20 }
  ↪ : Point Nat
```

```
■ { x := 10, y := 20 } : Point Nat
```

```
#check ({ x := 10, y := 20 } : Point Nat) -- { x := 10, y :=
  ↪ 20 } : Point Nat
```

```
■ { x := 10, y := 20 } : Point Nat
```

```
example : Point Nat :=
  { y := 20, x := 10 }
```

字段可以用花括号标记为隐式。隐式字段会成为构造子的隐式参数。

如果某个字段的值没有指定，Lean 会尝试推断它。如果未指定的字段无法被推断，Lean 会报告错误，指出相应的占位符无法被合成。

```
structure MyStruct where
```

```
  { $\alpha$  : Type u}
  { $\beta$  : Type v}
  a :  $\alpha$ 
```

$b : \beta$

```
#check { a := 10, b := true : MyStruct }
| {  $\alpha$  := Nat,  $\beta$  := Bool, a := 10, b := true } : MyStruct
```

记录更新是另一种常见操作，它相当于通过修改旧记录中一个或多个字段的值来创建新的记录对象。**Lean** 允许你指定：记录说明中未赋值的字段应从先前定义的结构对象 **s** 中取得；做法是在字段赋值之前添加注记 **s with**。如果提供了多个记录对象，那么 **Lean** 会按顺序访问它们，直到找到包含该未指定字段的对象。如果在访问所有对象后仍有字段名未被指定，**Lean** 就会报错。

```
structure Point ( $\alpha$  : Type u) where
  x :  $\alpha$ 
  y :  $\alpha$ 
deriving Repr
```

```
def p : Point Nat :=
  { x := 1, y := 2 }
```

```
#eval { p with y := 3 } -- { x := 1, y := 3 }
| { x := 1, y := 3 }
```

```
#eval { p with x := 4 } -- { x := 4, y := 2 }
| { x := 4, y := 2 }
```

```
structure Point3 ( $\alpha$  : Type u) where
  x :  $\alpha$ 
  y :  $\alpha$ 
  z :  $\alpha$ 
```

```
def q : Point3 Nat :=
  { x := 5, y := 5, z := 5 }
```

```
def r : Point3 Nat :=
  { p, q with x := 6 }
```

```
example : r.x = 6 := rfl
```

```
example : r.y = 5 := rfl
```

```
example : r.z = 5 := rfl
```

9.3 继承

我们可以通过添加新字段来扩展既有结构。这个特性使我们能够模拟一种继承形式。

```
structure Point ( $\alpha$  : Type u) where
  x :  $\alpha$ 
  y :  $\alpha$ 
```

```
inductive Color where
  | red | green | blue
```

```
structure ColorPoint ( $\alpha$  : Type u) extends Point  $\alpha$  where
  c : Color
```

在下一个例子中，我们使用多重继承定义一个结构，然后利用父结构的对象来定义一个对象。

```
structure Point ( $\alpha$  : Type u) where
  x :  $\alpha$ 
  y :  $\alpha$ 
  z :  $\alpha$ 
```

```
structure RGBValue where
  red : Nat
  green : Nat
  blue : Nat
```

```
structure RedGreenPoint ( $\alpha$  : Type u) extends Point  $\alpha$ , RGBValue
 $\hookrightarrow$  where
  no_blue : blue = 0
```

```
def p : Point Nat :=
  { x := 10, y := 10, z := 20 }
```

```
def rgp : RedGreenPoint Nat :=
  { p with red := 200, green := 40, blue := 0, no_blue := rfl
 $\hookrightarrow$  }
```

```
example : rgp.x = 10 := rfl
```

```
example : rgp.red = 200 := rfl
```

10 类型类

类型类最初作为一种原则化的方法被引入，用以在函数式编程语言中实现特设多态。首先注意，如果一个特设多态函数（如加法）只是把某个类型专属的加法实现作为参数，并随后把该实现应用于其余参数，那么实现它将很容易。例如，假设我们在 Lean 中声明一个结构来保存加法的实现。

```
structure Add ( $\alpha$  : Type) where
  add :  $\alpha \rightarrow \alpha \rightarrow \alpha$ 
```

```
#check @Add.add -- @Add.add : { $\alpha$  : Type}  $\rightarrow$  Add  $\alpha \rightarrow \alpha \rightarrow \alpha \rightarrow \alpha$ 
```

```
| @Add.add : { $\alpha$  : Type}  $\rightarrow$  Add  $\alpha \rightarrow \alpha \rightarrow \alpha \rightarrow \alpha$ 
```

在上面的 Lean 代码中，字段 `add` 的类型为 `Add.add : { $\alpha$  : Type}  $\rightarrow$  Add  $\alpha \rightarrow \alpha \rightarrow \alpha \rightarrow \alpha$` ，其中类型 α 外的花括号表示它是一个隐式参数。我们可以如下实现 `double`：

```
def double ( $s$  : Add  $\alpha$ ) ( $x$  :  $\alpha$ ) :  $\alpha$  :=
   $s.add$   $x$   $x$ 
```

```
#eval double { add := Nat.add } 10 -- 20
```

```
| 20
```

```
#eval double { add := Nat.mul } 10 -- 100
```

```
| 100
```

```
#eval double { add := Int.add } 10 -- 20
```

```
| 20
```

注意，可以用 `double { add := Nat.add }  $n$` 将自然数 n 加倍。当然，让用户以这种方式手动传递实现会非常繁琐。事实上，这会抵消特设多态的大部分潜在益处。

类型类背后的主要思想，是使诸如 `Add  $\alpha$` 这样的参数成为隐式参数，并使用用户定义实例的数据库，通过称为类型类解析的过程自动合成所需实例。在 Lean 中，把上例中的 `structure` 改为 `class` 后，`Add.add` 的类型变为：

```
class Add ( $\alpha$  : Type) where
  add :  $\alpha \rightarrow \alpha \rightarrow \alpha$ 
```

```
#check @Add.add -- @Add.add : { $\alpha$  : Type}  $\rightarrow$  [self : Add  $\alpha$ ]  $\rightarrow \alpha$ 
   $\hookrightarrow \rightarrow \alpha \rightarrow \alpha$ 
```

```
| @Add.add : { $\alpha$  : Type}  $\rightarrow$  [self : Add  $\alpha$ ]  $\rightarrow \alpha \rightarrow \alpha \rightarrow \alpha$ 
```

其中方括号表示类型为 `Add  $\alpha$` 的参数是实例隐式参数，也就是说它应当通过类型类解析来合成。这个版本的 `add` 是 Lean 中对应于 Haskell 项 `add :: Add a => a -> a -> a` 的类似物。类似地，我们可以如下注册实例：

```
instance : Add Nat where
  add := Nat.add
```

```
instance : Add Int where
  add := Int.add
```

```
instance : Add Float where
  add := Float.add
```

于是对于 $n : \text{Nat}$ 和 $m : \text{Nat}$ ，项 `Add.add  $n$   $m$` 会触发以 `Add Nat` 为目标的类型类解析，而类型类解析会合成上面关于 `Nat` 的实例。现在我们可以使用实例隐式参数来重新实现 `double`：

```
def double [Add  $\alpha$ ] ( $x : \alpha$ ) :  $\alpha$  :=
  Add.add  $x$   $x$ 
```

```
#check @double -- @double : { $\alpha$  : Type} → [Add  $\alpha$ ] →  $\alpha$  →  $\alpha$ 
```

```
| @double : { $\alpha$  : Type} → [Add  $\alpha$ ] →  $\alpha$  →  $\alpha$ 
```

```
#eval double 10 -- 20
```

```
| 20
```

```
#eval double (10 : Int) -- 20
```

```
| 20
```

```
#eval double (7 : Float) -- 14.000000
```

```
| 14.000000
```

```
#eval double (239.0 + 2) -- 482.000000
```

```
| 482.000000
```

一般而言，实例可以以复杂方式依赖于其他实例。例如，可以声明一个实例，说明如果 α 具有加法，那么 `Array  $\alpha$` 也具有加法：

```
instance [Add  $\alpha$ ] : Add (Array  $\alpha$ ) where
  add  $x$   $y$  := Array.zipWith ( $\cdot$  +  $\cdot$ )  $x$   $y$ 
```

```
#eval Add.add #[1, 2] #[3, 4] -- #[4, 6]
```

```
| #[4, 6]
```

```
#eval #[1, 2] + #[3, 4] -- #[4, 6]
```

```
| #[4, 6]
```

注意，`( $\cdot$  +  $\cdot$ )` 是 Lean 中 `fun  $x$   $y$  =>  $x$  +  $y$` 的记法。

上面的例子展示了类型类如何用于重载记法。现在，我们探讨另一种应用。我们经常需要某个给定类型的任意元素。回忆一下，在 Lean 中类型可能没有任何元素。我们常常希望某个定义在“边界情形”下返回一个任意元素。例如，当 xs 的类型为 `List  $\alpha$` 时，我们可能希望表达式 `head  $xs$` 的类型为 α 。类似地，

许多定理在附加假设某个类型非空时才成立。例如，如果 α 是一个类型，那么 $\exists x : \alpha, x = x$ 只有在 α 非空时才为真。标准库定义了类型类 `Inhabited`，使类型类推断能够推断有默认元素类型的“默认”元素。让我们从上述方案的第一步开始，即声明一个适当的类：

```
class Inhabited ( $\alpha$  : Type u) where
  default :  $\alpha$ 
```

```
#check @Inhabited.default -- @Inhabited.default : { $\alpha$  : Type
  ↪ u_1} → [self : Inhabited  $\alpha$ ] →  $\alpha$ 
| @Inhabited.default : { $\alpha$  : Type u_1} → [self : Inhabited  $\alpha$ ] →
  ↪  $\alpha$ 
```

注意，`Inhabited.default` 没有任何显式参数。类 `Inhabited  $\alpha$` 的一个元素只是形如 `Inhabited.mk x` 的表达式，其中 $x : \alpha$ 。投影 `Inhabited.default` 允许我们从 `Inhabited  $\alpha$` 的元素中“提取”这样一个 α 的元素。现在我们用一些实例填充该类：

```
instance : Inhabited Bool where
  default := true
```

```
instance : Inhabited Nat where
  default := 0
```

```
instance : Inhabited Unit where
  default := ()
```

```
instance : Inhabited Prop where
  default := True
```

```
#eval (Inhabited.default : Nat) -- 0
```

```
| 0
```

```
#eval (Inhabited.default : Bool) -- true
```

```
| true
```

可以使用命令 `export` 为 `Inhabited.default` 创建别名 `default`。

```
export Inhabited (default)
```

```
#eval (default : Nat) -- 0
```

```
| 0
```

```
#eval (default : Bool) -- true
```

```
| true
```

10.1 实例的链接

如果类型类推断仅止于此，那么它并不算十分令人印象深刻：它不过是把一系列实例存储起来，供精化器在查找表中查找的机制。类型类推断的强大之处在于

可以链接实例。也就是说，一个实例声明本身可以依赖某个类型类的隐式实例。这会使类推断通过实例进行递归链接，并在必要时回溯，形成一种类似 Prolog 的搜索。

例如，下面的定义表明，如果两个类型 α 和 β 都有默认元素，那么它们的乘积类型也有默认元素：

```
instance [Inhabited  $\alpha$ ] [Inhabited  $\beta$ ] : Inhabited ( $\alpha \times \beta$ ) where
  default := (default, default)
```

把它加入前面的实例声明后，类型类实例推断便可以推断出例如 `Nat × Bool` 的默认元素：

```
instance [Inhabited  $\alpha$ ] [Inhabited  $\beta$ ] : Inhabited ( $\alpha \times \beta$ ) where
  default := (default, default)
```

```
#eval (default : Nat × Bool) -- (0, true)
```

```
| (0, true)
```

类似地，我们可以用合适的常值函数使函数类型具有默认元素：

```
instance [Inhabited  $\beta$ ] : Inhabited ( $\alpha \rightarrow \beta$ ) where
  default := fun _ => default
```

作为练习，请尝试为其他类型定义默认实例，例如 `List` 和 `Sum` 类型。

Lean 标准库包含定义 `inferInstance`。它的类型是 $\{\alpha : \text{Sort } u\} \rightarrow [i : \alpha] \rightarrow \alpha$ ，当期望类型是一个实例时，它可用于触发类型类解析过程。

```
#check (inferInstance : Inhabited Nat) -- inferInstance :
  ↪ Inhabited Nat
```

```
| inferInstance : Inhabited Nat
```

```
def foo : Inhabited (Nat × Nat) := inferInstance
```

```
theorem ex : foo.default = (default, default) :=
  rfl
```

可以使用命令 `#print` 来查看 `inferInstance` 有多么简单。

```
#print inferInstance
```

```
| @[reducible] def inferInstance.{u} : { $\alpha : \text{Sort } u$ } → [ $i : \alpha$ ] →
  ↪  $\alpha :=$ 
  | fun { $\alpha$ } [ $i : \alpha$ ] => i
```

10.2 ToString (转为字符串)

多态方法 `toString` 的类型是 $\{\alpha : \text{Type } u\} \rightarrow [\text{ToString } \alpha] \rightarrow \alpha \rightarrow \text{String}$ 。你可以为自己的类型实现其实例，并通过链接把复杂值转换为字符串。Lean 为大多数内建类型提供了 `ToString` 实例。

```

structure Person where
  name : String
  age  : Nat

instance : ToString Person where
  toString p := p.name ++ "@" ++ toString p.age

#eval toString { name := "Leo", age := 542 : Person } --
↪ "Leo@542"
| "Leo@542"
#eval toString ({ name := "Daniel", age := 18 : Person },
↪ "hello") -- "(Daniel@18, hello)"
| "(Daniel@18, hello)"

```

10.3 数值字面量

数值字面量在 Lean 中是多态的。你可以使用一个数值字面量（例如 2）来表示任何实现了类型类 `OfNat` 的类型的元素。

```

structure Rational where
  num : Int
  den : Nat
  inv : den ≠ 0

instance : OfNat Rational n where
  ofNat := { num := n, den := 1, inv := by decide }

instance : ToString Rational where
  toString r := s! "{r.num}/{r.den}"

#eval (2 : Rational) -- 2/1
| 2/1
#check (2 : Rational) -- 2 : Rational
| 2 : Rational
#check (2 : Nat)      -- 2 : Nat
| 2 : Nat

```

Lean 分别将项 `(2 : Nat)` 和 `(2 : Rational)` 精化为 `@OfNat.ofNat Nat 2 (@instOfNatNat 2)` 与 `@OfNat.ofNat Rational 2 (@instOfNatRational 2)`。我们称精化后项中出现的数值字面量 2 为原始自然数。可以使用宏 `nat_lit` 2 输入原始自然数 2。

```

#check nat_lit 2 -- 2 : Nat
| 2 : Nat

```

原始自然数 不是多态的。

`OfNat` 实例以数值字面量为参数。因此，可以为特定数值字面量定义实例。第二个参数通常像上例那样是变量，或者是一个 原始自然数。

```
class Monoid ( $\alpha$  : Type u) where
  unit :  $\alpha$ 
  op    :  $\alpha \rightarrow \alpha \rightarrow \alpha$ 

instance [s : Monoid  $\alpha$ ] : OfNat  $\alpha$  (nat_lit 1) where
  ofNat := s.unit

def getUnit [Monoid  $\alpha$ ] :  $\alpha$  :=
  1
```

10.4 输出参数

默认情况下，Lean 只有在项 T 已知且不包含缺失部分时，才会尝试合成实例 `Inhabited  $T$` 。下面的命令会产生错误 `typeclass instance problem is stuck, it is often due to metavariables`，因为该类型有一个缺失部分（即 `_`）。

```
/--
error: typeclass instance problem is stuck, it is often due to
  ↪ metavariables
    Inhabited (Nat × ?m.2)
-/
#guard_msgs (error) in
#eval (inferInstance : Inhabited (Nat × _))
```

可以把类型类 `Inhabited` 的参数看作类型类合成器的一个输入值。当类型类有多个参数时，可以把其中一些标记为输出参数。即便这些参数含有缺失部分，Lean 也会启动类型类合成器。在下面的例子中，我们使用输出参数来定义异质多态乘法。

```
class HMul ( $\alpha$  : Type u) ( $\beta$  : Type v) ( $\gamma$  : outParam (Type w))
  ↪ where
    hMul :  $\alpha \rightarrow \beta \rightarrow \gamma$ 

export HMul (hMul)

instance : HMul Nat Nat Nat where
  hMul := Nat.mul

instance : HMul Nat (Array Nat) (Array Nat) where
  hMul a bs := bs.map (fun b => hMul a b)

#eval hMul 4 3          -- 12
```

12

```
#eval hMul 4 #[2, 3, 4] -- #[8, 12, 16]
```

#[8, 12, 16]

参数 α 和 β 被视为输入参数，而 γ 被视为输出参数。给定应用 `hMul a b`，在 a 和 b 的类型已知后，类型类合成器会被调用，并从输出参数 γ 得到结果类型。在上面的例子中，我们定义了两个实例。第一个是自然数的同质乘法；第二个是数组的标量乘法。注意，你可以链接实例并推广第二个实例。

```
class HMul ( $\alpha$  : Type u) ( $\beta$  : Type v) ( $\gamma$  : outParam (Type w))
```

```
  ↪ where
```

```
    hMul :  $\alpha \rightarrow \beta \rightarrow \gamma$ 
```

```
export HMul (hMul)
```

```
instance : HMul Nat Nat Nat where
```

```
  hMul := Nat.mul
```

```
instance : HMul Int Int Int where
```

```
  hMul := Int.mul
```

```
instance [HMul  $\alpha$   $\beta$   $\gamma$ ] : HMul  $\alpha$  (Array  $\beta$ ) (Array  $\gamma$ ) where
```

```
  hMul a bs := bs.map (fun b => hMul a b)
```

```
#eval hMul 4 3 -- 12
```

12

```
#eval hMul 4 #[2, 3, 4] -- #[8, 12, 16]
```

#[8, 12, 16]

```
#eval hMul (-2) #[3, -1, 4] -- #[-6, 2, -8]
```

#[-6, 2, -8]

```
#eval hMul 2 #[#[2, 3], #[0, 4]] -- #[#[4, 6], #[0, 8]]
```

#[#[4, 6], #[0, 8]]

只要有实例 `HMul  $\alpha$   $\beta$   $\gamma$` ，就可以把我们新的标量数组乘法实例用于类型为 `Array  $\beta$` 的数组和类型为 α 的标量。在最后一个 `#eval` 中，注意该实例在数组的数组上被使用了两次。

输出参数在实例合成过程中会被忽略。即使实例合成发生在输出参数的值已经确定的上下文中，这些值也会被忽略。一旦 `Lean` 使用输入参数找到了一个实例，它会确保输出参数的既有已知值与所找到的值相匹配。

`Lean` 还具有半输出参数，它们兼有输入参数和输出参数的一些特征。像输入参数一样，半输出参数在选择实例时会被考虑；像输出参数一样，它们可用于实例化未知值。然而，它们并不会唯一地完成这一点。带有半输出参数的实例合成可能更难预测，因为实例被考虑的顺序可能决定所选实例，但它也更加灵活。

10.5 默认实例

在类 `HMul` 中, 参数 α 和 β 被当作输入值。因此, 类型类合成只有在这两个类型已知后才会开始。这常常可能过于受限。

```
class HMul ( $\alpha$  : Type u) ( $\beta$  : Type v) ( $\gamma$  : outParam (Type w))
  ↪ where
    hMul :  $\alpha \rightarrow \beta \rightarrow \gamma$ 
```

```
export HMul (hMul)
```

```
instance : HMul Int Int Int where
  hMul := Int.mul
```

```
def xs : List Int := [1, 2, 3]
```

```
/--
error: typeclass instance problem is stuck
  HMul Int ?m.2 (?m.11 y)
```

```
Note: Lean will not try to resolve this typeclass instance
  ↪ problem because the second type argument to `HMul` is a
  ↪ metavariable. This argument must be fully determined
  ↪ before Lean will try to resolve the typeclass.
```

```
Hint: Adding type annotations and supplying implicit arguments
  ↪ to functions can give Lean more information for typeclass
  ↪ resolution. For example, if you have a variable `x` that
  ↪ you intend to be a `Nat`, but Lean reports it as having an
  ↪ unresolved type like `?m`, replacing `x` with `(x : Nat)`
  ↪ can get typeclass resolution un-stuck.
```

```
-/
#guard_msgs (error) in
#eval fun y => xs.map (fun x => hMul x y)
```

Lean 没有合成实例 `HMul`, 因为尚未给出 y 的类型。然而, 在这种情形下, 自然会假定 y 和 x 的类型应当相同。我们可以用默认实例精确地实现这一点。

```
class HMul ( $\alpha$  : Type u) ( $\beta$  : Type v) ( $\gamma$  : outParam (Type w))
  ↪ where
    hMul :  $\alpha \rightarrow \beta \rightarrow \gamma$ 
```

```
export HMul (hMul)
```

```
@[default_instance]
instance : HMul Int Int Int where
  hMul := Int.mul
```

```
def xs : List Int := [1, 2, 3]
```

```
#check fun y => xs.map (fun x => hMul x y) -- fun y =>
  ↳ List.map (fun x => hMul x y) xs : Int → List Int
| fun y => List.map (fun x => hMul x y) xs : Int → List Int
```

通过给上面的实例加上属性 `[default_instance]`，我们指示 Lean 在待处理的类型类合成问题上使用此实例。Lean 的实际实现为算术运算符定义了同质和异质的类。此外， $a + b$ 、 $a * b$ 、 $a - b$ 、 a / b 和 $a \% b$ 都是异质版本的记法。实例 `OfNat Nat n` 是 `OfNat` 类的默认实例（优先级为 100）。这就是为什么在期望类型未知时，数值字面量 2 的类型为 `Nat`。可以定义更高优先级的默认实例来覆盖内建实例。

```
structure Rational where
  num : Int
  den : Nat
  inv : den ≠ 0

@[default_instance 200]
instance : OfNat Rational n where
  ofNat := { num := n, den := 1, inv := by decide }
```

```
instance : ToString Rational where
  toString r := s! "{r.num}/{r.den}"
```

```
#check 2 -- 2 : Rational
```

```
| 2 : Rational
```

优先级也有助于控制不同默认实例之间的交互。例如，假设 `xs` 的类型为 `List α`。在精化 `xs.map (fun x => 2 * x)` 时，我们希望乘法的同质实例具有比 `OfNat α 2` 的默认实例更高的优先级。当我们只实现了实例 `HMul α α α` 而没有实现 `HMul Nat α α` 时，这一点尤其重要。现在，我们展示 Lean 中记法 $a * b$ 是如何定义的。

```
class OfNat (α : Type u) (n : Nat) where
  ofNat : α

@[default_instance]
instance (n : Nat) : OfNat Nat n where
  ofNat := n

class HMul (α : Type u) (β : Type v) (γ : outParam (Type w))
  ↳ where
  hMul : α → β → γ

class Mul (α : Type u) where
  mul : α → α → α

@[default_instance 10]
instance [Mul α] : HMul α α α where
```

```
hMul a b := Mul.mul a b
```

```
infixl:70 " * " => hMul.hMul
```

Mul 类对于只实现同质乘法的类型很方便。

10.6 局部实例

在 Lean 中，类型类是使用属性实现的。因此，可以使用 **local** 修饰符表示它们只在当前 **section** 或 **namespace** 关闭之前有效，或一直有效到当前文件末尾。

```
structure Point where
```

```
  x : Nat
```

```
  y : Nat
```

```
section
```

```
local instance : Add Point where
```

```
  add a b := { x := a.x + b.x, y := a.y + b.y }
```

```
def double (p : Point) :=
```

```
  p + p
```

```
end -- instance `Add Point` is not active anymore
```

```
/--
```

```
error: failed to synthesize instance of type class
```

```
  HAdd Point Point ?m.5
```

```
Hint: Type class instance resolution failures can be inspected
```

```
↪ with the `set_option trace.Meta.synthInstance true`
```

```
↪ command.
```

```
-/
```

```
#guard_msgs in
```

```
def triple (p : Point) :=
```

```
  p + p + p
```

也可以使用 **attribute** 命令临时禁用一个实例，直到当前 **section** 或 **namespace** 关闭，或直到当前文件末尾。

```
structure Point where
```

```
  x : Nat
```

```
  y : Nat
```

```
instance addPoint : Add Point where
```

```
  add a b := { x := a.x + b.x, y := a.y + b.y }
```



```

def double (p : Point) :=
  p + p

attribute [-instance] addPoint

/--
error: failed to synthesize instance of type class
  HAdd Point Point ?m.5

Hint: Type class instance resolution failures can be inspected
↪ with the `set_option trace.Meta.synthInstance true`
↪ command.
-/
#guard_msgs in
def triple (p : Point) :=
  p + p + p -- Error: failed to synthesize instance

```

我们建议你只在诊断问题时使用此命令。

10.7 作用域实例

也可以在命名空间中声明作用域实例。这类实例只有在你位于该命名空间内部或打开该命名空间时才处于活动状态。

```

structure Point where
  x : Nat
  y : Nat

namespace Point

scoped instance : Add Point where
  add a b := { x := a.x + b.x, y := a.y + b.y }

def double (p : Point) :=
  p + p

end Point
-- instance `Add Point` is not active anymore

/--
error: failed to synthesize instance of type class
  HAdd Point Point ?m.3

Hint: Type class instance resolution failures can be inspected
↪ with the `set_option trace.Meta.synthInstance true`
↪ command.
-/
#guard_msgs (error) in
#check fun (p : Point) => p + p + p

```

```

| fun p => sorry : (p : Point) → ?m.6 p
namespace Point
-- instance `Add Point` is active again
#check fun (p : Point) => p + p + p
| fun p => p + p + p : Point → Point
end Point

open Point -- activates instance `Add Point`
#check fun (p : Point) => p + p + p
| fun p => p + p + p : Point → Point

```

可以使用命令 **open scoped** <namespace> 来激活作用域属性，但不会“打开”该命名空间中的名称。

```

structure Point where
  x : Nat
  y : Nat

namespace Point

scoped instance : Add Point where
  add a b := { x := a.x + b.x, y := a.y + b.y }

def double (p : Point) :=
  p + p

end Point

open scoped Point -- activates instance `Add Point`
#check fun (p : Point) => p + p + p
| fun p => p + p + p : Point → Point
/--
error: Unknown identifier `double`
-/
#guard_msgs (error) in
#check fun (p : Point) => double p
| fun p => sorry : (p : Point) → ?m.2 p

```

10.8 可判定命题

让我们考虑标准库中定义的另一个类型类示例，即 **Decidable** 命题的类型类。粗略地说，如果我们能够判定一个 **Prop** 的元素是真还是假，就说它是可判定的。这一区分只在构造性数学中 useful；在经典意义下，每个命题都是可判定的。

但是，如果我们使用经典原则，例如按情况定义函数，那么该函数将不是可计算的。从算法角度看，**Decidable** 类型类可用于推断一个能有效判断该命题是否为真的过程。因此，该类型类在可能时支持这类计算性定义，同时又允许平滑过渡到经典定义和经典推理。

在标准库中，**Decidable** 形式上定义如下：

```
class inductive Decidable (p : Prop) where
| isFalse (h : ¬p) : Decidable p
| isTrue  (h : p)  : Decidable p
```

从逻辑上说，拥有一个元素 $t : \text{Decidable } p$ 强于拥有一个元素 $t' : p \vee \neg p$ ；它使我们能够根据 p 的真值定义任意类型的值。例如，要使表达式 **if** p **then** a **else** b 有意义，我们需要知道 p 是可判定的。该表达式是 **ite** p a b 的语法糖，而 **ite** 定义如下：

```
def ite {α : Sort u}
  (c : Prop) [h : Decidable c]
  (t e : α) : α :=
  h.casesOn (motive := fun _ => α) (fun _ => e) (fun _ => t)
```

标准库还包含 **ite** 的一个变体，称为 **dite**，即依值的 if-then-else 表达式。它定义如下：

```
def dite {α : Sort u}
  (c : Prop) [h : Decidable c]
  (t : c → α) (e : Not c → α) : α :=
  Decidable.casesOn (motive := fun _ => α) h e t
```

也就是说，在 **dite** c t e 中，我们可以在“then”分支中假设 $hc : c$ ，并在“else”分支中假设 $hnc : \neg c$ 。为了使 **dite** 更便于使用，Lean 允许我们写 **if** $h : c$ **then** t **else** e ，而不是 **dite** c (fun $h : c \Rightarrow t$ h) (fun $h : \neg c \Rightarrow e$ h)。

没有经典逻辑，我们无法证明每个命题都是可判定的。但我们可以证明某些命题是可判定的。例如，可以证明自然数和整数上的相等、比较等基本运算的可判定性。此外，可判定性在命题联结词下保持：

```
#check @instDecidableAnd -- @instDecidableAnd : {p q : Prop} →
  ↪ [dp : Decidable p] → [dq : Decidable q] → Decidable (p ∧
  ↪ q)
| @instDecidableAnd : {p q : Prop} → [dp : Decidable p] → [dq :
  ↪ Decidable q] → Decidable (p ∧ q)
#check @instDecidableOr
| @instDecidableOr : {p q : Prop} → [dp : Decidable p] → [dq :
  ↪ Decidable q] → Decidable (p ∨ q)
#check @instDecidableNot
| @instDecidableNot : {p : Prop} → [dp : Decidable p] →
  ↪ Decidable ¬p
```

因此，我们可以在自然数上的可判定谓词上按情况进行定义：

```
def step (a b x : Nat) : Nat :=
  if  $x < a \vee x > b$  then 0 else 1
```

```

set_option pp.explicit true
#print step
def step : Nat → Nat → Nat → Nat :=
fun a b x =>
  @ite Nat (Or (@LT.lt Nat instLTNat x a) (@GT.gt Nat
    ↳ instLTNat x b))
    (@instDecidableOr (@LT.lt Nat instLTNat x a) (@GT.gt Nat
      ↳ instLTNat x b) (Nat.decLt x a) (Nat.decLt b x))
    (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0)))
    ↳ (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit
      ↳ 1))))

```

打开隐式参数会显示，精化器仅通过应用适当的实例，便已推断出命题 $x < a \vee x > b$ 的可判定性。

借助经典公理，我们可以证明每个命题都是可判定的。可以导入经典公理，并通过打开 `Classical` 命名空间使通用的可判定性实例可用。

```
open Classical
```

此后，对于每个 p ，`Decidable p` 都有一个实例。因此，当你想进行经典推理时，库中所有依赖可判定性假设的定理都可以自由使用。在 chapter 12 (p. 199) 中，我们将看到，使用排中律来定义函数可能会阻止它们以计算方式使用。因此，标准库为 `propDecidable` 实例赋予较低优先级。

```
open Classical
```

```
noncomputable scoped
```

```
instance (priority := low) propDecidable (a : Prop) :
```

```

↳ Decidable a :=
  choice <| match em a with
    | Or.inl h => {isTrue h}
    | Or.inr h => {isFalse h}

```

这保证了 Lean 会优先采用其他实例，只有在推断可判定性的其他尝试都失败后，才退回到 `propDecidable`。

`Decidable` 类型类还为证明定理提供了一点小规模自动化。标准库引入了策略 `decide`，它使用 `Decidable` 实例来解决简单目标；还引入了函数 `decide`，它使用 `Decidable` 实例来计算相应的 `Bool`。

```
example : 10 < 5 ∨ 1 > 0 := by
  decide
```

```
example : ¬(True ∧ False) := by
  decide
```

```
example : 10 * 20 = 200 := by
  decide
```

```
theorem ex : True ∧ 2 = 1 + 1 := by
  decide
```

```
#print ex
| theorem ex : True ∧ 2 = 1 + 1 :=
| of_decide_eq_true (id (Eq.refl true))
#check @of_decide_eq_true -- @of_decide_eq_true : ∀ {p : Prop}
↪ [inst : Decidable p], decide p = true → p
| @of_decide_eq_true : ∀ {p : Prop} [inst : Decidable p],
| ↪ decide p = true → p
#check @decide -- decide : (p : Prop) → [h : Decidable p] →
↪ Bool
| decide : (p : Prop) → [h : Decidable p] → Bool
```

它们的工作方式如下。表达式 `decide p` 尝试推断 p 的判定过程；若成功，则求值为 `true` 或 `false`。特别地，如果 p 是一个为真的闭表达式，那么 `decide p` 会按定义归约为布尔值 `true`。在假设 `decide p = true` 成立时，`of_decide_eq_true` 产生 p 的证明。策略 `decide` 将这些合在一起以证明目标 p 。根据前面的观察，只要为 p 推断出的判定过程有足够信息按定义求值到 `isTrue` 情形，`decide` 就会成功。

10.9 管理类型类推断

如果你遇到需要提供一个 Lean 能够通过类型类推断推得的表达式的情形，可以要求 Lean 使用 `inferInstance` 执行该推断：

```
def foo : Add Nat := inferInstance
def bar : Inhabited (Nat → Nat) := inferInstance

#check @inferInstance -- @inferInstance : {α : Sort u_1} → [i
↪ : α] → α
| @inferInstance : {α : Sort u_1} → [i : α] → α
```

事实上，可以使用 Lean 的 $(t : T)$ 记法以简洁方式指定你要寻找其实例的类：

```
#check (inferInstance : Add Nat)
| inferInstance : Add Nat
```

也可以使用辅助定义 `inferInstanceAs`：

```
#check (inferInstanceAs (Add Nat) : Add Nat)
| { add := Nat.add } : Add Nat
```

有时 Lean 找不到实例，是因为该类隐藏在某个定义之下。例如，Lean 无法找到 `Inhabited (Set α)` 的实例。我们可以显式声明一个：

```
def Set (α : Type u) := α → Prop
```

```

/--
error: failed to synthesize instance of type class
      Inhabited (Set  $\alpha$ )

Hint: Type class instance resolution failures can be inspected
      ↪ with the `set_option trace.Meta.synthInstance true`
      ↪ command.
-/
#guard_msgs in
example : Inhabited (Set  $\alpha$ ) :=
  inferInstance

instance : Inhabited (Set  $\alpha$ ) :=
  inferInstanceAs (Inhabited ( $\alpha \rightarrow \text{Prop}$ ))

```

有时，你可能会发现类型类推断无法找到期望的实例；更糟的是，它可能陷入无限循环并超时。为帮助调试这些情形，Lean 允许你请求搜索轨迹：

```
set_option trace.Meta.synthInstance true
```

如果使用 VS Code，可以将鼠标悬停在相关定理或定义上读取结果，或者用 Ctrl Shift Enter 打开消息窗口。

也可以使用以下选项限制搜索：

```
set_option synthInstance.maxHeartbeats 10000
set_option synthInstance.maxSize 400
```

选项 `synthInstance.maxHeartbeats` 指定每个类型类解析问题的最大心跳数。一个心跳指（小规模）内存分配的次数（以千为单位）；0 表示没有限制。选项 `synthInstance.maxSize` 是在类型类实例合成过程中用于构造解的最大实例数。

还要记住，在 VS Code 和 Emacs 两种编辑器模式中，`set_option` 中都可以使用 Tab 补全来帮助你找到合适的选项。

如上所述，给定上下文中的类型类实例表示一个类似 Prolog 的程序，由此产生回溯搜索。程序的效率以及找到的解都可能依赖于系统尝试实例的顺序。后声明的实例会先被尝试。此外，如果实例在其他模块中声明，则它们被尝试的顺序取决于命名空间打开的顺序。较晚打开的命名空间中声明的实例会较早被尝试。

可以通过给类型类实例指定 优先级来改变尝试它们的顺序。声明实例时，它会被赋予一个默认优先级值。定义实例时可以指定其他优先级。下面的例子说明了如何做到这一点：

```

class Foo where
  a : Nat
  b : Nat

instance (priority := default + 1) i1 : Foo where
  a := 1

```

```

b := 1

instance i2 : Foo where
  a := 2
  b := 2

example : Foo.a = 1 :=
  rfl

instance (priority := default + 2) i3 : Foo where
  a := 3
  b := 3

example : Foo.a = 3 :=
  rfl

```

10.10 使用类型类的强制转换

最基本的强制转换把一个类型的元素映射到另一个类型。例如，从 `Nat` 到 `Int` 的强制转换允许我们把任意元素 $n : \text{Nat}$ 看作 `Int` 的元素。但有些强制转换依赖于参数；例如，对于任意类型 α ，我们可以把任意元素 $as : \text{List } \alpha$ 看作 `Set  $\alpha$` 的元素，即列表中出现的元素所组成的集合。相应的强制转换定义在由 α 参数化的类型“族” `List  $\alpha$` 上。

Lean 允许我们声明三类强制转换：

- 从一个类型族到另一个类型族
- 从一个类型族到 `sort` 类
- 从一个类型族到函数类型类

第一类强制转换允许我们把源族某个成员的任意元素看作目标族相应成员的元素。第二类强制转换允许我们把源族某个成员的任意元素看作一个类型。第三类强制转换允许我们把源族的任意元素看作一个函数。下面依次考察这三类。

在 Lean 中，强制转换建立在类型类解析框架之上。通过声明 `Coe  $\alpha$   $\beta$` 的实例，我们定义从 α 到 β 的强制转换。例如，可以如下定义从 `Bool` 到 `Prop` 的强制转换：

```

instance : Coe Bool Prop where
  coe b := b = true

```

这使我们能够在 `if-then-else` 表达式中使用布尔项：

```

#eval if true then 5 else 3
| 5
#eval if false then 5 else 3

```

3

可以如下定义从 $\text{List } \alpha$ 到 $\text{Set } \alpha$ 的强制转换:

```
def List.toSet : List  $\alpha$   $\rightarrow$  Set  $\alpha$ 
| []      => Set.empty
| a::as => {a}  $\cup$  as.toSet
```

```
instance : Coe (List  $\alpha$ ) (Set  $\alpha$ ) where
  coe a := a.toSet
```

```
def s : Set Nat := {1}
```

```
#check s  $\cup$  [2, 3] -- s  $\cup$  [2, 3].toSet : Set Nat
```

```
| s  $\cup$  [2, 3].toSet : Set Nat
```

可以使用记法 $\uparrow$ 强制在特定位置引入强制转换。这也有助于明确我们的意图，并绕过强制转换解析系统的局限。

```
def s : Set Nat := {1}
```

```
#check let x :=  $\uparrow$ [2, 3]; s  $\cup$  x -- let x := [2, 3].toSet; s  $\cup$  x
   $\hookrightarrow$  : Set Nat
```

```
| let x := [2, 3].toSet;
  s  $\cup$  x : Set Nat
```

```
#check let x := [2, 3]; s  $\cup$  x -- let x := [2, 3]; s  $\cup$  x.toSet
   $\hookrightarrow$  : Set Nat
```

```
| let x := [2, 3];
  s  $\cup$  x.toSet : Set Nat
```

Lean 还通过类型类 `CoeDep` 支持依值强制转换。例如，不能把任意命题强制转换为 `Bool`，只能转换那些实现了 `Decidable` 类型类的命题。

```
instance (p : Prop) [Decidable p] : CoeDep Prop p Bool where
  coe := decide p
```

必要时，Lean 还会链接（非依值的）强制转换。实际上，类型类 `CoeT` 是 `Coe` 的传递闭包。

现在考虑第二类强制转换。所谓 *sort* 类，是指宇宙 `Type u` 的集合。第二类强制转换具有如下形式：

$$c : (x_1 : A_1) \rightarrow \dots \rightarrow (x_n : A_n) \rightarrow F \ x_1 \ \dots \ x_n \rightarrow \text{Type } u$$

其中 F 是如上的一个类型族。这允许我们在 t 的类型为 $F \ a_1 \ \dots \ a_n$ 时写 $s : t$ 。换言之，该强制转换允许我们把 $F \ a_1 \ \dots \ a_n$ 的元素看作类型。这在定义代数结构时非常有用，因为其中一个组成部分，即结构的载体，是一个 `Type`。例如，可以如下定义半群：


```

structure Semigroup where
  carrier : Type u
  mul : carrier → carrier → carrier
  mul_assoc (a b c : carrier) : mul (mul a b) c = mul a (mul b
    ↪ c)

instance (S : Semigroup) : Mul S.carrier where
  mul a b := S.mul a b

```

换言之，一个半群由一个类型 `carrier` 和一个乘法 `mul` 组成，并满足乘法结合律这一性质。只要有 `a b : S.carrier`，**instance** 命令就允许我们写 `a * b`，而不是 `Semigroup.mul S a b`；注意，Lean 可以从 `a` 和 `b` 的类型推断出参数 `S`。函数 `Semigroup.carrier` 将类 `Semigroup` 映射到 sort `Type u`：

```

#check Semigroup.carrier -- Semigroup.carrier.{u} (self :
  ↪ Semigroup) : Type u
| Semigroup.carrier.{u} (self : Semigroup) : Type u

```

如果把这个函数声明为强制转换，那么每当有一个半群 `S : Semigroup` 时，我们就可以写 `a : S`，而不是 `a : S.carrier`：

```

instance : CoeSort Semigroup (Type u) where
  coe s := s.carrier

```

```

example (S : Semigroup) (a b c : S) : (a * b) * c = a * (b *
  ↪ c) :=
  Semigroup.mul_assoc _ a b c

```

正是这个强制转换使得写 `(a b c : S)` 成为可能。注意，我们定义的是 `CoeSort Semigroup (Type u)` 的实例，而不是 `Coe Semigroup (Type u)`。

所谓函数类型类，是指 Π 类型 $(z : B) \rightarrow C$ 的集合。第三类强制转换具有如下形式：

$$c : (x_1 : A_1) \rightarrow \dots \rightarrow (x_n : A_n) \rightarrow (y : F \ x_1 \ \dots \ x_n) \rightarrow (z : B) \rightarrow C$$

其中 `F` 同样是一个类型族，而 `B` 和 `C` 可以依赖于 `x1, ..., xn, y`。这使得每当 `t` 是 `F a1 ... an` 的元素时，都可以写 `t s`。换言之，该强制转换使我们能够把 `F a1 ... an` 的元素看作函数。继续上面的例子，我们可以定义半群 `S1` 与 `S2` 之间的态射概念。也就是说，这是一个从 `S1` 的载体到 `S2` 的载体的函数（注意这里的隐式强制转换），并且它保持乘法。投影 `Morphism.mor` 将一个态射映射到底层函数：

```

structure Morphism (S1 S2 : Semigroup) where
  mor : S1 → S2
  resp_mul : ∀ a b : S1, mor (a * b) = (mor a) * (mor b)

```

```

#check @Morphism.mor

```

```
@Morphism.mor : {S1 : Semigroup} → {S2 : Semigroup} →
  ↪ Morphism S1 S2 → S1.carrier → S2.carrier
```

因此，它是第三类强制转换的天然候选。

```
instance (S1 S2 : Semigroup) :
  CoeFun (Morphism S1 S2) (fun _ => S1 → S2) where
  coe m := m.mor
```

```
theorem resp_mul {S1 S2 : Semigroup} (f : Morphism S1 S2) (a b
  ↪ : S1)
  : f (a * b) = f a * f b :=
  f.resp_mul a b
```

```
example (S1 S2 : Semigroup) (f : Morphism S1 S2) (a : S1) :
  f (a * a * a) = f a * f a * f a :=
  calc f (a * a * a)
    _ = f (a * a) * f a := by rw [resp_mul f]
    _ = f a * f a * f a := by rw [resp_mul f]
```

有了这个强制转换后，我们可以写 $f (a * a * a)$ ，而不是 $f.mor (a * a * a)$ 。当 `Morphism` 即 f 被用于期望函数的地方时，`Lean` 会插入该强制转换。类似于 `CoeSort`，我们还有另一个类 `CoeFun` 用于这一类强制转换。参数 γ 用于指定要强制转换到的函数类型。该类型可以依赖于被强制转换来源的类型。

11 转换策略模式

在策略块中，可以使用关键字 **conv** 进入 转换模式。该模式允许在假设和目标的内部移动，甚至可以进入函数抽象和依值箭头的内部，以应用重写或化简步骤。

11.1 基本导航与重写

作为第一个例子，我们来证明 $(a\ b\ c : \text{Nat}) : a * (b * c) = a * (c * b)$ 这个例子（本文件中的例子都有些人为构造，因为其他策略可以立即完成它们）。朴素的首次尝试是进入策略模式并尝试 `rw [Nat.mul_comm]`。但这会交换项中出现的第一个乘法，从而把目标变为 $b * c * a = a * (c * b)$ 。有几种方法可以修正这个问题，其中一种是使用更精确的工具：转换模式。下面的代码块展示了每一行之后的当前目标。

```
#guard_msgs (drop all) in
example (a b c : Nat) : a * (b * c) = a * (c * b) := by
  rw [Nat.mul_comm]

example (a b c : Nat) : a * (b * c) = a * (c * b) := by
  conv =>
    lhs
    congr
    rfl
    rw [Nat.mul_comm]
```

上面的片段展示了三个导航命令：

- **lhs** 导航到关系的左侧（在这里是等式）。也有一个 **rhs** 用来导航到右侧。
- **congr** 会按照当前头函数的（非依值且显式的）参数数量创建相应数量的目标（这里的头函数是乘法）。
- **rfl** 使用自反性关闭目标。

一旦到达相关目标，就可以像在普通策略模式中那样使用 **rw**。

使用转换模式的第二个主要原因是在绑定器下进行重写。假设我们想证明例子 $(\text{fun } x : \text{Nat} \Rightarrow 0 + x) = (\text{fun } x \Rightarrow x)$ 。朴素的首次尝试是进入策略模式并尝试 `rw [Nat.zero_add]`。但这会失败，并给出令人沮丧的

error: tactic 'rewrite' failed, did not find instance of the pattern
in the target expression

```
0 + ?n
⊢ (fun x => 0 + x) = fun x => x
```

解决方法是:

```
example : (fun x : Nat => 0 + x) = (fun x => x) := by
  conv =>
    lhs
    intro x
    rw [Nat.zero_add]
```

其中 **intro x** 是进入 **fun** 绑定器内部的导航命令。注意, 这个例子有些人
为构造, 也可以这样做:

```
example : (fun x : Nat => 0 + x) = (fun x => x) := by
  funext x; rw [Nat.zero_add]
```

或者直接

```
example : (fun x : Nat => 0 + x) = (fun x => x) := by
  simp
```

conv 也可以使用 **conv at h** 来重写局部上下文中的假设 **h**。

11.2 模式匹配

使用上述命令进行导航可能会很繁琐。可以如下使用模式匹配作为捷径:

```
example (a b c : Nat) : a * (b * c) = a * (c * b) := by
  conv in b * c =>
    rw [Nat.mul_comm]
```

这只是以下写法的语法糖:

```
example (a b c : Nat) : a * (b * c) = a * (c * b) := by
  conv =>
    pattern b * c
    rw [Nat.mul_comm]
```

当然, 也允许使用通配符:

```
example (a b c : Nat) : a * (b * c) = a * (c * b) := by
  conv in _ * c => rw [Nat.mul_comm]
```

11.3 组织转换策略

在 `conv` 模式中，也可以使用花括号和 `.` 来组织策略：

```
example (a b c : Nat) : (0 + a) * (b * c) = a * (c * b) := by
  conv =>
    lhs
    congr
    . rw [Nat.zero_add]
    . rw [Nat.mul_comm]
```

11.4 转换模式中的其他策略

- **arg** *i* 进入应用的第 *i* 个非依值显式参数。

```
example (a b c : Nat) : a * (b * c) = a * (c * b) := by
  conv =>
    lhs
    arg 2
    rw [Nat.mul_comm]
```

- **args** 是 **congr** 的另一个名称。
- **simp** 将化简器应用于当前目标。它支持普通策略模式中可用的相同选项。

```
def f (x : Nat) :=
  if x > 0 then x + 1 else x + 2

example (g : Nat → Nat)
  (h1 : g x = x + 1) (h2 : x > 0) :
  g x = f x := by
  conv =>
    rhs
    simp [f, h2]
  exact h1
```

- **enter** [1, x, 2, y] 用给定的参数依次执行 **arg** 和 **intro**。
- **done** 会在存在未解决目标时失败。
- **trace_state** 显示当前策略状态。
- **whnf** 将项置于弱头范式。
- **tactic** => <tactic sequence> 返回普通策略模式。这对于解决 **conv** 模式不支持的目标，以及应用自定义的同余性和外延性引理很有用。

```

example (g : Nat → Nat → Nat)
  (h1 : ∀ x, x ≠ 0 → g x x = 1)
  (h2 : x ≠ 0)
  : g x x + x = 1 + x := by
  conv =>
    lhs
    arg 1
    rw [h1]
    . skip
    . tactic =>
      exact h2

```

- **apply** <term> 是 **tactic** => apply <term> 的语法糖。

```

example (g : Nat → Nat → Nat)
  (h1 : ∀ x, x ≠ 0 → g x x = 1)
  (h2 : x ≠ 0)
  : g x x + x = 1 + x := by
  conv =>
    lhs
    arg 1
    rw [h1]
    . skip
    . apply h2

```

12 公理与计算

我们已经看到，Lean 中实现的构造演算（Calculus of Constructions）的版本包含依值函数类型、归纳类型，以及一个宇宙层级；该层级以位于底部的非直谓、证明无关性 **Prop** 为起点。在本章中，我们考察用额外公理和规则扩展 **CIC** 的方式。以这种方式扩展一个基础系统通常很方便；它既可能使我们证明更多定理，也可能使本来可以证明的定理更易于证明。但是，加入额外公理也可能产生负面后果，而且这些后果可能超出对其正确性的担忧。特别地，公理的使用会影响定义和定理的计算内容；这正是我们将在这里探讨的问题。

Lean 旨在同时支持计算式推理和经典推理。愿意这样做的用户可以坚持使用一个“计算上纯净”的片段，该片段保证系统中的闭表达式会求值为规范范式。特别地，例如，任何类型为 **Nat** 的、计算上纯净的闭表达式都会规约为一个数码。

Lean 的标准库定义了一个额外公理，即命题外延性，以及一个商构造；后者又蕴含函数外延性原则。例如，这些扩展被用于发展集合和有限集合的理论。我们将在下文看到，使用这些定理可能会阻塞 Lean 内核中的求值，使得类型为 **Nat** 的闭项不再求值为数码。不过，Lean 在将定义编译为可执行代码时会擦除类型和命题信息；而这些公理只加入新的命题，因此它们与这种计算解释相容。即使偏重计算的用户，也可能希望使用经典的排中律来推理计算。这同样会阻塞内核中的求值，但与编译后的代码相容。

标准库还定义了一个选择原则；它与计算解释完全相悖，因为它会从断言某对象存在的命题中神奇地产生“数据”。它的使用对某些经典构造至关重要，用户可以在需要时导入它。但是，使用这一构造来产生数据的表达式没有计算内容；在 Lean 中，我们必须把这样的定义标记为 **noncomputable**，以表明这一事实。

借助一个巧妙的技巧（称为 **Diaconescu** 定理），可以从命题外延性、函数外延性和选择原则推出排中律。然而，如上所述，只要排中律以及其他经典原则不被用来制造数据，它们的使用仍然与编译相容。

概括地说，在宇宙、依值函数类型和归纳类型这一底层框架之上，标准库又加入了三个组成部分：

- 命题外延性公理
- 一个商构造，它蕴含函数外延性
- 一个选择原则，它从存在命题中产生数据。

前两者会阻塞 Lean 内部的规范化，但与代码生成相容；而第三者则不适合计算解释。下面我们将更精确地说明这些细节。

12.1 历史与哲学背景

在其历史的大部分时期，数学在本质上是计算性的：几何研究几何对象的构造，代数关注方程组的算法解法，而分析则提供计算随时间演化的系统未来行为的方法。从一个表明“对每个 x ，存在一个 y 使得……”的定理证明中，通常可以直接提取出一个算法，用于在给定 x 时计算这样的 y 。

然而，在十九世纪，数学论证复杂性的增长促使数学家发展出新的推理风格：这些风格压制算法信息，并使用对数学对象的描述，而这些描述抽象掉了对象表示方式的细节。其目标是在不陷入计算细节的情况下获得有力的“概念性”理解；但其结果是允许了一些在直接计算性解读下根本为假的数学定理。

今天，人们仍然相当一致地认为计算对数学很重要。但关于如何最好地处理计算方面的关切，却存在不同看法。从构造性的观点看，把数学同其计算根源分离开来是错误的：每一个有意义的数学定理都应当具有直接的计算解释。从经典的观点看，保持关注点分离更有成效：我们可以使用一种语言和一套方法来编写计算机程序，同时保留使用非构造性理论和方法来推理这些程序的自由。Lean 被设计为支持这两种进路。库的核心部分以构造性方式发展，但系统也提供了进行经典数学推理的支持。

从计算的角度看，依值类型论中最纯粹的部分完全避免使用 **Prop**。归纳类型和依值函数类型可以被视为数据类型，而这些类型的项可以通过不断应用规约规则来“求值”，直到没有更多规则可用为止。原则上，任何类型为 **Nat** 的闭项（也就是没有自由变量的项）都应当求值为一个数码，`succ (... (succ zero) ...)`。

引入证明无关的 **Prop** 并将定理标记为不可规约，是迈向关注点分离的第一步。其意图是，类型 $p : \mathbf{Prop}$ 的元素不应在计算中扮演任何角色；因此，从这个意义上说，项 $prf : p$ 的具体构造是“无关的”。我们仍然可以定义包含 **Prop** 类型元素的计算对象；要点在于，这些元素可以帮助我们推理计算的效果，但在从项中提取“代码”时可以被忽略。然而，**Prop** 类型的元素并非完全无害。它们包括任意类型 α 上的等式 $s = t : \alpha$ ，而这样的等式可作为强制转换使用，以便对项进行类型检查。下面我们将看到这类强制转换如何阻塞系统中的计算。不过，在一种会擦除命题内容、忽略中间类型约束并将项规约到范式的求值方案下，计算仍然是可能的。这正是 Lean 的虚拟机所做的事情。一旦采用证明无关的 **Prop**，人们就可能认为使用例如排中律 $p \vee \neg p$ 是正当的，其中 p 是任意命题。当然，按照 CIC 的规则，这同样可能阻塞计算；但如上所述，它并不妨碍生成可执行代码。只有在 section 12.5 (p. 208) 中讨论的选择原则，才会完全抹去理论中证明无关部分与数据相关部分之间的区别。

12.2 命题外延性

命题外延性是如下公理：

```
axiom propext {a b : Prop} : (a ↔ b) → a = b
```


它断言：当两个命题相互蕴含时，它们实际上相等。这与集合论解释相容；在这种解释中，任意元素 $a : \text{Prop}$ 要么为空，要么是单元素集合 $\{*\}$ ，其中 $*$ 是某个指定元素。该公理的效果是，等价命题可以在任何语境中相互替换：

```
variable (a b c d e : Prop)
```

```
theorem thm1 (h : a ↔ b) : (c ∧ a ∧ d → e) ↔ (c ∧ b ∧ d → e)
  ↪ :=
  propext h ▸ Iff.refl _
```

```
theorem thm2 (p : Prop → Prop) (h : a ↔ b) (h1 : p a) : p b :=
  propext h ▸ h1
```

12.3 函数外延性

与命题外延性类似，函数外延性断言：任意两个类型为 $(x : \alpha) \rightarrow \beta$ 的函数，只要它们在所有输入上的取值都相同，就相等：

```
funext.{u, v}
  {α : Sort u} {β : α → Sort v}
  {f g : (x : α) → β x}
  (h : ∀ (x : α), f x = g x) :
  f = g
```

从经典的集合论观点看，这正是两个函数相等的含义。这被称为函数的“外延”观点。然而，从构造性观点看，把函数看作以某种显式方式给出的算法或计算机程序，有时更为自然。当然，两个计算机程序虽然语法上相当不同，却完全可能对每个输入计算出相同答案。与此类似，你也许希望保持一种函数观，它并不强迫你把具有相同输入/输出行为的两个函数等同起来。这被称为函数的“内涵”观点。

事实上，函数外延性可以从商的存在推出；我们将在下一节描述商。因此，在 Lean 标准库中，funext 是由商构造证明的¹。

假设对 $\alpha : \text{Type } u$ ，我们定义 $\text{Set } \alpha := \alpha \rightarrow \text{Prop}$ 来表示 α 的子集类型，本质上是把子集同谓词等同起来。结合 funext 和 propext，我们便得到这类集合的外延理论：

```
def Set (α : Type u) := α → Prop
```

```
namespace Set
```

```
def mem (x : α) (a : Set α) := a x
```

```
infix:50 (priority := high) "∈" => mem
```

```
theorem setext {a b : Set α} (h : ∀ x, x ∈ a ↔ x ∈ b) : a = b
  ↪ :=
  funext (fun x => propext (h x))
```

¹<https://github.com/leanprover/lean4/blob/master/src/Init/Core.lean>

end Set

于是, 例如, 我们可以继续定义空集和集合交, 并证明集合恒等式:

```
def empty : Set  $\alpha$  := fun _ => False

notation (priority := high) "∅" => empty

def inter (a b : Set  $\alpha$ ) : Set  $\alpha$  :=
  fun x => x ∈ a ∧ x ∈ b

infix:70 " n " => inter

theorem inter_self (a : Set  $\alpha$ ) : a n a = a :=
  setext fun x => Iff.intro
    (fun ⟨h, _⟩ => h)
    (fun h => ⟨h, h⟩)

theorem inter_empty (a : Set  $\alpha$ ) : a n ∅ = ∅ :=
  setext fun _ => Iff.intro
    (fun ⟨_, h⟩ => h)
    (fun h => False.elim h)

theorem empty_inter (a : Set  $\alpha$ ) : ∅ n a = ∅ :=
  setext fun _ => Iff.intro
    (fun ⟨h, _⟩ => h)
    (fun h => False.elim h)

theorem inter.comm (a b : Set  $\alpha$ ) : a n b = b n a :=
  setext fun _ => Iff.intro
    (fun ⟨h1, h2⟩ => ⟨h2, h1⟩)
    (fun ⟨h1, h2⟩ => ⟨h2, h1⟩)
```

下面的例子说明函数外延性如何阻塞 Lean 内核内部的计算:

```
def f (x : Nat) := x
def g (x : Nat) := 0 + x

theorem f_eq_g : f = g :=
  funext fun x => (Nat.zero_add x).symm

def val : Nat :=
  Eq.recOn (motive := fun _ _ => Nat) f_eq_g 0

-- 不会规约为 0
#reduce val
| f_eq_g ▶ 0
-- 求值为 0
#eval val
```

0

首先，我们用函数外延性说明两个函数 f 和 g 相等；然后在类型中把 f 替换为 g ，从而对类型为 Nat 的 0 作强制转换。当然，这个强制转换是空洞的，因为 Nat 并不依赖于 f 。但这已足以造成影响：在系统的计算规则下，我们现在得到一个类型为 Nat 的闭项，它不会规约为数码。在这个例子中，我们也许想把该表达式规约为 0 。但在非平凡的例子中，消去强制转换会改变项的类型，从而可能使外围表达式的类型不正确。不过，虚拟机可以毫无困难地把该表达式求值为 0 。下面是一个类似的人为例子，展示 `propext` 如何造成阻碍：

```
theorem tteq : (True ∧ True) = True :=
  propext (Iff.intro (fun ⟨h, _⟩ => h) (fun h => ⟨h, h⟩))
```

```
def val : Nat :=
  Eq.recOn (motive := fun _ _ => Nat) tteq 0
```

```
-- 不会规约为 0
```

```
#reduce val
```

```
0 tteq ▶ 0
```

```
-- 求值为 0
```

```
#eval val
```

```
0
```

当前的一些研究计划，包括关于观测类型论和立方类型论的工作，旨在以允许涉及函数外延性、商以及更多构造的强制转换进行规约的方式扩展类型论。但是，解决方案并不那么清晰明确，而且 `Lean` 底层演算的规则并不认可这样的规约。

然而，从某种意义上说，强制转换并不改变表达式的含义。它更像是一种用于推理表达式类型的机制。给定合适的语义，就可以按保持含义的方式规约项，而忽略为使规约保持类型正确所需的中间簿记。在这种情况下，在 `Prop` 中加入新公理并不重要；由于证明无关性，`Prop` 中的表达式不携带信息，可以被规约过程安全地忽略。

12.4 商

令 α 为任意类型，令 r 为 α 上的等价关系。在数学中，形成“商” α / r 是很常见的；也就是说，形成 α 的元素“模” r 所得到的类型。从集合论角度看，可以把 α / r 看作 α 关于 r 的等价类集合。如果 $f : \alpha \rightarrow \beta$ 是任意尊重该等价关系的函数，即对每个 $x y : \alpha$ ， $r x y$ 都蕴含 $f x = f y$ ，那么 f 就“提升”为定义在各个等价类上的函数 $f' : \alpha / r \rightarrow \beta$ ，满足对 $[x]$ 有 $f' [x] = f x$ 。`Lean` 的标准库用额外常量扩展构造演算，正是为了实现这些构造，并将最后这个等式安装为定义性规约规则。在最基本的形式中，商构造甚至不要求 r 是等价关系。以下常量内建于 `Lean`：

```
universe u v
```

```

axiom Quot : { $\alpha$  : Sort u}  $\rightarrow$  ( $\alpha \rightarrow \alpha \rightarrow \text{Prop}$ )  $\rightarrow$  Sort u

axiom Quot.mk : { $\alpha$  : Sort u}  $\rightarrow$  ( $r : \alpha \rightarrow \alpha \rightarrow \text{Prop}$ )  $\rightarrow$   $\alpha \rightarrow$  Quot r

axiom Quot.ind :
   $\forall$  { $\alpha$  : Sort u} { $r : \alpha \rightarrow \alpha \rightarrow \text{Prop}$ } { $\beta$  : Quot r  $\rightarrow$  Prop},
    ( $\forall a, \beta$  (Quot.mk r a))  $\rightarrow$  ( $q : \text{Quot } r$ )  $\rightarrow$   $\beta$  q

axiom Quot.lift :
  { $\alpha$  : Sort u}  $\rightarrow$  { $r : \alpha \rightarrow \alpha \rightarrow \text{Prop}$ }  $\rightarrow$  { $\beta$  : Sort u}  $\rightarrow$  ( $f : \alpha$ 
     $\hookrightarrow$   $\rightarrow \beta$ )
     $\rightarrow$  ( $\forall a b, r a b \rightarrow f a = f b$ )  $\rightarrow$  Quot r  $\rightarrow$   $\beta$ 

  第一个常量在给定类型  $\alpha$  以及  $\alpha$  上任意二元关系  $r$  时形成类型 Quot r。第
  二个常量把  $\alpha$  映到 Quot  $\alpha$ , 因此若  $r : \alpha \rightarrow \alpha \rightarrow \text{Prop}$  且  $a : \alpha$ , 则 Quot.mk
   $r a$  是 Quot r 的元素。第三个原则 Quot.ind 说明, Quot.mk  $r a$  的每个元
  素都具有这种形式。至于 Quot.lift, 给定函数  $f : \alpha \rightarrow \beta$ , 若  $h$  是  $f$  尊重关
  系  $r$  的证明, 则 Quot.lift  $f h$  是 Quot r 上相应的函数。其想法是, 对  $\alpha$  中
  的每个元素  $a$ , 函数 Quot.lift  $f h$  把 Quot.mk  $r a$  (包含  $a$  的  $r$ -类) 映到  $f$ 
   $a$ , 而  $h$  说明这个函数是良定义的。事实上, 计算原则被声明为规约规则, 如下
  证明所示。

def mod7Rel (x y : Nat) : Prop :=
  x % 7 = y % 7

-- 商类型
#check (Quot mod7Rel : Type)
| Quot mod7Rel : Type
-- 与 4 等价的数所在的类
#check (Quot.mk mod7Rel 4 : Quot mod7Rel)
| Quot.mk mod7Rel 4 : Quot mod7Rel

def f (x : Nat) : Bool :=
  x % 7 = 0

theorem f_respects (a b : Nat) (h : mod7Rel a b) : f a = f b
 $\hookrightarrow$  := by
  simp [mod7Rel, f] at *
  rw [h]

#check (Quot.lift f f_respects : Quot mod7Rel  $\rightarrow$  Bool)
| Quot.lift f f_respects : Quot mod7Rel  $\rightarrow$  Bool
-- 计算原则
example (a : Nat) : Quot.lift f f_respects (Quot.mk mod7Rel a)
 $\hookrightarrow$  = f a :=
  rfl

```

这四个常量 `Quot`、`Quot.mk`、`Quot.ind` 和 `Quot.lift` 本身并不很强。你可以检查，如果简单地把 `Quot r` 取为 α ，并把 `Quot.lift` 取为恒等函数（忽略 h ），那么 `Quot.ind` 仍然成立。因此，这四个常量并不被视为额外公理。

它们和归纳定义的类型及其相关构造子与递归子一样，被视为逻辑框架的一部分。

使 `Quot` 构造成为真正商的是下面这个额外公理：

```
axiom Quot.sound :
  ∀ {α : Type u} {r : α → α → Prop} {a b : α},
    r a b → Quot.mk r a = Quot.mk r b
```

这个公理断言： α 中任何两个由 r 关联的元素，在商中都会被等同。如果某个定理或定义使用了 `Quot.sound`，它就会出现在 `#print axioms` 命令的输出中。

当然，商构造最常用于 r 是等价关系的情形。给定如上的 r ，如果我们按规则“ $r' a b$ 当且仅当 `Quot.mk r a = Quot.mk r b`”来定义 r' ，那么显然 r' 是一个等价关系。事实上， r' 是函数 `fun a => Quot.mk r a` 的核。公理 `Quot.sound` 表明 $r a b$ 蕴含 $r' a b$ 。使用 `Quot.lift` 和 `Quot.ind`，我们可以说明 r' 是包含 r 的最小等价关系；也就是说，如果 r'' 是任何包含 r 的等价关系，那么 $r' a b$ 蕴含 $r'' a b$ 。特别地，如果 r 一开始就是等价关系，那么对所有 a 和 b ，都有 $r a b$ 当且仅当 $r' a b$ 。

为了支持这种常见用法，标准库定义了 *setoid* 的概念；它不过是一个带有关联等价关系的类型：

```
class Setoid (α : Sort u) where
  r : α → α → Prop
  iseqv : Equivalence r

instance {α : Sort u} [Setoid α] : HasEquiv α :=
  (Setoid.r)

namespace Setoid

variable {α : Sort u} [Setoid α]

theorem refl (a : α) : a ≈ a :=
  iseqv.refl a

theorem symm {a b : α} (hab : a ≈ b) : b ≈ a :=
  iseqv.symm hab

theorem trans {a b c : α} (hab : a ≈ b) (hbc : b ≈ c) : a ≈ c
  ↪ :=
  iseqv.trans hab hbc

end Setoid
```

给定一个类型 α 、 α 上的关系 r ，以及证明 `iseqv` 表明 r 是等价关系，我们就可以定义 `Setoid` 类的一个实例。

```
def Quotient { $\alpha$  : Sort u} (s : Setoid  $\alpha$ ) :=
  @Quot  $\alpha$  Setoid.r
```

常量 `Quotient.mk`、`Quotient.ind`、`Quotient.lift` 和 `Quotient.sound` 无非是 `Quot` 中相应元素的特化。类型类推断能够找到与类型 α 关联的 `setoid`，这带来若干好处。首先，我们可以用记号 $a \approx b$ （通过 `\approx` 输入）表示 `Setoid.r a b`；在记号 `Setoid.r` 中，`Setoid` 的实例是隐式的。我们可以使用通用定理 `Setoid.refl`、`Setoid.symm`、`Setoid.trans` 来推理该关系。特别是对商而言，我们可以使用定理 `Quotient.exact`：

```
Quotient.exact { $\alpha$  : Sort u} {s : Setoid  $\alpha$ } {a b :  $\alpha$ } :
  Quotient.mk s a = Quotient.mk s b  $\rightarrow$ 
  a  $\approx$  b
```

结合 `Quotient.sound`，这表明商的元素与 α 中元素的等价类精确对应。

回忆一下，在标准库中， $\alpha \times \beta$ 表示类型 α 和 β 的笛卡尔积。为了说明商的用法，我们把类型 α 的元素的无序对类型定义为类型 $\alpha \times \alpha$ 的一个商。首先，我们定义相关的等价关系：

```
private def eqv (p1 p2 :  $\alpha \times \alpha$ ) : Prop :=
  (p1.1 = p2.1  $\wedge$  p1.2 = p2.2)  $\vee$  (p1.1 = p2.2  $\wedge$  p1.2 = p2.1)
```

```
infix:50 " ~ " => eqv
```

下一步是证明 `eqv` 确实是等价关系；也就是说，它是自反的、对称的和传递的。我们可以用依值模式匹配进行分类讨论，并把假设拆成若干部分，再重新组合以得到结论；这样能以方便且可读的方式证明这三个事实。

```
private theorem eqv.refl (p :  $\alpha \times \alpha$ ) : p ~ p :=
  Or.inl {rfl, rfl}
```

```
private theorem eqv.symm :  $\forall \{p1 p2 : \alpha \times \alpha\}, p1 \sim p2 \rightarrow p2 \sim$ 
 $\hookrightarrow p1$ 
| (a1, a2), (b1, b2), (Or.inl {a1b1, a2b2}) =>
  Or.inl (by simp_all)
| (a1, a2), (b1, b2), (Or.inr {a1b2, a2b1}) =>
  Or.inr (by simp_all)
```

```
private theorem eqv.trans :  $\forall \{p1 p2 p3 : \alpha \times \alpha\}, p1 \sim p2 \rightarrow p2$ 
 $\hookrightarrow \sim p3 \rightarrow p1 \sim p3$ 
| (a1, a2), (b1, b2), (c1, c2), Or.inl {a1b1, a2b2}, Or.inl
 $\hookrightarrow \{b1c1, b2c2\} =>$ 
  Or.inl (by simp_all)
| (a1, a2), (b1, b2), (c1, c2), Or.inl {a1b1, a2b2}, Or.inr
 $\hookrightarrow \{b1c2, b2c1\} =>$ 
  Or.inr (by simp_all)
| (a1, a2), (b1, b2), (c1, c2), Or.inr {a1b2, a2b1}, Or.inl
 $\hookrightarrow \{b1c1, b2c2\} =>$ 
  Or.inr (by simp_all)
| (a1, a2), (b1, b2), (c1, c2), Or.inr {a1b2, a2b1}, Or.inr
 $\hookrightarrow \{b1c2, b2c1\} =>$ 
  Or.inl (by simp_all)
```

```
private theorem is_equivalence : Equivalence (@eqv  $\alpha$ ) :=
  { refl := eqv.refl, symm := eqv.symm, trans := eqv.trans }
```

既然已经证明 `eqv` 是等价关系，我们就可以构造一个 `Setoid ( $\alpha \times \alpha$ )`，并用它定义无序对类型 `UProd  $\alpha$` 。

```
instance uprodSetoid ( $\alpha$  : Type u) : Setoid ( $\alpha \times \alpha$ ) where
  r      := eqv
  iseqv := is_equivalence
```

```
def UProd ( $\alpha$  : Type u) : Type u :=
  Quotient (uprodSetoid  $\alpha$ )
```

```
namespace UProd
```

```
def mk { $\alpha$  : Type} ( $a_1 a_2$  :  $\alpha$ ) : UProd  $\alpha$  :=
  Quotient.mk' ( $a_1, a_2$ )
```

```
notation "{ "  $a_1$  ", "  $a_2$  " }" => mk  $a_1 a_2$ 
```

```
end UProd
```

注意，我们在局部把无序对的记号 $\{a_1, a_2\}$ 定义为 `Quotient.mk' ( $a_1, a_2$ )`。这对说明例子很有用，但通常并不是好主意，因为该记号会遮蔽花括号的其他用途，例如用于记录和集合。我们可以很容易地用 `Quot.sound` 证明 $\{a_1, a_2\} = \{a_2, a_1\}$ ，因为有 $(a_1, a_2) \sim (a_2, a_1)$ 。

```
theorem mk_eq_mk ( $a_1 a_2$  :  $\alpha$ ) : { $a_1, a_2$ } = { $a_2, a_1$ } :=
  Quot.sound (Or.inr (rfl, rfl))
```

为了完成这个例子，给定 $a : \alpha$ 和 $u : \text{UProd } \alpha$ ，我们定义命题 $a \in u$ ；当 a 是无序对 u 的元素之一时，它应当成立。首先，我们在（有序）对上定义一个类似命题 `mem_fnau`；然后用引理 `mem_respects` 说明 `mem_fn` 尊重等价关系 `eqv`。这是 Lean 标准库中广泛使用的一种惯用法。

```
private def mem_fn ( $a$  :  $\alpha$ ) :  $\alpha \times \alpha \rightarrow \text{Prop}$ 
  | ( $a_1, a_2$ ) =>  $a = a_1 \vee a = a_2$ 
```

```
-- 用于证明 mem_respects 的辅助引理
```

```
private theorem mem_swap { $a$  :  $\alpha$ } :
   $\forall \{p : \alpha \times \alpha\}, \text{mem\_fn } a \ p = \text{mem\_fn } a \ (\{p.2, p.1\})$ 
  | ( $a_1, a_2$ ) => by
    apply propext
    apply Iff.intro
    . intro
      | Or.inl  $h$  => exact Or.inr  $h$ 
      | Or.inr  $h$  => exact Or.inl  $h$ 
    . intro
      | Or.inl  $h$  => exact Or.inr  $h$ 
      | Or.inr  $h$  => exact Or.inl  $h$ 
```

```

private theorem mem_respects : {p1 p2 :  $\alpha \times \alpha$ } → (a :  $\alpha$ ) → p1
  ↪ ~ p2 → mem_fn a p1 = mem_fn a p2
  | (a1, a2), (b1, b2), a, Or.inl ⟨a1b1, a2b2⟩ => by
    simp_all
  | (a1, a2), (b1, b2), a, Or.inr ⟨a1b2, a2b1⟩ => by
    simp_all only
    apply mem_swap

```

```

def mem (a :  $\alpha$ ) (u : UProd  $\alpha$ ) : Prop :=
  Quot.liftOn u (fun p => mem_fn a p) (fun p1 p2 e =>
    ↪ mem_respects a e)

```

```

infix:50 (priority := high) " ∈ " => mem

```

```

theorem mem_mk_left (a b :  $\alpha$ ) : a ∈ {a, b} :=
  Or.inl rfl

```

```

theorem mem_mk_right (a b :  $\alpha$ ) : b ∈ {a, b} :=
  Or.inr rfl

```

```

theorem mem_or_mem_of_mem_mk {a b c :  $\alpha$ } : c ∈ {a, b} → c = a
  ↪  $\forall$  c = b :=
  fun h => h

```

为方便起见，标准库还定义了 `Quotient.lift2`，用于提升二元函数；并定义了 `Quotient.ind2`，用于对两个变量进行归纳。

本节最后，我们提示一下为什么商构造蕴含函数外延性。要说明 $(x : \alpha) \rightarrow \beta$ 上的外延相等是一个等价关系并不困难，因此我们可以考虑函数“模等价”得到的类型 `extfun  $\alpha$   $\beta$` 。当然，函数应用尊重这种等价：如果 f_1 等价于 f_2 ，那么 f_1 a 等于 f_2 a 。因此，应用诱导出一个函数 `extfun_app : extfun  $\alpha$   $\beta$  → ( $x : \alpha$ ) →  $\beta$   $x$` 。但对每个 f ，`extfun_app (.mk _ f)` 按定义等于 `fun x => f x`，而后者又按定义等于 f 。所以，当 f_1 和 f_2 外延相等时，我们有如下等式链：

```

example (f1 f2 : ( $x : \alpha$ ) →  $\beta$   $x$ ) (h :  $\forall$  x, f1 x = f2 x) :=
  calc f1
    _ = extfun_app (.mk _ f1) := rfl
    _ = extfun_app (.mk _ f2) := by rw [Quot.sound]; trivial
    _ = f2 := rfl

```

因此， f_1 等于 f_2 。

12.5 选择

为了陈述标准库中定义的最后—个公理，我们需要 `Nonempty` 类型，其定义如下：

```

class inductive Nonempty ( $\alpha$  : Sort u) : Prop where
  | intro (val :  $\alpha$ ) : Nonempty  $\alpha$ 

```


由于 `Nonempty  $\alpha$` 的类型是 `Prop`，而它的构造子包含数据，因此它只能消去到 `Prop`。事实上，`Nonempty  $\alpha$` 等价于 $\exists x : \alpha, \text{True}$ ：

```
example ( $\alpha$  : Type u) : Nonempty  $\alpha$   $\leftrightarrow$   $\exists x : \alpha, \text{True}$  :=
  Iff.intro (fun (a) => (a, trivial)) (fun (a, h) => (a))
```

现在，选择公理可以简单地表述如下：

```
axiom choice { $\alpha$  : Sort u} : Nonempty  $\alpha$   $\rightarrow$   $\alpha$ 
```

仅给定断言 h ，即 α 非空，`choice h` 就会神奇地产生 α 的一个元素。当然，这会阻塞任何有意义的计算：按照 `Prop` 的解释， h 完全不包含如何找到一个元素的信息。

它位于 `Classical` 命名空间中，因此该定理的全名是 `Classical.choice`。选择原则等价于 不定描述原则，后者可以用子类型表述如下：

```
noncomputable def indefiniteDescription { $\alpha$  : Sort u}
  ( $p$  :  $\alpha \rightarrow \text{Prop}$ ) ( $h$  :  $\exists x, p x$ ) : { $x$  //  $p x$ } :=
  choice <| let (x, px) := h; (x, px)
```

由于 `indefiniteDescription` 依赖于 `choice`，Lean 无法为它生成可执行代码，因此要求我们把该定义标记为 **noncomputable**。同样在 `Classical` 命名空间中，函数 `choose` 和性质 `choose_spec` 分解 `indefiniteDescription` 输出的两个部分：

```
variable { $\alpha$  : Sort u} { $p$  :  $\alpha \rightarrow \text{Prop}$ }
```

```
noncomputable def choose ( $h$  :  $\exists x, p x$ ) :  $\alpha$  :=
  (indefiniteDescription p h).val
```

```
theorem choose_spec ( $h$  :  $\exists x, p x$ ) :  $p$  (choose h) :=
  (indefiniteDescription p h).property
```

`choice` 原则还抹去了作为 `Nonempty` 的性质与更具构造性的作为 `Inhabited` 的性质之间的区别：

```
noncomputable def inhabited_of_nonempty ( $h$  : Nonempty  $\alpha$ ) : Inhabited  $\alpha$  := choice (let (a) :=
```

在下一节中，我们将看到 `propext`、`funext` 和 `choice` 合在一起蕴含排中律以及所有命题的可判定性。利用这些原则，可以把不定描述原则加强如下：

```
strongIndefiniteDescription { $\alpha$  : Sort u} ( $p$  :  $\alpha \rightarrow \text{Prop}$ )
  ( $h$  : Nonempty  $\alpha$ ) :
  { $x$  // ( $\exists (y : \alpha), p y$ )  $\rightarrow p x$ }
```

假设外围类型 α 非空，若存在满足 p 的 α 的元素，则 `strongIndefiniteDescription p` 会产生这样一个元素。该定义的数据部分通常称为 **Hilbert** 的 **epsilon** 函数：

```
Classical.epsilon { $\alpha$  : Sort u} [ $h$  : Nonempty  $\alpha$ ] ( $p$  :  $\alpha \rightarrow \text{Prop}$ )
   $\hookrightarrow$  :  $\alpha$ 
```

```
#check @Classical.epsilon_spec
```

```
@Classical.epsilon_spec :  $\forall$  { $\alpha$  : Sort u_1} { $p$  :  $\alpha \rightarrow \text{Prop}$ }
   $\hookrightarrow$  (hex :  $\exists y, p y$ ),  $p$  (Classical.epsilon p)
```

12.6 排中律

排中律如下：

`Classical.em :  $\forall (p : \text{Prop}), p \vee \neg p$`

Diaconescu 定理²表明，选择公理足以推出排中律。更准确地说，它说明排中律可由 `Classical.choice`、`propext` 和 `funext` 推出。我们概述标准库中的证明。

首先，我们导入必要的公理，并定义两个谓词 U 和 V ：

`open Classical`

`theorem em (p : Prop) : p  $\vee$   $\neg$ p := by`

`let U (x : Prop) : Prop := x = True  $\vee$  p`

`let V (x : Prop) : Prop := x = False  $\vee$  p`

`have exU :  $\exists x, U x := \langle \text{True}, 0r.inl \text{ rfl} \rangle$`

`have exV :  $\exists x, V x := \langle \text{False}, 0r.inl \text{ rfl} \rangle$`

如果 p 为真，那么 `Prop` 的每个元素都同时属于 U 和 V 。如果 p 为假，那么 U 是单元素集合 `True`，而 V 是单元素集合 `False`。

接下来，我们使用 `choose` 分别从 U 和 V 中选择一个元素：

`let u : Prop := choose exU`

`let v : Prop := choose exV`

`have u_def : U u := choose_spec exU`

`have v_def : V v := choose_spec exV`

U 和 V 都是析取式，因此 u_def 和 v_def 表示四种情况。在其中一种情况下， $u = \text{True}$ 且 $v = \text{False}$ ；而在所有其他情况下， p 为真。因此我们有：

`have not_uv_or_p : u  $\neq$  v  $\vee$  p := by`

`match u_def, v_def with`

`| 0r.inr h, _ => exact 0r.inr h`

`| _, 0r.inr h => exact 0r.inr h`

`| 0r.inl hut, 0r.inl hvf =>`

`apply 0r.inl`

`simp [hvf, hut, true_ne_false]`

另一方面，如果 p 为真，那么由函数外延性和命题外延性， U 与 V 相等。根据 u 和 v 的定义，这又蕴含它们也相等。

`have p_implies_uv : p  $\rightarrow$  u = v :=`

`fun hp =>`

`have hpred : U = V :=`

`funext fun x =>`

`have hl : (x = True  $\vee$  p)  $\rightarrow$  (x = False  $\vee$  p) :=`

`fun _ => 0r.inr hp`

`have hr : (x = False  $\vee$  p)  $\rightarrow$  (x = True  $\vee$  p) :=`

`fun _ => 0r.inr hp`

`show (x = True  $\vee$  p) = (x = False  $\vee$  p) from`

`propext (Iff.intro hl hr)`

`have h $\theta$  :  $\forall exU exV, @choose \_ U exU = @choose \_ V exV := by$`

`rw [hpred]; intros; rfl`

`show u = v from h $\theta$  _ _`

把最后这两个事实合在一起，就得到所需结论：

²https://en.wikipedia.org/wiki/Diaconescu%27s_theorem

```

match not_uv_or_p with
| Or.inl hne =>
  exact Or.inr (mt p_implies_uv hne)
| Or.inr h    =>
  exact Or.inl h

```

排中律的后果包括双重否定消去、分类证明和反证法；这些都在 section 3.5 (p. 40) 一节中描述。排中律和命题外延性蕴含命题完备性：

```

open Classical
theorem propComplete (a : Prop) : a = True ∨ a = False :=
  match em a with
  | Or.inl ha =>
    Or.inl (propext (Iff.intro (fun _ => True.intro) (fun _ =>
      ↪ ha)))
  | Or.inr hn =>
    Or.inr (propext (Iff.intro (fun h => hn h) (fun h =>
      ↪ False.elim h)))

```

再结合选择原则，我们还得到更强的原则：每个命题都是可判定的。回忆一下，Decidable 命题类定义如下：

```

class inductive Decidable (p : Prop) where
| isFalse (h : ¬p) : Decidable p
| isTrue  (h : p)  : Decidable p

```

与只能消去到 Prop 的 $p \vee \neg p$ 不同，类型 Decidable p 等价于和类型 Sum $p (\neg p)$ ，而后者可以消去到任意类型。编写 if-then-else 表达式所需要的正是这种数据。作为经典推理的一个例子，我们使用 choose 说明：如果 $f : \alpha \rightarrow \beta$ 是单射且 α 是有居留元的，那么 f 有左逆。为了定义左逆 linv，我们使用依值的 if-then-else 表达式。回忆 **if** $h : c$ **then** t **else** e 是 $\text{dite } c \text{ (fun } h : c \Rightarrow t) \text{ (fun } h : \neg c \Rightarrow e)$ 的记号。在 linv 的定义中，选择被使用了两次：首先，用来说明 $(\exists a : \alpha, f a = b)$ 是“可判定的”；然后，用来选择一个满足 $f a = b$ 的 a 。注意，propDecidable 是一个作用域实例，由 open Classical 命令激活。我们用这个实例来为 if-then-else 表达式提供依据。（另见 section 10.8 (p. 186) 中的讨论。）

```

open Classical

noncomputable def linv [Inhabited  $\alpha$ ] (f :  $\alpha \rightarrow \beta$ ) :  $\beta \rightarrow \alpha$  :=
  fun b :  $\beta$  => if ex : ( $\exists a : \alpha, f a = b$ ) then choose ex else
    ↪ default

```

```

theorem linv_comp_self {f :  $\alpha \rightarrow \beta$ } [Inhabited  $\alpha$ ]
  (inj :  $\forall \{a\ b\}, f a = f b \rightarrow a = b$ )
  : linv f  $\circ$  f = id :=

```

```

  funext fun a =>
    have ex :  $\exists a_1 : \alpha, f a_1 = f a$  := (a, rfl)
    have feq : f (choose ex) = f a := choose_spec ex
    calc linv f (f a)
      _ = choose ex := rfl
      _ = a         := inj feq

```

从经典观点看，`linv` 是一个函数。从构造性观点看，它不可接受；因为一般而言无法实现这样的函数，所以该构造并不提供信息。